

linked lists

(i) why we need linked list?

array initialization and printing its content:

```
#include <stdio.h>
#define size 10 // Macro for array size

void main() {
    int A[size] = {10, 20, 30, 5}; // Initialize the first four elements explicitly
    // The remaining elements are implicitly initialized to 0

    // Printing the array elements
    printf("Array elements are:\n");
    for (int i = 0; i < size; i++) {
        printf("%d ", A[i]);
    }
    printf("\n");
}
```

(i) we have to define the size of array, size here represents that how much elements can be stored in an array

The array `A[size]` is declared with a size of 10.

```
#include <stdio.h>
#define size 10 // Macro for array size
```

0	1	2	3	4	5	6	7	8	9

(ii) fill in the elements.

When initializing an array in C, you can specify values for some indices, and the remaining indices are automatically set to 0.

For example: `int A[10] = {10, 20, 30, 5};` results in `A[0] = 10`, `A[1] = 20`, `A[2] = 30`, `A[3] = 5`, and `A[4]` to `A[9] = 0`.

10	20	30	5						
0	1	2	3	4	5	6	7	8	9

```
void main() {
    int A[size] = {10, 20, 30, 5}; // Initialize the first four elements explicitly
    // The remaining elements are implicitly initialized to 0
```

to analysis:

`n`: number of elements in the array.

`size`: number of elements that can be stored in an array.

(iii) perform the operation:

(i) i want to insert a new element `x=100` at the end of the array.

```

#include <stdio.h>
#define size 10 // Macro for array size

void main() {
    int A[size] = {10, 20, 30, 5}; // Initialize the array with 4 elements
    int current_size = 4; // Track the number of elements in the array
    int x = 100; // Element to insert

    // Check if there is space in the array
    if (current_size < size) {
        A[current_size] = x; // Insert x at the end
        current_size++; // Update the current size
    } else {
        printf("Array is full. Cannot insert element.\n");
    }

    // Print the updated array
    printf("Updated array elements are:\n");
    for (int i = 0; i < current_size; i++) {
        printf("%d ", A[i]);
    }
    printf("\n");
}

```

value of n(current size) is 4 here.

so last index will be n-1

x = 100

- to insert the element at the end.

$a[n(\text{current size})] = x$

add the element to the nth (current) index.

$a[4] = 100$

now, in the 4th block there is 100.

10	20	30	5	100					
0	1	2	3	4	5	6	7	8	9

we will update the current size for future reference.

$n(\text{current size}) = n(\text{current size}) + 1$

$n(\text{current size}) = 4 + 1$

$n(\text{current size}) = 5$

(ii) i want to insert the element $x=100$ at index [1] instead of last.

Logic :

(a) the array firstly must have space to store the new element incoming.

(b) we need to shift the elements to right until we do not reach the targeted index

(c) then we will insert the desired element $x=100$

10	20	30	5						
0	1	2	3	4	5	6	7	8	9

10		20	30	5					
0	1	2	3	4	5	6	7	8	9

```
#include <stdio.h>
#define size 10 // Macro for array size

void main() {
    // Initialize the array with 4 elements, the rest are 0
    int A[size] = {10, 20, 30, 5}; // Initial values
    int current_size = 4; // Track the number of elements in the array
    int x = 100; // Element to insert
    int index = 1; // Target index for insertion

    // Check if there is space in the array
    if (current_size >= size) {
        printf("Array is full. Cannot insert element.\n");
    } else if (index < 0 || index > current_size) {
        printf("Invalid index. Cannot insert element.\n");
    } else {
        // Shift elements to the right, starting from the last element
        for (int i = current_size; i >= index + 1; i--) {
            A[i] = A[i - 1]; // Move each element one position to the right
        }

        // Insert the new element at the target index
        A[index] = x;

        // Increment the current size to reflect the new element
        current_size++;

        // Print the updated array
        printf("Updated array elements are:\n");
        for (int i = 0; i < current_size; i++) {
            printf("%d ", A[i]);
        }
        printf("\n");
    }
}
```

if ($4 \geq 10$)
array is full. (will not be printed)

else if ($i < 0 \parallel i > 4$)
i: cannot be -1
 $i > 4$: valid indices for insertion are from 0 to 4.

if the index is negative (index < 0) or the index exceeds the current size (index > current_size), the condition will evaluate to true.

- print an error message indicating the index is invalid.
- not perform the insertion, because it's not possible to insert at an invalid position.

Scenarios:

Index (e.g., index = 2):

if current_size = 4, the program allows insertion at index 2 (between the second and third elements).

Invalid Index (e.g., index = -1):

- if current_size = 4, index = -1 is invalid because indices cannot be negative. The program will print an error message:

```
lua
Invalid index. Cannot insert element.
```

3. Index Beyond Current Size (e.g., index = 5):

- If current_size = 4, index = 5 is invalid because the maximum valid index is 4. The program will print:

```
lua
Invalid index. Cannot insert element.
```

Initial Array:

```
c
int a[5] = {10, 20, 30, 5}; // current_size = 4
```

We want to insert the value 100 at index 1.

Step 1: Shifting Elements

- current_size = 4, so the loop starts from i = 4 and goes to i = index + 1 = 2:
- Iteration 1 (i = 4): Shift element at index 3 (which is 5) to index 4.

```
c
a[4] = a[3]; // a = {10, 20, 30, 5, 5}
```

```
c
a[4] = a[3]; // a = {10, 20, 30, 5, 5}
```

```
for (i= current_size; i>=index+1; i--){
a[i]= a[i-1];
}
```

```
for (i=4; i>=1+1; i--){
a[4]= a[4-1];
a[4]= a[3];
}
```

10	20	30	5						
----	----	----	---	--	--	--	--	--	--

0 1 2 3 4 5 6 7 8 9

10	20	30	5	5					
----	----	----	---	---	--	--	--	--	--

0 1 2 3 4 5 6 7 8 9

- **Iteration 2 (i = 3):** Shift element at index 2 (which is 30) to index 3 .

```
c
a[3] = a[2]; // a = {10, 20, 30, 30, 5}
```

```
for (i=3; i>=1+1; i--){
a[3]= a[3-1];
a[3]= a[2];
}
```

10	20	30	5	5					
----	----	----	---	---	--	--	--	--	--

0 1 2 3 4 5 6 7 8 9

10	20	30	30	5					
----	----	----	----	---	--	--	--	--	--

0 1 2 3 4 5 6 7 8 9

- **Iteration 3 (i = 2):** Shift element at index 1 (which is 20) to index 2 .

```
c
a[2] = a[1]; // a = {10, 20, 20, 30, 5}
```

```
for (i=2; i>=1+1; i--){
a[2]= a[2-1];
a[2]= a[1];
}
```

10	20	30	30	5					
----	----	----	----	---	--	--	--	--	--

0 1 2 3 4 5 6 7 8 9

10	20	20	30	5					
----	----	----	----	---	--	--	--	--	--

0 1 2 3 4 5 6 7 8 9

```
for (i=1; i>=1+1; i--){
will not run
}
```

At this point, the array looks like this:

 Copy code

```
{10, 20, 20, 30, 5}
```

Step 2: Insert the New Element

Now, we insert the new element `100` at `index = 1`:

 Copy code

```
c  
  
a[index] = 100; // a = {10, 100, 20, 30, 5}
```

```
a[index]=x;
```

```
a[1]=100;
```

10	20	30	30	5					
----	----	----	----	---	--	--	--	--	--

0 1 2 3 4 5 6 7 8 9

10	100	20	30	5					
----	-----	----	----	---	--	--	--	--	--

0 1 2 3 4 5 6 7 8 9

Explanation of Time Complexity:

In the worst case:

- You are inserting an element at the **beginning** of the array (i.e., at `index = 0`).
- To make room for the new element, **every element in the array** (except the one at the target index) must be shifted one position to the right.

1. Loop to Shift Elements:

The loop for shifting elements starts from `current_size` (the last index with a valid element) and moves backwards toward the target `index`. In the worst case, you would have to shift **all the elements** (when inserting at index 0).

- If there are `n` elements in the array, the loop will run for each element from `current_size` down to `index + 1`.
- Each element from the last position to the insertion point is shifted, which involves copying the element to the next index.

2. Shifting Elements:

- The number of shifts in the worst case is equal to the number of elements in the array, i.e., `n`.
- In the worst case (inserting at the start of the array, at index `0`), you shift **all `n` elements** by one position.

3. Insertion:

- After the shift, the new element is inserted at the target index. This operation is constant time: $O(1)$.

Worst-Case Complexity:

The main contributor to the time complexity is the loop that shifts the elements. Since this loop runs at most n times (in the worst case), the overall time complexity is:

SCSS

Copy code

$O(n)$

so, in arrays

(i) size are fixed (static)

(ii) insertion and deletion both are time consuming.

advantage:

(i) random access: constant time

(ii) cache friendliness.

concept: self referential structure.

A **self-referential structure** (or **recursive structure**) is a structure in C (or similar languages) that contains a pointer to an instance of the same type as one of its members. In simpler terms, it is a structure that points to itself.

Self-referential structures are commonly used to create linked data structures such as **linked lists**, **trees**, and **graphs**, where each element (node) points to the next element or some other related element of the same type.

self-referential structure named pankaj, which represents a linked list-like structure where each node contains an integer (data) and a pointer (ptr) to the next node of the same type.

```

#include <stdio.h>

// Define the self-referential structure
struct pankaj {
    int data;           // Data to store the integer value
    struct pankaj *ptr; // Pointer to the next structure (of same type)
};

void main() {
    // Declare three instances of the structure
    struct pankaj p1, p2, p3;

    // Assign values to the data fields
    p1.data = 10;
    p2.data = 20;
    p3.data = 30;

    // Link the nodes using the ptr pointer
    p1.ptr = &p2; // p1 points to p2
    p2.ptr = &p3; // p2 points to p3
    p3.ptr = NULL; // p3 is the last node, so it points to NULL

    // Print the linked list
    struct pankaj *current = &p1; // Start with the first node (p1)
    while (current != NULL) {
        printf("%d → ", current->data); // Print the current node's data
        current = current->ptr;          // Move to the next node
    }
    printf("NULL\n"); // Indicate the end of the list
}

```

1. Structure Definition:

```

struct pankaj {
    int data;
    struct pankaj *ptr;
};

```

Yes, exactly! In the structure definition `struct pankaj`, there are two members:

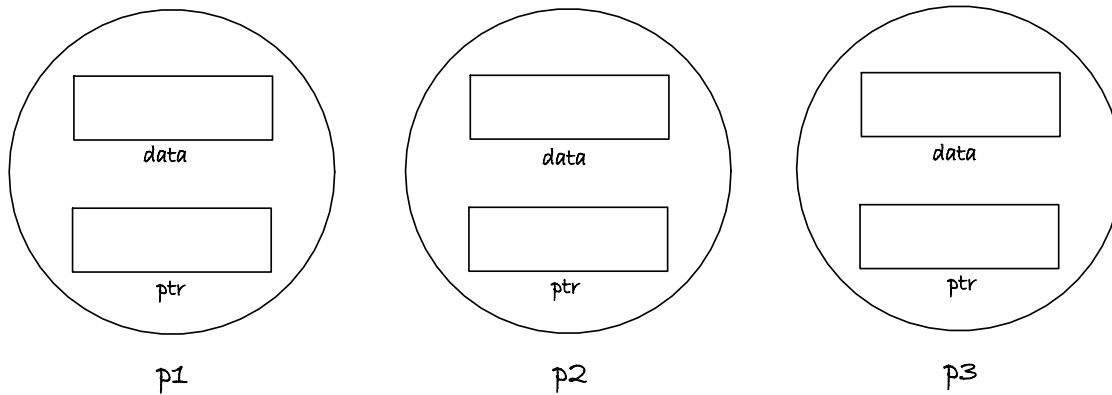
Explanation of the Two Members:

1. `int data;`
 - This is just an integer variable that stores some value. In a linked list, it could represent the actual data that you want to store in each node of the list.
2. `struct pankaj * ptr;`
 - This is a pointer of type `struct pankaj*`. It points to another `struct pankaj` instance, essentially linking each structure instance to the next one. The pointer enables the creation of a chain or list of such structures, which is the core concept behind a **linked list**.

2. Creating Instances:

```
struct pankaj p1, p2, p3;
```

- We create three variables `p1`, `p2`, and `p3`, which are instances of the `struct pankaj`.



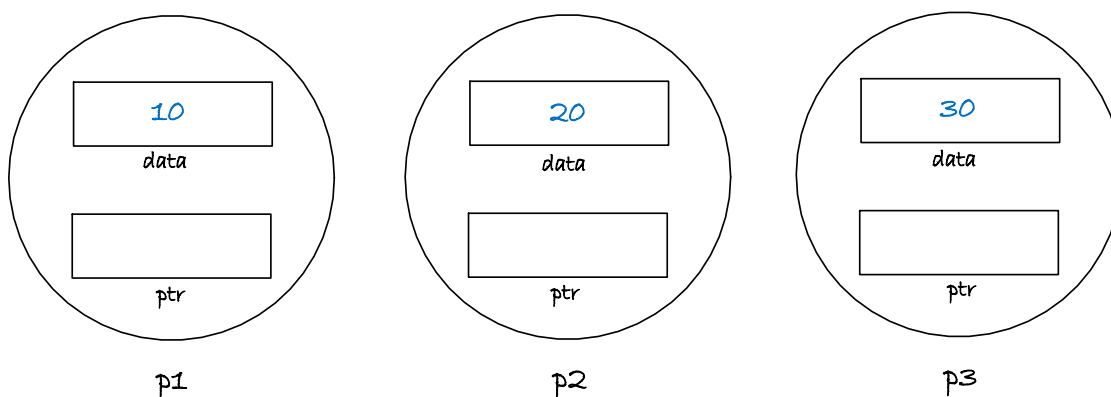
3. Assigning Data Values:

```
c
p1.data = 10;
p2.data = 20;
p3.data = 30;
```

membership operator: to access or "check" the members of a structure, you use the dot operator (.)

Copy code

- Assign integer values to the `data` field of each structure.



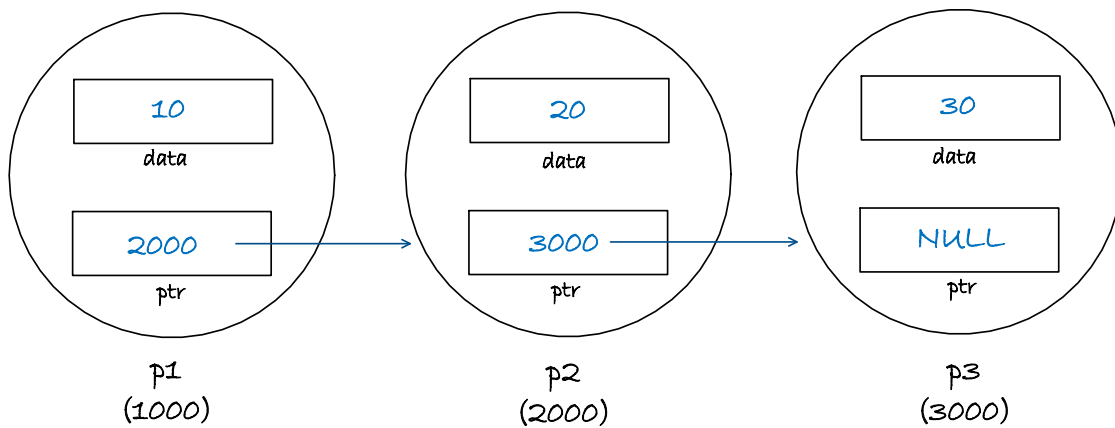
4. Linking the Nodes:

c

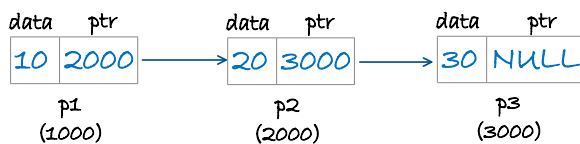
Copy code

```
p1.ptr = &p2; // p1 points to p2 p1.ptr = 2000
p2.ptr = &p3; // p2 points to p3 p2.ptr = 3000
p3.ptr = NULL; // p3 is the last node, so it points to NULL p3.ptr = NULL
```

- `p1.ptr = &p2;` makes `p1` point to `p2`. This means that `p1` is linked to `p2`.
- `p2.ptr = &p3;` makes `p2` point to `p3`, linking `p2` to `p3`.
- `p3.ptr = NULL;` marks the end of the list. Since `p3` is the last node, its `ptr` points to `NULL`, indicating the end of the linked list.



linked list:



topic: linked list

a linked list is a linear data structure, which is collection of elements called nodes, in which every node is divided into two parts

(i) data

(ii) contains address of next node in linked list (pointer to next node)

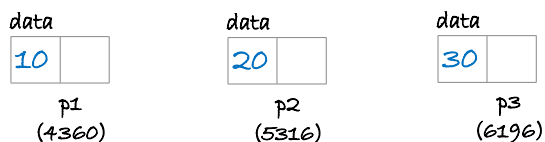
A **linked list** is a linear data structure in which elements (called nodes) are stored in separate memory locations and connected via pointers. Unlike arrays, linked lists allow for efficient insertion and deletion of elements, as elements are not stored in contiguous memory locations. Linked lists are fundamental in many algorithms and systems, such as implementing stacks, queues, and dynamic memory allocation.

Basic Terminology:

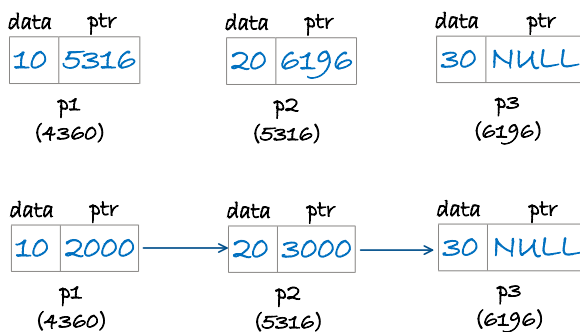
1. **Node:** A basic element of a linked list. It contains:
 - **Data:** The actual data stored in the node.
 - **Pointer:** A reference (or pointer) to the next node in the list.
2. **Head**^(start): The first node in the linked list. It serves as the entry point to the list. If the list is empty, the head is `NULL`.
3. **Tail:** The last node in the list, which points to `NULL` (indicating the end of the list).

A **linear data structure** is a type of data structure in which the elements are arranged in a sequential or linear order. In such data structures, each element is connected to its previous and next element (except for the first and last elements), forming a straightforward sequence.

(i) there is no linear order in memory



(ii) to maintain the linear order i have to put address in the node, to hold the address we need pointer.



so, linear order is maintain explicitly using pointers.

concept: how to implement a node

To implement a **node** in a linked list, you typically define a **struct** in languages like C or C++. A node in a linked list contains two main parts:

1. **Data:** This is the actual value stored in the node.
2. **Pointer:** This is a reference (or pointer) to the next node in the list.

Steps to Implement a Node:

1. **Define a Structure:** The structure (`struct`) will represent a node. It contains:

- A data field to store the value. (`int data;`)
- A pointer to the next node, which links it to the next element in the list. (`struct node *next;`)

2. **Create a Node:** To create a node, we typically allocate memory for it dynamically (using `malloc()` in C). After that, we assign values to the node's data and set the next pointer. `struct node *ptr;`
`ptr=malloc(sizeof(structnode));`

1. Defining the Node Structure:

In C, a node can be defined using a `struct`. Here's how you define a node structure for an integer data type:

```
#include <stdio.h>
#include <stdlib.h>

// Define the structure of a node in a singly linked list
struct Node {
    int data;           // The data part of the node
    struct Node *next; // Pointer to the next node
};
```

node: `int data next`

Node Structure: The `Node` structure contains two fields:

- `int data` : The actual value (or data) stored in the node.
- `struct Node* next` : A pointer to the next node in the list.

2. Creating a New Node:

To create a node, you would allocate memory for it using `malloc()` and then assign values to the `data` and `next` fields.

```
// Function to create a new node
struct Node* createNode(int data) {
    // Allocate memory for a new node
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
```

pointer to store the
returned address

return the block
address equal to the
size of struct node

```
malloc(sizeof(struct Node)):
```

- `malloc()` allocates the required amount of memory (the size of the `Node` structure) on the heap. This memory is not automatically initialized, so you would typically assign values to the fields later.
- The cast `(struct Node*)` is necessary in C to tell the compiler that the returned `void*` pointer from `malloc()` should be treated as a pointer to `struct Node`.

Why use `malloc(sizeof())`? `ptr=malloc(sizeof(structnode));`

1. **Dynamically Allocating Memory:**

- The main purpose of `malloc()` is to allocate a specific amount of memory from the heap at runtime.
- Unlike arrays, where the size is determined at compile-time, `malloc()` allows you to allocate memory based on the size of a type or structure at runtime.

2. `sizeof()` **Ensures Correct Memory Allocation:**

- The `sizeof()` operator in C returns the size (in bytes) of the given data type or structure. This is important because the size of different types (such as `int`, `float`, `struct`, etc.) may vary depending on the system architecture (32-bit vs 64-bit) and compiler.
- By using `sizeof()`, we make sure that we allocate enough memory to store the correct number of elements, regardless of the system's architecture.

```
ptr=malloc(sizeof(structnode));
```

size equal to node

```
ptr=malloc(sizeof(structnode));
```

returns
a block

size equal to node

`malloc` returns the address, so to save the address we need a pointer.

code: new node

```
#include <stdio.h>
#include <stdlib.h>
```

```
// Define the structure of the node
struct node {
    int data;           // To store the value of the node
    struct node *next; // Pointer to the next node in the list
};
```

memory
is not
allocated
here

```
int data; // To store the value of the node
struct node *next; // Pointer to the next node in the list
};
```

is not
allocated
here

```
void main() {
    // Declare a pointer to a node
    struct node *temp;

    // Dynamically allocate memory for a new node
    temp = (struct node *)malloc(sizeof(struct node)); // Allocating memory for one node

    // Check if memory allocation was successful
    if (temp == NULL) {
        printf("Memory allocation failed\n");
        return; // Exit the program if malloc fails
    }

    // Assign values to the newly created node
    temp->data = 10; // Assigning value 10 to the data of the node
    temp->next = NULL; // This node is the last node, so its next is NULL

    // Print the data of the node
    printf("Node Data: %d\n", temp->data);

    // Free the dynamically allocated memory
    free(temp); // Free the allocated memory for the node
}
```

pointer: temp that is pointing to node

temp 5012

node: int data next

pointer to node 5012

temp 5012

temp is a
pointer to
structure

data next
node: 10 NULL

pointer to node 5012

temp 5012

- how to access the members?

In C, when you're working with structures, there are two main ways to access the members of a structure:

1. Using the dot operator (`.`) for structure instances.
2. Using the arrow operator (`->`) for pointers to structures.

1. The Dot Operator (`.`)

- The dot operator is used when you have an instance (i.e., a variable) of a structure.

```
#include <stdio.h>

struct node {
    int data;
    struct node *next;
};

int main() {
    // Declare a structure instance
    struct node n1;

    // Access members of the structure using the dot operator
    n1.data = 10;
    n1.next = NULL;

    // Print data
    printf("Data: %d\n", n1.data);

    return 0;
}
```

memory
is not
allocated
here

node n1:

int data	next
5012	

node n1:

data	next
10	NULL
	5012

10

Explanation:

- `n1` is an instance of the structure `node`.
- To access the `data` field of `n1`, we use the dot operator (`n1.data`).

2. The Arrow Operator (`->`)

- The arrow operator is used when you have a **pointer** to a structure. It is essentially a shorthand for dereferencing the pointer and then accessing the member.

```

#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

int main() {
    // Declare a pointer to a structure
    struct node *n2;

    // Dynamically allocate memory for the structure
    n2 = (struct node *)malloc(sizeof(struct node));

    // Access members of the structure using the arrow operator
    n2->data = 20; (assign 20 to the data of n2)
    n2->next = NULL; (assign null to the next of n2)

    // Print data
    printf("Data: %d\n", n2->data);

    // Free dynamically allocated memory
    free(n2);

    return 0;
}

```

memory is not allocated here

n2

n2 5012

node: int data next 5012

pointer to node

n2 5012

node: 20 NULL 5012

data next

pointer to node

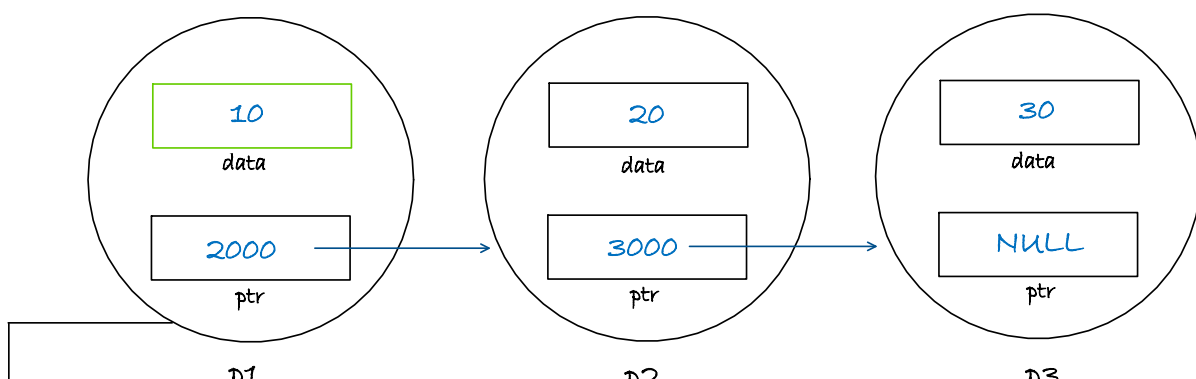
20

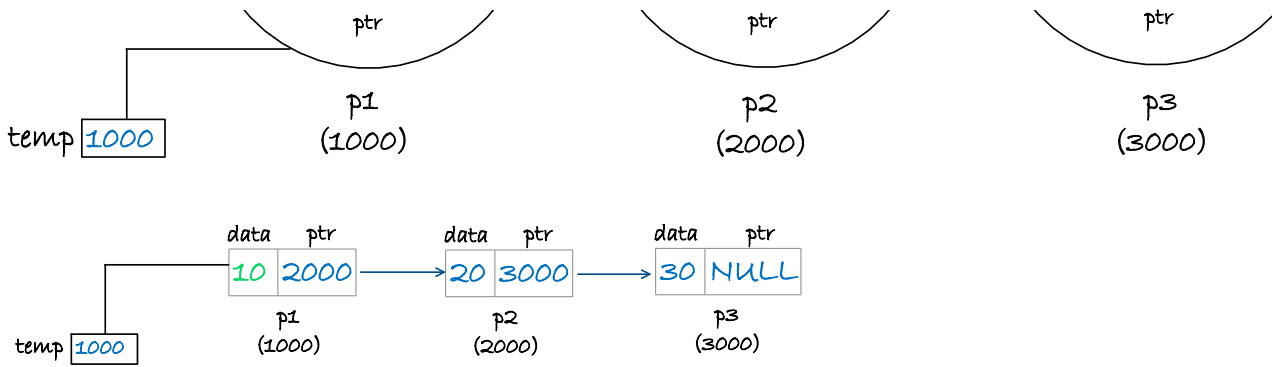
Explanation:

- `n2` is a pointer to a `struct node`.
- To access the `data` field of `n2`, we use the arrow operator (`n2->data`), which is equivalent to `(*n2).data`.
- `n2->data` is a shorthand for `(*n2).data`, where `*n2` dereferences the pointer `n2` to get the structure it points to.

(i) `temp->data`

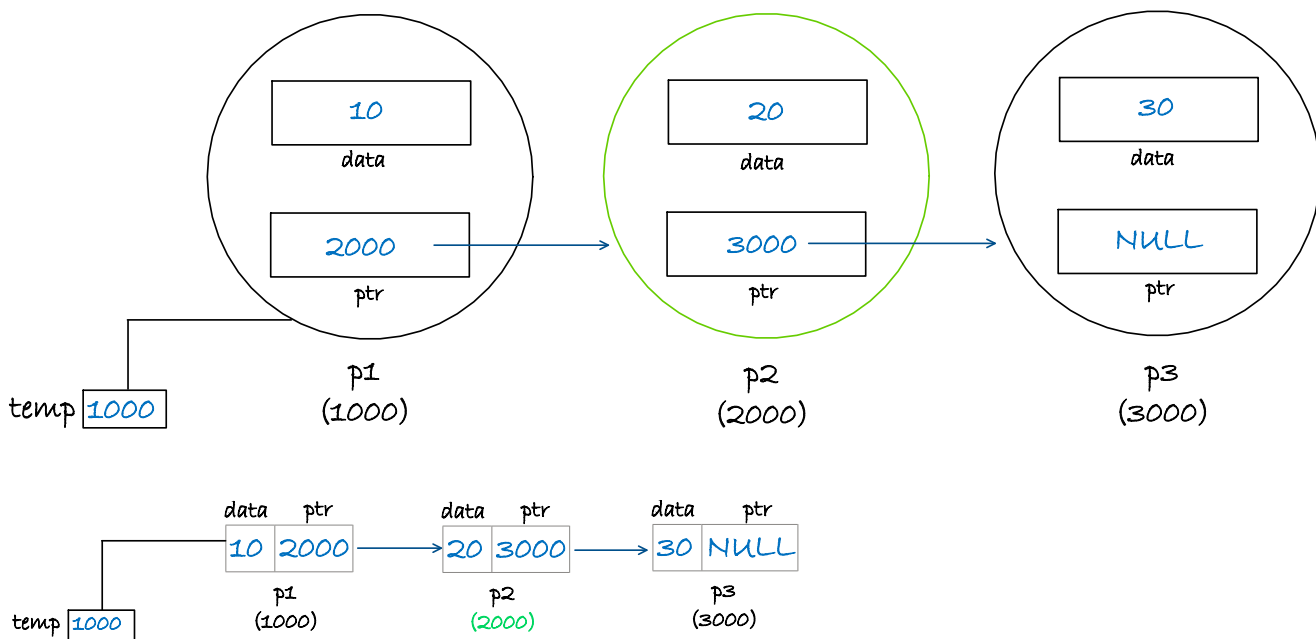
`temp` is pointer pointing to structure `p1`
 so, if i want to access data of `p1`
`temp->data: 10`





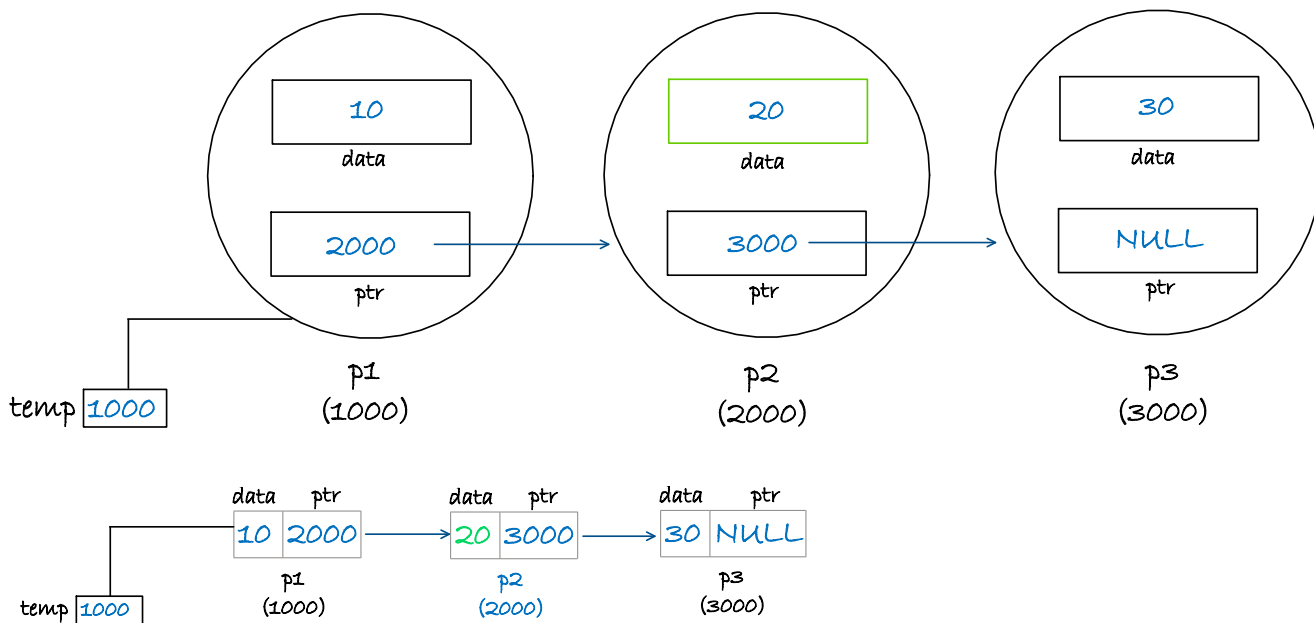
(ii) `temp->ptr`

`temp` is pointer pointing to structure `p1`
`ptr` is pointer storing the address of `p2`
`temp->ptr`: 2000 (standing outside)



(iii) `(temp->ptr)->ptr`

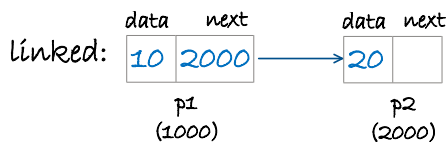
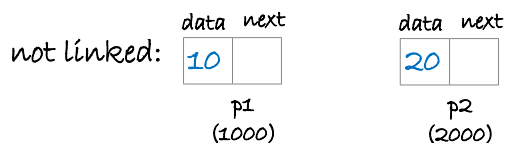
`temp` is pointer pointing to structure `p1`
`ptr` is pointer storing the address of `p2`
`temp->ptr`: 2000 (standing outside)
`temp->ptr->data`: 20



summary:

(a) how to link?

put the address of second node in the ptr field of first node.

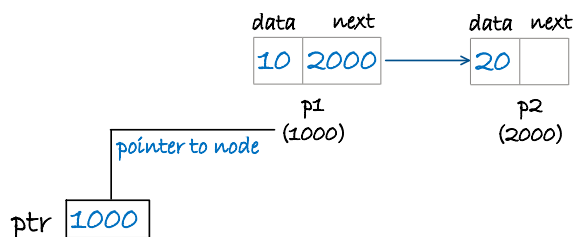


(b) how to handle node?

we need pointer to node.

declare a pointer to structure

`structnode *ptr`



(c) how to access member?

- if i want the data

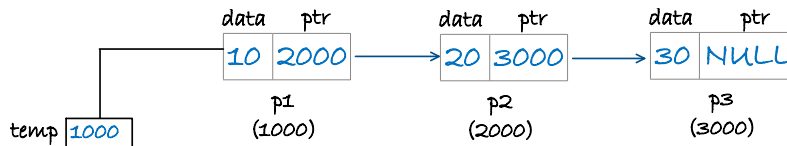
`ptr->data : 10`

- if i want to move to the next node

ptr->next : 2000

the next field of first node, memory location 2000 (address of second node)

accessing whole node:



(i) temp->data: 10

(ii) temp->ptr: memory location 2000

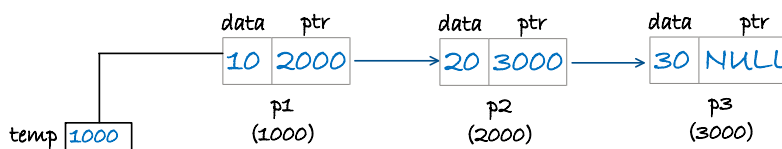
(iii) (temp->ptr)->data: 20

(iv) (temp->ptr)->ptr: memory location 3000

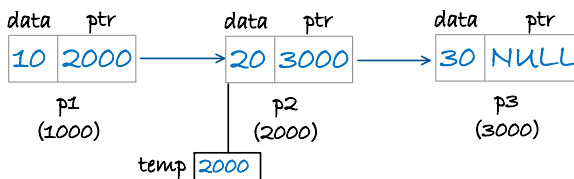
(v) ((temp->ptr)->ptr)->data: 30

(vi) ((temp->ptr)->ptr)->ptr: null

concept: how to move in linked list (next node)



temp=temp->ptr: store memory location 2000 in temp



concept: head/start

1. What is head / start ?

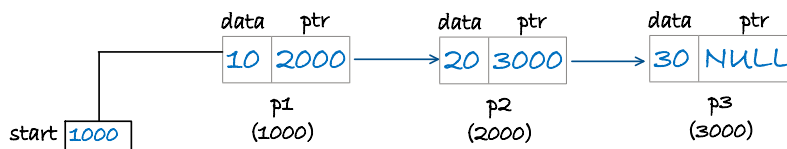
- **head** or **start** is a **pointer** that always points to the **first node** of a linked list.
- It acts as the entry point to the list, as it is typically the only way to access the entire linked list.
- Without the **head**, the entire list may become inaccessible, especially if nodes are allocated dynamically.

2. Why is head Important?

- It helps traverse the linked list from the beginning.
- If we lose the reference to the `head`, we lose the reference to the entire linked list.
- In most implementations of linked lists, the `head` pointer is maintained separately to keep track of the list.

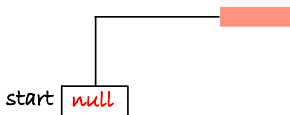
(i) start/head hold the address of first node

`structnode *start`

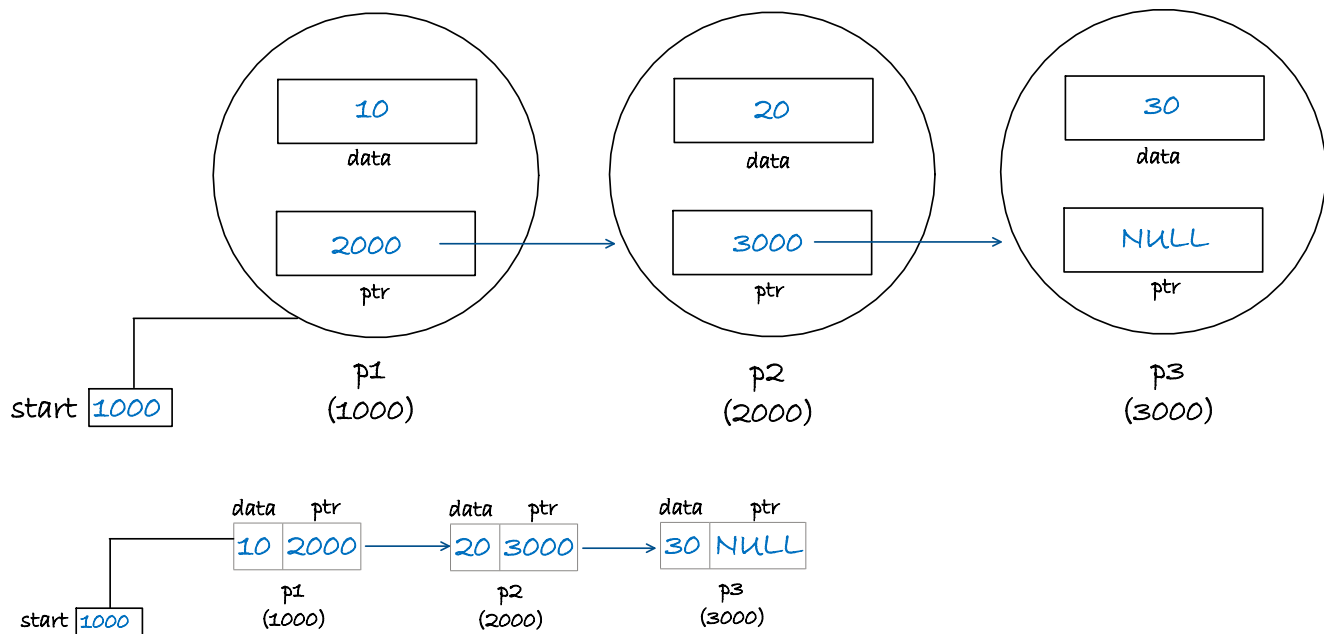


(ii) what if linked list is empty?

no valid address hence null.

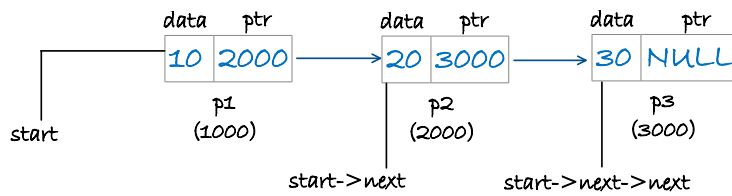


example:

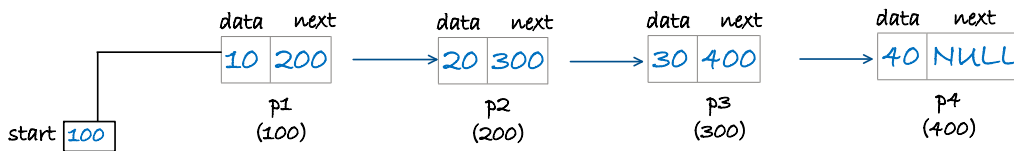


start: address of 1st node (pointer to 1st node), pointer to structure.

- start->data: 10
- start->next: memory location 2000, (pointer to 2nd node), pointer to second structure.
- (start->next)->data: data at memory location 2000: 20
- (start->next)->next: memory location 3000, (pointer to 3rd node), pointer to third structure.



code analysis (i):



```
c
Copy code

struct node *ptr,
ptr = start->next->next;
ptr->next = start;
printf("%d", start->next->next->next->data);
```

execution line by line:

(i) declaration of pointer to structure.

```
c
Copy code

struct node *ptr;
```

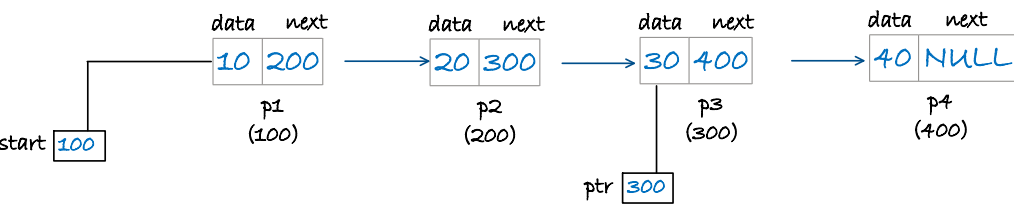
Explanation: Declares a pointer `ptr` of type `struct node *`. This pointer will be used to navigate or modify the linked list.

(ii) `ptr=(start->next)->next`

```
c
Copy code

ptr = start->next->next;
```

`(start->next)->next: 300`
`ptr=300`



Explanation: Initializes `ptr` to point to the third node in the linked list by traversing from the `start` node through two `next` pointers.

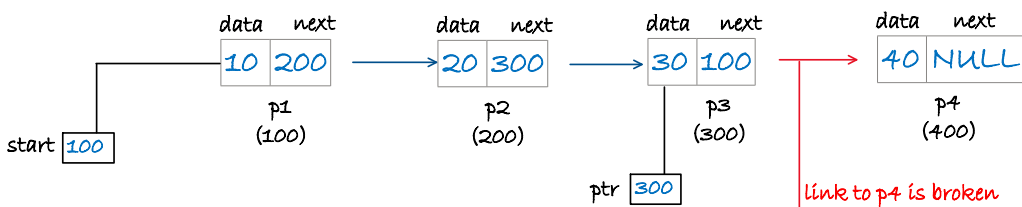
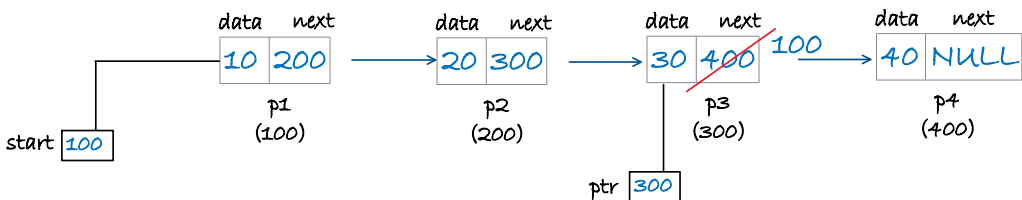
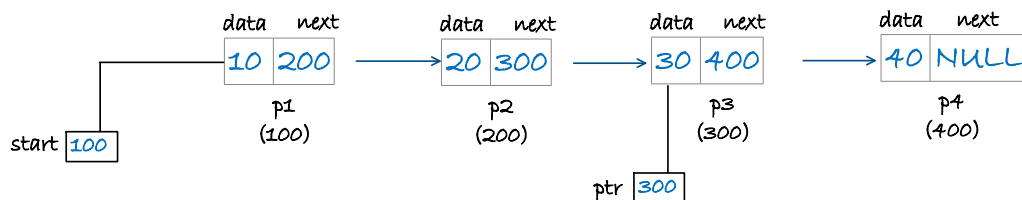
(iii) `ptr->next=start;`

```
c
ptr->next = start;
```

start: 100

`ptr->next=100;`

`300->next=100;`



Explanation: Modifies the `next` pointer of the node pointed to by `ptr` (third node) to point back to `start`, creating a circular reference in the linked list.

(iv) `((((start->next)->next)->next)->data);`

```
c
printf("%d", start->next->next->next->data);
```

start->next: 200

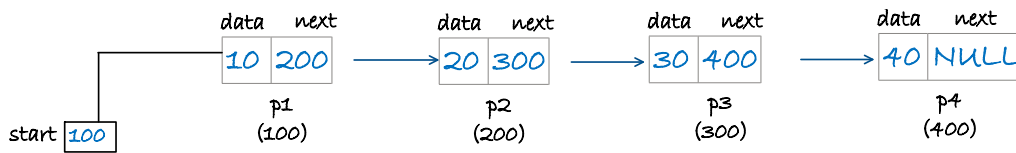
200->next: 300

300->next: 100

100->data: 10

Explanation: Accesses the `data` of the node pointed to by `start->next->next->next`. Due to the circular reference, this resolves to the `data` of the `start` node.

code analysis (ii):



```
c
Copy code

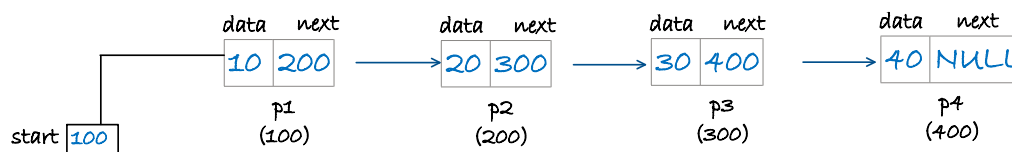
struct node *ptr;
ptr = start;
for (i = 1; i <= 3; i++)
    ptr = ptr->next;
ptr->next = start->next->next;
printf("%d", start->next->next->data);
```

1. Declare `ptr` and initialize it to `start`:

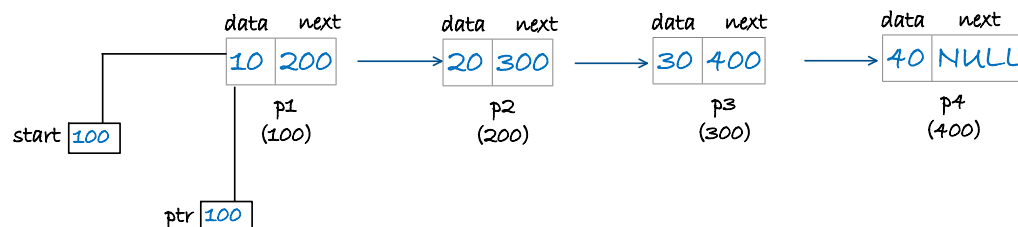
```
c
Copy code

struct node *ptr;
ptr = start;
```

- A pointer `ptr` of type `struct node *` is declared and initialized to point to the `start` node (the head of the list).



`ptr=start;`
`ptr=100`



2. Traverse the list using the `for` loop:

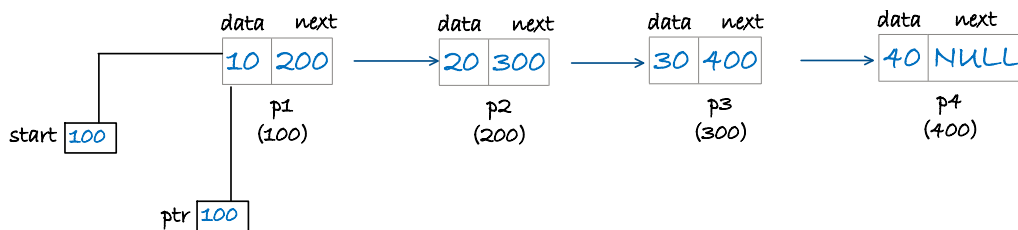
c

 Copy code

```
for (i = 1; i <= 3; i++)  
    ptr = ptr->next;
```

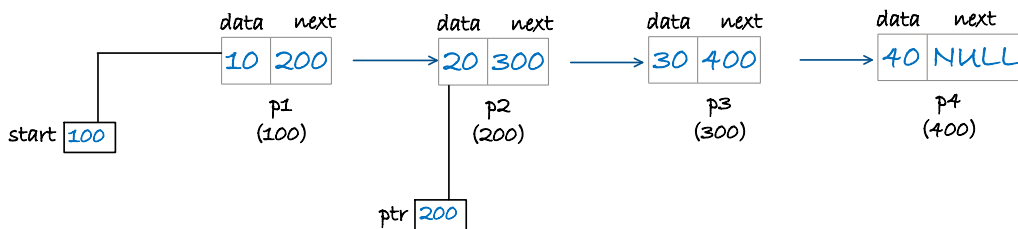
- The `for` loop moves the `ptr` pointer to the next node three times:
 - After the 1st iteration: `ptr = start->next` (2nd node).
 - After the 2nd iteration: `ptr = start->next->next` (3rd node).
 - After the 3rd iteration: `ptr = start->next->next->next` (4th node, assuming the list is long enough).

`for (i=1; i<=3; i++)`
`ptr=ptr->next;`



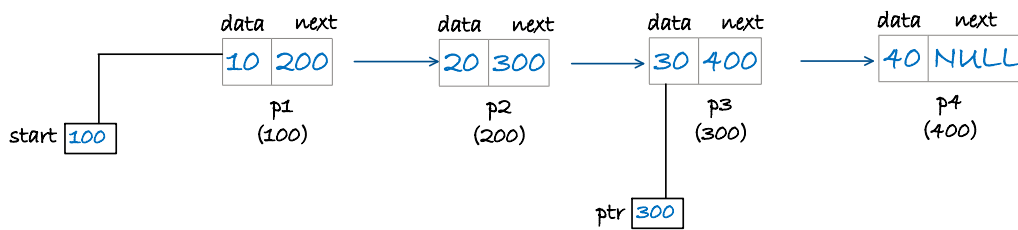
(i) 1st iteration:

`ptr = 100->next`
`ptr = 200`



(ii) 2nd iteration:

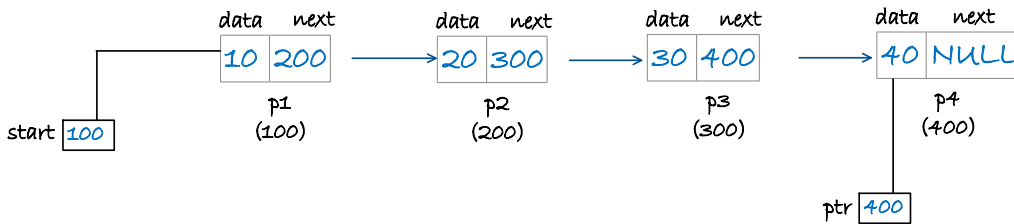
`ptr = 200->next`
`ptr = 300`



(iii) 3rd iteration:

`ptr = 300->next`

ptr = 400



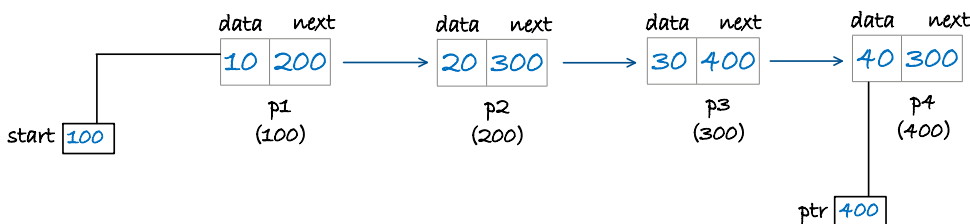
3. Modify the next pointer of the current ptr :

```
c
ptr->next = start->next->next;
```

Copy code

- After the loop, ptr points to the 4th node.
- This line sets the next pointer of the 4th node to point to the 3rd node (start->next->next).

ptr->next = start->next->next;
200->next = 300



4. Access and print data:

```
c
printf("%d", start->next->next->data);
```

Copy code

- This accesses the data field of the 3rd node (start->next->next) and prints it.
- The modifications in ptr->next do not affect this specific access, as it directly reads from the 3rd node.

start->next->next->data
30

topic: linked lists problems

statement: given a linked lists
this basically means a start pointer is given.

assume start is a global variable.

point to remember:
if ptr = NULL

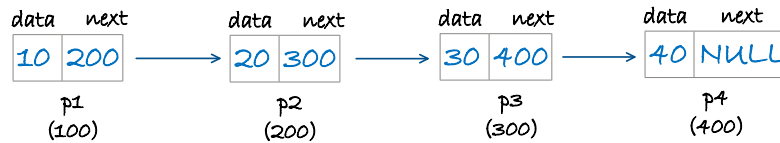
and
ptr->data
ptr->next

both are invalid as
NULL->data
NULL->next

we cannot fetch a value from invalid address.

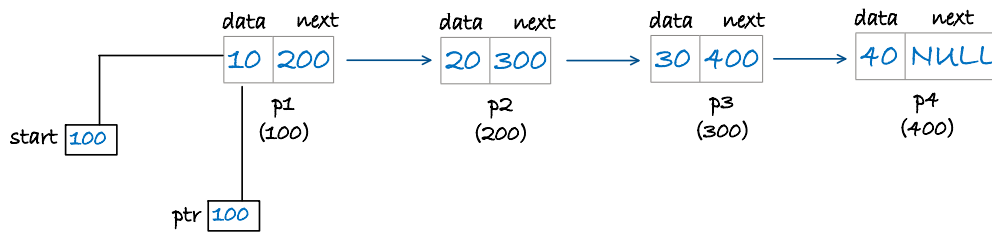
problem (i) given a linked list, traverse it (visit each element exactly once).

analysis:



(i) if i want to traverse the linked list the first thing i need here is a pointer.

structnode *ptr
ptr=start;



(ii) if ptr is pointing to a valid address then print the data.

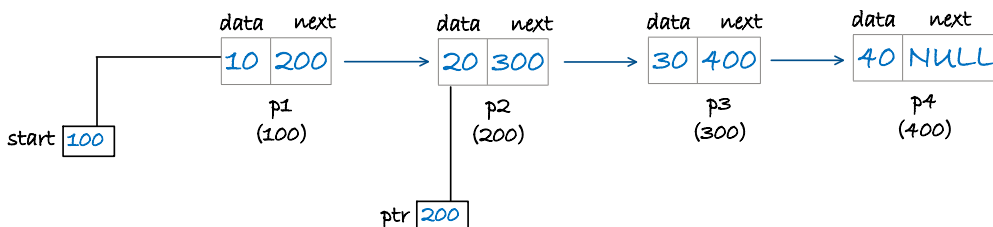
valid address?

pd("%d", ptr->data); 10

(iii) i want to move to the next node

ptr=ptr->next; 200

- moves to the second node



(ii) if ptr is pointing to a valid address then print the data.

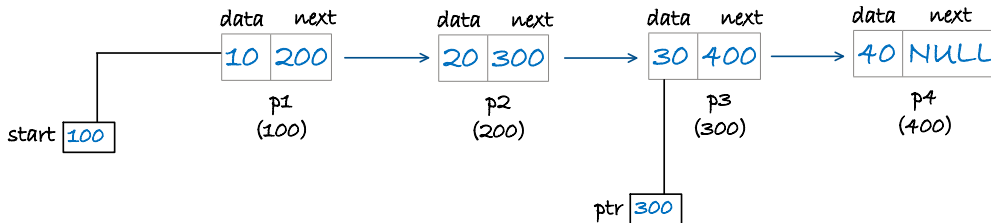
valid address?

```
pd("%d", ptr->data); 20
```

(iii) i want to move to the next node

```
ptr=ptr->next; 300
```

- moves to the third node



(ii) if ptr is pointing to a valid address then print the data.

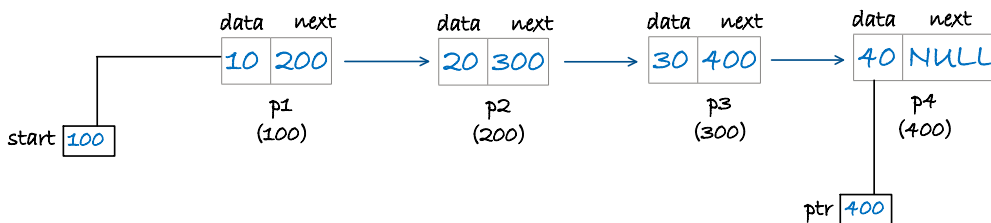
valid address?

```
pd("%d", ptr->data); 30
```

(iii) i want to move to the next node

```
ptr=ptr->next; 400
```

- moves to the fourth node



(ii) if ptr is pointing to a valid address then print the data.

valid address?

```
pd("%d", ptr->data); 40
```

(iii) i want to move to the next node

```
ptr=ptr->next; null
```

now we encountered null, there is no valid address.

so basically we are repeating step (ii) and step (iii) till ptr is not pointing to null.

so,

```
while (ptr!=null){
    pf("%d", ptr->data);
    ptr=ptr->next;
}
```

but what if linked list is empty?

- that means start contains null

so,

ptr!=null

null!=null

false, hence prints nothing.

final code:

c

 Copy code

```
// General function to traverse a Linked List
void traverse(struct node *head) {
    struct node *ptr = head; // Start from the head node

    while (ptr != NULL) { // Traverse until the end of the List
        printf("%d -> ", ptr->data); // Print the data of the current node
        ptr = ptr->next;           // Move to the next node
    }

    printf("NULL\n"); // End of the List
}
```

Explanation:

- The function `traverse` takes a pointer to the head node of a linked list (`struct node *head`).
- It then uses the pointer `ptr` to iterate through each node of the linked list until it reaches the end (i.e., when `ptr == NULL`).
- `printf("%d -> ", ptr->data);` prints the data of each node followed by an arrow (`->`).
- Once the end of the list is reached (when `ptr` becomes `NULL`), it prints `NULL` to signify the end of the list.

```

#include <stdio.h>
#include <stdlib.h>

// Define the structure for a node
struct node {
    int data;
    struct node *next;
};

void traverse(struct node *head) {
    struct node *ptr = head; // Start from the head node
    while (ptr != NULL) {    // Traverse until the end of the list
        printf("%d → ", ptr->data); // Print the current node's data
        ptr = ptr->next;           // Move to the next node
    }
    printf("NULL\n"); // End of the list
}

int main() {
    // Create sample nodes for demonstration
    struct node *head = (struct node *)malloc(sizeof(struct node));
    struct node *second = (struct node *)malloc(sizeof(struct node));
    struct node *third = (struct node *)malloc(sizeof(struct node));

    // Assign data and link nodes
    head->data = 10;
    head->next = second;

    second->data = 20;
    second->next = third;

    third->data = 30;
    third->next = NULL; // End of the list

    // Traverse the linked list
    printf("Linked List: ");
    traverse(head);

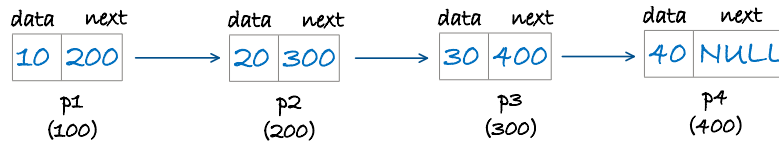
    // Free allocated memory
    free(third);
    free(second);
    free(head);

    return 0;
}

```

problem (ii) given a linked list, count the number of nodes in linked list.

analysis:



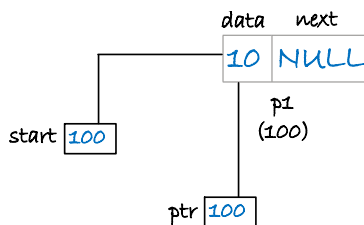
(i) we have to count the number of nodes in linked list so we need a variable that can store the number of count.

(ii) we have to count the number of nodes till we have not reached the end, as we know the last node always points to null so we must traverse the linked list until ptr is not pointing to null.

the traversal code is programmed to traverse all the nodes, so we can simply count how many times traversal code runs and that is equal to the number of nodes.

for example:

- there is only one node in linked list



code runs: one time, count =1.

this code will run one time, so the first time we are going into the iteration we must have some function that increases the count value from 0 to 1, which is count++.

```
void traverse(struct node *start) {
    int count = 0; (initialise the count)
    struct node *ptr = start; (start from the head/start of the list)

    while (ptr != NULL) {
        printf("%d -> ", ptr->data); 10
        c++; (increment the count)
        ptr = ptr->next; (move to the next node)
    }

    printf("NULL\n");
    return count;
}
```

we are not only traversing but also counting the nodes so lets change the function name to int countnodes.

we don't want this because we are not printing the node data, we need to count the nodes.

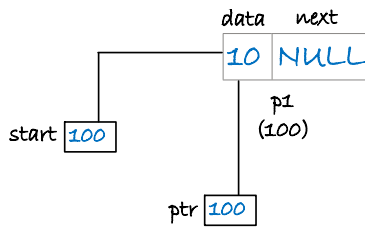
don't print NULL once the ptr is pointing to NULL, the program exists the loop so it must return the value of count

- return the total count.

```
int countNodes(struct node *start) {
    int count = 0;
    struct node *ptr = start;
```

```
while (ptr != NULL) {  
    count++;  
    ptr = ptr->next;  
}  
  
return count;  
}
```

- there is only one node in linked list



```
int countNodes(struct node *start) {
    int count = 0;
    struct node *ptr = start;
```

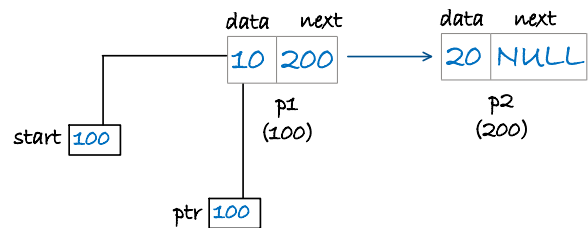
```
(i) iteration 1st
    while (ptr != NULL) {
        count++; 1
        ptr = ptr->next; NULL
    }
```

```
    return count;
}
```

```
(ii) iteration 1nd: returns count (1)
    while (ptr != NULL) {
        count++;
        ptr = ptr->next;
    }
```

```
    return count; 1
}
```

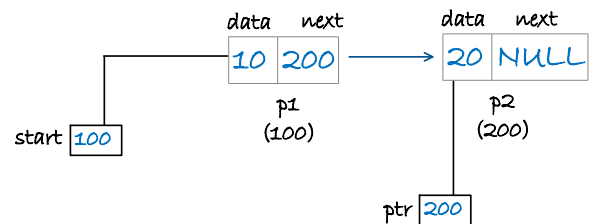
- there are two nodes in linked list



```
int countNodes(struct node *start) {
    int count = 0;
    struct node *ptr = start;
```

```
(i) iteration 1st
    while (ptr != NULL) {
        count++; 1
        ptr = ptr->next; 200
    }
```

```
    return count;
}
```



```
(ii) iteration 1nd:
    while (ptr != NULL) {
        count++; 2
        ptr = ptr->next; NULL
    }
```

```
    return count;
}
```

```
(iii) iteration 1st:
    while (ptr != NULL) {
        count++;
        ptr = ptr->next;
    }
```

```
    return count; 2
}
```

final code:

final code:

```
int countNodes(struct node *start) {
    int count = 0;           // Initialize count to 0
    struct node *ptr = start; // Start from the head of the list.

    while (ptr != NULL) {    // Traverse until the end of the list
        count++;             // Increment the count
        ptr = ptr->next;      // Move to the next node
    }

    return count;           // Return the total count
}
```

Explanation of Code:

1. Start Traversal:

- Initialize `ptr` to `start` (head of the list).

2. Count Nodes:

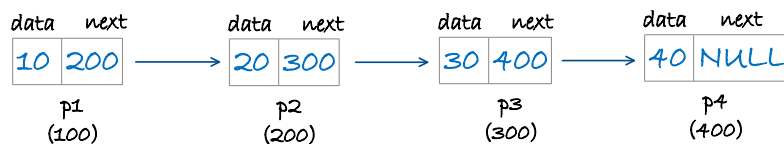
- As long as `ptr != NULL`, increment the `count` variable and move to the next node using `ptr = ptr->next`.

3. Return Result:

- Once traversal is complete, return the value of `count`.

problem (iii) given a linked list, print last node data.

analysis:



(i) to print the last node data we must traverse through the linked list to reach the last node, for that we need a pointer (`ptr`)

(ii) so the `ptr` traverse through and reaches the last node.

check, is `ptr=last node`, no? then move to next node; `ptr=ptr->next;`

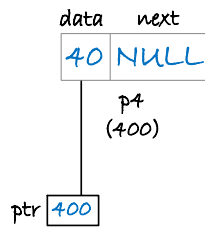
while (`ptr` has not reached the last node) {

`ptr=ptr->next;`

}

we keep repeating this in loop until have not reached the last node

(iii) when we reach the last node we must print the data, what is the property of last node?
the last node contains NULL in the next section of the node.
so, `ptr->next==NULL`, it means we have reached the last node.



last node and also the only node.

(iv) so we can say that while we have not reached the last node means
`ptr->next!=NULL`
we should continue to move to the next node
`ptr=ptr->next;`

```

struct node *ptr = start;

while (ptr->next != NULL) {
    ptr = ptr->next;
}
printf("The last node's data is: %d\n", ptr->data);
}

```

but there is a problem with this code,

case(i) what if the linked list is empty?

```

ptr[NULL]

while (ptr->next != NULL)
while (NULL->next != NULL) : invalid

```

so we must assure that if the linked list is empty we must return instead of moving to while code.
so, if `start==NULL` we should return.

```

void printlastnode(struct node *start) {
    if (start == NULL) { (check if the list is empty)
        printf("the list is empty.\n");
        return;
    }
}

```

```

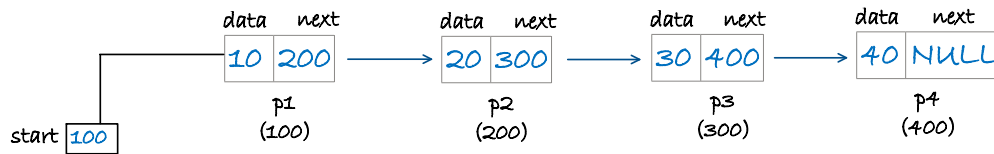
void printlastnode(struct node *start) {
    if (start == NULL) {
        printf("the list is empty.\n");
    }
}

```

```

    return;
}
struct node *ptr = start;
while (ptr->next != NULL) {
    ptr = ptr->next;
}
printf("the last node's data is: %d\n", ptr->data);
}

```



```

void printlastnode(struct node *start) {
    if (start == NULL) { (start is not null)
        printf("the list is empty.\n");
        return;
    }
    struct node *ptr = start; (ptr initialised with 100)
}

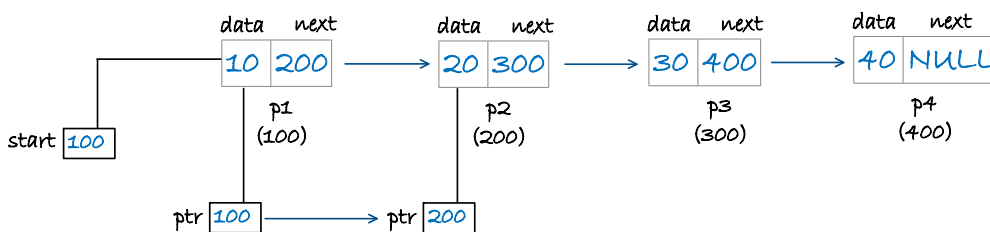
```

(i) iteration 1st

```

while (ptr->next != NULL) { (not NULL)
    ptr = ptr->next; (moves to second node (200))
}
printf("the last node's data is: %d\n", ptr->data);
}

```

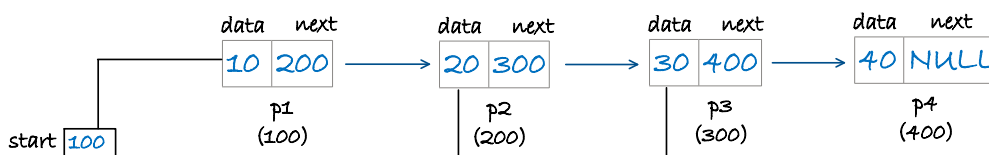


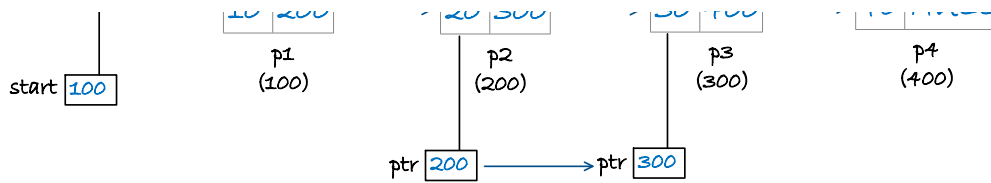
(ii) iteration 2nd

```

while (ptr->next != NULL) { (not NULL)
    ptr = ptr->next; (moves to third node (300))
}
printf("the last node's data is: %d\n", ptr->data);
}

```



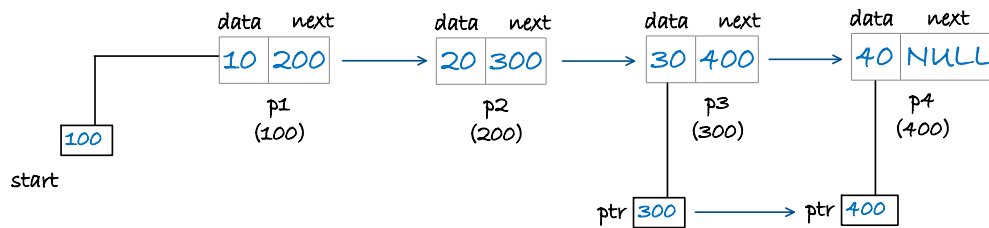


(iii) iteration iiird

```

while (ptr->next != NULL) { (not NULL)
    ptr = ptr->next; (moves to fourth node)
}
printf("the last node's data is: %d\n", ptr->data);
}

```

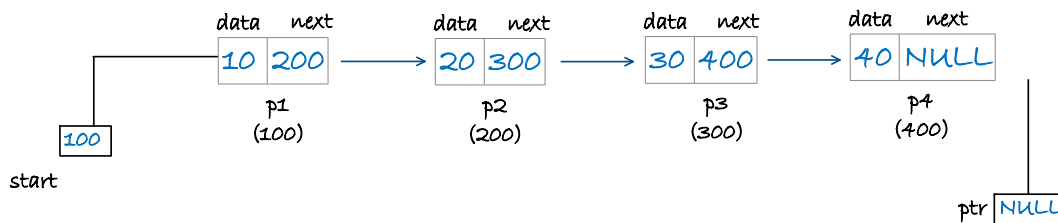


(iv) iteration ivth

```

while (ptr->next != NULL) { (NULL); exit the code
    ptr = ptr->next;
}
printf("the last node's data is: %d\n", ptr->data); 40
}

```



```

void printLastNode(struct node *start) {
    if (start == NULL) { // Check if the list is empty
        printf("The list is empty.\n");
        return;
    }

    struct node *ptr = start; // Initialize pointer to traverse the list

    // Traverse to the Last node
    while (ptr->next != NULL) {
        ptr = ptr->next;
    }

    // Print the data of the Last node
    printf("The last node's data is: %d\n", ptr->data);
}

```

Explanation:

1. Check for Empty List:

- If `start == NULL`, the linked list is empty, and there's no last node to print.

2. Traverse the List:

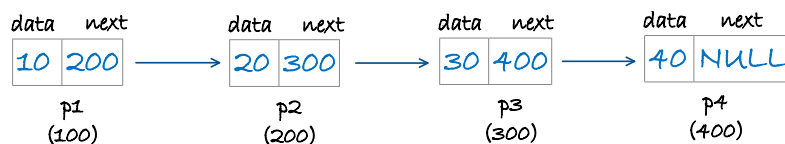
- Use a pointer `ptr` starting at the head (`start`).
- Continue traversing while `ptr->next != NULL`. This stops when `ptr` points to the last node.

3. Print Last Node's Data:

- After the loop, `ptr` will point to the last node.
- Print the `data` of the last node using `ptr->data`.

problem (iv) given a linked list, print second last node data.

analysis:



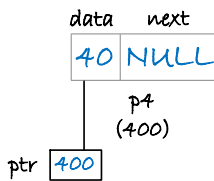
(i) to print the second last node we must traverse through all the nodes to reach the second last node.

(ii) to reach the second last node we must understand the property of second last node

- property of last node:

the next part of last node is basically NULL

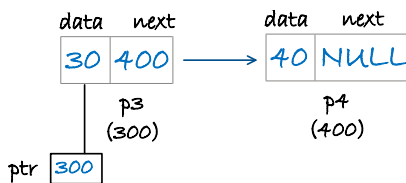
`ptr->next=NULL`



- property of second last node:

the next part of second last node basically points to the last node

`ptr->next->next=NULL`



(iii) so we continue to traverse or move to the next node until we have not reached second last node

`while (ptr->next->next!=NULL) {`

`ptr=ptr->next;`

`}`

but again, this could be wrong because we must ensure that there are atleast two nodes in the linked list, what if linked list is empty?

so ensure this by:

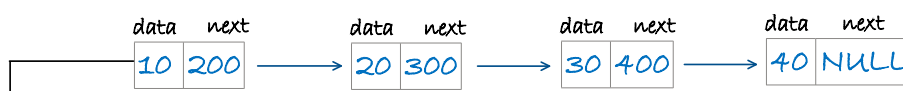
`if (start==NULL || start->next==NULL)`

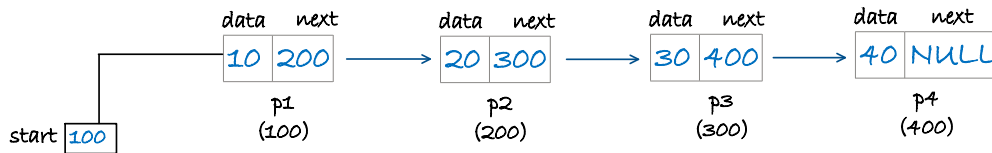
we must return.

```

void printsecondlastnode(struct node *start) {
    if (start == NULL || start->next == NULL) {
        printf("the list has fewer than two nodes.\n");
        return;
    }
    struct node *ptr = start;
    while (ptr->next->next != NULL) {
        ptr = ptr->next;
    }
    printf("the second last node's data is: %d\n", ptr->data);
}

```





```

void printsecondlastnode(struct node *start) {
    if (start == NULL || start->next == NULL) { (there are two nodes available)
        printf("the list has fewer than two nodes.\n");
        return;
    }
    struct node *ptr = start; (initialise ptr with 100)

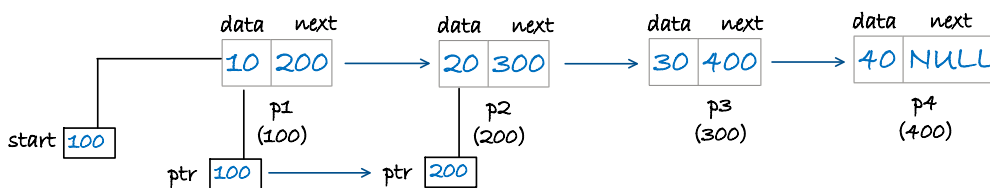
```

(i) iteration ist

```

    while (ptr->next->next != NULL) { (no, points to first node currently)
        ptr = ptr->next; (moves to second node)
    }
    printf("the second last node's data is: %d\n", ptr->data);
}

```

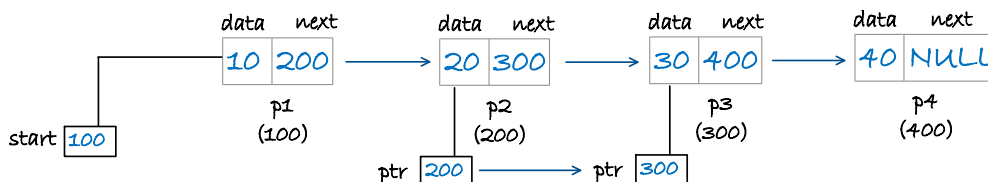


(ii) iteration iind

```

    while (ptr->next->next != NULL) { (no, points to second node currently)
        ptr = ptr->next; (moves to third node)
    }
    printf("the second last node's data is: %d\n", ptr->data);
}

```

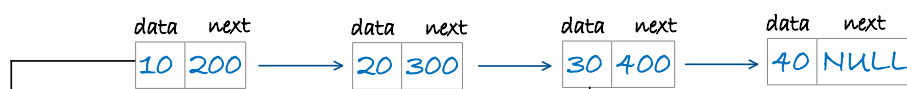


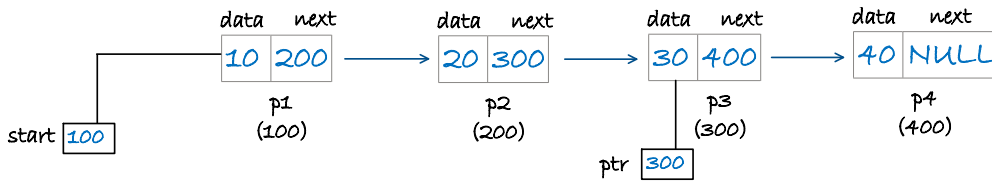
(iii) iteration iiird

```

    while (ptr->next->next != NULL) { (yes, points to third (second last) node currently)
        ptr = ptr->next;
    } (exits the code)
    printf("the second last node's data is: %d\n", ptr->data); 30
}

```





final code:

```

void printSecondLastNode(struct node *start) {
    if (start == NULL || start->next == NULL) {
        // If the list is empty or has only one node
        printf("The list has fewer than two nodes.\n");
        return;
    }

    struct node *ptr = start; // Initialize pointer to traverse the list

    // Traverse until the second last node
    while (ptr->next->next != NULL) {
        ptr = ptr->next;
    }

    // Print the data of the second last node
    printf("The second last node's data is: %d\n", ptr->data);
}
  
```

Explanation:

1. Edge Cases:

- If the list is empty (`start == NULL`) or contains only one node (`start->next == NULL`), the second last node does not exist.
- Print a message indicating the list is too short.

2. Traverse to Second Last Node:

- Use the condition `ptr->next->next != NULL` to stop the loop when `ptr` is pointing to the second last node.

3. Print Data:

- After the loop, `ptr` points to the second last node. Print its `data` using `ptr->data`.

concept: what if start is a local variable?

Key Considerations for a Local `start` Variable:

1. Scope of `start` :

- A local `start` is only accessible within the function where it is declared.
- Changes made to `start` inside a function (e.g., reassigning it) won't affect the original list outside that function unless you return it or pass a pointer to it.

2. When Traversing the List:

- You can still use `start` to traverse the list within the function. The traversal operations don't alter the original `start` pointer itself, so the list structure remains unchanged.

3. Passing `start` to Functions:

- When `start` is local, you can pass it as an argument to functions that operate on the list (e.g., `printSecondLastNode(start)`).

(function to traverse the linked list) (call by value)

```
void traverse(struct node *start) {  
    struct node *ptr = start; (local pointer initialized to start)  
  
    while (ptr != NULL) { (traverse until the end of the list)  
        printf("%d -> ", ptr->data); (print current node's data)  
        ptr = ptr->next; (move to the next node)  
    }  
    printf("NULL\n"); (end of the list)  
}
```

```
int main() {  
    (create a linked list locally)  
    struct node *start = createNode(10); (head node)  
    start->next = createNode(20); (second node)  
    start->next->next = createNode(30); (third node)  
  
    call the traverse function  
    traverse(start);  
  
    return 0;  
}
```

problem with `start` as local:

when the traverse function is called, the traverse function will take the copy of `start` and initialises `ptr` and executes the code.

as desired, it completes the execution and traverse the whole linked list and prints the node of each data.

now when we exit the traverse function return to the original function to print the values, the

values will not be accessible anymore because start was local and it has destroyed once we exit the block

so, any changes made to the traverse function will not be reflected in the main because of the local start.

Key Points to Clarify:

1. Local Variables in C:

- A local variable like `start` exists only within the function where it is defined (e.g., `main()` in this case).
- When you pass it by value to another function like `traverse`, a copy of the pointer `start` is made, not the actual linked list.

2. Traverse Function Behavior:

- The `traverse` function operates on the copied pointer (`ptr`), and since it does not modify the structure of the list, the functionality works as intended.
- No permanent changes are made to the list or the `start` pointer in `main()`.

3. Why This Doesn't Cause a Problem for Traversal:

- The list nodes themselves are dynamically allocated using `malloc` and exist in the heap. They are not destroyed when a function exits unless explicitly freed.
- The local `start` in `main()` is merely a pointer to the head of the list. It remains valid after returning from the `traverse` function.

4. When Problems Occur:

- If `traverse` or any other function modifies the `start` pointer (e.g., changing its value to point elsewhere), those changes won't reflect in `main()` because only a copy of the pointer was passed.

what is the possible solution when start is a local variable?
call by reference, we must pass the address of start.

understand that start is already a pointer that is holding the address already that points to the starting of first node.

- now we are passing the address of start means address of a pointer.
`&start`

- so to store the address of a pointer we must have a double pointer in the function.
`void traverse(struct node **start)`

- as we are traversing in linked lists so there should be a node, so we must give a condition that ensure that atleast one node exist
`if (*start == NULL)`

function to traverse the linked list (call by reference)

```

void traverse(struct node **start) {
    if (*start == NULL) { (check if the list is empty)
        printf("the list is empty.\n");
        return;
    };

    struct node *ptr = *start; (initialize pointer to head of the list)

    while (ptr != NULL) { (traverse until the end of the list)
        printf("%d -> ", ptr->data); (print current node's data)
        ptr = ptr->next; (move to the next node)
    }
    printf("NULL\n"); (end of the list)
}

int main() {
    (create a linked list locally)
    struct node *start = createNode(10); (head node)
    start->next = createNode(20); (second node)
    start->next->next = createNode(30); (third node)

    call the traverse function with reference to start
    traverse(&start);

    return 0;
}

```

Explanation:

1. Pass by Reference:

- The function `traverse` takes a double pointer (`struct node **start`) as a parameter.
- We pass the address of `start` from `main()` using `&start`.

2. Dereferencing `start`:

- Inside `traverse`, we use `*start` to access the actual head of the list.
- A pointer `ptr` is initialized to `*start` for traversal.

3. Traverse Logic:

- The function traverses the list and prints each node's data.
- The structure of the list remains unchanged since no modifications are made.

4. Why Pass by Reference Here?

- For traversal alone, passing by reference is not strictly necessary.
- However, if we wanted to modify the head node (`start`), passing by reference ensures changes persist in `main()`.

```

#include <stdio.h>
#include <stdlib.h>

// Define the structure of the node
struct node {
    int data;
    struct node *next;
};

// Function to traverse the linked list (call by reference)
void traverse(struct node **start) {
    if (*start == NULL) { // Check if the list is empty
        printf("The list is empty.\n");
        return;
    }

    struct node *ptr = *start; // Initialize pointer to head of the list

    // Traverse the list and print the data
    while (ptr != NULL) {
        printf("%d → ", ptr->data);
        ptr = ptr->next;
    }

    printf("NULL\n"); // End of the list
}

// Helper function to create a new node
struct node *createNode(int value) {
    struct node *newNode = (struct node *)malloc(sizeof(struct node));
    newNode->data = value;
    newNode->next = NULL;
    return newNode;
}

int main() {
    // Create a linked list
    struct node *start = createNode(10); // Head node
    start->next = createNode(20); // Second node
    start->next->next = createNode(30); // Third node

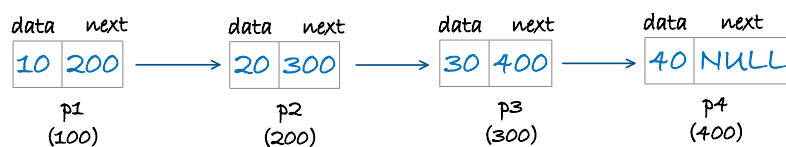
    // Call the traverse function with reference to start
    traverse(&start);

    return 0;
}

```

problem(v) given a linked list, find whether the key is present in linked list or not.

analysis:



(i) the idea is to find the key in the linked list

for example;

key: 20

output: yes

key: 50

output: no

(ii) so there are two conditions if key is present then print output as yes or else no.

(iii) to check the nodes we must travel through the whole linked list so we can use the traverse code here as we have to check till last node.

```
while (ptr != null)
```

```
ptr = ptr->next;
```

(iv) to check whether key is present or not

```
if (ptr->data == key) { (if key is present)
```

```
    return 1; (return 1 (true) if key is found after traversing the whole linked list)
```

```
} (else)
```

```
ptr = ptr->next; (move to the next node)
```

```
}
```

```
return 0; (return 0 (false) if key is not found after traversing the whole linked list)
```

(v) also make sure that should be at-least one node, if the linked list is empty we must return.

function to check if a key is present in the linked list

```
int findKey(struct node *start, int key) {
```

```
    if (start == NULL) { (if the list is empty)
```

```
        printf("the linked list is empty.\n");
```

```
        return 0; // Key not found
```

```
}
```

function to check if a key is present in the linked list:

```
int findKey(struct node *start, int key) {
```

```
    struct node *ptr = start;
```

```
    while (ptr != NULL) {
```

```
        if (ptr->data == key) {
```

```
            return 1;
```

```
        }
```

```
        ptr = ptr->next;
```

```
    }
```

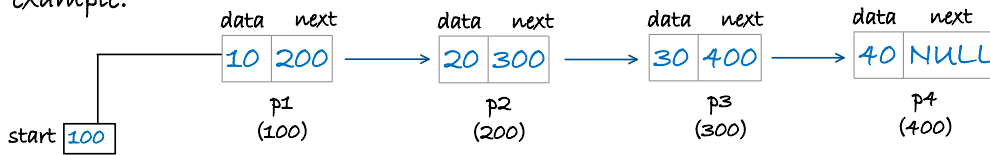
```
    return 0;
```

```
}
```

example:



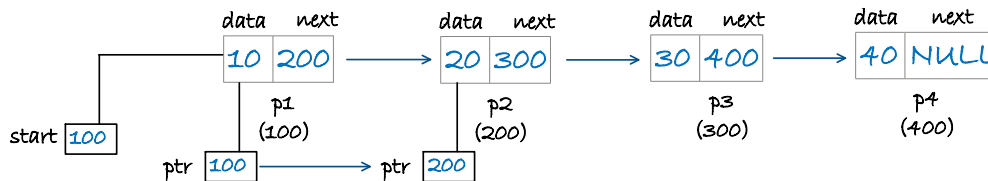
example:



key: 40

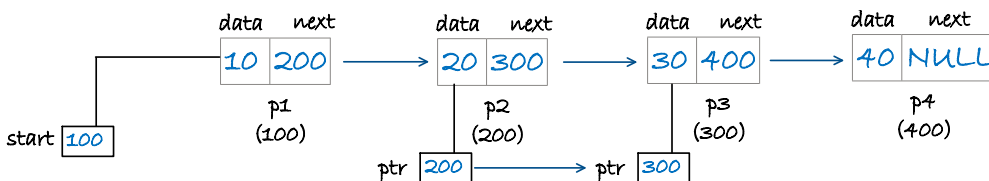
```

int findKey(struct node *start, int key) {
    struct node *ptr = start; (initialised pointer with start 100)
    (i) iteration ist
    while (ptr != NULL) { (not NULL)
        if (ptr->data == 40) { (no)
            return 1;
        }
        ptr = ptr->next; (moves to the next node, 200)
    }
    return 0;
}
  
```



```

(ii) iteration iind
while (ptr != NULL) { (not NULL)
    if (ptr->data == 40) { (no)
        return 1;
    }
    ptr = ptr->next; (moves to the next node, 300)
}
return 0;
}
  
```



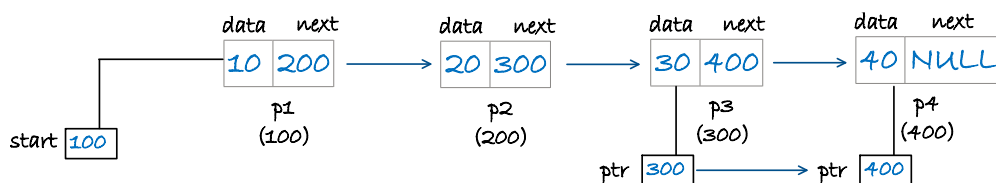
```

(iii) iteration iiird
while (ptr != NULL) { (not NULL)
    if (ptr->data == 40) { (no)
        return 1;
    }
}
  
```

```

    ptr = ptr->next; (moves to the next node, 400)
}
return 0;
}

```

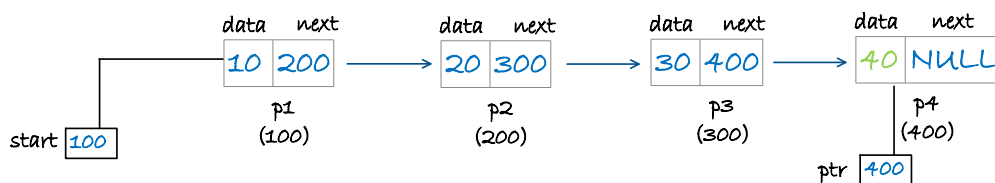


(iii) iteration iiird

```

while (ptr != NULL) { (not NULL)
    if (ptr->data == 40) { (yes)
        return 1; (returns 1)
    }
    ptr = ptr->next;
}
return 0;
}

```



final code:

```

// Function to check if a key is present in the linked list
int findKey(struct node *start, int key) {
    struct node *ptr = start; // Initialize pointer to head of the list

    // Traverse the list until the end (ptr is NULL)
    while (ptr != NULL) {
        // If the data in the current node matches the key
        if (ptr->data == key) {
            return 1; // Key found, return 1
        }
        ptr = ptr->next; // Move to the next node
    }

    // If the end of the list is reached and the key wasn't found
    return 0; // Key not found, return 0
}

```

Detailed Walkthrough:

1. Function Definition:

- The function takes two parameters:
 - `start` : A pointer to the head of the linked list.
 - `key` : The value you're looking for in the linked list.

2. Pointer Initialization:

- `struct node *ptr = start;` — A pointer `ptr` is initialized to point to the head of the linked list. This will help in traversing the list.

3. While Loop (Traversal):

- `while (ptr != NULL)` : This loop keeps running as long as `ptr` is not `NULL`, meaning it continues until the end of the linked list is reached.
- Inside the loop:
 - `if (ptr->data == key)` : This checks if the data in the current node (`ptr->data`) matches the key.
 - If a match is found, `return 1;` is executed, indicating that the key exists in the linked list.
- `ptr = ptr->next;` : If the key isn't found in the current node, we move to the next node in the list.

4. End of List:

- Once the loop ends (i.e., `ptr` becomes `NULL`), it means the key was not found in the entire list. In that case, the function returns `0`, indicating the key is absent.

```

#include <stdio.h>
#include <stdlib.h>

// Define the structure of the node
struct node {
    int data;
    struct node *next;
};

// Function to check if a key is present in the linked list
int findKey(struct node *start, int key) {
    if (start == NULL) { // If the list is empty
        printf("The linked list is empty.\n");
        return 0; // Key not found
    }

    struct node *ptr = start; // Initialize pointer to head

    while (ptr != NULL) { // Traverse until the end of the list
        if (ptr->data == key) { // Compare node's data with key
            return 1; // Key found
        }
        ptr = ptr->next; // Move to the next node
    }

    return 0; // Key not found
}

// Helper function to create a new node
struct node *createNode(int value) {
    struct node *newNode = (struct node *)malloc(sizeof(struct node));
    newNode->data = value;
    newNode->next = NULL;
    return newNode;
}

int main() {
    // Create a linked list
    struct node *start = createNode(10); // Head node
    start->next = createNode(20); // Second node
    start->next->next = createNode(30); // Third node

    int key = 20; // Key to search for

    // Call the findKey function
    if (findKey(start, key)) {
        printf("Key %d is present in the linked list.\n", key);
    } else {
        printf("Key %d is not present in the linked list.\n", key);
    }

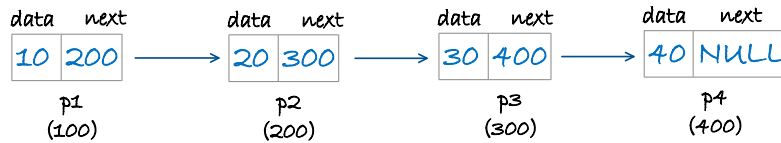
    return 0;
}

```

problem (vi) given a linked list, print alternate node data.

problem (vi) given a linked list, print alternate node data.

analysis:



(i) our target is to print the alternate node data. so the basic idea that we must traverse through every linked list and print the alternate data.

for example:

if i start with first node then i should skip the second node and print the data of third node

`ptr->data`

move to the third node:

`ptr=ptr->next->next`

- another intuition here is that linked list should not be empty means start should not point to null (`start==NULL`);

`return;`

(case:a) atleast one node exist, means ptr should not point to NULL

- `ptr!=NULL`

(case:b) we are also printing the third node

`ptr=ptr->next->next`

which means that third node should exist there.

- how to check if third node exists?

there should not be null in the next section of second node.

ensure that there is a valid address means not null in the next section of second node

`ptr->next!=null`

so we can run the loop until ptr is not pointing to NULL and the second node does not contain NULL in next section.

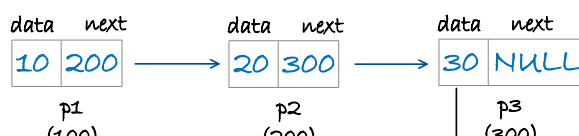
`while (ptr != NULL && ptr->next != NULL) {` (if any condition is false, the loop will not execute)

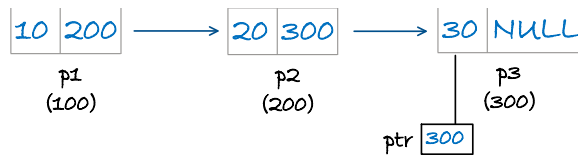
`printf("%d", ptr->data);` (print the current node's data)

`ptr = ptr->next->next;` (move ptr two steps forward)

`}`

(case:c) the another problem here is that if we try to print last node, it will not print as it contains NULL.





the third node should be printed but the code is written in a way that it will not.

```

while (ptr != NULL && ptr->next != NULL) { (the second condition will become true)
    printf("%d", ptr->data);
    ptr = ptr->next->next;
}

```

so the corrected way:

function to print alternate node data

approach (i)

```

void printAlternateNodes(struct node *start) {
    struct node *ptr = start; (initialize pointer to head)

    traverse the linked list and print alternate node data
    while (ptr != NULL && ptr->next != NULL) { (check if we can move two steps)
        printf("%d ", ptr->data); (print current node's data)
        ptr = ptr->next->next; (move two steps forward)
    }
    if the loop ends and we are on the last node
    if (ptr == NULL) { (this means there is no node left)
        return; (do nothing and return)
    }
    if (ptr->next == NULL) { (this means there is one node left (reached the last node))
        printf("%d\n", ptr->data); (print data of the last node)
    }
}

```

approach (ii)

```

void printAlternateNodes(struct node *start) {
    struct node *ptr = start; (initialize pointer to head)

    traverse the linked list and print alternate node data
    while (ptr != NULL && ptr->next != NULL) { (check if we can move two steps)
        printf("%d ", ptr->data); (print current node's data)
        ptr = ptr->next->next; (move two steps forward)
    }
    if the loop ends and we are on the last node
    if (ptr != NULL) { (this means there is one node left (odd length list))
        printf("%d\n", ptr->data); (print data of the last node)
    }
}

```

final code:

```

void printAlternateNodes(struct node *start) {
    struct node *ptr = start; // Initialize pointer to head

    // Traverse the linked list and print alternate node data
    while (ptr != NULL && ptr->next != NULL) { // Check if we can move two steps
        printf("%d ", ptr->data); // Print current node's data
        ptr = ptr->next->next; // Move two steps forward
    }

    // If the loop ends and we are on the last node
    if (ptr != NULL) { // This means there is one node left (odd-length list)
        printf("%d\n", ptr->data); // Print data of the last node
    }
}

```

Function Logic:

1. Initialization:

- A pointer `ptr` is set to the start of the linked list, pointing to the head node.

2. Traversal and Printing:

- A `while` loop is used to traverse the list. The condition `ptr != NULL && ptr->next != NULL` ensures that the loop only executes if:
 - `ptr` points to a valid node (not NULL).
 - There is another valid node to step forward (`ptr->next != NULL`).
- Inside the loop:
 - The data of the current node pointed to by `ptr` is printed using `printf`.
 - `ptr` is advanced two steps forward (`ptr = ptr->next->next`) to skip the next node and move to the alternate one.

3. Handling the Last Node:

- After the loop, if `ptr` is still pointing to a valid node (`ptr != NULL`), it indicates the list had an odd number of nodes.
- In this case, the data of the last node is printed.

```

#include <stdio.h>
#include <stdlib.h>

// Define the structure of a node
struct node {
    int data;

```

```

// Define the structure of a node
struct node {
    int data;
    struct node *next;
};

// Function to print alternate nodes in the linked list
void printAlternateNodes(struct node *start) {
    struct node *ptr = start; // Initialize pointer to head

    // Traverse the linked list and print alternate node data
    while (ptr != NULL && ptr->next != NULL) { // Check if we can move two steps
        printf("%d ", ptr->data); // Print current node's data
        ptr = ptr->next->next; // Move two steps forward
    }

    // If the loop ends and we are on the last node
    if (ptr != NULL) { // This means there is one node left (odd-length list)
        printf("%d\n", ptr->data); // Print data of the last node
    }
}

// Function to create a new node
struct node* createNode(int value) {
    struct node *newNode = (struct node*)malloc(sizeof(struct node));
    if (newNode == NULL) {
        printf("Memory allocation failed\n");
        exit(1);
    }
    newNode->data = value;
    newNode->next = NULL;
    return newNode;
}

// Function to append a node to the linked list
struct node* appendNode(struct node *start, int value) {
    struct node *newNode = createNode(value);

    if (start == NULL) {
        return newNode; // If the list is empty, return the new node as the start
    }

    struct node *ptr = start;
    while (ptr->next != NULL) {
        ptr = ptr->next; // Traverse to the end of the list
    }
    ptr->next = newNode; // Append the new node
    return start;
}

// Main function
int main() {
    struct node *start = NULL;

    // Create a linked list: 10 → 20 → 30 → 40
    start = appendNode(start, 10);
    start = appendNode(start, 20);
    start = appendNode(start, 30);
    start = appendNode(start, 40);

    printf("Alternate nodes in the linked list: ");
    printAlternateNodes(start);

    return 0;
}

```

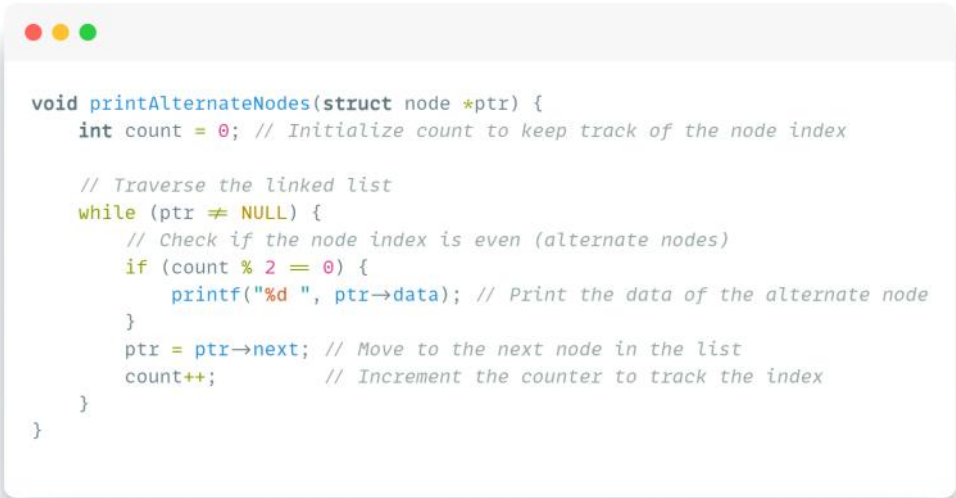
approach (iii)

```
void printAlternateNodes(struct node *ptr) {
```

```

int count = 0; (initialize count to keep track of the node index)
while (ptr != NULL) {
    if (count % 2 == 0) { (check if the node is alternate (0, 2, 4, ...))
        printf("%d ", ptr->data);
    }
    ptr = ptr->next; (move to the next node)
    count++; (increment the counter)
}
}

```



```

void printAlternateNodes(struct node *ptr) {
    int count = 0; // Initialize count to keep track of the node index

    // Traverse the linked list
    while (ptr != NULL) {
        // Check if the node index is even (alternate nodes)
        if (count % 2 == 0) {
            printf("%d ", ptr->data); // Print the data of the alternate node
        }
        ptr = ptr->next; // Move to the next node in the list
        count++; // Increment the counter to track the index
    }
}

```

Explanation of Key Steps:

- Initialization (`int count = 0;`):**
 - A counter variable `count` is initialized to track the position of each node in the linked list.
- Traversal (`while (ptr != NULL)`):**
 - The loop continues until `ptr` (the current node pointer) becomes `NULL`, which indicates the end of the linked list.
- Condition Check (`if (count % 2 == 0)`):**
 - For every alternate node, the condition `count % 2 == 0` evaluates to `true` (nodes at positions 0, 2, 4, ...).
 - When the condition is satisfied, the node's data is printed using `printf`.
- Pointer and Counter Update:**
 - `ptr = ptr->next;` moves the pointer to the next node in the linked list.
 - `count++;` increments the counter for the next iteration.

topic: operations with linked list

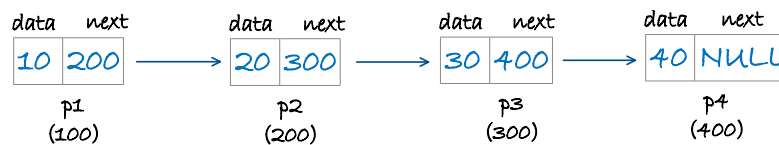
operation(i) insertion

topic: operations with linked list

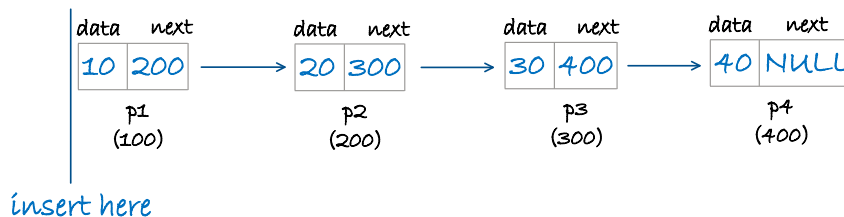
operation (i) insertion

problem (vii): given a linked list and a key, insert the key at the beginning of linked list.

analysis:



(i) we want to insert the key at beginning of the linked list, there are multiple nodes in linked list and there is a start pointer that points to the first node of the linked list.



(ii) the possible way to insert the new key here is to create a new node.

if we create a new node and initialise the next section of the node with the address of p1 then this newly created node become the first node of the linked list.

create new node:

- `struct node *newnode = (struct node*) malloc(sizeof(struct node));`
malloc returns the address of block equal to the size of struct node and as void is returned it will be casted structnode* type.

(iii) we will initialise the start with the address of newly created node.

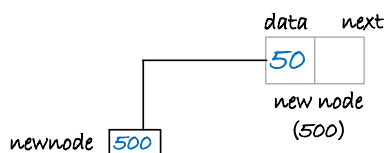
so this is the way we can insert the key at the beginning.

before insertion make sure that new node is allocated.

if (`newnode != null`); (means there is a valid address in newnode)

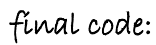
initialise new node:

- `newnode->data = key(50);` (set the data of the new node)



- `newnode->next = start;` (point the new node to the current first node)

data next



Chapter - 2 (Linked List) Page 55

Explanation of Fixes:

1. `struct node *newNode` Declaration:

- The declaration of the `newNode` pointer was missing a semicolon. It's fixed now as `struct node *newNode;`.

2. `if (newNode != NULL) :`

- The check for `malloc` failure should compare against `NULL` using `if (newNode != NULL)`.
- If memory allocation fails, a message is printed to indicate failure. If successful, the node data is set, and the `start` pointer is updated.

3. Code Flow:

- The function allocates memory for a new node.
- It assigns the `key` to `newNode->data` and links it to the existing first node (if any).
- Finally, the `start` pointer is updated to point to the newly created node.


```

#include <stdio.h>
#include <stdlib.h>

// Define the structure of the node
struct node {
    int data;
    struct node *next;
};

// Global variable to represent the head of the linked list
struct node *start = NULL;

// Function to insert at the beginning of the linked list
void insertAtBeginning(int key) {
    // Create a new node
    struct node *newNode = (struct node*)malloc(sizeof(struct node));

    // Check if memory allocation is successful
    if (newNode != NULL) {
        newNode->data = key; // Set the data of the new node
        newNode->next = start; // Point the new node to the current first node
        start = newNode; // Update the global start to point to the new node
    } else {
        printf("Memory allocation failed.\n");
    }
}

// Function to print the linked list
void printList() {
    struct node *ptr = start;
    while (ptr != NULL) {
        printf("%d → ", ptr->data);
        ptr = ptr->next;
    }
    printf("NULL\n");
}

int main() {
    // Insert 50 at the beginning
    insertAtBeginning(50);

    // Print the linked list to check the result
    printList(); // Expected output: 50 → NULL

    return 0;
}

```

concept: what if start is a local variable here

```

#include <stdio.h>
#include <stdlib.h>

```

(define the structure of the node)

```

struct node {
    int data;
    struct node *next;
}

```

```
};
```

(Global variable to represent the head of the linked list)

```
struct node *start = NULL;
```

(function to insert at the beginning of the linked list)

```
void insertatbeginning(int key) {
```

(create a new node)

```
struct node *newNode = (struct node*) malloc(sizeof(struct node));
```

(check if memory allocation is successful)

```
if (newNode != NULL) {
```

newNode->data = key; (set the data of the new node)

newNode->next = start; (point the new node to the current first node)

start = newNode; (update the global start to point to the new node)

```
} else {
```

```
printf("memory allocation failed.\n");
```

```
}
```

```
}
```

if start is a local variable, then as soon as the function exits, the memory allocated for start will be removed from the stack.

consequently, any changes made to start within the function will not be reflected in the main function, as the local start variable will no longer exist after the function completes.

(function to print the linked list)

```
void printList() {
```

```
struct node *ptr = start;
```

```
while (ptr != NULL) {
```

```
printf("%d -> ", ptr->data);
```

```
ptr = ptr->next;
```

```
}
```

```
printf("NULL\n");
```

```
}
```

```
int main() {
```

(insert 50 at the beginning)

```
insertatbeginning(50);
```

(print the linked list to check the result)

```
printList(); (expected output: 50 -> NULL)
```

```
return 0;
```

```
}
```

Why does this happen?

When a variable is declared as **local** inside a function:

- The memory for that variable is allocated on the **stack** when the function is called.
- Once the function **returns**, the memory is **deallocated** and the local variable is no longer accessible.
- Therefore, changes made to this local variable (like modifying the `next` pointers of nodes) will be lost as soon as the function exits. The changes do not affect the original variable (such as `start`) in the `main` function.

How to Fix This Issue (Pass by Reference):

To modify the `start` pointer globally (so changes persist in `main`), you need to pass a pointer to the `start` variable (i.e., a **pointer to a pointer**).

function to insert at the beginning of the linked list

(ii) we pass the address of `start` as `start` is a local variable so we need to ensure that it is pass by reference

- `struct node *start = NULL;` (local variable `start` in `main`)
- `insertatbeginning(&start, 50);` (pass address of `start` to the function)

(ii) we must have double pointer in the `insertatbeginning` function to receive the address of `start` because `start` is a pointer and we are passing the address of pointer that is already storing the address

- `void insertatbeginning(struct node **start, int key)`

(iii) we have to change `start=newnode;` to `*start=newnode;`

why?

- in the `main` function:
`start` is a pointer to the first node of the linked list.
- in the `insertatbeginning` function
when we pass `&start` (the address of `start`) to `insertatbeginning`, we're telling the function to work directly with the pointer to the pointer (`struct node **start`).

inside the function, `start` is treated as a pointer to the original `start` pointer in the `main` function. so, `*start` dereferences the pointer and accesses the original `start` variable in `main`.

- what happens when we do `*start = newNode;`
`*start` accesses the actual `start` pointer in `main`, so when we set `*start = newNode;`, we are actually modifying the `start` pointer in `main` to point to the new node (`newNode`).

this is important because we want the changes to be reflected outside the function, in the `main` function, where we are managing the linked list.

```

#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

// Function to insert at the beginning of the linked list
void insertAtBeginning(struct node **start, int key) {
    struct node *newNode = (struct node*)malloc(sizeof(struct node));
    if (newNode != NULL) {
        newNode->data = key; // Set the new node's data to key
        newNode->next = *start; // New node points to the current start
        *start = newNode; // Modify the actual 'start' in main
    }
}

int main() {
    struct node *start = NULL; // Local variable start in main

    insertAtBeginning(&start, 50); // Pass address of start to the function

    // Check if the list has been updated
    if (start != NULL) {
        printf("List after insertion: %d\n", start->data); // The list is updated with new node
    }

    return 0;
}

```

Code Explanation:

1. **Structure Definition (struct node)**: The structure `node` defines the node of the linked list with two fields:

- `data` : to store the data (integer).
- `next` : a pointer to the next node in the list.

2. **Function insertAtBeginning** : This function takes two parameters:

- `struct node **start` : a pointer to a pointer to the start of the linked list (so the function can modify the actual `start` pointer).
- `int key` : the data (key) to be inserted into the new node.

Inside the function:

- A new node is created using `malloc`.
- The `data` of the new node is set to the given `key`.
- The `next` pointer of the new node is set to point to the current start of the list (`*start`).
- The `start` pointer is updated to point to the new node.

3. **Function main** :

- The `start` pointer is initialized as `NULL` to represent an empty linked list.
- `insertAtBeginning(&start, 50)` is called to insert a new node with the value `50` at the beginning of the list.
- After insertion, the code checks if `start` is `NULL` (i.e., if the insertion was successful). If `start` is not `NULL`, it prints the data of the first node.

```

#include <stdio.h>
#include <stdlib.h>

// Define the structure of a node in the linked list
struct node {
    int data;
    struct node *next;
};

// Function to insert a node at the beginning of the linked list
void insertAtBeginning(struct node **start, int key) {
    // Create a new node
    struct node *newNode = (struct node*)malloc(sizeof(struct node));

    // Check if memory allocation was successful
    if (newNode != NULL) {
        newNode->data = key; // Set the data of the new node to the provided key
        newNode->next = *start; // Point the new node to the current first node
        *start = newNode; // Update the start pointer to point to the new node
    }
    else {
        printf("Memory allocation failed!\n");
    }
}

// Function to print the linked list
void printList(struct node *start) {
    struct node *ptr = start; // Initialize a pointer to the head of the list

    // Traverse and print each node in the linked list
    while (ptr != NULL) {
        printf("%d → ", ptr->data);
        ptr = ptr->next;
    }
    printf("NULL\n"); // Indicate the end of the list
}

int main() {
    struct node *start = NULL; // Local pointer 'start' for the linked list

    // Insert nodes at the beginning of the list
    insertAtBeginning(&start, 50); // Insert 50 at the beginning
    insertAtBeginning(&start, 30); // Insert 30 at the beginning
    insertAtBeginning(&start, 10); // Insert 10 at the beginning

    // Print the updated linked list
    printf("Linked List after insertion: \n");
    printList(start); // This will display the linked list

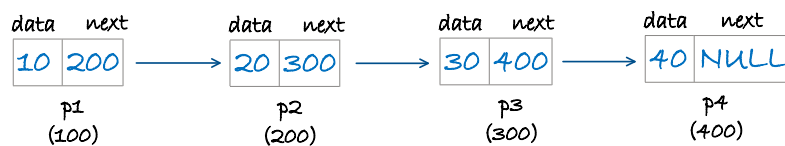
    return 0;
}

```

operation(ii): insert at the end

problem (viii): given a linked list and a key (60), insert the key at the end of the linked list.

analysis:



(i) to insert the key at the end of the linked list.

(a) we will create the newnode.

```
structnode *newnode;  
newnode=(structnode*) malloc(sizeof(newnode));
```

(b) assign the key to the new node.

```
structnode *newnode;  
newnode=(structnode*) malloc(sizeof(newnode));  
newnode->data=key;  
newnode->next=NULL (to make new node as a last node its next part should contain NULL)
```

case (i) what if the linked list is empty?

- if the linked list is empty then the newly created node becomes the last node itself.

so,

```
if (start==null){ (linked list is empty)  
    start=newNode; (assign start with the address of new node)  
    return;
```

case (ii) what if the linked list is not empty?

- if the linked list is not empty then we have to assign newly created node as the last node.

so,

(a) reach to the last node of current linked list

```
ptr=ptr->next;
```

(b) once reached the last node fill the next with the address of new node

```
ptr->next= newnode;
```

function to insert a key at the end of the linked list

```
void insertatend(int key) {  
    struct node *newNode = (struct node*)malloc(sizeof(struct node)); (create a new node)  
    if (newNode == NULL) { (makes sure that memory is allocated)  
        printf("memory allocation failed.\n");  
        return;  
    }  
    newNode->data = key; (set the key)  
    newNode->next = NULL; (this will be the last node, so next is NULL)
```

```
    if (start == NULL) { (if start is not pointing to any valid address, linked list is empty)  
        (if the list is empty, the new node becomes the head)  
        start = newNode;
```

```
    } else {
```

(traverse to the last node)

```
    struct node *ptr = start; (assign start to ptr)
```

```
    while (ptr->next != NULL) { (exits when we reach the last node)
```

```
        ptr = ptr->next; (runs until we reach the last node)
```

```
    }
```

(append the new node to the end)

```
ptr->next = newNode;  
}  
}
```

working of code:

(i)

```
void insertatend(int key) {  
    struct node *newNode = (struct node*) malloc(sizeof(struct node)); (create a new node)  
    if (newNode == NULL) { (memory allocated hence not null)  
        printf("memory allocation failed.\n");  
        return;  
    }  
}
```



(ii)

```
newNode->data = key; (set the key)  
newNode->next = NULL; (this will be the last node, so next is NULL)
```

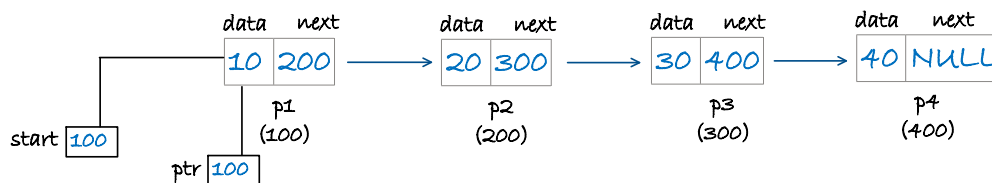


(iii)

```
if (start == NULL) { (start is pointing to valid address, hence not NULL)  
    start = newNode;  
}
```

(iv)

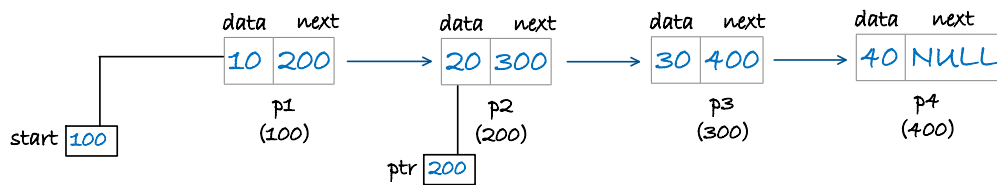
```
else {  
    (traverse to the last node)  
    struct node *ptr = start; (assign 100 to ptr)
```



(v)

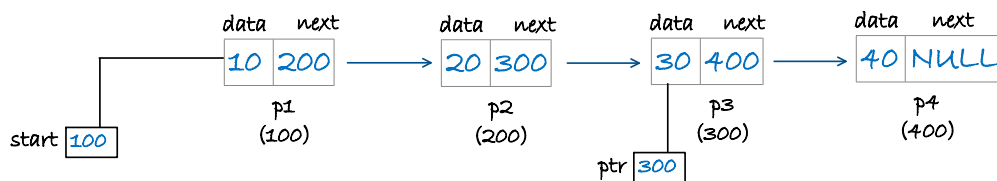
iteration ist:

```
while (ptr->next != NULL) { (not NULL)
    ptr = ptr->next; (ptr becomes 200)
}
```



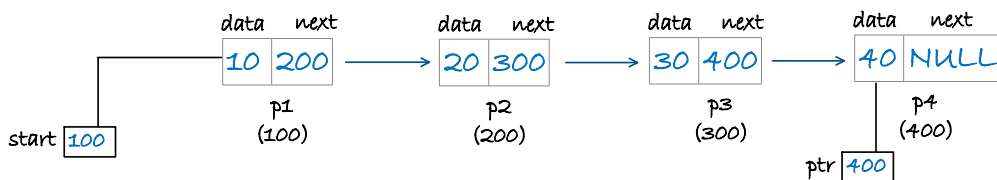
iteration iind:

```
while (ptr->next != NULL) { (not NULL)
    ptr = ptr->next; (ptr becomes 300)
}
```



iteration iiird:

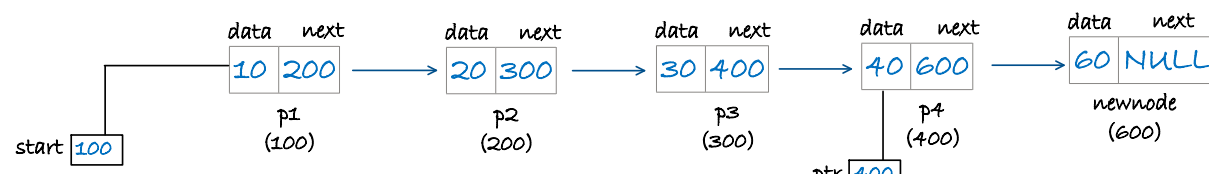
```
while (ptr->next != NULL) { (not NULL)
    ptr = ptr->next; (ptr becomes 400)
}
```



iteration ivth: becomes NULL and exits the iteration

```
while (ptr->next != NULL) { (becomes NULL)
    ptr = ptr->next;
}
```

(vi) ptr->next = newNode;
(append the new node to the end)





final code:

```
// Function to insert a key at the end of the linked list
void insertAtEnd(int key) {
    struct node *newNode = (struct node*)malloc(sizeof(struct node)); // Create a new node
    if (newNode == NULL) {
        printf("Memory allocation failed.\n");
        return;
    }
    newNode->data = key; // Set the key
    newNode->next = NULL; // This will be the last node, so next is NULL

    if (start == NULL) {
        // If the list is empty, the new node becomes the head
        start = newNode;
    } else {
        // Traverse to the last node
        struct node *ptr = start;
        while (ptr->next != NULL) {
            ptr = ptr->next;
        }
        // Append the new node to the end
        ptr->next = newNode;
    }
}
```

The function `insertAtEnd(int key)` adds a new node with the given `key` at the end of a linked list:

1. **Create a New Node:** Dynamically allocate memory for a new node, set its data to `key`, and its `next` pointer to `NULL`.
2. **Handle Empty List:** If the list is empty (`start == NULL`), make the new node the head of the list by setting `start = newNode`.
3. **Traverse to the Last Node:** If the list is not empty, use a pointer to traverse to the last node.
4. **Append the New Node:** Set the `next` pointer of the last node to point to the new node.

Result: The new node is added to the end of the list. If the list was empty, it becomes the first and only node.

```
#include <stdio.h>
#include <stdlib.h>

// Define the structure for a linked list node
struct node {
    int data;
    struct node *next;
};

// Declare a global pointer to the start of the linked list
```

```

    struct node *next;
};

// Declare a global pointer to the start of the linked list
struct node *start = NULL;

// Function to insert a key at the end of the linked list
void insertAtEnd(int key) {
    // Create a new node
    struct node *newNode = (struct node*)malloc(sizeof(struct node));
    if (newNode == NULL) {
        printf("Memory allocation failed.\n");
        return;
    }
    newNode->data = key; // Set the key
    newNode->next = NULL; // This will be the last node, so next is NULL

    if (start == NULL) {
        // If the list is empty, the new node becomes the head
        start = newNode;
    } else {
        // Traverse to the last node
        struct node *ptr = start;
        while (ptr->next != NULL) {
            ptr = ptr->next;
        }
        // Append the new node to the end
        ptr->next = newNode;
    }
}

// Function to display the linked list
void displayList() {
    struct node *ptr = start;
    if (ptr == NULL) {
        printf("The list is empty.\n");
        return;
    }
    printf("Linked list: ");
    while (ptr != NULL) {
        printf("%d → ", ptr->data);
        ptr = ptr->next;
    }
    printf("NULL\n");
}

int main() {
    // Insert 60 at the end of the linked list
    insertAtEnd(60);

    // Display the linked list
    displayList();

    return 0;
}

```

operation(iii): deletion from a linked list

basic idea:

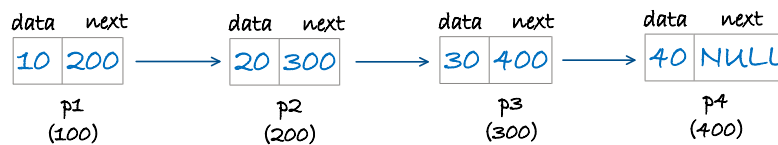
- insertion: in linked list, when we create new node we allocate the memory through dynamic memory allocation like malloc(), this memory is allocated on the heap. and the newly node created address is given to the function.
- deletion: now when we are performing the deletion operation, when the node is deleted the memory still remains and it is not freed automatically. this can cause memory leaks and program to run out over time.

concept: free()

when we delete a node, we will free the memory using "free()" function to release the memory that was allocated to that node.

problem (ix): given a linked list, delete the starting node of the linked list.

analysis:



to delete the first node

(a) check if linked list is not empty

```

if (start == NULL) {
    printf("the list is empty. nothing to delete.\n");
    return;
}
  
```

(b) when we ensure there exist a node

create a pointer to store the address of the start node: we need a pointer (let's call it temp) that will store the address of the first node. this pointer will be used to free the memory allocated to the first node.

move the start pointer to the second node: to delete the first node, we need to adjust the start pointer so that it points to the second node in the list

```
start = start->next
```

this removes the connection between the start pointer and the first node.

free the memory: once we've updated the start pointer, we can free the memory that was allocated to the first node

```
(free(ptr))
```

function to delete the starting node

```

void deletestartnode() {
    (check if the list is empty)
    if (start == NULL) {
        printf("the list is empty. nothing to delete.\n");
    }
}
  
```

```

    return;
}

```

(create a temporary pointer to store the current start node)

```

struct node *temp = start;

```

(update the start pointer to the second node)

```

start = start->next;

```

(free the memory of the old start node)

```

free(temp);

```

```

printf("the first node has been deleted.\n");

```

```

}

```

working:

(i)

```

void deletestartnode() {

```

(check if the list is empty)

```

if (start == NULL) { (no, linked list is not empty)

```

```

    printf("the list is empty. nothing to delete.\n");

```

```

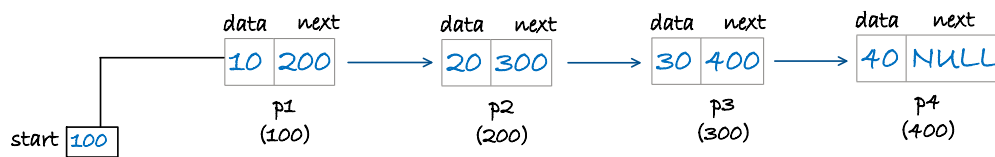
    return;

```

```

}

```



(ii)

```

struct node *temp = start;

```

(create a temporary pointer to store the current start node)

```

temp [ ]

```

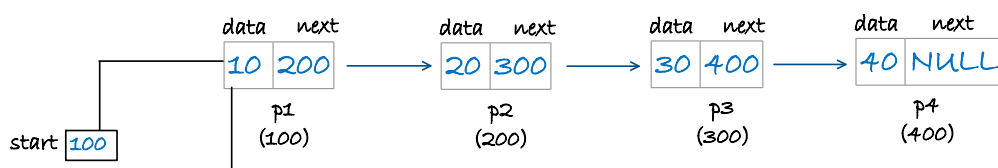
```

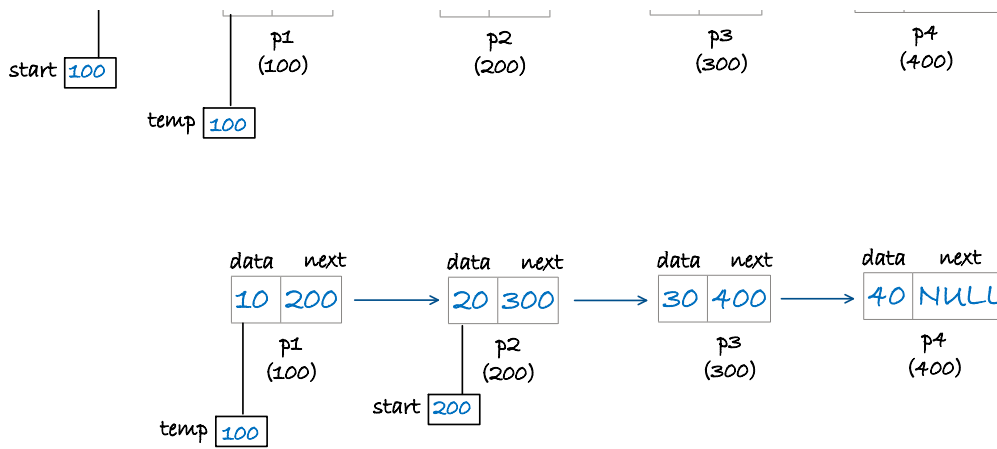
temp [100]    start [100]

```

(iii) start = start->next;

(update the start pointer to the second node)



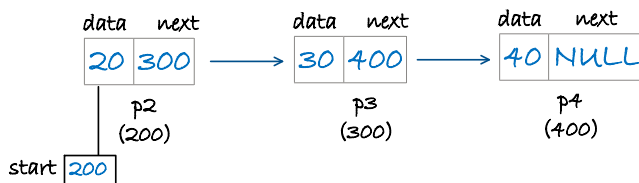
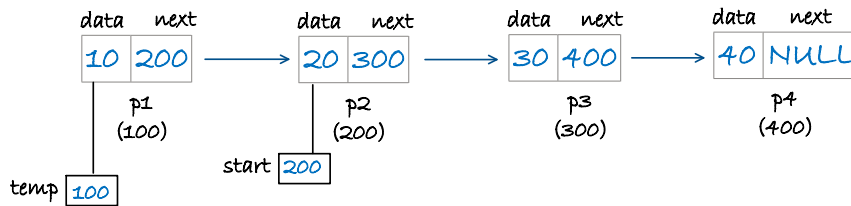


(iv)

`free(temp);`

`printf("the first node has been deleted.\n");`

(free the memory of the old start node)



the first node has been deleted

final code:

```
// Function to delete the starting node
void deleteStartNode() {
    // Check if the list is empty
    if (start == NULL) {
        printf("The list is empty. Nothing to delete.\n");
        return;
    }

    // Create a temporary pointer to store the current start node
    struct node *temp = start;
```

```

// Create a temporary pointer to store the current start node
struct node *temp = start;

// Update the start pointer to the second node
start = start->next;

// Free the memory of the old start node
free(temp);
printf("The first node has been deleted.\n");
}

```

Explanation of Code:

1. **Check if the list is empty:** The first thing we do is check if the list is empty by verifying if the `start` pointer is `NULL`. If it is, we print a message saying "The list is empty. Nothing to delete." and exit the function using the `return` statement. This ensures that we do not attempt to delete a node from an empty list.
2. **Create a temporary pointer:** A temporary pointer, `temp`, is created and set to point to the current `start` node. This pointer will store the address of the node we want to delete. By keeping this reference, we can later free the memory of this node after updating the list.
3. **Update the `start` pointer:** The `start` pointer is updated to point to the second node in the list. We do this by setting `start` to `start->next`. After this step, the first node is no longer part of the list, and the second node becomes the new "start" or first node of the list.
4. **Free the memory of the old start node:** After the `start` pointer has been updated, the first node is no longer referenced by any pointer. At this point, we can safely free the memory allocated to the old first node by calling `free(temp)`. This prevents a memory leak by ensuring that the memory occupied by the deleted node is properly released.
5. **Print confirmation message:** Finally, a message is printed to confirm that the first node has been successfully deleted from the list.

```

#include <stdio.h>
#include <stdlib.h>

// Define the structure for a linked list node
struct node {
    int data;
    struct node *next;
};

// Declare a global pointer to the start of the linked list
struct node *start = NULL;

// Function to insert a key at the end of the linked list
void insertAtEnd(int key) {
    // Create a new node
    struct node *newNode = (struct node*)malloc(sizeof(struct node));
    if (newNode == NULL) {
        printf("Memory allocation failed.\n");
        return;
    }
}

```

```

    struct node *newNode = (struct node*)malloc(sizeof(struct node));
    if (newNode == NULL) {
        printf("Memory allocation failed.\n");
        return;
    }
    newNode->data = key; // Set the key
    newNode->next = NULL; // This will be the last node, so next is NULL

    if (start == NULL) {
        // If the list is empty, the new node becomes the head
        start = newNode;
    } else {
        // Traverse to the last node
        struct node *ptr = start;
        while (ptr->next != NULL) {
            ptr = ptr->next;
        }
        // Append the new node to the end
        ptr->next = newNode;
    }
}

// Function to delete the starting node
void deleteStartNode() {
    // Check if the list is empty
    if (start == NULL) {
        printf("The list is empty. Nothing to delete.\n");
        return;
    }

    // Create a temporary pointer to store the current start node
    struct node *temp = start;

    // Update the start pointer to the second node
    start = start->next;

    // Free the memory of the old start node
    free(temp);
    printf("The first node has been deleted.\n");
}

// Function to display the linked list
void displayList() {
    struct node *ptr = start;
    if (ptr == NULL) {
        printf("The list is empty.\n");
        return;
    }
    printf("Linked list: ");
    while (ptr != NULL) {
        printf("%d → ", ptr->data);
        ptr = ptr->next;
    }
    printf("NULL\n");
}

int main() {
    // Insert some keys at the end of the linked list
    insertAtEnd(10);
    insertAtEnd(20);
    insertAtEnd(30);
    insertAtEnd(40);

    // Display the list
    displayList();

    // Delete the first node
    deleteStartNode();

    // Display the updated list

```

```

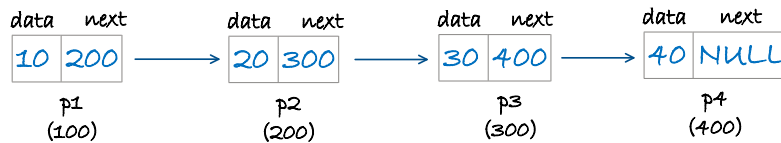
// Display the updated list
displayList();

return 0;
}

```

problem (x): given a linked list, delete the last node of the linked list.

analysis:



to delete the last node of linked list

(a) check if linked list is not empty

```

if (start == NULL) {
    printf("the list is empty. nothing to delete.\n");
    return;
}

```

(b) if there is one node in the linked list

the same code will work because there is only single node, no starting node and no last node.

function to delete the starting node

```

void deletestartnode() {
    (check if the list is empty)
    if (start == NULL) {
        printf("the list is empty. nothing to delete.\n");
        return;
    }
}

```

(create a temporary pointer to store the current start node)

```

struct node *temp = start;

```

(update the start pointer to the second node)

```

start = start->next;

```

(free the memory of the old start node)

```

free(temp);
printf("the first node has been deleted.\n");
}

```

(c) if there are more than one node.

so if there are more than one node, at first we should traverse to reach the second last node.

why not last node, why second last?

the logic here is the once we reach the second last node, we will set NULL in the next field so that second last automatically becomes the last node.

we created a pointer temp to traverse through the linked list.

```
while (temp->next!=NULL && temp->next->next!=NULL){  
temp=temp->next;  
}
```

(d) now as we reached the second last node, we created a new pointer where we store the address of last node and assign the next section of second last node with NULL so the connection between second last node and last node will be removed.

```
lastnode=temp->next;  
temp->next=NULL
```

(d) now free (lastnode)

```
free (lastnode);
```

function to delete the last node

```
void deleteLastNode() {  
    (check if the list is empty)  
    if (start == NULL) {  
        printf("The list is empty. Nothing to delete.\n");  
        return;  
    }  
}
```

(if the list has only one node, delete it)

```
if (start->next == NULL) {  
    free(start);  
    start = NULL; (set start to NULL as the list is now empty)  
    printf("The last node has been deleted.\n");  
    return;  
}
```

(traverse the list to find the second-to-last node)

```
struct node *temp = start;  
while (temp->next != NULL && temp->next->next != NULL) {  
    temp = temp->next;  
}
```

(delete the last node)

```
struct node *lastNode = temp->next;  
temp->next = NULL;  
free(lastNode); (free memory of the last node)
```

```
printf("The last node has been deleted.\n");
```

}

working:

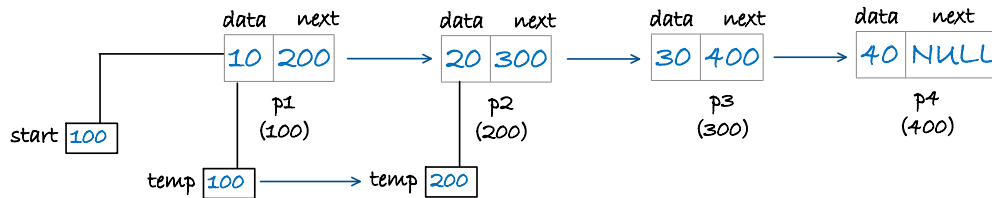
(i)

struct node *temp = start; (temp initialised with 100)

iteration ist:

while (temp->next != NULL && temp->next->next != NULL) { (not NULL)

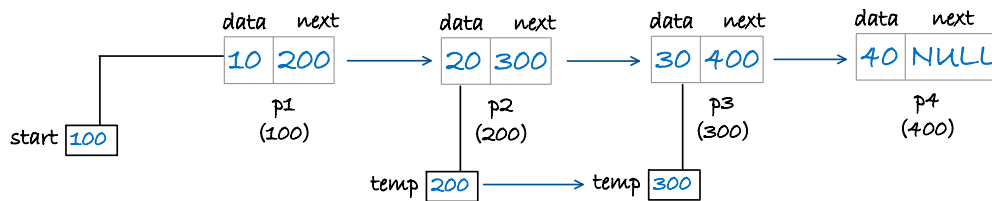
temp = temp->next; (moves to second node)



iteration iind:

while (temp->next != NULL && temp->next->next != NULL) { (not NULL)

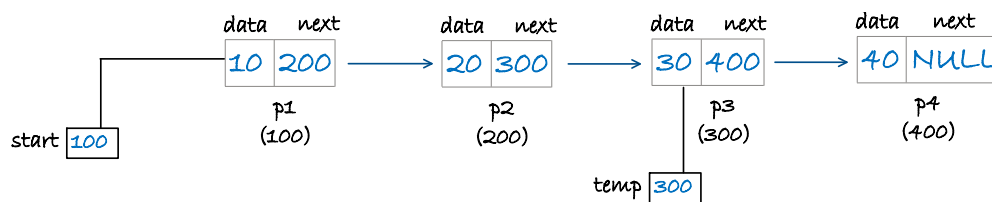
temp = temp->next; (moves to third node)



iteration iiird:

while (temp->next != NULL && temp->next->next != NULL) { (NULL)

temp = temp->next; (does not execute)



(ii)

struct node *lastNode = temp->next;

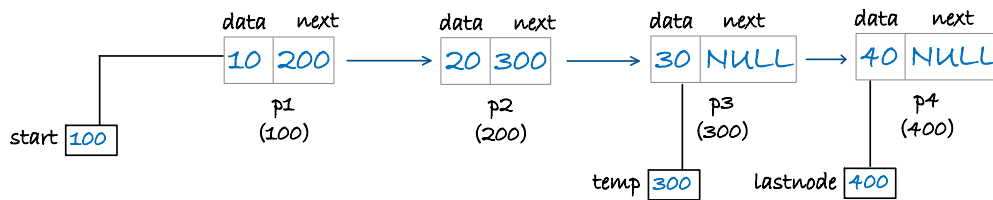
temp->next = NULL;

free(lastNode); (free memory of the last node)

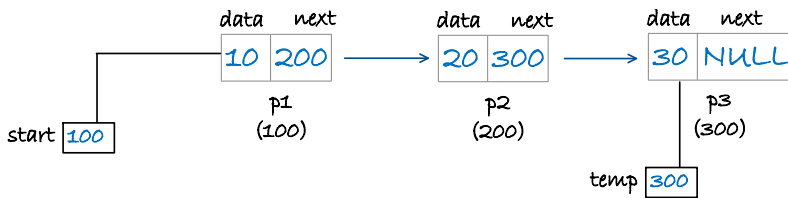
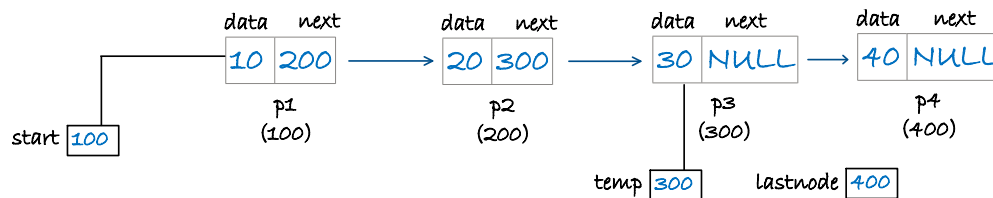
printf("The last node has been deleted.\n");

lastnode 400

temp->next = NULL;



free(lastNode); (free memory of the last node)
 printf("The last node has been deleted.\n");



The last node has been deleted.

final code:

```

// Function to delete the last node
void deleteLastNode() {
    // Check if the list is empty
    if (start == NULL) {
        printf("The list is empty. Nothing to delete.\n");
        return;
    }

    // If the list has only one node, delete it
    if (start->next == NULL) {
        free(start);
        start = NULL; // Set start to NULL as the list is now empty
        printf("The last node has been deleted.\n");
        return;
    }

    // Traverse the list to find the second-to-last node
    struct node *temp = start;
    while (temp->next != NULL && temp->next->next != NULL) {
        temp = temp->next;
    }

    // Delete the last node
  
```

```

    }

    // Delete the last node
    struct node *lastNode = temp->next;
    temp->next = NULL;
    free(lastNode); // Free memory of the last node

    printf("The last node has been deleted.\n");
}

```

Breakdown of the `deleteLastNode` function:

1. Check if the list is empty:

- If the `start` pointer is `NULL`, it means the list is empty. In this case, a message is printed and the function exits without doing anything.

2. Check if the list has only one node:

- If the list has only one node (`start->next == NULL`), we free the memory of that single node, set `start` to `NULL` (making the list empty), and print a message confirming the deletion.

3. Traverse the list to find the second-to-last node:

- If the list has more than one node, we need to traverse the list to find the second-to-last node (the node before the last one). The condition `temp->next != NULL && temp->next->next != NULL` ensures that we stop at the second-to-last node.

4. Delete the last node:

- Once the second-to-last node is found (`temp`), we:
 - Store the last node in the `lastNode` pointer (`struct node *lastNode = temp->next`).
 - Set `temp->next` to `NULL`, disconnecting the last node from the list.
 - Finally, we free the memory of the last node (`free(lastNode)`).

```

#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *start = NULL;

// Function to delete the last node
void deleteLastNode() {
    if (start == NULL) {
        printf("The list is empty. Nothing to delete.\n");
        return;
    }
}

```

```

        if (start == NULL) {
            printf("The list is empty. Nothing to delete.\n");
            return;
        }
        if (start->next == NULL) {
            free(start);
            start = NULL;
            printf("The last node has been deleted.\n");
            return;
        }

        struct node *temp = start;
        while (temp->next != NULL && temp->next->next != NULL) {
            temp = temp->next;
        }

        struct node *lastNode = temp->next;
        temp->next = NULL;
        free(lastNode);
        printf("The last node has been deleted.\n");
    }

    // Function to insert a new node at the end
    void insertAtEnd(int key) {
        struct node *newNode = (struct node*)malloc(sizeof(struct node));
        if (newNode == NULL) {
            printf("Memory allocation failed.\n");
            return;
        }
        newNode->data = key;
        newNode->next = NULL;

        if (start == NULL) {
            start = newNode;
        } else {
            struct node *temp = start;
            while (temp->next != NULL) {
                temp = temp->next;
            }
            temp->next = newNode;
        }
    }

    // Function to display the list
    void displayList() {
        struct node *temp = start;
        if (temp == NULL) {
            printf("The list is empty.\n");
            return;
        }
        while (temp != NULL) {
            printf("%d → ", temp->data);
            temp = temp->next;
        }
        printf("NULL\n");
    }

    int main() {
        insertAtEnd(10);
        insertAtEnd(20);
        insertAtEnd(30);

        printf("Before deleting last node:\n");
        displayList();
    }

```

```

deleteLastNode(); // Delete last node

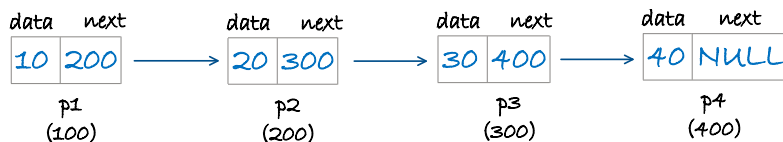
printf("After deleting last node:\n");
displayList();

return 0;
}

```

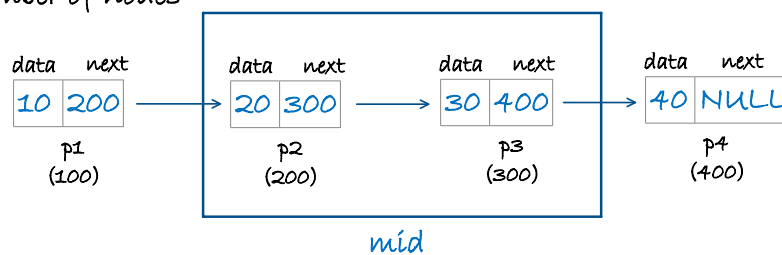
problem (xi): given a linked list, print mid node data.

analysis:

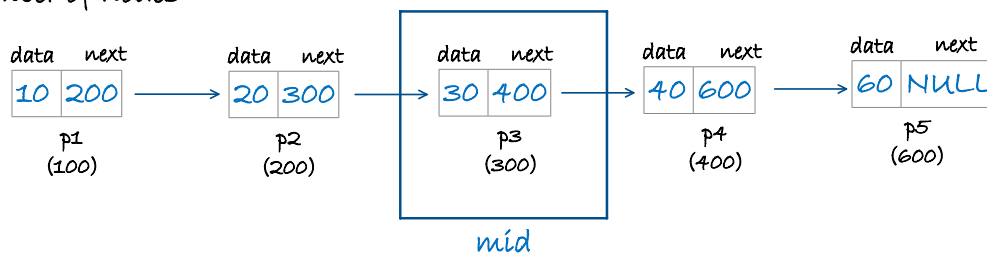


there are two cases possible in this scenario:

case(i): even number of nodes



case(ii): odd number of nodes



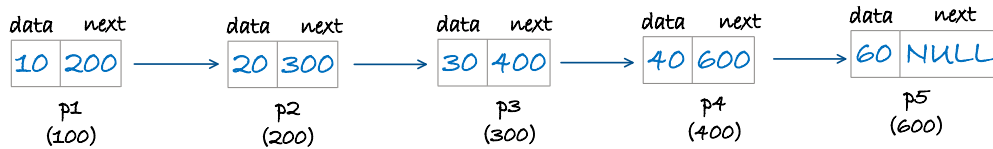
concept: slow and fast pointer (tortoise and hare algorithm)

to find the middle node of a linked list, you can use the two-pointer technique (also known as the tortoise and hare algorithm)

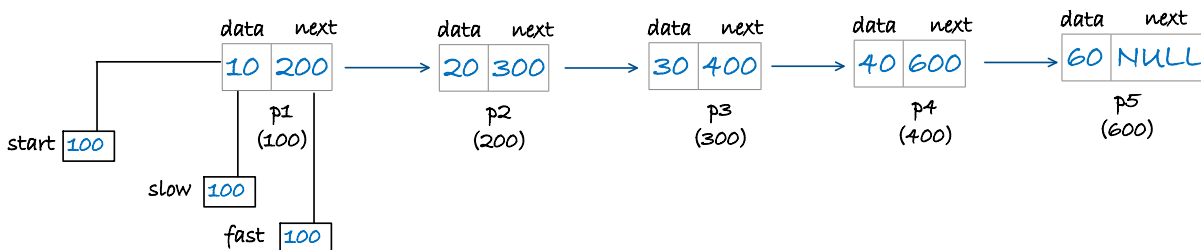
slow pointer: this pointer moves one step at a time.

fast pointer: this pointer moves two steps at a time.

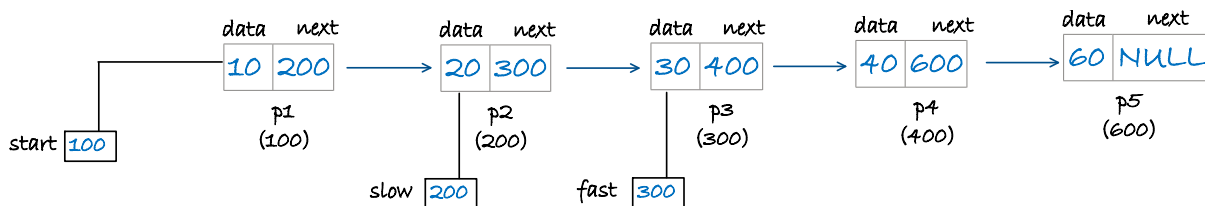
by the time the fast pointer reaches the end of the list, the slow pointer will be at the middle.



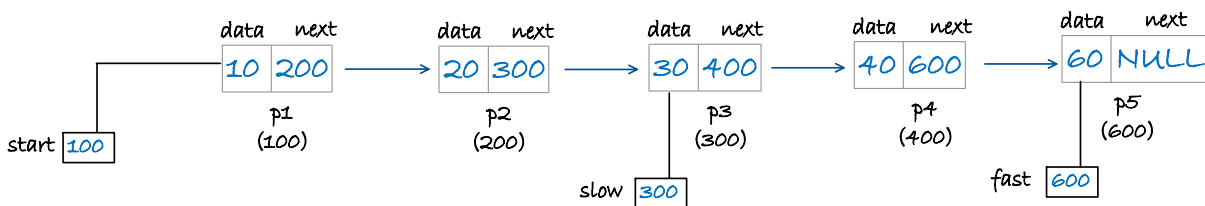
(i) firstly we will create two pointers slow and fast,
 we want to move slow means the next node
 and with fast pointer we want to move to third node
 slow=start;
 fast=start;



slow=slow->next;
 fast=fast->next->next;



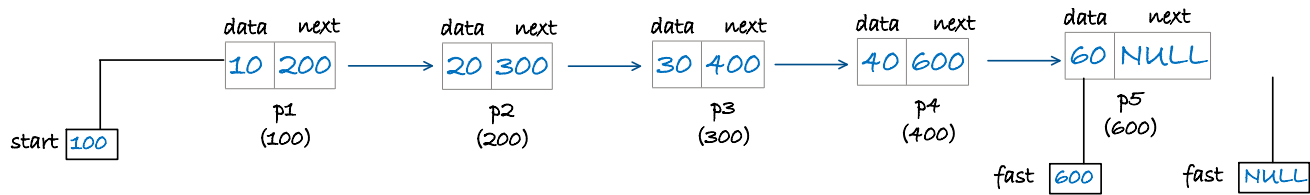
slow=slow->next;
 fast=fast->next->next;



problem: in the case of even nodes
 fast in on NULL.

so we have to stop when either
 fast=NULL or fast->next=NULL;

so we have to stop when either
fast=NULL or fast->next=NULL;



so,
while (fast!=null fast->next!=NULL){
slow=slow->next;
fast=fast->next->next;
}

final code:

```
void printMiddleNode() {
    // Check if the list is empty
    if (start == NULL) {
        printf("The list is empty.\n");
        return;
    }

    struct node *slow = start; // Slow pointer (moves one step at a time)
    struct node *fast = start; // Fast pointer (moves two steps at a time)

    // Traverse the list
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next; // Move slow pointer by one step
        fast = fast->next->next; // Move fast pointer by two steps
    }

    // At this point, the slow pointer is at the middle node
    printf("The middle node's data is: %d\n", slow->data);
}
```


Explanation:

1. Initial Setup:

- We initialize two pointers, `slow` and `fast`, both starting at the `start` of the linked list.
- `slow` will move one step at a time, while `fast` will move two steps at a time.

2. List Traversal:

- The `while` loop continues as long as `fast` is not NULL and `fast->next` is not NULL. This ensures that we don't run into errors if the list has fewer than two nodes.
- In each iteration of the loop:
 - `slow` moves one step forward (`slow = slow->next`).
 - `fast` moves two steps forward (`fast = fast->next->next`).

3. Middle Node Logic:

- Since `fast` is moving two steps at a time and `slow` is moving one step at a time, when `fast` reaches the end of the list, `slow` will be in the middle of the list.
- If the list has an odd number of nodes, `slow` will be exactly at the middle node. If the list has an even number of nodes, `slow` will be at the second middle node (this is common in many implementations).

4. Print Middle Node:

- Once the loop exits, `slow` is pointing to the middle node. The data of the middle node is then printed (`printf("The middle node's data is: %d\n", slow->data)`).

```
#include <stdio.h>
#include <stdlib.h>

// Define the structure for a linked list node
struct node {
    int data;
    struct node *next;
};

// Declare a global pointer to the start of the linked list
struct node *start = NULL;

// Function to insert a node at the end of the linked list
void insertAtEnd(int key) {
    struct node *newNode = (struct node*)malloc(sizeof(struct node));
    if (newNode == NULL) {
        printf("Memory allocation failed.\n");
        return;
    }
    newNode->data = key;
    newNode->next = NULL;

    // If the list is empty, make the new node the head
    if (start == NULL) {
        start = newNode;
    } else {
        // Traverse to the last node and append the new node
        struct node *temp = start;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
}
```

```

        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
}

// Function to print the middle node's data
void printMiddleNode() {
    // Check if the list is empty
    if (start == NULL) {
        printf("The list is empty.\n");
        return;
    }

    struct node *slow = start; // Slow pointer (moves one step at a time)
    struct node *fast = start; // Fast pointer (moves two steps at a time)

    // Traverse the list
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next; // Move slow pointer by one step
        fast = fast->next->next; // Move fast pointer by two steps
    }

    // At this point, the slow pointer is at the middle node
    printf("The middle node's data is: %d\n", slow->data);
}

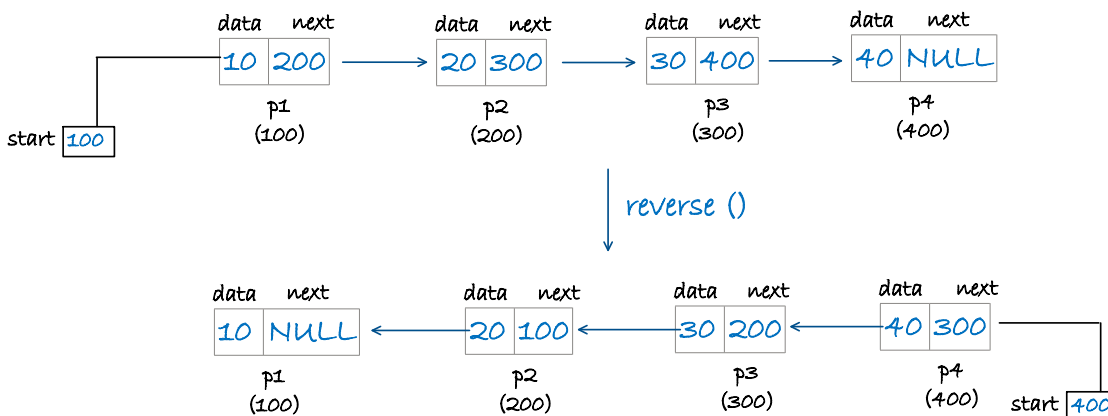
int main() {
    // Insert some nodes at the end of the linked list
    insertAtEnd(10);
    insertAtEnd(20);
    insertAtEnd(30);
    insertAtEnd(40);
    insertAtEnd(60); // Insert 60 instead of 50

    // Call the function to print the middle node's data
    printMiddleNode();

    return 0;
}

```

problem (xii): reverse a linked list.



(100)

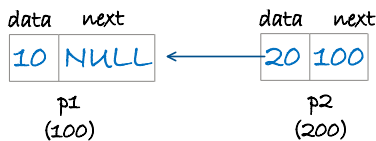
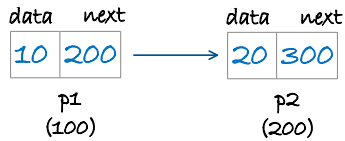
(200)

(300)

(400) start 400

reversing two nodes

(a) let's say we are reversing two nodes and we will analyse how this works and what needs to be done.



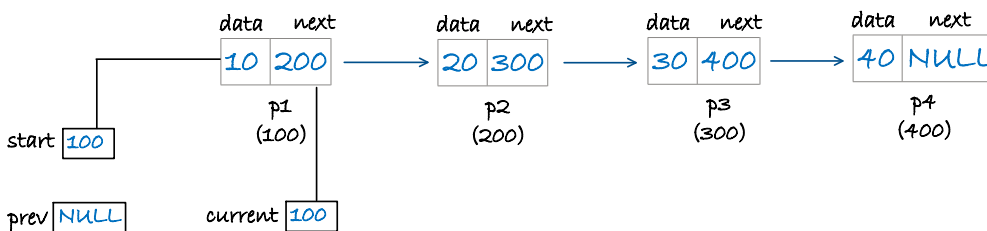
the first node becomes the last node and the last node becomes the first node, so as per last node it contains NULL

now the second node becomes first node so the next field contains the address of last node which is 100.

so we need two pointers to store the both nodes address and reverse them.

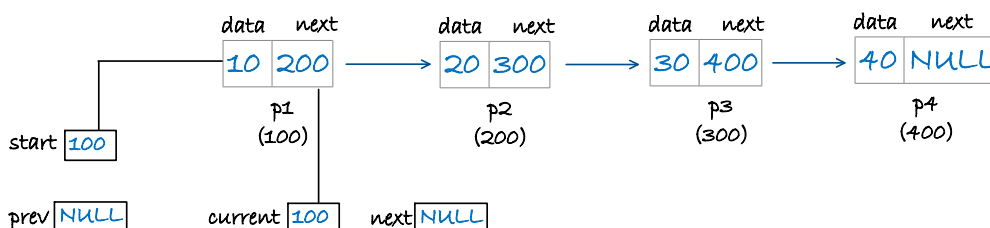
let's say;

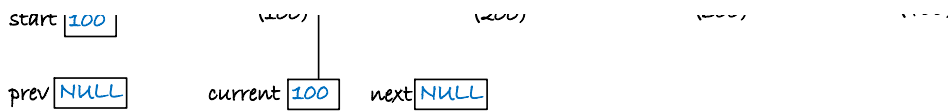
`*prev = NULL, *current = start`



but if we perform this in more than two nodes, the connection will be terminated for rest of the nodes and if we try to reverse two nodes, we must store the address of third node while we are reversing first two nodes.

`*next_node = NULL`





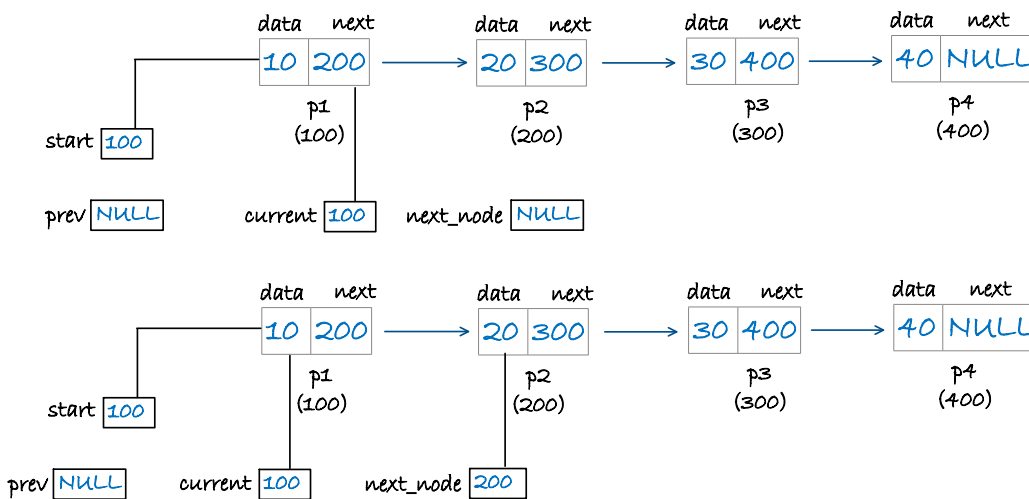
the current should not be NULL

```

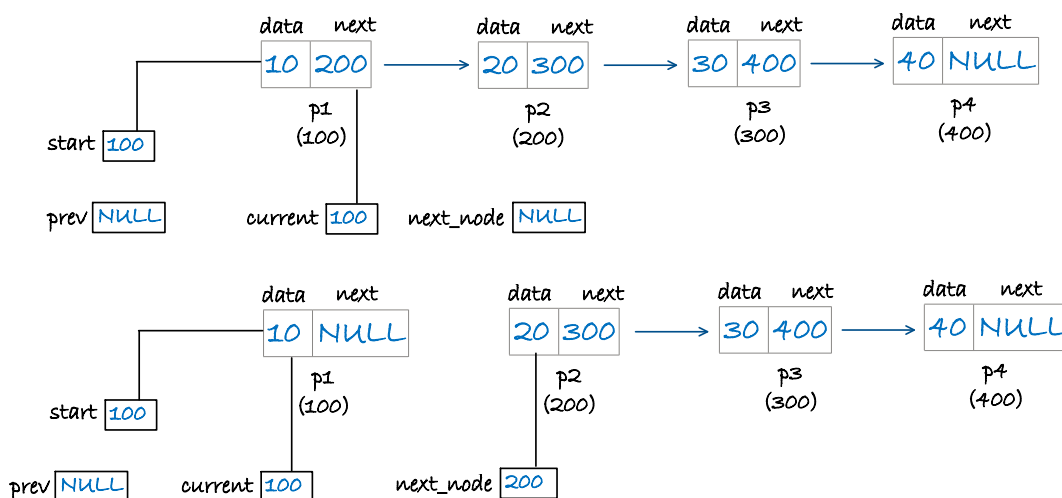
while (current != NULL) {
    next_node = current->next; (save the next node temporarily)
    current->next = prev; (reverse the current node's next pointer)
    prev = current; (move prev and current one step forward)
    current = next_node;
}
start = prev; (update the start to the new head (prev will be the new head))
}

```

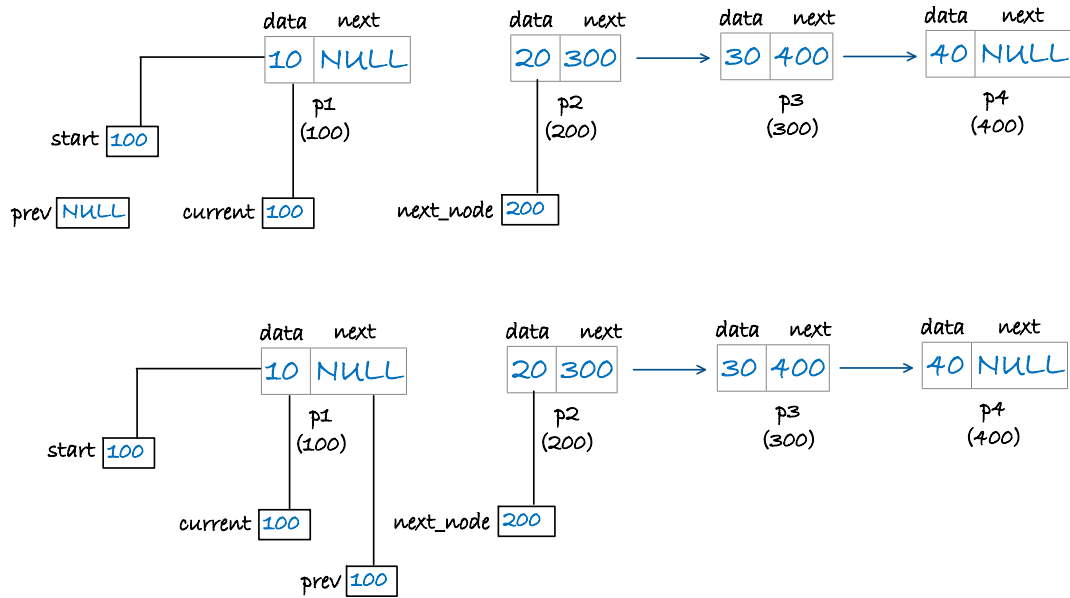
(i) `next_node = current->next;` (save the next node temporarily)



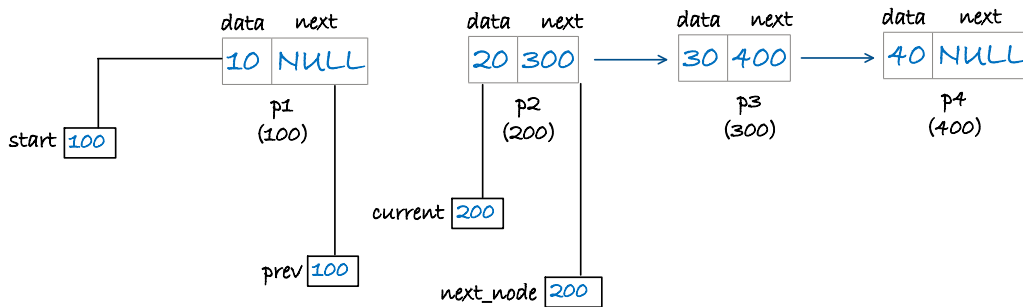
(ii) `current->next = prev;` (reverse the current node's next pointer)



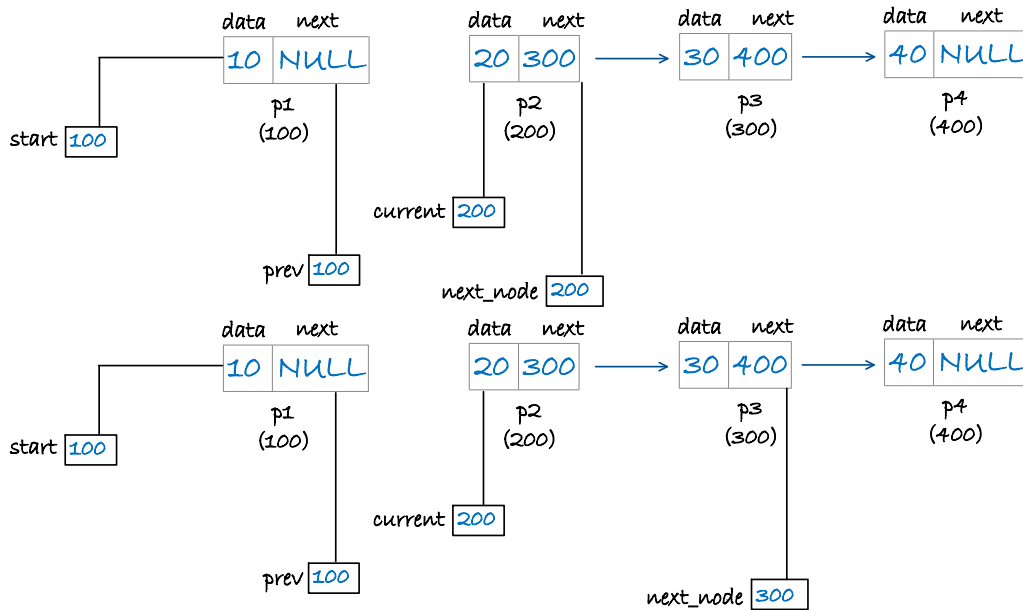
(iii) prev = current; (move prev and current one step forward)



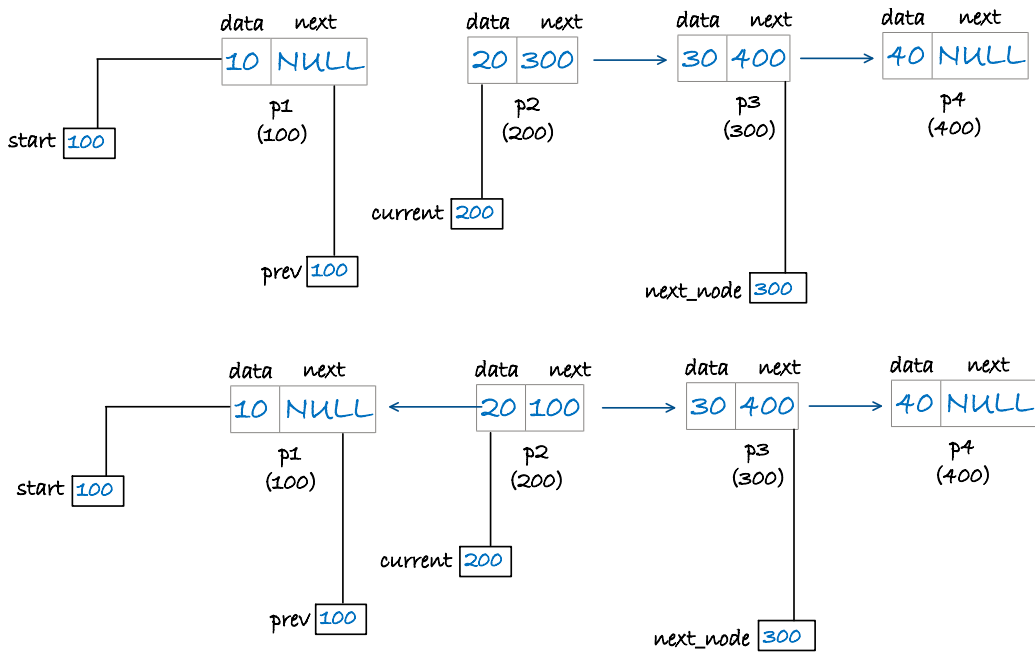
(iv) current = next_node;



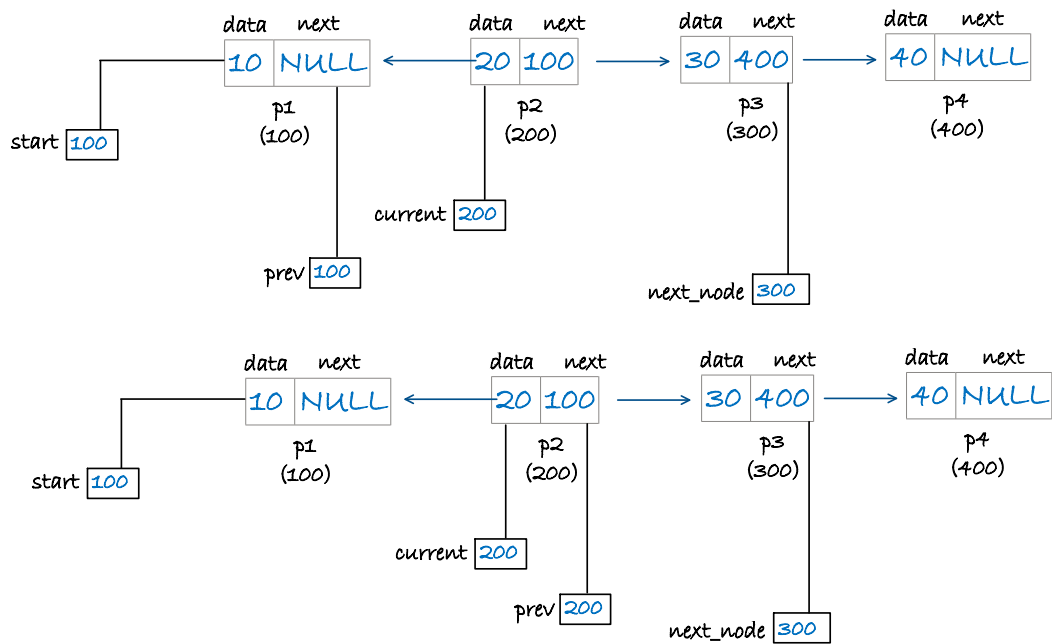
(i) next_node = current->next; (save the next node temporarily)



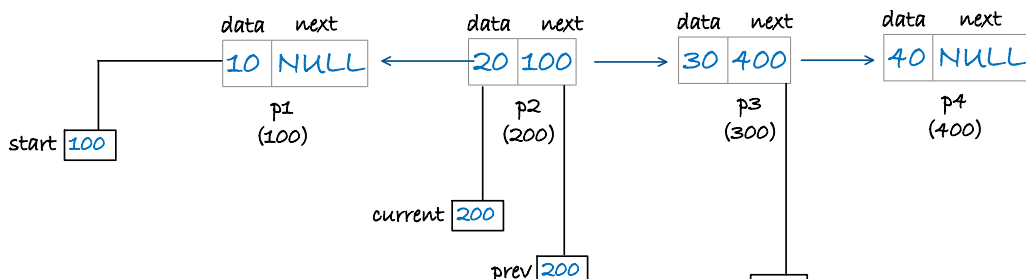
(ii) `current->next = prev;` (reverse the current node's next pointer)

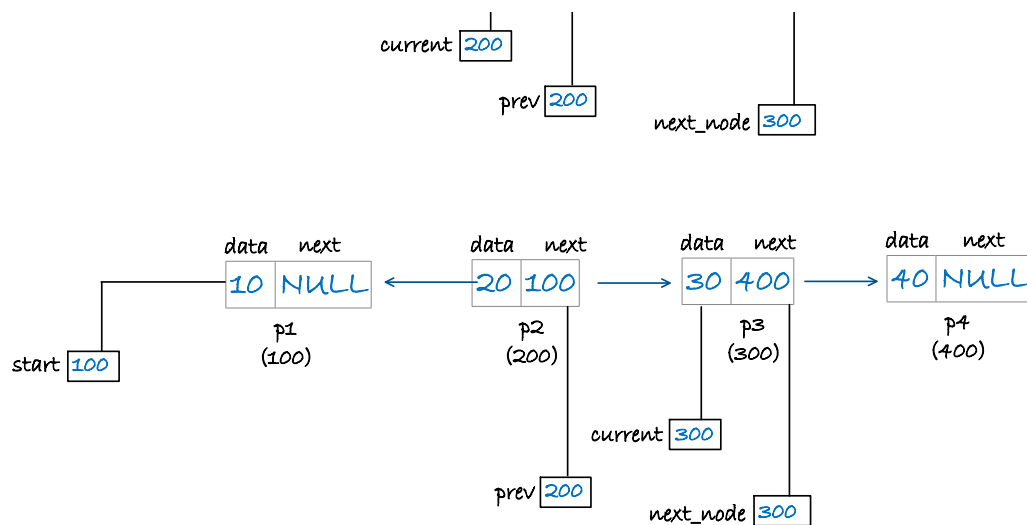


(iii) `prev = current;` (move prev and current one step forward)



(iv) `current = next_node;`





```
// Function to reverse the linked list
void reverseList() {
    struct node *prev = NULL, *current = start, *next_node = NULL;

    // Traverse the list and reverse the links
    while (current != NULL) {
        next_node = current->next; // Save the next node temporarily
        current->next = prev;      // Reverse the current node's next pointer
        prev = current;           // Move prev and current one step forward
        current = next_node;
    }
}
```

Explanation:

1. Initialization:

- `prev` is initialized as `NULL` because the first node's next pointer should point to `NULL` after reversal.
- `current` is initialized to `start`, the head of the list, to start traversing the linked list.
- `next_node` is a temporary pointer that helps us preserve the link to the next node while we reverse the `current` node's `next` pointer.

2. While Loop:

- The `while (current != NULL)` loop iterates through the linked list.
- In each iteration:
 - We temporarily store the next node (`current->next`) in `next_node` because we will change `current->next` in the next step.
 - We reverse the `current` node's `next` pointer by setting it to point to `prev`.
 - We move `prev` to the current node because after the reversal, `prev` will become the new "previous" node for the next iteration.
 - We then move `current` to the next node using `next_node` to continue traversing the list.

3. Updating `start`:

- After the loop, `current` will be `NULL`, and `prev` will be pointing to the last node, which is now the first node in the reversed list.
- We set `start = prev` to update the head of the list.

```
#include <stdio.h>
#include <stdlib.h>

// Define the structure for a linked list node
struct node {
    int data;
    struct node *next; // This stores the address of the next node in the list
};

// Global pointer to the start of the linked list
struct node *start = NULL;

// Function to insert a key at the end of the linked list
void insertAtEnd(int key) {
    struct node *newNode = (struct node*)malloc(sizeof(struct node));
    if (newNode == NULL) {
        printf("Memory allocation failed.\n");
        return;
    }
    newNode->data = key;
    newNode->next = NULL; // This will be the last node, so next is NULL

    if (start == NULL) {
        start = newNode; // If the list is empty, set start to the new node
    } else {
        struct node *ptr = start;
        while (ptr->next != NULL) { // Traverse to the last node
            ptr = ptr->next;
        }
        ptr->next = newNode; // Append the new node to the end
    }
}
```



```

    }
    ptr->next = newNode; // Append the new node to the end
}
}

// Function to reverse the linked list
void reverseList() {
    struct node *prev = NULL, *current = start, *next_node = NULL;

    // Traverse the list and reverse the links
    while (current != NULL) {
        next_node = current->next; // Save the next node temporarily
        current->next = prev;      // Reverse the current node's next pointer
        prev = current;           // Move prev and current one step forward
        current = next_node;
    }

    start = prev; // Update the start to the new head (prev will be the new head)
}

// Function to display the linked list
void displayList() {
    struct node *ptr = start;
    if (ptr == NULL) {
        printf("The list is empty.\n");
        return;
    }
    printf("Linked list: ");
    while (ptr != NULL) {
        printf("%d → ", ptr->data);
        ptr = ptr->next; // Move to the next node
    }
    printf("NULL\n");
}

int main() {
    // Insert elements at the end of the linked list
    insertAtEnd(10);
    insertAtEnd(20);
    insertAtEnd(30);
    insertAtEnd(40);
    insertAtEnd(50);

    // Display the original list
    printf("Original list:\n");
    displayList();

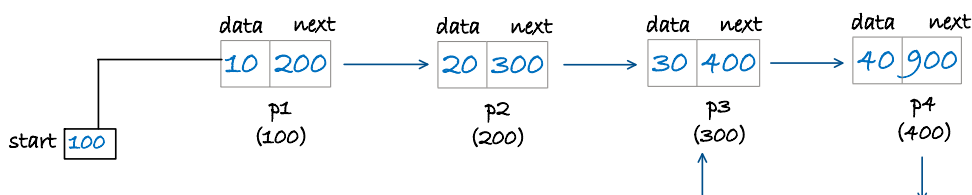
    // Reverse the linked list
    reverseList();

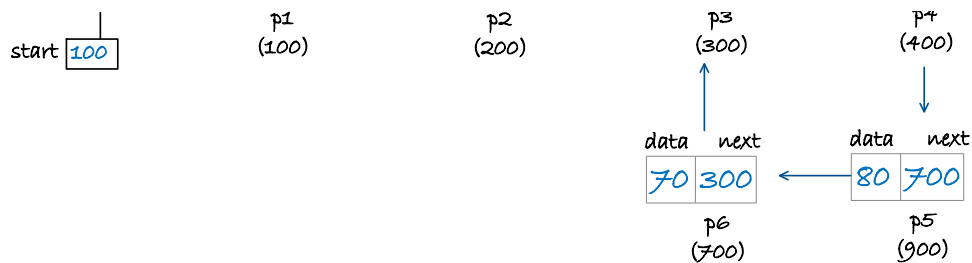
    // Display the reversed list
    printf("Reversed list:\n");
    displayList();

    return 0;
}

```

problem (xiii): given a linked list, detect whether it contains a loop or not.

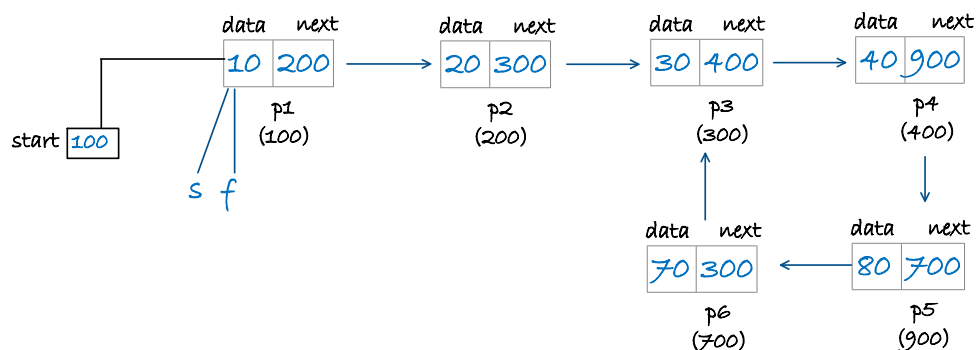




lets try to understand through slow and fast pointers.

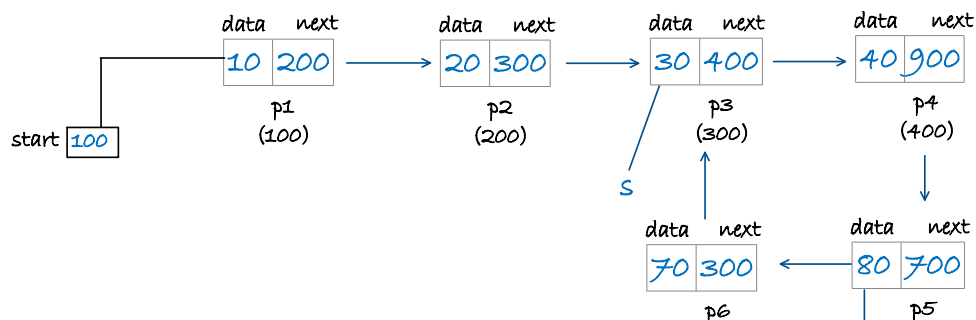
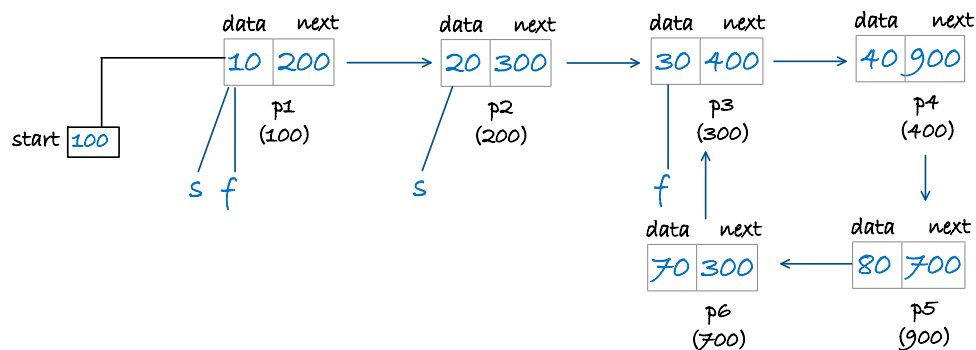
*slow=start;

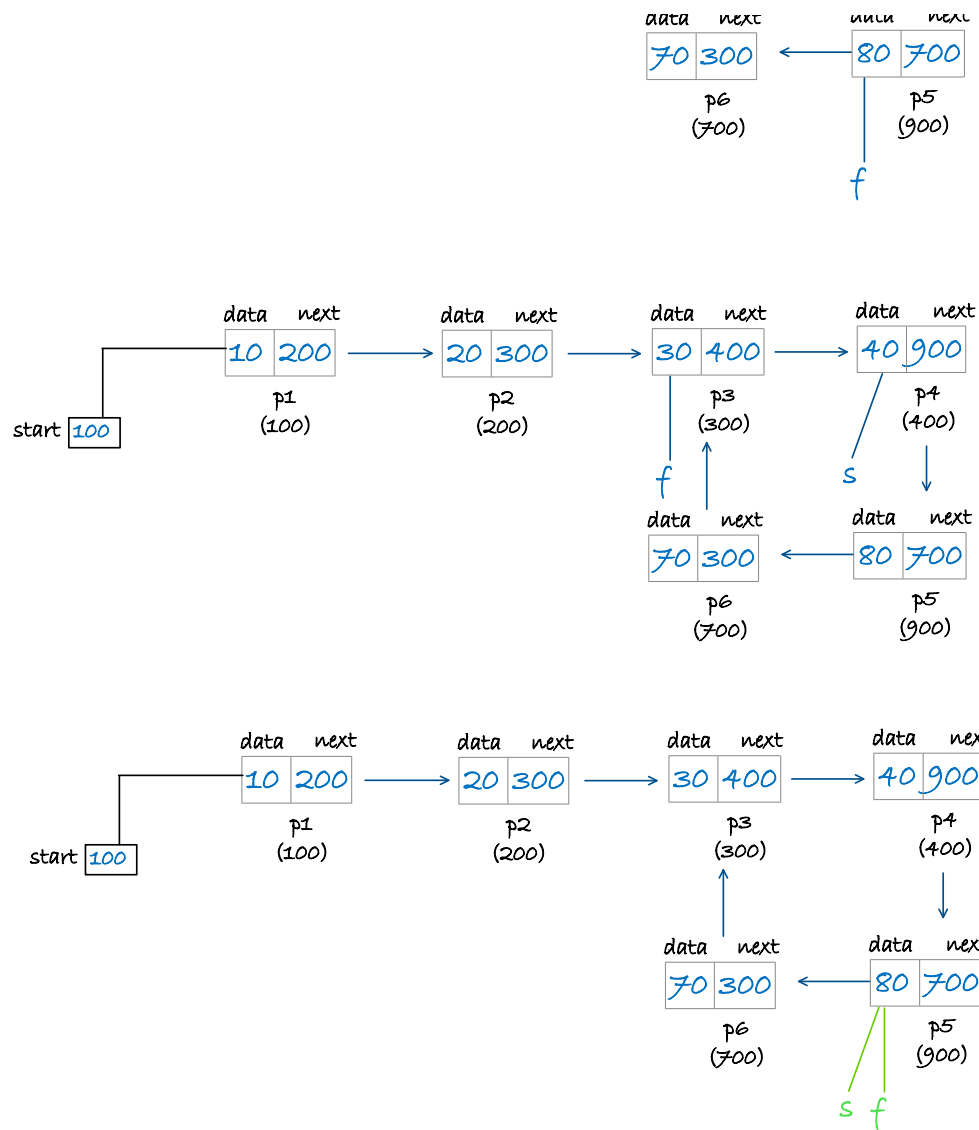
*fast=start;



slow=slow->next;

fast=fast->next->next;





the point here is that if loop exists in the linked list then the fast and slow pointer will collide

```

so,
slow=slow->next;
fast=fast->next->next;
if(slow==fast){
return 1;
}

```

now the another analysis is that when there is cycle, there is no NULL in any next field
but what if linked list does not cycle?

then it the last node will have NULL in their next section

so to ensure we must check

fast!=NULL (moves 2 node ahead so it may reach NULL if there no loop)

fast->next!=NULL (if the next node is NULL then no loop exists)

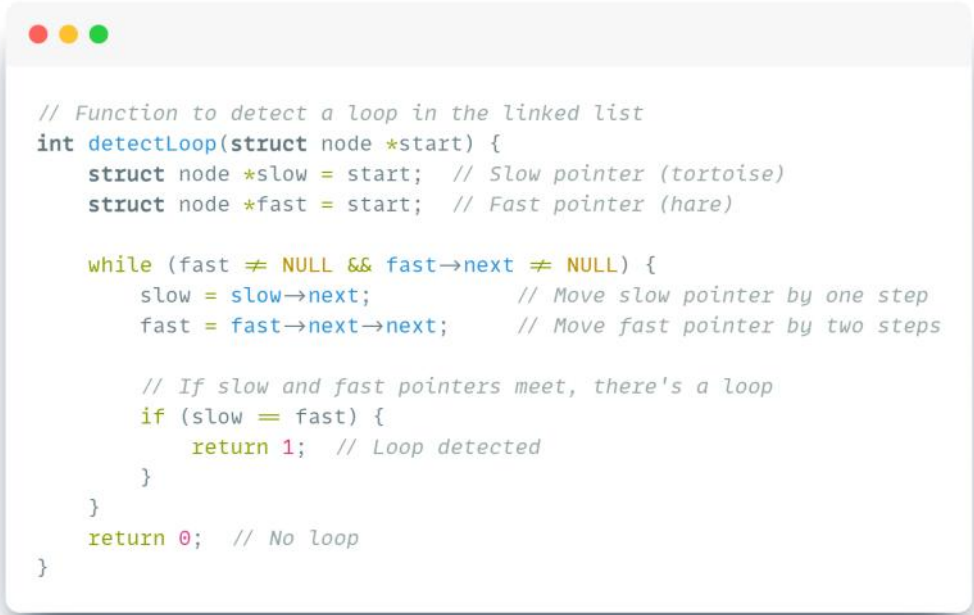
but if any condition gets false then we should return false

so,

fast!=NULL && fast->next!=NULL

function to detect a loop in the linked list

```
int detectLoop(struct node *start) {  
    struct node *slow = start; (slow pointer (tortoise))  
    struct node *fast = start; (fast pointer (hare))  
  
    while (fast != NULL && fast->next != NULL) {  
        slow = slow->next; (move slow pointer by one step)  
        fast = fast->next->next; (move fast pointer by two steps)  
  
        (if slow and fast pointers meet, there's a loop)  
        if (slow == fast) {  
            return 1; (loop detected)  
        }  
    }  
    return 0; (no loop)  
}
```



```
// Function to detect a loop in the linked list  
int detectLoop(struct node *start) {  
    struct node *slow = start; // Slow pointer (tortoise)  
    struct node *fast = start; // Fast pointer (hare)  
  
    while (fast != NULL && fast->next != NULL) {  
        slow = slow->next; // Move slow pointer by one step  
        fast = fast->next->next; // Move fast pointer by two steps  
  
        // If slow and fast pointers meet, there's a loop  
        if (slow == fast) {  
            return 1; // Loop detected  
        }  
    }  
    return 0; // No loop  
}
```

Step-by-Step Explanation:

1. Initial Setup:

- Two pointers are initialized at the start of the linked list:
 - `slow` (tortoise) moves one step at a time.
 - `fast` (hare) moves two steps at a time.
- Both pointers start at the same position (`start`).

2. The Loop:

- The `while` loop checks if `fast` and `fast->next` are not `NULL`. This ensures that the list is long enough to move the `fast` pointer two steps at a time.
- Inside the loop:
 - The `slow` pointer advances by one node (`slow = slow->next`).
 - The `fast` pointer advances by two nodes (`fast = fast->next->next`).

3. Cycle Detection:

- If the list contains a loop, eventually, the `slow` pointer and `fast` pointer will meet at the same node within the loop. This is because the `fast` pointer moves twice as fast as the `slow` pointer.
- If they meet (`slow == fast`), the function returns `1`, indicating that a loop has been detected.

4. No Loop:

- If the `fast` pointer reaches the end of the list (`NULL`), it means there is no loop in the list, and the function returns `0`.

```
#include <stdio.h>
#include <stdlib.h>

// Define the structure for a linked list node
struct node {
    int data;
    struct node *next;
};

// Function to insert a node at the end of the list
void insertAtEnd(struct node **start, int key) {
    struct node *newNode = (struct node*)malloc(sizeof(struct node));
    if (newNode == NULL) {
        printf("Memory allocation failed.\n");
        return;
    }
    newNode->data = key;
    newNode->next = NULL;

    if (*start == NULL) {
        *start = newNode; // If the list is empty, set start to the new node
    } else {
        struct node *ptr = *start;
        while (ptr->next != NULL) {
            ptr = ptr->next;
        }
        ptr->next = newNode; // Append the new node to the end
    }
}

// Function to detect a loop in the linked list
int detectLoop(struct node *start) {
```

```

}

// Function to detect a loop in the linked list
int detectLoop(struct node *start) {
    struct node *slow = start; // Slow pointer (tortoise)
    struct node *fast = start; // Fast pointer (hare)

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next; // Move slow pointer by one step
        fast = fast->next->next; // Move fast pointer by two steps

        // If slow and fast pointers meet, there's a loop
        if (slow == fast) {
            return 1; // Loop detected
        }
    }
    return 0; // No loop
}

// Function to create a loop for testing (for demonstration)
void createLoop(struct node *start) {
    struct node *temp = start;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    // Create a loop by connecting the last node to the second node
    temp->next = start->next;
}

// Function to display the linked list (just for testing without an infinite loop)
void displayList(struct node *start) {
    struct node *ptr = start;
    if (ptr == NULL) {
        printf("The list is empty.\n");
        return;
    }
    printf("Linked list: ");
    while (ptr != NULL) {
        printf("%d → ", ptr->data);
        ptr = ptr->next;
    }
    printf("NULL\n");
}

int main() {
    struct node *start = NULL;

    // Insert elements into the linked list
    insertAtEnd(&start, 10);
    insertAtEnd(&start, 20);
    insertAtEnd(&start, 30);
    insertAtEnd(&start, 40);

    // Create a loop for testing (comment this line if no loop is needed)
    createLoop(start);

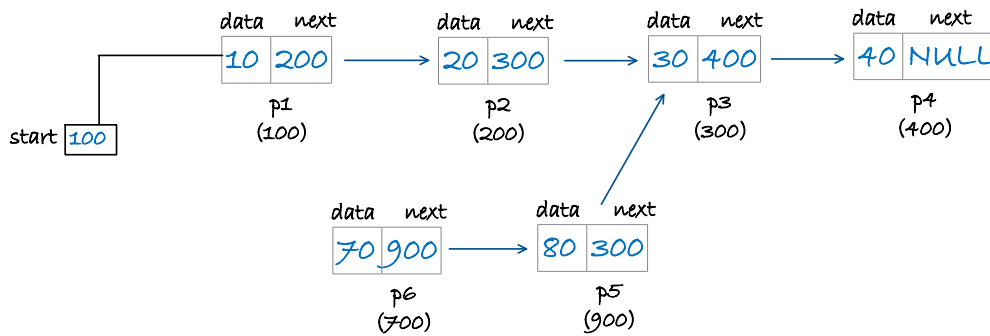
    // Detect loop in the linked list
    if (detectLoop(start)) {
        printf("Loop detected in the linked list.\n");
    } else {
        printf("No loop detected in the linked list.\n");
    }

    return 0;
}

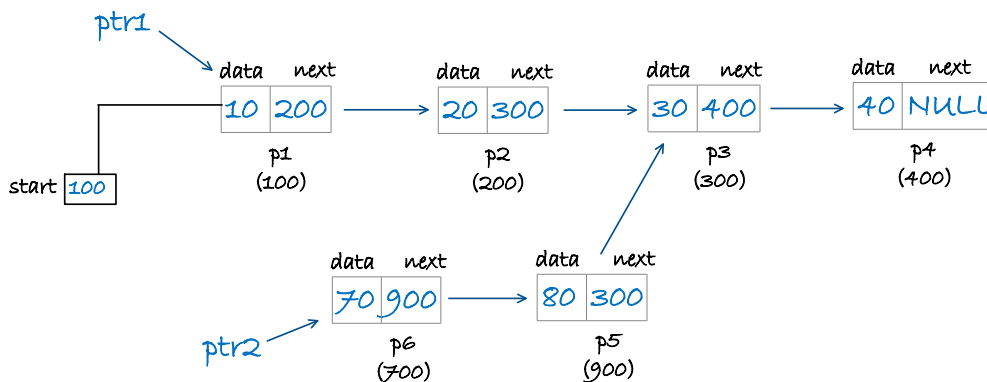
```

problem (xiv): there are two linked lists given; m and n. find the intersection point between two linked

lists.



(i) the basic idea here is to understand the difference because lets say there are two pointer and both are pointing to different linked lists.



the rough idea here is that first we will get the length of the both of the nodes and then we will find the difference between both nodes.

we will make both of pointer traverse when the length is common so both pointer will intersect at common point.

(i) get the length of both nodes

```
int getLength(struct node *head) {
    int length = 0; (length starts from 0)
    struct node *current = head;
    while (current != NULL) { (until the current is not NULL means we not reached last node, traverse)
        length++; (increase the count of the length)
        current = current->next; (moves to the next node)
    }
    return length; (once the function reaches NULL it will return the length)
}
```

(ii) as we get the length both nodes we will now find the difference between them.

```
struct node* findIntersection(struct node *head1, struct node *head2) {
    int len1 = getLength(head1); (store the length of first node into intlen1 variable)
    int len2 = getLength(head2); (store the length of second node into intlen2 variable)
```

(find the difference in lengths)

`int diff = abs(len1 - len2);` (this will give us the difference between length)

`struct node *ptr1 = head1;` (to move through the first linked list we need a pointer)

`struct node *ptr2 = head2;` (to move through the second linked list we need a pointer)

(move the pointer of the longer list by the difference in lengths)

(i)

`if (len1 > len2) {` (if length 1 is greater than length 2)

`for (int i = 0; i <= diff; i++) {` (iterate until i is not greater than difference, move ptr1 to next node)

`ptr1 = ptr1->next;`

`}`

`} else {`

`for (int i = 0; i <= diff; i++) {`

`ptr2 = ptr2->next;`

`}`

`}`

(ii)

`if (len1 > len2) {` (if length 1 is not greater than length 2)

`for (int i = 0; i <= diff; i++) {`

`ptr1 = ptr1->next;`

`}`

`} else {`

`for (int i = 0; i <= diff; i++) {` (iterate until i is not greater than difference, move ptr2 to next node)

`ptr2 = ptr2->next;`

`}`

`}`

final code:

```
// Function to calculate the length of a linked list
int getLength(struct node *head) {
    int length = 0;
    struct node *current = head;
    while (current != NULL) {
        length++;
        current = current->next;
    }
    return length;
}

// Function to find the intersection point of two linked lists
struct node* findIntersection(struct node *head1, struct node *head2) {
    int len1 = getLength(head1);
    int len2 = getLength(head2);
```



```

struct node* findIntersection(struct node *head1, struct node *head2) {
    int len1 = getLength(head1);
    int len2 = getLength(head2);

    // Find the difference in lengths
    int diff = abs(len1 - len2);

    struct node *ptr1 = head1;
    struct node *ptr2 = head2;

    // Move the pointer of the longer list by the difference in lengths
    if (len1 > len2) {
        for (int i = 0; i < diff; i++) {
            ptr1 = ptr1->next;
        }
    } else {
        for (int i = 0; i < diff; i++) {
            ptr2 = ptr2->next;
        }
    }

    // Traverse both lists simultaneously and find the intersection point
    while (ptr1 != NULL && ptr2 != NULL) {
        if (ptr1 == ptr2) {
            return ptr1; // Intersection point found
        }
        ptr1 = ptr1->next;
        ptr2 = ptr2->next;
    }

    return NULL; // No intersection point
}

```

Step-by-Step Explanation:

1. Calculate the Length of Both Lists:

- First, we calculate the length of each linked list by traversing through each list and counting the number of nodes. This can be done using a function that starts from the head of the list and keeps moving to the next node, increasing a counter for each node, until we reach the end (`NULL`).

2. Find the Length Difference:

- After calculating the lengths of both lists, we find the difference in their lengths. This helps us identify how much longer one list is compared to the other. This difference is important because the lists might be of different lengths, but the intersection (if it exists) will be at the same distance from the end of both lists. We use this length difference to align the two lists.

3. Align the Two Lists:

- To ensure that both lists are equidistant from the intersection point (if it exists), we move the pointer of the longer list forward by the difference in lengths. This way, the remaining nodes in both lists will be aligned, and we can compare them without considering the extra nodes in the longer list.

4. Traverse Simultaneously:

- Once the lists are aligned, we start traversing both lists simultaneously, comparing each pair of nodes one by one. If at any point the two pointers meet at the same node, it means we have found the intersection point. This node is the common node that exists in both lists.

5. No Intersection:

- If the pointers reach the end of the lists without meeting, it means there is no intersection point between the two lists. We return `NULL` to indicate that no intersection exists.

topic: types of linked list

concept: single linked list

a single linked list is a type of data structure in which each element (node) contains two parts:

data: the actual value or information that the node holds.

next: a pointer (or reference) to the next node in the sequence.

in a singly linked list, each node points only to the next node in the list, and the last node points to `NULL`, indicating the end of the list. unlike arrays, linked lists do not store elements in contiguous memory locations; instead, each node points to the next node, forming a chain of nodes.

key characteristics of a singly linked list:

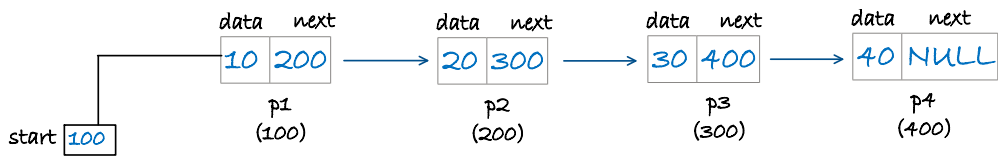
linear structure: the nodes are connected in a linear fashion, where each node is linked to the next one.

dynamic size: the size of the linked list is dynamic and can grow or shrink as needed, unlike arrays which have a fixed size.

sequential access: you can only traverse the list in one direction (from the head to the tail), making access to elements slower than arrays for certain operations (like searching).

no direct access: to access an element, you have to traverse the list from the head until you reach the desired node.

traverse in one direction.



concept: doubly linked list

unlike a singly linked list, where each node points only to the next node, in a doubly linked list, each node has two pointers: one pointing to the next node and one pointing to the previous node. This allows traversal in both directions- forward and backward.

structure of a doubly linked list node

each node in a doubly linked list typically contains:

data: the value that the node stores.

next: a pointer to the next node in the list.

prev: a pointer to the previous node in the list.

advantages of a doubly linked list:

bidirectional traversal: you can traverse the list in both directions (forward and backward), which can be useful in certain operations like undo functionality or backtracking.

easier deletion: deletion of a node is easier compared to a singly linked list because we have access to the previous node via the Prev pointer, making it possible to directly adjust pointers without needing to traverse the list to find the previous node.

disadvantages:

extra memory: each node requires extra memory to store the prev pointer, increasing the memory usage.

complexity: operations like insertion, deletion, and traversal can become more complex due to the need to manage two pointers (previous and next).

operations on a doubly linked list:

common operations that can be performed on a doubly linked list include:

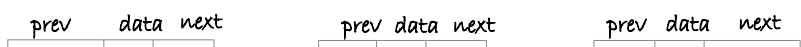
insertion: adding a node at the beginning, end, or middle of the list.

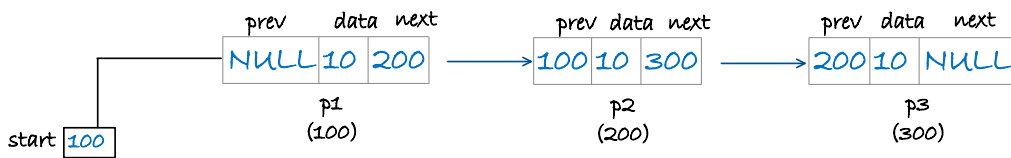
deletion: removing a node from the beginning, end, or middle.

traversal: visiting each node to perform operations (such as printing the data).

reversal: reversing the order of the nodes in the list.

search: finding a node with a specific value.



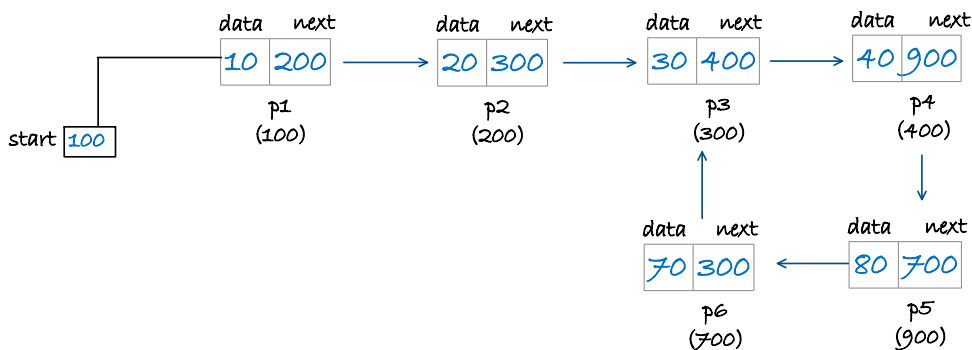


concept: circular linked list

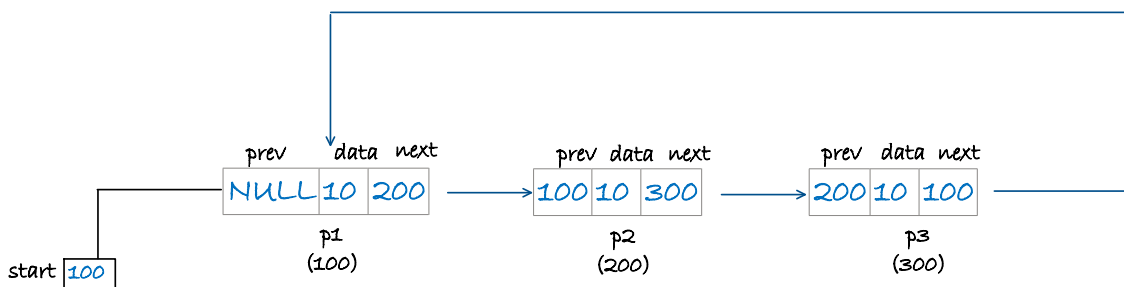
a circular linked list is a variation of a regular linked list where the last node points back to the first node instead of having a NULL reference. this creates a loop, allowing for continuous traversal of the list. circular linked lists can be either singly or doubly circular, where the last node's next points to the first node in a singly circular list, or both the last node's next and the first node's prev point to each other in a doubly circular list.

types of circular linked lists:

singly circular linked list: in this type, each node has a next pointer that points to the next node. the last node's next pointer points back to the first node, creating a circle.



doubly circular linked list: this type has two pointers in each node (next and prev), and both the first and last nodes point to each other, forming a circle.



advantages of circular linked lists:

you can easily traverse the entire list starting from any node.

useful for applications like round-robin scheduling, continuous traversal, and buffer management.

stack and queues

topic: two views of data structures

(i) abstract view

the abstract view defines the conceptual structure, features, and operations of the data structure without considering implementation details or coding. it focuses on what the data structure does rather than how it does it.

key features of abstract view:

(a) no coding or implementation:

it describes the behavior and functionality, not the underlying implementation.

(b) operations:

describes operations the data structure supports (e.g., insert, delete, search, traverse).

example: in a queue, the operations are enqueue (add an item) and dequeue (remove an item).

(c) features:

describes key properties such as ordering, circularity, indexing, or dynamic resizing.

example: a stack follows the last-in-first-out (LIFO) principle.

(d) real-world analogy:

explains the data structure in simple terms, using real-world analogies for better understanding.

example:

- a circular linked list consists of nodes connected in a circular manner, where the last node points back to the first.

- operations:

insert a node at the beginning.

delete a node from any position.

traverse nodes in a loop.

(ii) concrete view

the concrete view focuses on the implementation details of the data structure, including how its operations are executed in terms of programming and memory management.

key features of concrete view:

(a) coding and programming:

implementation involves choosing an appropriate programming language and writing algorithms.

focuses on data structure design, algorithms, and performance optimization.

(b) memory management:

explains how data is stored in memory (e.g., pointers, arrays).

example: linked lists use dynamic memory allocation.

(c) implementation-specific operations:

operations are detailed with algorithmic steps.

example: in a doubly linked list, adding a node involves adjusting pointers in both directions.

example:

- implemented using a structure or class with pointers to connect nodes.
- algorithm for insertion:
 - allocate memory for a new node.
 - update pointers to ensure circularity is maintained.

topic: stack

a stack is a linear data structure that follows the last in, first out (lifo) principle. this means the last element added to the stack is the first one to be removed.

key features of a stack

(i) lifo order:

the most recently added (or "top") item is the first to be removed.

example: think of a stack of plates in a cafeteria. the plate added last is the one removed first.

(ii) fixed or dynamic size:

stacks can be implemented with a fixed size (using arrays) or can grow dynamically (using linked lists).

(iii) restricted access:

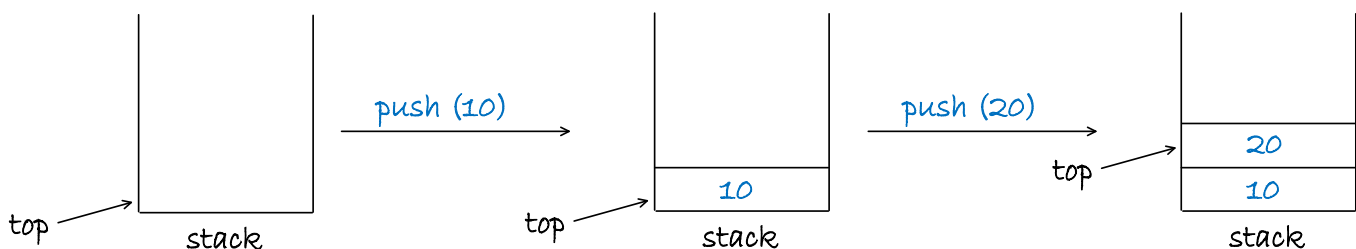
elements can only be added or removed from one end, known as the top of the stack.

basic operations

(i) push:

adds an element to the top of the stack.

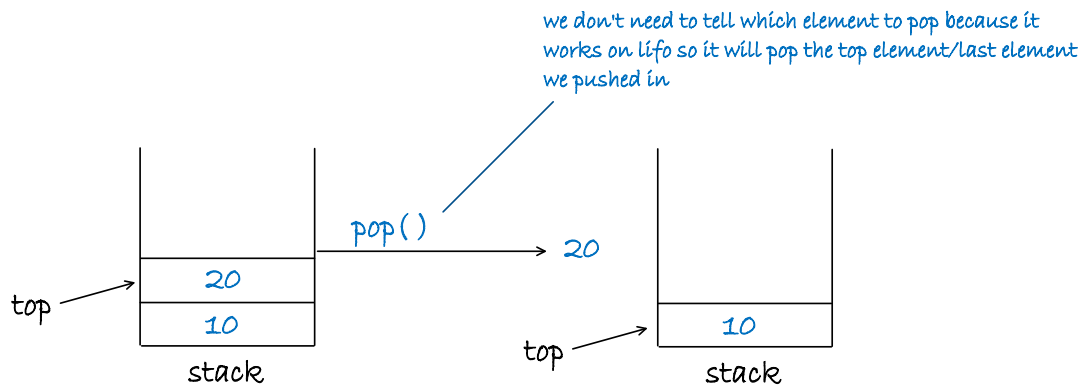
example: adding a new book to the top of a pile.



(ii) pop:

removes the top element of the stack.

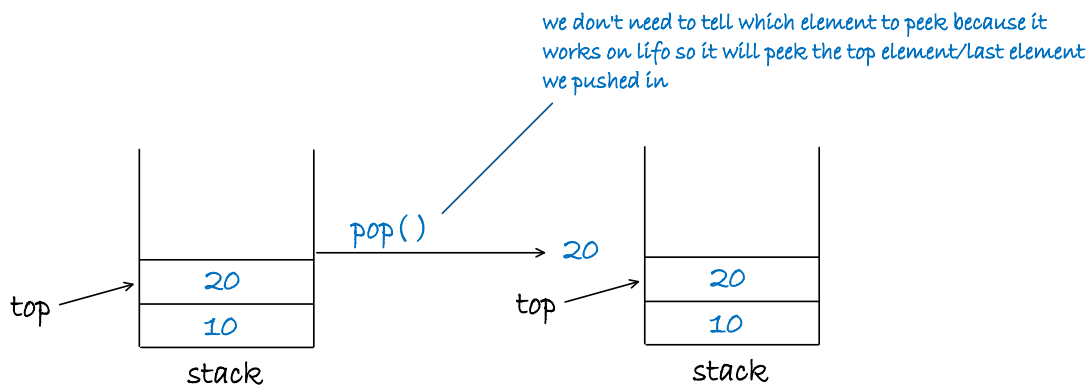
example: taking the topmost book from the pile.



(iii) `peek/top`:

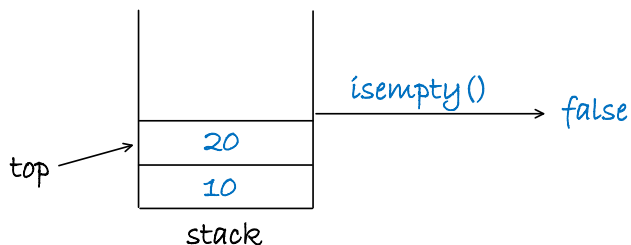
retrieves the top element without removing it.

example: looking at the top plate in a stack without taking it.



(iv) `isempty`:

checks whether the stack is empty.



(v) `isfull` (if implemented with a fixed size):

checks whether the stack is full.

not typically part of the ADT because it assumes implementation detail (fixed size stack). in an abstract sense, a stack is often conceptualized as having an unlimited size.

real-world examples

(a) undo operation in text editors:

each action is pushed onto a stack, and undoing pops the last action.

(b) browser back button:

the pages visited are stored in a stack. pressing back pops the current page.

(c) recursive function calls:

function calls are managed using the call stack, where the last called function must complete first.

advantages of stacks

- simple to implement.
- efficient for operations that require lifo behavior.
- useful in many applications like parsing, evaluating expressions, and backtracking algorithms.

disadvantages of stacks

- limited access: only the top element can be accessed directly.
- fixed size stacks (using arrays) can lead to overflow if not enough space is allocated.

topic: applications of stack

(i) expression evaluation and conversion

- infix to postfix/prefix conversion:

stacks are used to convert infix expressions (e.g., $a + b$) to postfix (e.g., $ab+$) or prefix ($+ab$).

- evaluation of postfix/prefix expressions:

stacks help evaluate postfix/prefix expressions efficiently.

(ii) function call management

- call stack:

used by programming languages to manage function calls.

stores local variables, return addresses, and more for each function call.

functions called most recently are completed first.

(iii) undo/redo operations

- text editors:

undo operations use a stack to track the sequence of changes.

redo uses another stack to reverse undo operations.

(iv) backtracking algorithms

examples:

maze solving: store paths taken; backtrack when a dead end is reached.

sudoku solving: store partially solved states to explore alternate paths.

(v) parentheses matching

- used to check if parentheses/brackets are balanced in expressions.

example: $\{ [()] \}$ is valid, while $\{ ([]) \}$ is not.

(vi) browser history

the "back" button in web browsers uses a stack to store visited pages.
the current page is pushed, and going back pops the last page visited.

(vii) parsing

- compilers and interpreters:
 - stacks are used to parse and evaluate syntax trees.
 - they help in matching braces, tags, or parentheses.

(viii) recursive problems

recursion internally uses the call stack to track function calls.

examples:

solving problems like the fibonacci sequence, tower of hanoi, or factorial computation.

(ix) stack-based algorithms

- depth-first search (dfs):
 - uses stacks to explore nodes in graph traversal.
- topological sorting:
 - implemented using a stack for directed acyclic graphs.

(x) operating systems

- runtime memory management:
 - stacks store temporary data such as function arguments, return values, and local variables.

(xi) string reversal

reversing a string can be achieved by pushing each character onto a stack and then popping them in reverse order.

(xii) editing tools

- syntax highlighting:
 - stacks are used to match opening and closing brackets, quotes, or tags in code editors.

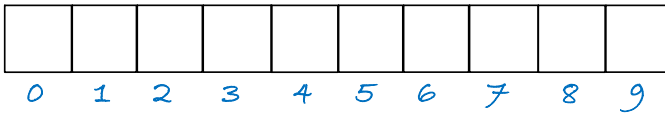
(xiii) virtual machines

- the java virtual machine (jvm) and postscript use stacks to execute instructions.

concept: array as a stack

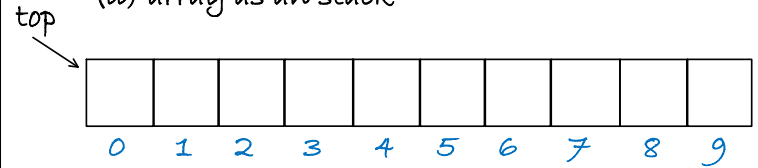
what is the difference in array and array as a stack?

(i) array



- a linear data structure where elements are stored at contiguous memory locations.
- any element can be accessed directly using its index.
- provides random access. you can access any element directly by its index.
example: `arr[3]` retrieves the 4th element.
- supports a wide range of operations like accessing, inserting, updating, and deleting at any index.
- typically has a fixed size defined at the time of its creation.
- resizing requires creating a new array if it's a static array.

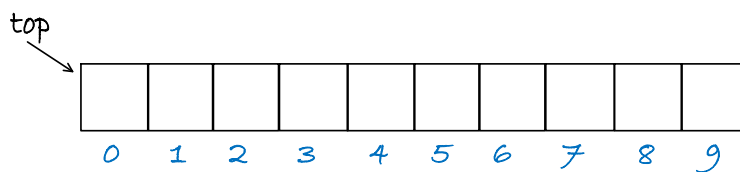
(ii) array as an stack



- an array used to implement a stack follows the last in, first out (lifo) principle.
- elements can only be added (pushed) or removed (popped) from one end (the top of the stack).
- access is restricted to the topmost element.
- you can only interact with the element on top of the stack.
- operations are limited to:
 - push: add an element to the top.
 - pop: remove the top element.
 - peek: view the top element without removing it.

so, the key difference is that in array i can access any index and add a new element but in array as an stack we can only add element to the top of the stack and retrieve the top/last added element.

(i) push



- operations to perform:

- push (10)
- push (20)
- push (30)
- push (40)
- push (50)
- push (60)
- push (70)
- push (80)
- push (90)

push (100)

(i) the top is initially empty pointing to nothing which means $\text{top} = -1$

(ii) the logic to fill the stack

(a) first we need to move the top to the stack

$\text{top} = \text{top} + 1$; (moves top to index 0)

(b) now we can push the element inside

$\text{stack}[\text{top}] = x$;

we can repeat this logic until stack is not full.

so, we have to create the logic for checking the condition also if stack is full or not.

(iii) condition to check if stack is full or not

if ($\text{top} == \text{size} - 1$) { $\text{size} - 1$: last index and if $\text{top} == \text{size}$ that means the stack is full)

pf ("overflow");

}return;

}

so code will be:

if ($\text{top} == \text{size} - 1$) {

pf ("overflow");

}return;

$\text{top} = \text{top} + 1$;

$\text{stack}[\text{top}] = x$;

}

```
// Function to push an element onto the stack
void push(int x) {
    if (top == SIZE - 1) { // Check for stack overflow
        printf("stack overflow\n");
        return;
    }
    top++;                // Increment the top pointer
    stack[top] = x;       // Insert the element at the new top position
    printf("%d pushed onto stack\n", x);
}
```

Explanation of the push function:

1. Check for stack overflow:

- if ($\text{top} == \text{SIZE} - 1$):
 - The `top` pointer is used to keep track of the index of the last element in the stack

Explanation of the push function:

1. Check for stack overflow:

- `if (top == SIZE - 1) :`
 - The `top` pointer is used to keep track of the index of the last element in the stack.
 - Since array indices start from 0, the last valid index in the stack is `SIZE - 1` (e.g., if `SIZE` is 10, the last index will be 9).
 - If `top` equals `SIZE - 1`, that means the stack is full and no more elements can be added. Thus, the condition checks for a stack overflow.

2. Stack overflow:

- If the condition `top == SIZE - 1` is true, the function prints `"stack overflow"` and returns. This means the operation stops here and does not proceed further to insert a new element.

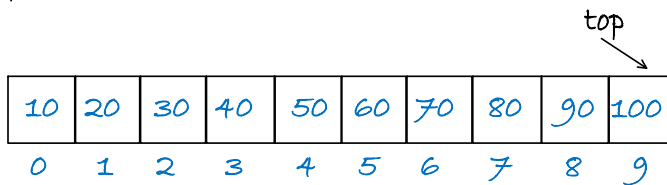
3. Increment the `top` pointer:

- `top++` : If there is no overflow, this increments the `top` pointer to point to the next available position in the stack. Initially, `top` is set to `-1`, so after the first push, it becomes `0`, pointing to the first index of the stack array.

4. Insert the element:

- `stack[top] = x` : The element `x` is inserted into the stack at the position pointed to by `top`.

(ii) pop



- operations to perform:

pop (10)
pop (20)
pop (30)
pop (40)
pop (50)
pop (60)
pop (70)
pop (80)
pop (90)
pop (100)

(i) as we know stack works on LIFO so the top is currently pointing to the last element added and we want to pop the elements

(ii) to pop the element think element as a coloured water, we need something to store coloured water so we need a bucket to store that coloured water and return.

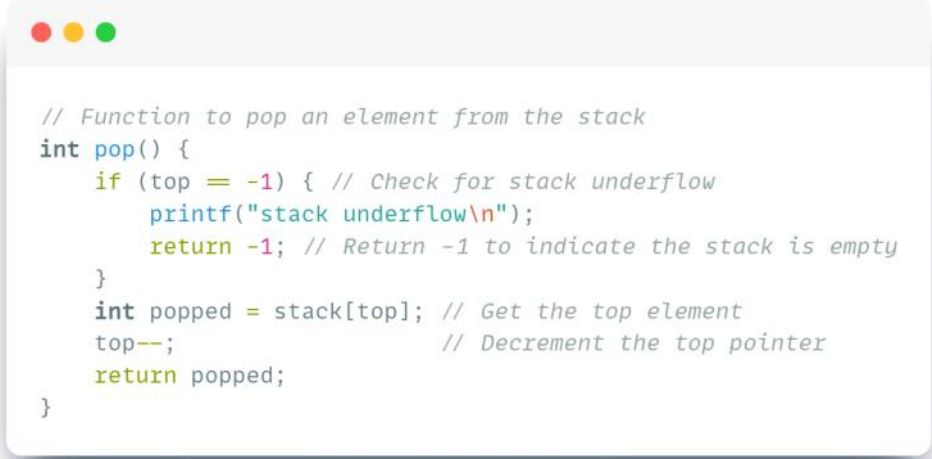
means to store the element we need a integer variable, lets name it temp.

```
int pop(){
int temp;
temp=stack[top]; (fill the temp with the top of the stack element)
top = top-1; (move the top to previous element)
return temp; (return the element we wanted to pop)
}
```

we need to repeat this code until there are no elements in the stack, so how we can ensure that the code stops once stack is empty.

(iii) to ensure that code stops when stack is empty

```
int pop (){
if (top == -1) { (if top is pointing to nothing)
return INT_MIN; (returns the value that is not in stack)
}
temp=stack[top];
top--;
return temp;
}
```



```
// Function to pop an element from the stack
int pop() {
    if (top == -1) { // Check for stack underflow
        printf("stack underflow\n");
        return -1; // Return -1 to indicate the stack is empty
    }
    int popped = stack[top]; // Get the top element
    top--; // Decrement the top pointer
    return popped;
}
```

Explanation of the pop function:

1. Check for stack underflow:

- `if (top == -1) :`
 - The `top` pointer keeps track of the index of the topmost element in the stack. Initially, `top` is set to `-1`, indicating that the stack is empty.
 - If `top` is `-1`, it means that the stack is empty, and attempting to pop an element from an empty stack will cause a **stack underflow**.
 - The condition checks for stack underflow, and if it is true, it prints `"stack underflow"` and returns `-1` to indicate that there are no elements to pop.

2. Get the top element:

- `int popped = stack[top]; :`
 - If the stack is not empty, this line retrieves the topmost element from the stack and stores it in the `popped` variable. The `stack[top]` represents the element at the index `top`.

3. Decrement the top pointer:

- `top--;`
 - After retrieving the top element, the `top` pointer is decremented by 1. This effectively moves the top pointer to the next element in the stack (the new top). If the stack becomes empty (i.e., `top` becomes `-1`), it indicates that the stack is empty.

4. Return the popped element:

- `return popped;`
 - The function returns the value of the popped element, which is stored in the `popped` variable.

```
#include <stdio.h>

#define SIZE 10 // Define the size of the stack

// Declare the stack array and the top pointer
int stack[SIZE];
int top = -1; // Initialize top to -1 to represent an empty stack

// Function to push an element onto the stack
void push(int x) {
    if (top == SIZE - 1) { // Check for stack overflow
        printf("stack overflow\n");
        return;
    }
    top++; // Increment the top pointer
    stack[top] = x; // Insert the element at the new top position
    printf("%d pushed onto stack\n", x);
}

// Function to pop an element from the stack
int pop() {
    if (top == -1) { // Check for stack underflow
        printf("stack underflow\n");
        return -1; // Return -1 to indicate the stack is empty
    }
    int popped = stack[top]; // Get the top element
    top--; // Decrement the top pointer
    return popped;
}
```

```

        top--;          // Decrement the top pointer
        return popped;
    }

    // Function to display the contents of the stack
    void display() {
        if (top == -1) { // Check if the stack is empty
            printf("stack is empty\n");
            return;
        }
        printf("stack contents: ");
        for (int i = 0; i <= top; i++) { // Print elements from bottom to top
            printf("%d ", stack[i]);
        }
        printf("\n");
    }

    // Main function to demonstrate stack operations
    int main() {
        push(10);
        push(20);
        push(30);
        push(40);
        push(50);
        push(60);
        push(70);
        push(80);
        push(90);
        push(100); // This fills the stack

        // Attempting to push when the stack is full
        push(110); // Should print "stack overflow"

        display(); // Display the stack contents

        // Popping elements from the stack
        printf("popped: %d\n", pop());
        printf("popped: %d\n", pop());

        display(); // Display the updated stack contents

        return 0;
    }

```

problem with this code:

we have not generalised the code, what if we want to perform for 2 stack, this code is just for a single stack and that too a particular stack.

solution:

this can be done by introducing an array of stacks or by implementing separate push and pop functions for each stack, where each stack has its own top pointer.

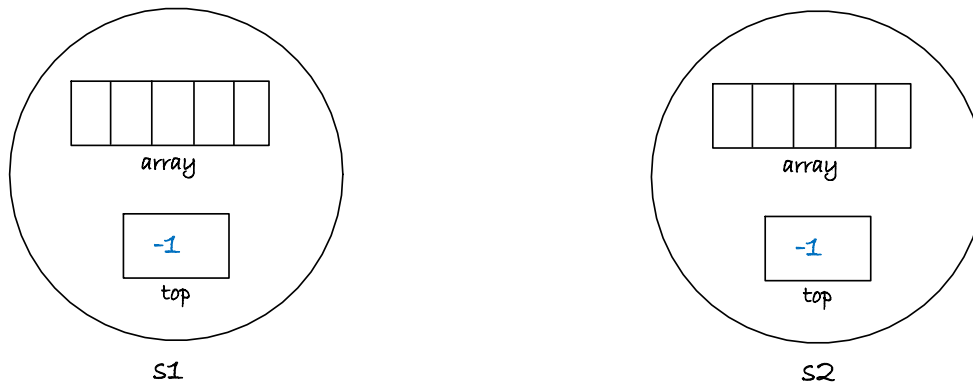
solution for handling multiple stacks:

we can represent multiple stacks using a single array. we need to have a separate top pointer for each stack. for this, we could either:

- (i) create a structure to manage multiple stacks.
- (ii) use an array of top pointers, where each stack will have its own top pointer.

```
#define SIZE 10 (define the size of the stack)
(define a structure for stack)
struct stack {
    int array[SIZE]; (array to hold stack elements)
    int top; (integer to track the top of the stack)
};
```

```
void main() {
    struct stack s1, s2;
    s.top = -1; (initialize the stack to be empty)
```



confusion:

- (a) push (): in which stack we want to insert?
- (b) pop (): from which stack we want to pop?

```
#define SIZE 10 (define the size of the stack)
(define a structure for the stack)
struct stack {
    int array[SIZE]; (array to hold stack elements)
    int top; (integer to track the top of the stack)
};
```

(function to push an element onto the stack (call by value))

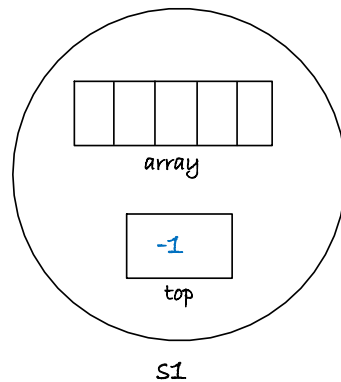
```
void push(struct stack s, int value) {
    if (s.top == SIZE - 1) {
        printf("Stack overflow\n");
        return;
    }
    s.top++; (increment the top pointer (this won't affect the original stack))
    s.array[s.top] = value; (insert the element at the new top position)
    printf("%d pushed onto stack\n", value);
}
```

```
int main() {
```



```
struct stack s1; (declare a stack variable 's1')
s1.top = -1; (initialize the stack to be empty (top = -1))
```

(test stack operations (pushing elements))
push(s1, 10);



the problem with this code (call by value)

when you pass a variable to a function in call by value, the function receives a copy of the variable. this means any changes made to the variable inside the function do not affect the original variable in the calling function. it operates on the copy, and once the function finishes, the copy is discarded.

In our code:

- in `main()`: we create `s1` and initialize it. this is the original stack.

```
int main() {
    struct stack s1; (declare a stack variable 's1')
```

- in `push(s1, 10)`:

(a) the copy of `s1` is passed to the `push()` function (the copy is called `s` inside the function).

```
void push(struct stack s, int value)
```

(b) the stack is modified inside the function (incrementing `top`, adding 10 to array, etc.).

```
s.top++; (increment the top pointer (this won't affect the original stack))
```

```
s.array[s.top] = value; (insert the element at the new top position)
```

(c) but these changes only affect the copy `s` inside the function.

(d) once the function finishes, the copy `s` is discarded, and `s1` remains unchanged in the `main()` function.

- how to reflect changes? (call by reference)

to reflect the changes in the original stack, we need to pass the stack by reference (i.e., by using a pointer). this way, the function will work on the original stack and any changes made to it will be reflected back to the original variable.

passing by reference (using pointers):

- pass the address of structure in which we want to make changes, to store the address of upcoming structure we need a pointer.

```
#define SIZE 10 (define the size of the stack)
```

```
(define a structure for the stack)
```

```
struct stack {  
    int array[SIZE]; (array to hold stack elements)  
    int top;          (integer to track the top of the stack)  
};
```

```
(function to push an element onto the stack (call by reference))
```

```
void push(struct stack *s, int value) {  
    if (s.top == SIZE - 1) { (there is no variable such as top in this function, use s.top)  
        printf("Stack overflow\n");  
        return;  
    }  
    s->top = s->top + 1; (increment the top pointer (this affect the original stack))  
    s->array[s->top] = value; (insert the element at the new top position)  
    printf("%d pushed onto stack\n", value);  
}
```

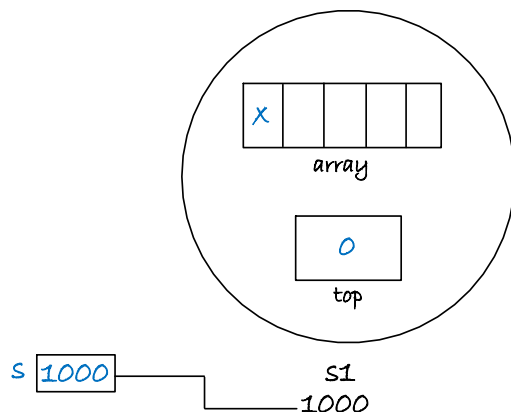
```
int main() {  
    struct stack s1; (declare a stack variable 's1')  
    s1.top = -1; (initialize the stack to be empty (top = -1))
```

```
(test stack operations (pushing elements))
```

```
push(&s1, 10); (pass stack by reference)
```

key changes:

(i) pass by reference: the function push() now takes a pointer to the stack (struct stack *s)



```

#include <stdio.h>

#define SIZE 10 // Define the size of the stack

// Define a structure for the stack
struct stack {
    int array[SIZE]; // Array to hold stack elements
    int top;         // Integer to track the top of the stack
};

// Function to push an element onto the stack (pass by reference)
void push(struct stack *s, int value) {
    if (s->top == SIZE - 1) {
        printf("Stack overflow\n");
        return;
    }
    s->top++; // Increment the top pointer
    s->array[s->top] = value; // Insert the element at the new top position
    printf("%d pushed onto stack\n", value);
}

int main() {
    struct stack s1; // Declare a stack variable 's1'
    s1.top = -1; // Initialize the stack to be empty (top = -1)

    // Test stack operations (pushing elements)
    push(&s1, 10); // Pass stack by reference
    return 0;
}

```

Code Explanation:

1. `#define SIZE 10 :`
 - This defines the size of the stack as 10. This means the stack will hold up to 10 elements.
2. `struct stack :`
 - This structure represents the stack and has two members:
 - `int array[SIZE]` : An array to hold the stack elements.
 - `int top` : A variable that keeps track of the top of the stack. It starts with `-1` to indicate that the stack is initially empty.
3. `void push(struct stack *s, int value) :`
 - This function adds an element (`value`) to the stack. It works by receiving a pointer to the stack (`struct stack *s`), which allows it to modify the original stack in `main()`.
 - The function first checks if the stack is full by comparing `s->top` with `SIZE - 1`. If the stack is full, it prints "Stack overflow" and exits.
 - If the stack isn't full, it increments the `top` pointer (i.e., `s->top++`) and adds the value to the `array` at the new top position (`s->array[s->top] = value`).
4. `main function:`
 - The `main` function declares a stack (`struct stack s1`), initializes it with `s1.top = -1` to represent an empty stack, and then calls the `push` function to add the element `10` to the stack.
5. `push(&s1, 10); :`
 - The stack `s1` is passed by reference to the `push` function using the address of `s1` (`&s1`), so any changes made in the `push` function will reflect in the original `s1` stack.

concept: stack permutation

stack permutation problem

in the stack permutation problem, you're given two sequences of integers:

- the in-order sequence
- the output sequence.

the task is to determine whether it is possible to obtain the output sequence by performing stack operations (push and pop) on the in-order sequence.

operations allowed:

push: push elements from the input sequence to the stack.

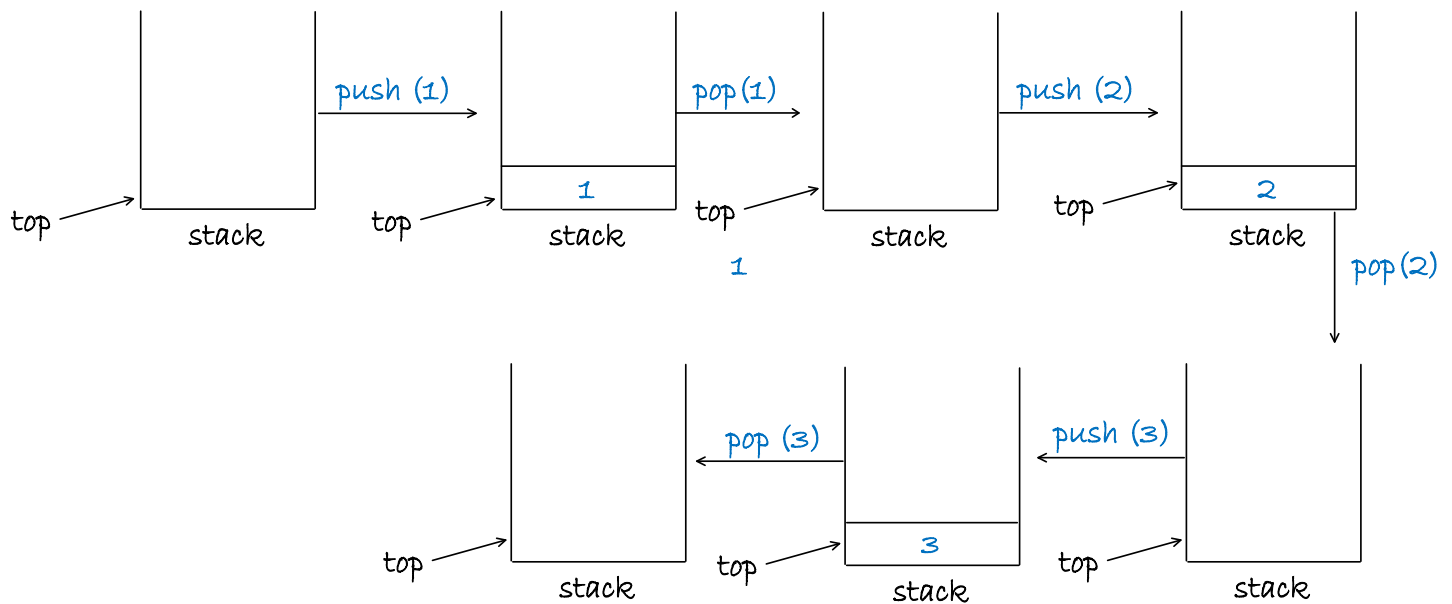
pop: pop elements from the stack to produce the output sequence.

conditions for stack permutation:

- the order of the output sequence must be maintained as you perform the operations.
- you cannot pop elements from the stack unless they are on the top, and you can only pop them in the order in which they appear.

example:

(i) insertion order: 1,2,3 and pop order: 1,2,3, is it possible?



push (1)
 pop (1)
 push (2)
 pop (2)
 push (3)
 pop (3)
 valid stack permutation.

(ii) pop order: 1,3,2, is it possible?

push (1)
 pop (1) : 1
 push (2)
 push (3)
 pop (3) : 3
 pop (2) : 2
 valid stack permutation.

(iii) pop order: 2,1,3, is it possible?

push (1)
 push (2)
 pop (2) : 2
 pop (1) : 1
 push (3)
 pop (3) : 3
 valid stack permutation.

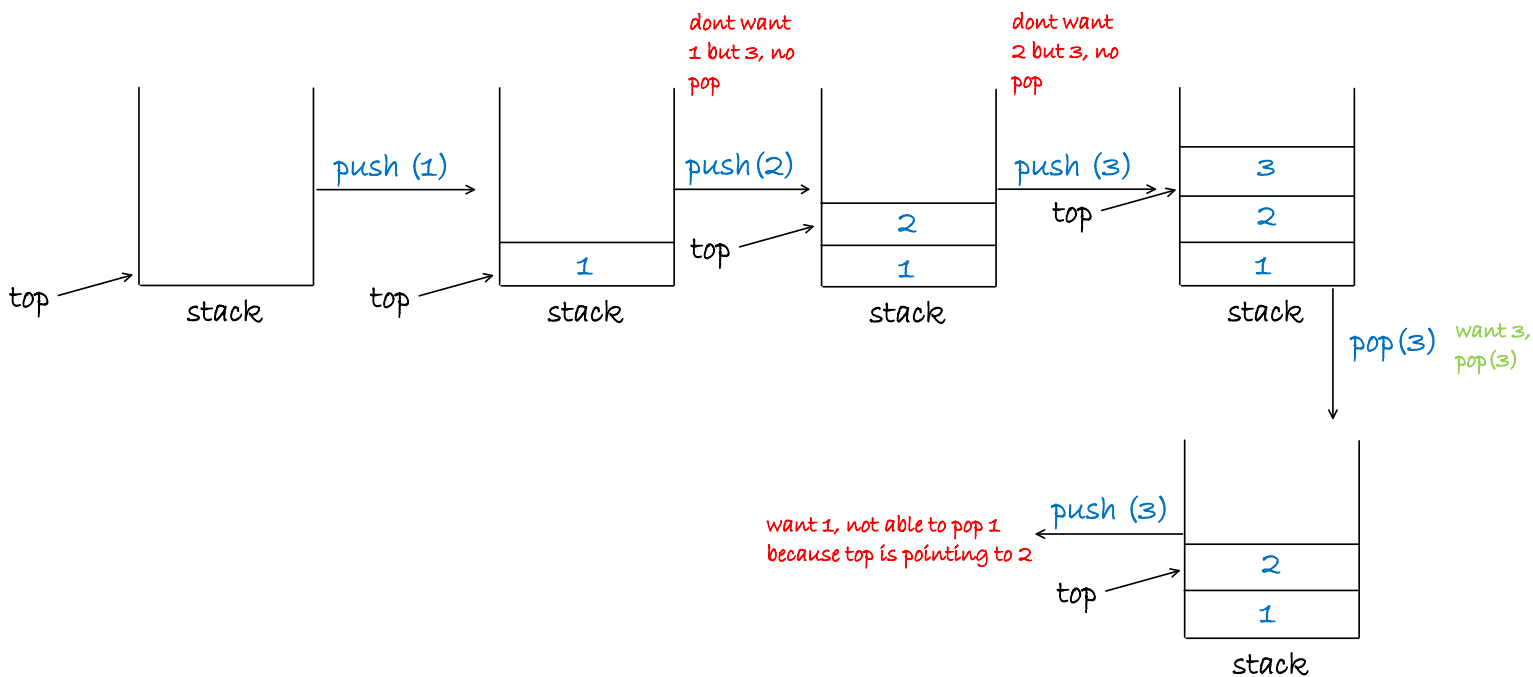
(iv) pop order: 2,3,1, is it possible?

push (1)
 push (2)

pop (2) : 2
 push (3)
 pop (3) : 3
 pop (1) : 1
 valid stack permutation.

(v) pop order: 3,1,2, is it possible?

push (1)
 push (2)
 push (3)
 pop (3) : 3
 pop (2) : 2
 pop (1) : 1
 not a valid stack permutation.



point to remember:

- as we have previously defined in definition that the insertion order is given and we should insert element according to insertion order only
- we cannot pop any element we want, we can only pop the element at top.

(vi) pop order: 3,2,1, is it possible?

push (1)
 push (2)
 push (3)
 pop (3) : 3
 pop (2) : 2
 pop (1) : 1
 valid stack permutation.

out of 6 total permutations, 5 are valid stack permutation.

$$\text{total valid stack permutations} = \frac{2\eta_{C_n}}{n+1}$$

concept: infix, postfix, prefix

in computer science and mathematics, infix, postfix, and prefix are different ways of writing expressions, especially arithmetic or logical expressions, involving operators and operands. these notations dictate how operators and operands are arranged and interpreted.

(i) infix notation (most common)

in infix notation, the operator is placed between the operands.

for example:

expression: $a + b$

meaning: add a and b.

(ii) postfix notation (also known as reverse polish notation (rpn))

in postfix notation, the operator comes after the operands. like prefix notation, postfix does not need parentheses for grouping, as the order of operations is implicitly determined by the position of the operators.

expression: $a b +$

meaning: add a and b.

other examples:

$a b c * +$ (equivalent to $a + (b * c)$)

$a b + c *$ (equivalent to $(a + b) * c$)

- why postfix?

computers evaluate expressions from left to right, but they don't inherently "know" when to perform an operation, as operators have different levels of precedence. in infix expressions, operators need to be evaluated based on their precedence, and parentheses also affect the order of evaluation. this creates the need for multiple scans and checks to determine which operator should be evaluated first, which increases the complexity.

postfix notation eliminates this complexity because, in postfix expressions, the order of operations is already built into the expression. there is no need to consider operator precedence or parentheses during evaluation. as you scan the postfix expression from left to right, you can immediately perform the operation when an operator is encountered, using operands that were pushed to the stack earlier. this allows for a single, efficient scan, avoiding the need for additional checks for precedence or parentheses, thus reducing time complexity.

when a computer is given an infix expression, it typically follows a two-step process to evaluate it using a stack:

(i) convert infix to postfix (using a stack)

first, the infix expression is converted into its equivalent postfix expression. this conversion is done using a stack to handle operator precedence and parentheses.

(ii) evaluate postfix expression (using a stack)

after the infix expression is converted to postfix, the postfix expression can be easily evaluated using a stack in one pass.

steps involved:

- scan the postfix expression from left to right.
- push operands (numbers or variables) onto the stack.
- when an operator is encountered:
 - pop the required number of operands (usually two) from the stack.
 - perform the operation using those operands.
 - push the result of the operation back onto the stack.
- when the scan is complete, the result will be the only element left in the stack.

concept: conversion from infix to postfix

here is a precedence table to help you convert an infix expression to postfix:

Operator	Precedence	Associativity
$^$	4	Right to Left
* , $/$, $\%$	3	Left to Right
$+$, $-$	2	Left to Right
$=$	1	Left to Right
$($, $)$	N/A	N/A

(i) without using stack

example: without stack

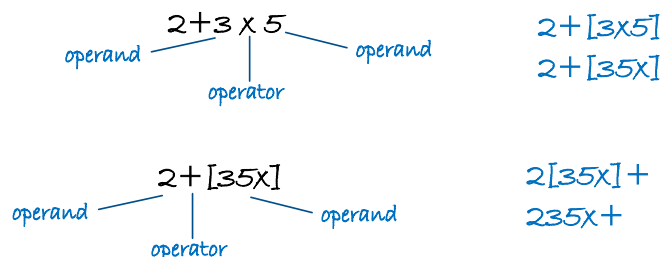
(i) infix: $2+3 \times 5$

postfix:

$2+[3 \times 5]$

$2+[35 \times]$

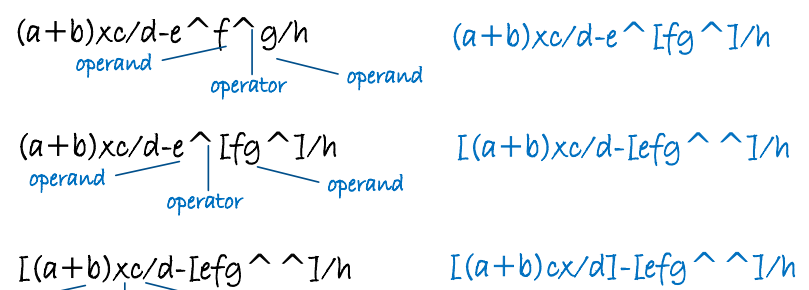
$235 \times +$



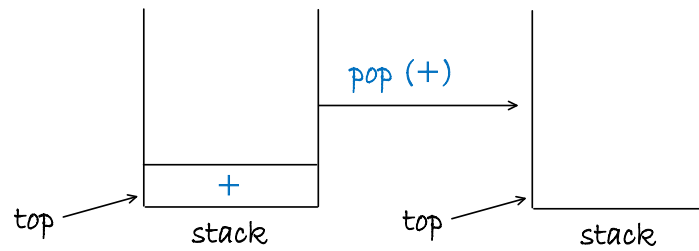
(ii) infix: $(a+b)xc/d-e^f \wedge g/h$

postfix:

$ab+cx/d/efg^{\wedge}h/-$



(d) whenever we reach the end, pop everything from the stack and add to the output one by one



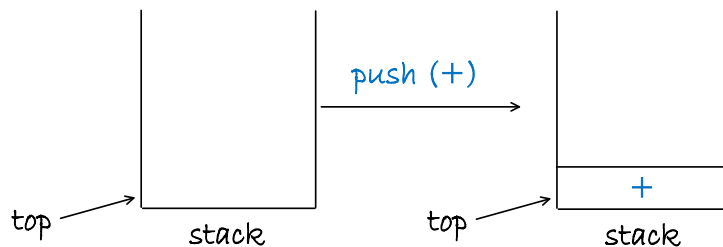
final postfix: $23+$

(ii) infix: $a+b-c$

(a) directly write the output when the operand comes.

postfix: a

(b) if an scanned element is an operator and the stack is empty, push the operator.



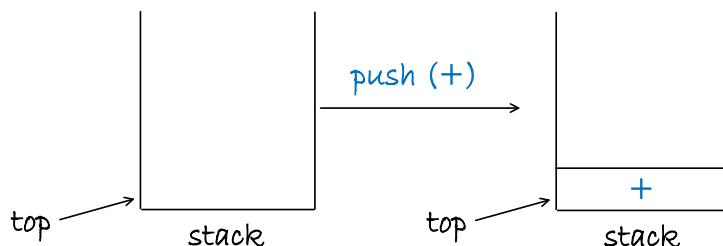
(c) directly write the output when the operand comes

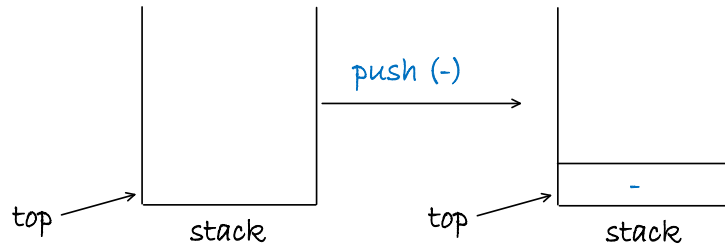
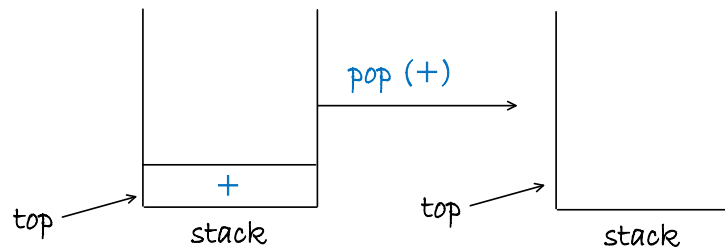
postfix: ab

(d) if an scanned element is an operator, push the operator.

but the priority is same of $+$ and $-$

so if on the top of stack, the operator stored is of same priority as new operator then old operator will be popped out and new operator will be pushed.



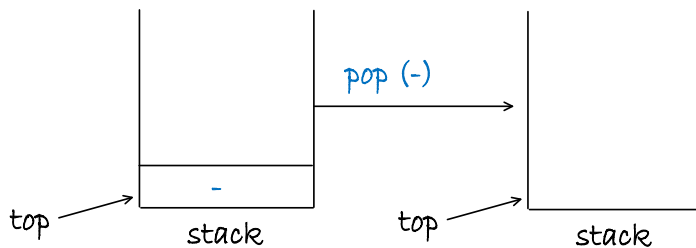


postfix: $ab+$

(e) directly write the output when the operand comes

postfix: $ab+c$

(f) whenever we reach the end, pop everything from the stack and add to the output one by one



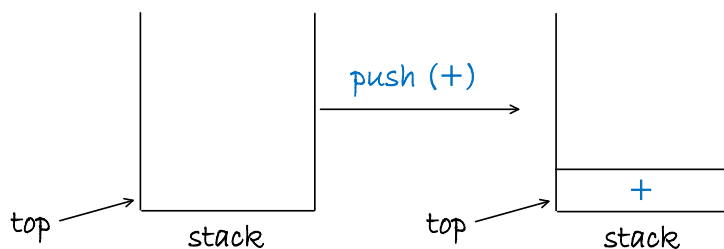
postfix: $ab+c-$

(iii) infix: $a+bx/c/d$

(a) directly write the output when the operand comes.

postfix: a

(b) if an scanned element is an operator and the stack is empty, push the operator.



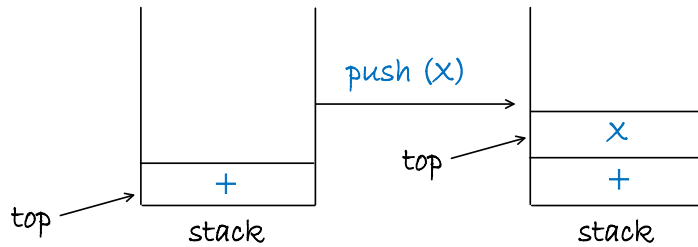
(c) directly write the output when the operand comes

postfix: *ab*

(d) if an scanned element is an operator, push the operator.

- the priority is different of + and x

so if on the top of stack, the operator stored is of less priority as comparable to new operator then new operator will be pushed without popping out the lower priority operator.



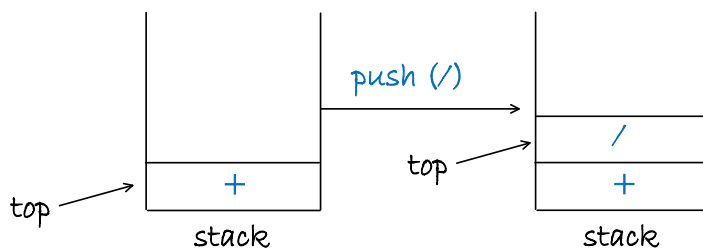
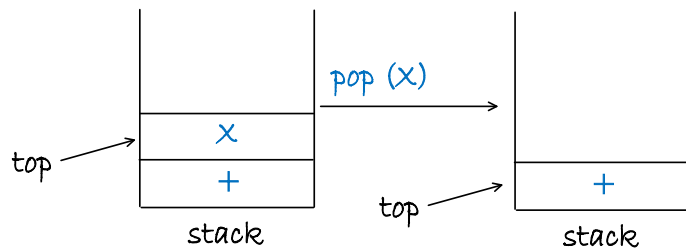
(e) directly write the output when the operand comes.

postfix: *abc*

(f) if an scanned element is an operator, push the operator.

- the priority is same of / and x

so if on the top of stack, the operator stored is of same priority as new operator then old operator will be popped out and new operator will be pushed.

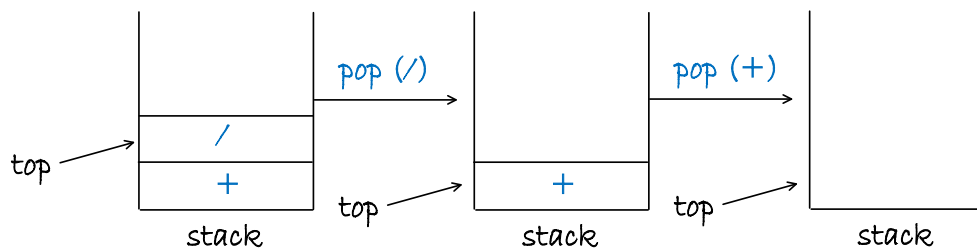


postfix: *abcx*

(g) directly write the output when the operand comes.

postfix: *abcxd*

(h) whenever we reach the end, pop everything from the stack and add to the output one by one



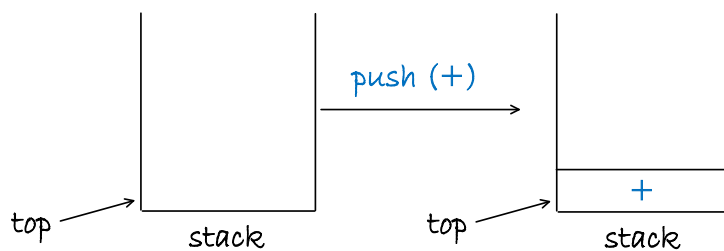
postfix: $abcxd/+$

(iv) infix: $2 + (3 \times 4 - 6 / 2)$

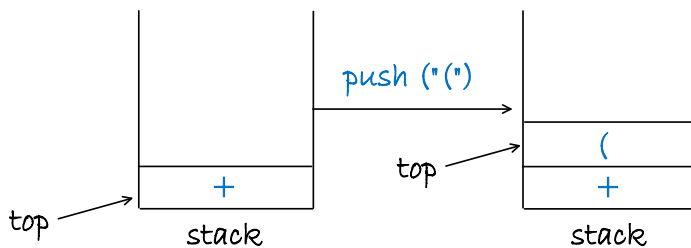
(a) directly write the output when the operand comes.

postfix: 2

(b) if an scanned element is an operator and the stack is empty, push the operator.



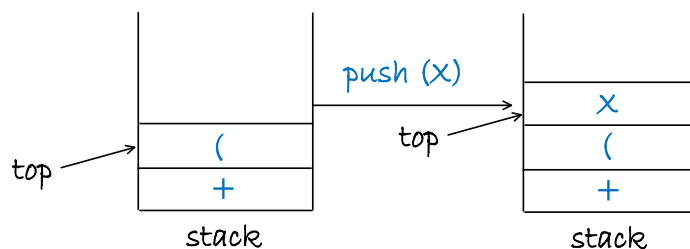
(c) left parenthesis so we will push parenthesis in stack because it is treated as new expression



(d) directly write the output when the operand comes.

postfix: 23

(e) if an scanned element is an operator, push the operator.



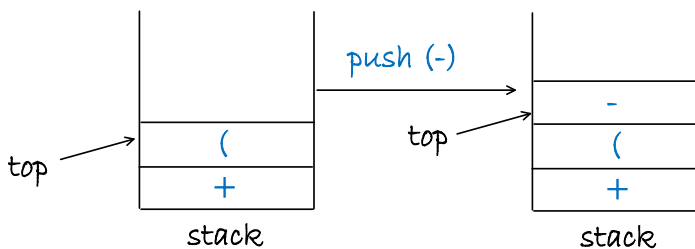
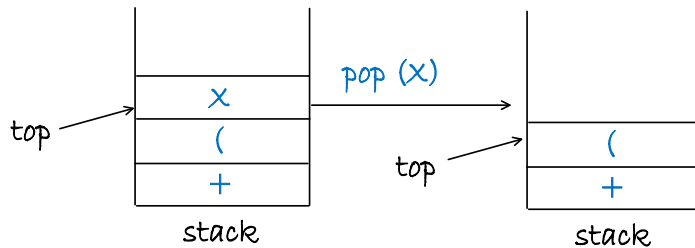
(e) directly write the output when the operand comes.

postfix: 234

(f) if an scanned element is an operator, push the operator.

- the priority is different of X and -

so if on the top of stack, the operator stored is of high priority as comparable to new operator then the old operator with more priority will be popped and new operator will be pushed.



postfix: 234X

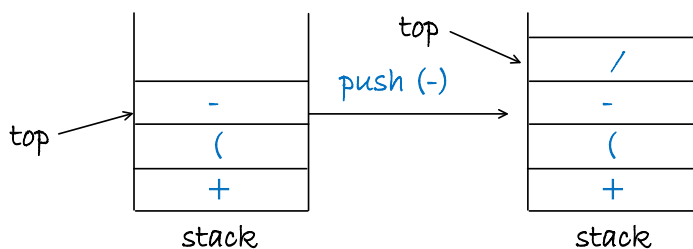
(g) directly write the output when the operand comes.

postfix: 234X6

(h) if an scanned element is an operator, push the operator.

- the priority is different of - and /

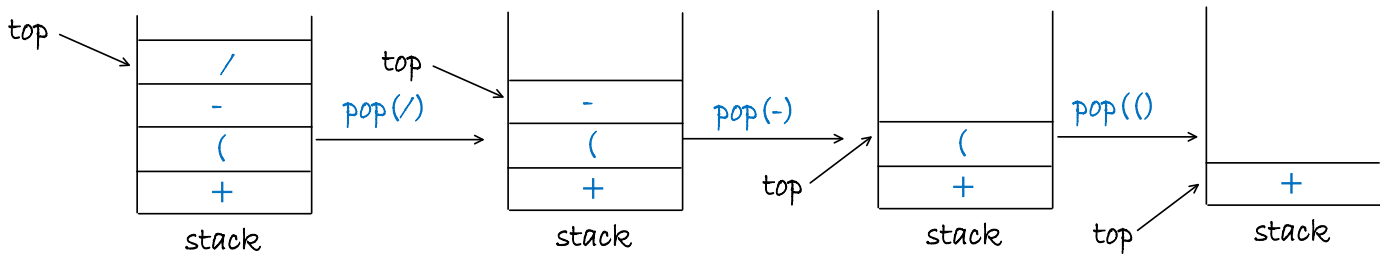
so if on the top of stack, the operator stored is of less priority as comparable to new operator then new operator will be pushed without popping out the lower priority operator.



(i) directly write the output when the operand comes.

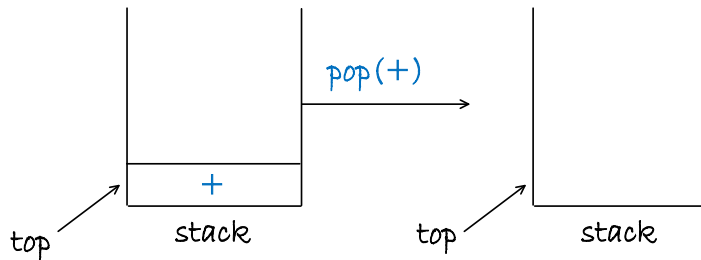
postfix: 234X62

(j) right parenthesis ')' encountered, pop and add operators to output until left parenthesis '(' is found



postfix: 234x62/-

(h) whenever we reach the end, pop everything from the stack and add to the output one by one



postfix: 234x62/-+

concept: behaviour of r to l associativity operator

usual behavior of same-priority operators (l-to-r associativity):

- for most operators (e.g., +, -, *, /), if a new operator with the same precedence enters the stack, the operator already in the stack is popped and added to the output before pushing the new one.
- this is because these operators are left-to-right (l-to-r) associative, meaning the first operator encountered is evaluated first.

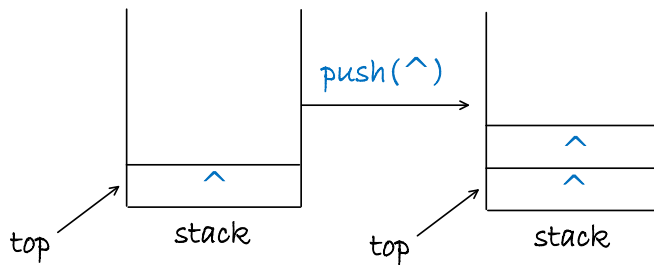
behavior of ^ (r-to-l associativity):

- when the ^ operator is already in the stack and a new ^ operator enters:
 - the old ^ is not popped.
 - instead, the new ^ is pushed onto the stack on top of the old one.
- this behavior respects right-to-left (r-to-l) associativity, where the most recent (rightmost) operator is evaluated first.

why the behavior?

this ensures the exponentiation operator is evaluated correctly according to its r-to-l associativity. if the old operator were popped, it would lead to incorrect evaluation like 2^3 first, then 4 .

example: right to left priority (^)



concept: parenthesis in stack

when converting infix to postfix using a stack, parentheses are treated specially to correctly manage operator precedence and grouping.

encountering a left parenthesis (()):

reason: a left parenthesis signifies the start of a sub-expression. operators within this sub-expression should only affect elements inside the parentheses, and they have higher precedence temporarily.

action: the left parenthesis is pushed onto the stack.

it acts as a marker for when to stop popping operators. when you later encounter a right parenthesis ()), the left parenthesis ensures you don't pop operators beyond the scope of the current sub-expression.

encountering a right parenthesis ())::

reason: a right parenthesis indicates the end of a sub-expression. at this point, you need to process all operators within the parentheses.

action:

- pop operators from the stack and add them to the output until a left parenthesis (()) is encountered.
- discard the left parenthesis after popping it (do not add it to the output).

(iii) prefix notation (also known as polish notation)

in prefix notation, the operator comes before the operands. it eliminates the need for parentheses for grouping, as the operator determines the order of evaluation. the operator precedes the operands.

for example:

expression: $+ a b$

meaning: add a and b .

other examples:

$+ a * b c$ (equivalent to $a + (b * c)$)

$* + a b c$ (equivalent to $(a + b) * c$)

conversion from infix to prefix:

to convert an infix expression to a prefix expression, follow these steps

(i) reverse the infix expression.

(ii) convert it to postfix notation.

(iii) reverse the postfix expression to get the prefix expression.

(iii) reverse the postfix expression to get the prefix expression.

concept: conversion from infix to prefix

Operator	Description	Associativity
()	Parentheses	None
^	Exponentiation	Right-to-Left
*, /	Multiplication, Division	Left-to-Right
+, -	Addition, Subtraction	Left-to-Right

(i) without using stack

example: without stack

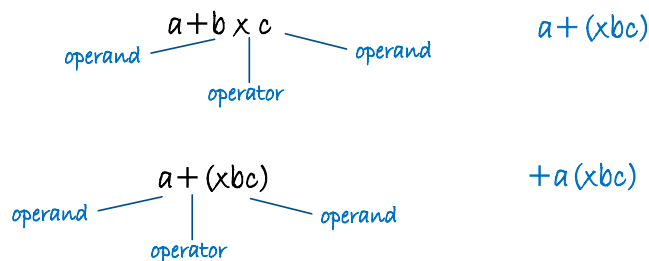
(i) infix: $a + bxc$

prefix:

$a + (xbc)$

$+a(xbc)$

$+axbc$



(ii) infix: $a + bxc - d/e^f^g$

postfix:

$a + bxc - d/e^f^g$

$a + bxc - d/e [^fg]$

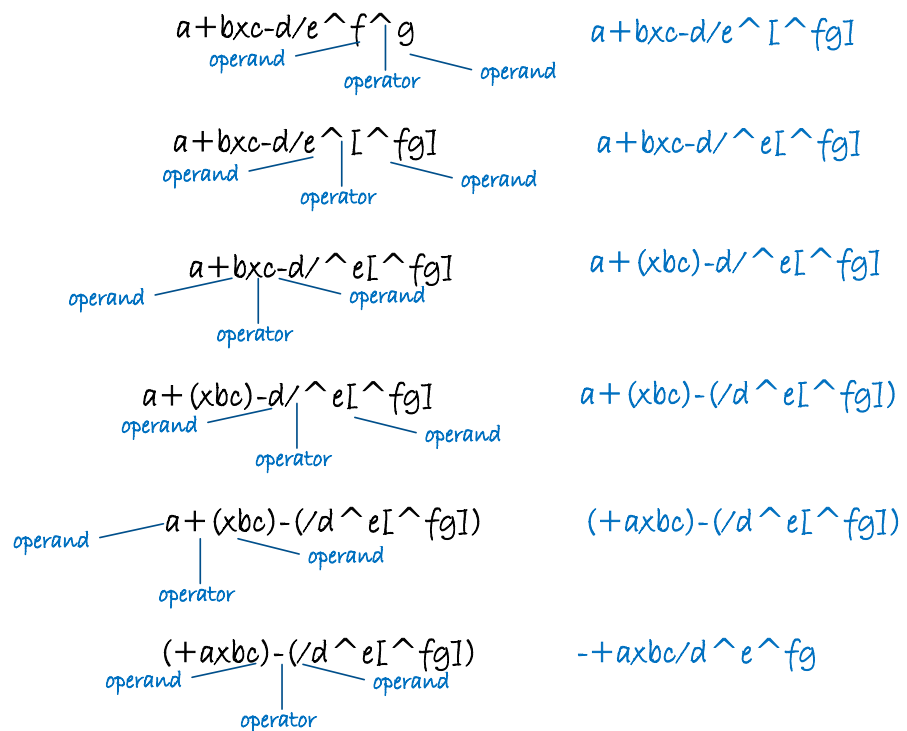
$a + bxc - d/ [^e [^fg]$

$a + (xbc) - d/ [^e [^fg]$

$a + (xbc) - (/d [^e [^fg])$

$(+axbc) - (/d [^e [^fg])$

$- +axbc/d [^e [^fg]$



(ii) with using stack

(i) reverse the infix notation

(ii) repeat the process same as postfix

- the same priority operator will not stay in the stack.
- if the lower priority operator comes, we will pop the higher priority operator same as we did in postfix

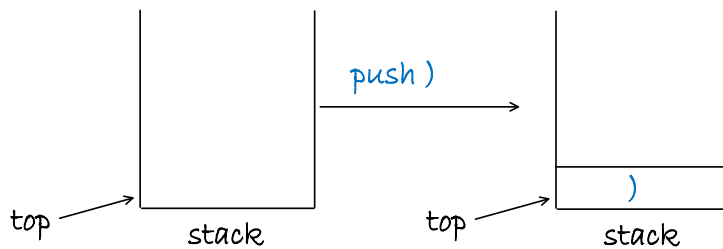
(i) infix: $a + (bxc^d^e - f)$

(a) reverse the infix

$a + (bxc^d^e - f)$

$)f - e^d^cxb(+a$

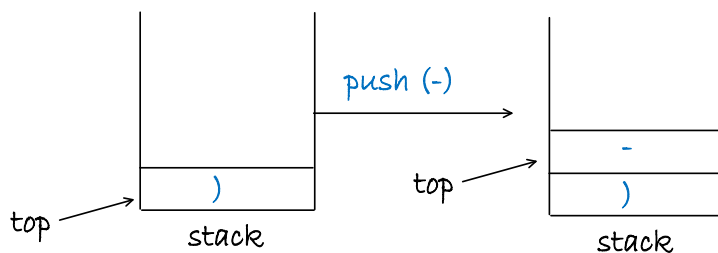
(b) push the left parenthesis operator, continue to repeat the same process but as soon as you encounter right parenthesis fetch the operators until left parenthesis and then delete.



(c) directly write the output when the operand comes

prefix: f

(d) if an scanned element is an operator, push the operator.

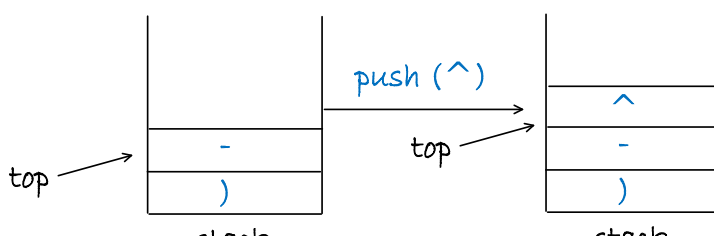


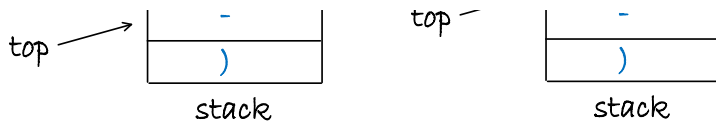
(e) directly write the output when the operand comes

prefix: fe

(f) if an scanned element is an operator, push the operator.

- the priority of $^$ is more than -
- push $^$





prefix: fe

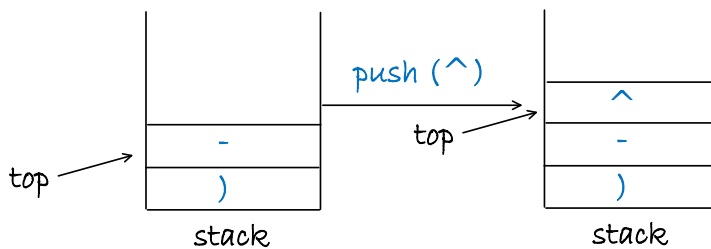
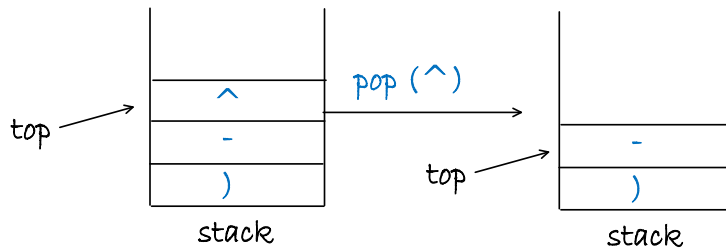
(g) directly write the output when the operand comes.

prefix: fed

(h) if an scanned element is an operator, push the operator.

- the priority is same of ^ and ^

but in this we will pop out the operator and push the new operator.



postfix: fed ^

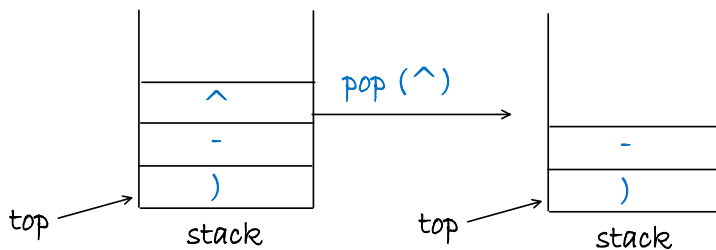
(i) directly write the output when the operand comes.

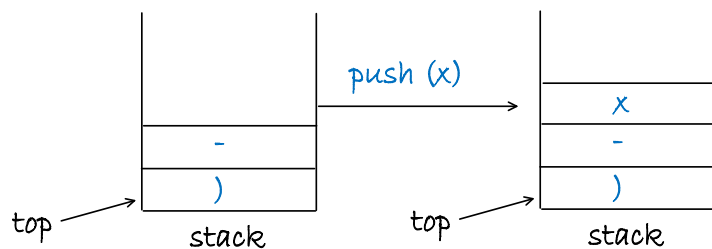
prefix: fed ^ c

(j) if an scanned element is an operator, push the operator.

- the priority is less of X as compared to ^

pop ^ and push X

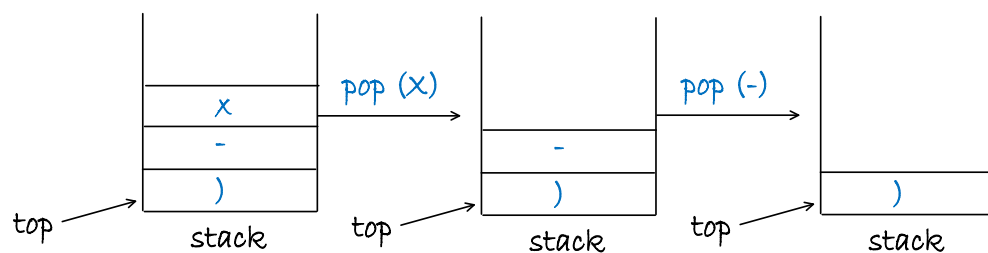




(k) directly write the output when the operand comes.

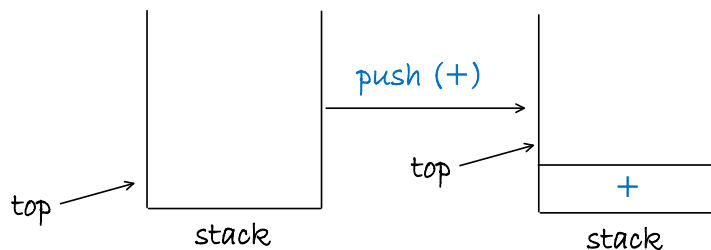
prefix: $fed^{\wedge}c^{\wedge}b$

(l) left parenthesis occurred so fetch all the operands.



prefix: $fed^{\wedge}c^{\wedge}bx-$

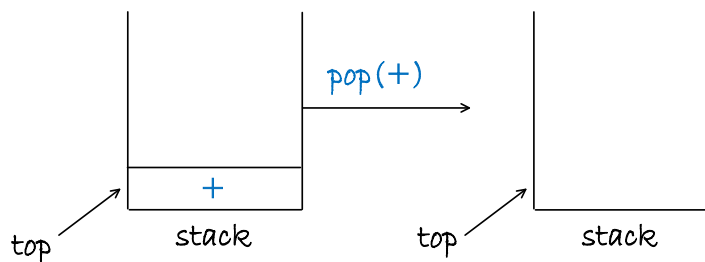
(m) if an scanned element is an operator, push the operator.



(n) directly write the output when the operand comes.

prefix: $fed^{\wedge}c^{\wedge}bx-a$

(o) whenever we reach the end, pop everything from the stack and add to the output one by one



prefix: $fed^{\wedge}c^{\wedge}bx-a+$

(p) reverse again

prefix: $fed^{\wedge}c^{\wedge}bx-a+$

prefix: $+a-xb^{\wedge}c^{\wedge}def$

concept: how to evaluate postfix and infix

(i) postfix

infix: $2+3 \times 4-6/2$

postfix: $234 \times + 62 / -$

$2+3 \times 4-62 /$

$2+(3 \times 4)-62 /$

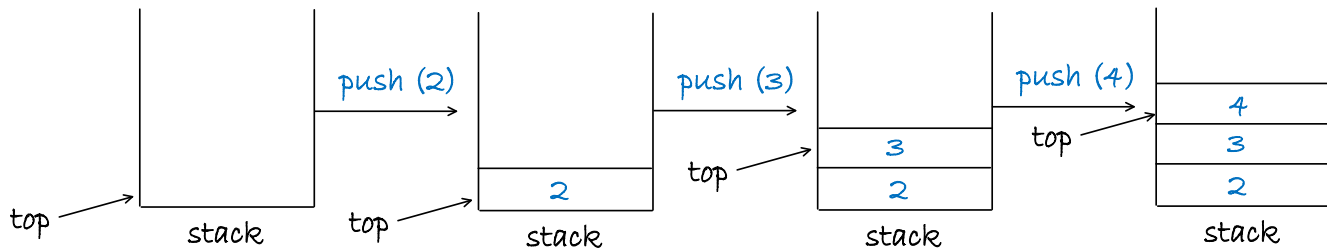
$234 \times + -62 /$

$234 \times + 62 / -$

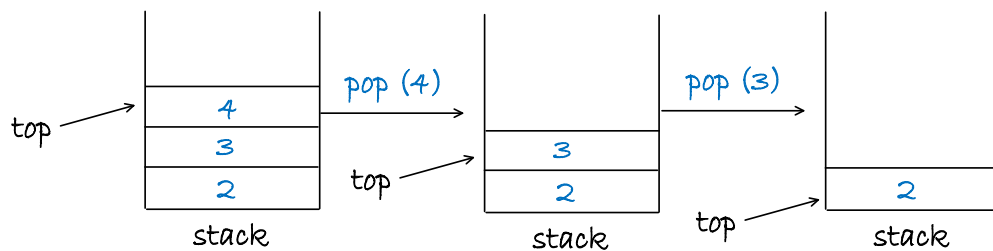
(a) using stack

in the evaluation we push operands in the stack and when we encounter the operator, we pop 2 elements out from the stack from top and evaluate them and push them back into the stack

(i) push 2, 3 and 4



(ii) encountered the operand so we will pop top elements and multiply them and push them back into the stack



pop order

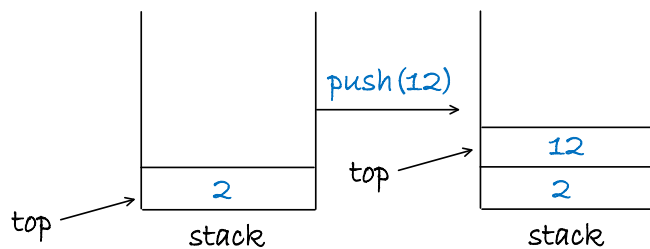
$\xrightarrow{\quad}$
4 3

calculation order

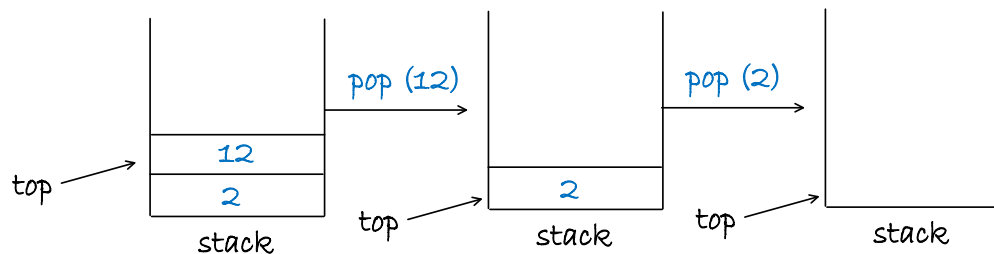
3 4

perform multiplication: $3 \times 4 = 12$

- push 12 back to stack.



(iii) encountered the operand so we will pop top elements and add them and push them back into the stack



pop order

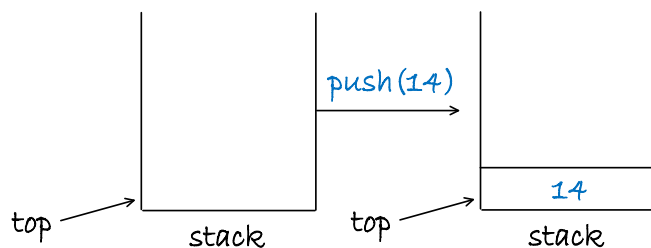
→
12 2

calculation order

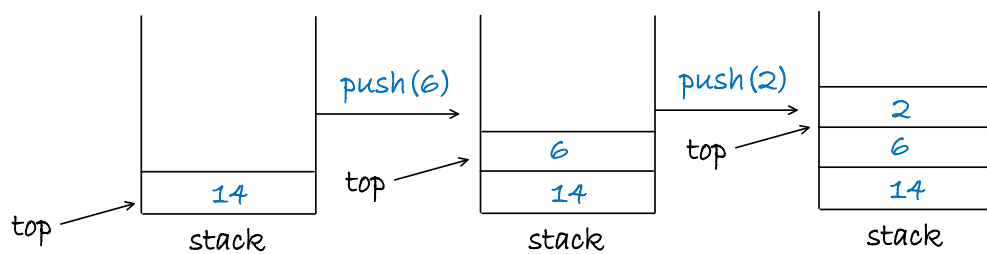
2 12

perform addition: $2 + 12 = 14$

- push 14 back to stack.

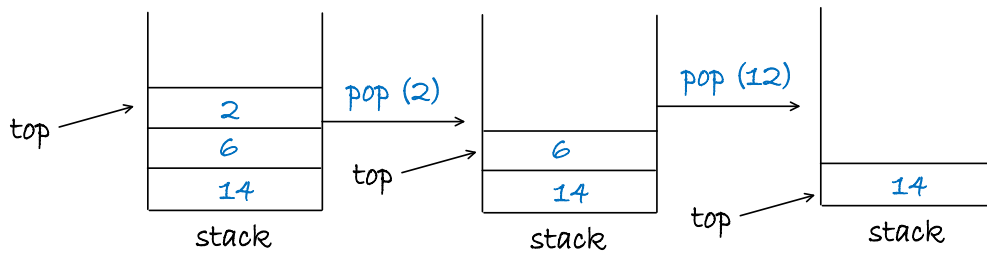


(iv) push 6 and 2



(v) encountered the operand so we will pop top elements and divide them and push them back into the

stack



pop order

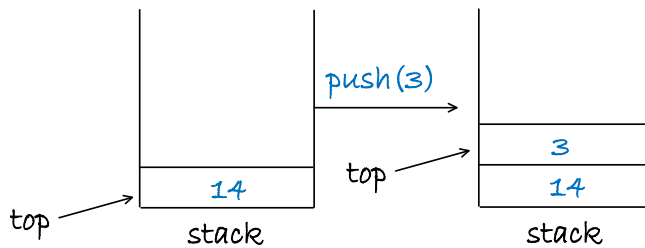
$\xrightarrow{\quad}$
2 6

calculation order

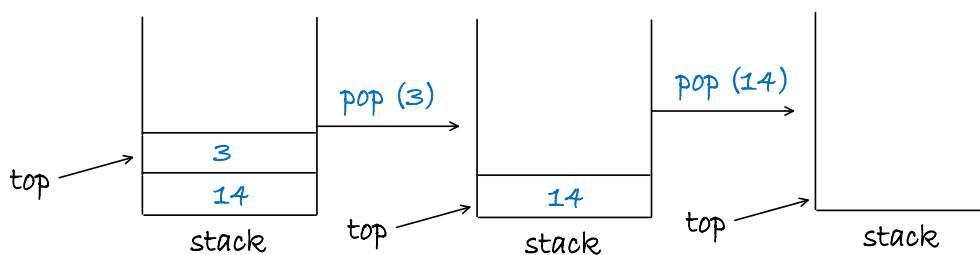
6 2

perform division : $6/2=3$

- push 3 back to stack.



(vi) encountered the operand so we will pop top elements and subtract them and push them back into the stack



pop order

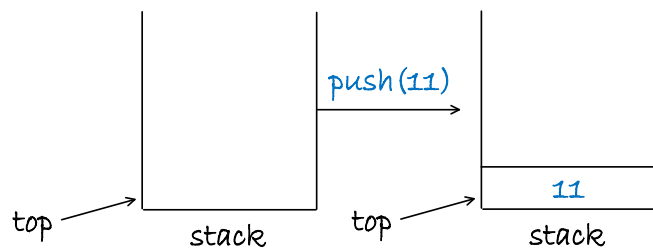
$\xrightarrow{\quad}$
3 14

calculation order

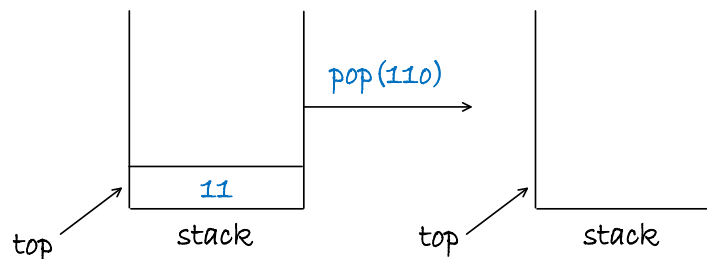
14 3

perform subtraction: $14-3=11$

- push 11 back to stack.



(vii) whenever we reach the end, pop everything from the stack and add to the output one by one



output: 11

(b) not using stack

infix: $2+3 \times 4-6/2$

postfix: $234 \times + 62 / -$

$2+3 \times 4-62 /$

$2+(3 \times 4)-62 /$

$234 \times + -62 /$

$234 \times + 62 / -$

postfix: $234 \times + 62 / -$

$2(3 \times 4) + 62 / -$

$212 + (62 /) -$

$(212 +) 3 -$

$143 -$

11

(ii) prefix

infix: $2+3 \times 5$

prefix: $+2 \times 35$

(a) using stack

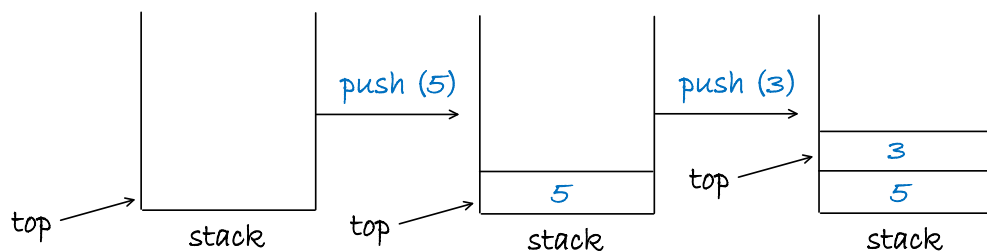
in the evaluation we push operands in the stack and when we encounter the operator, we pop 2 elements out from the stack from top and evaluate them and push them back into the stack

(i) reverse

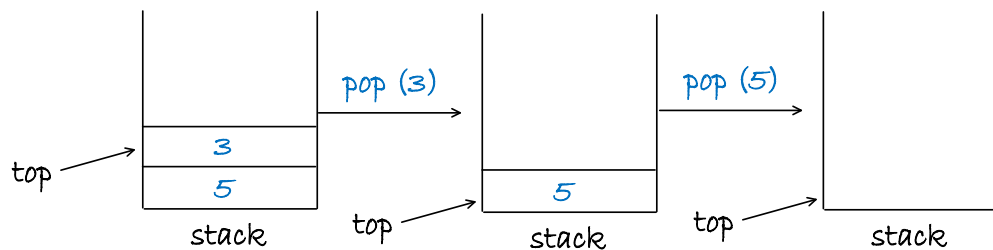
$+2 \times 35$

$53 \times 2 +$

(ii) push 5 and 3



(iii) encountered the operand so we will pop top elements and multiply them and push them back into the stack



pop order

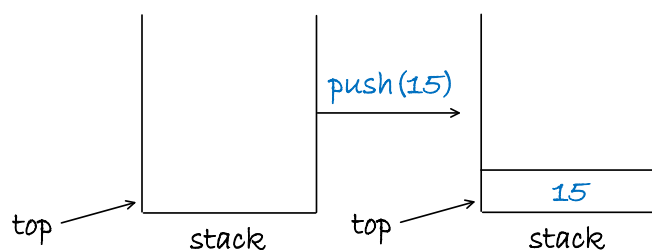
→
3 5

calculation order

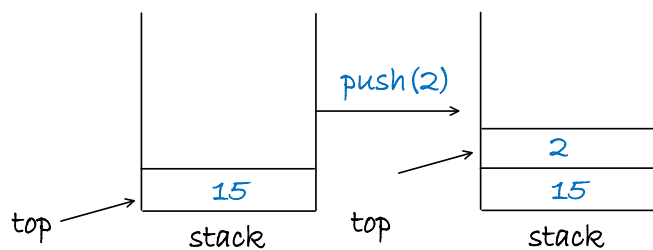
3 5

perform multiplication: $3 \times 5 = 15$

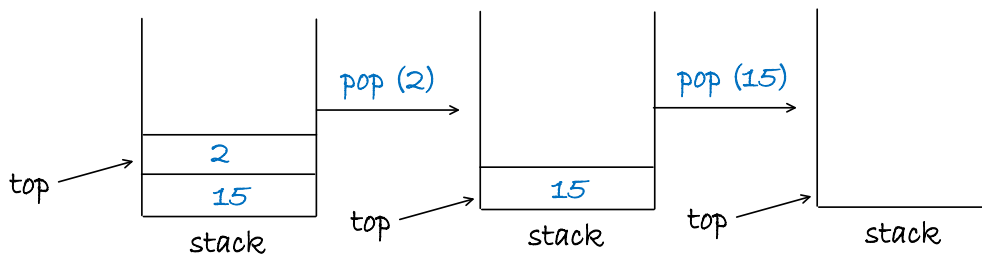
- push 15 back to stack.



(iv) push 2



(v) encountered the operand so we will pop top elements and add them and push them back into the stack



pop order

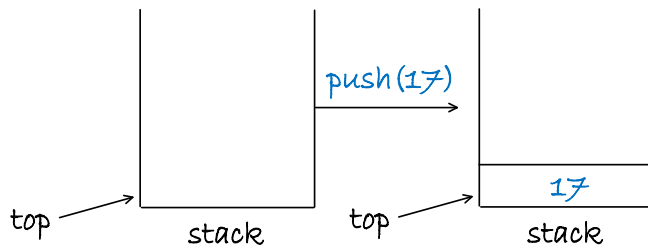
→
2 15

calculation order

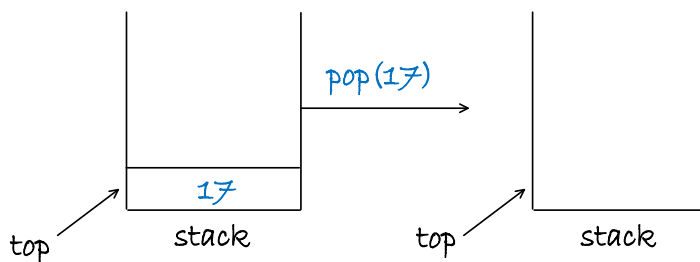
2 15

perform addition: $2 + 15 = 17$

- push 17 back to stack.



(vii) whenever we reach the end, pop everything from the stack and add to the output one by one



output: 17

concept: balanced parenthesis

to determine whether a parenthesis sequence like () () is balanced,
the following rule applies:

- for every right parenthesis), there should be a corresponding left parenthesis (that appears earlier in the sequence.

steps to check balance:

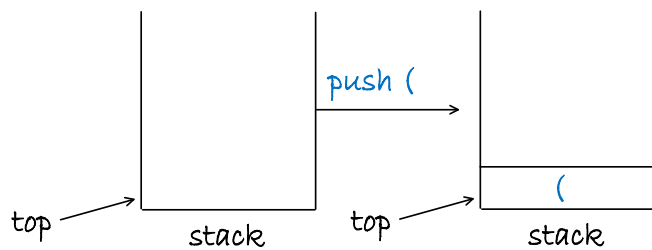
(i) use a stack:

- push each left parenthesis (onto a stack.

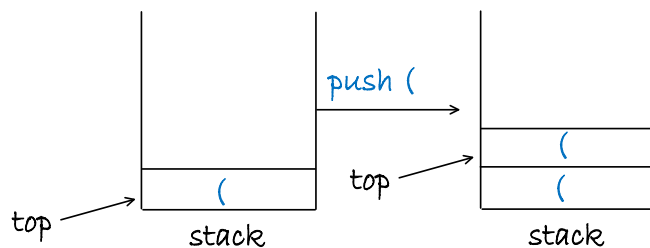
- when encountering a right parenthesis), check:
 - if the stack is empty: the sequence is unbalanced (no matching ()).
 - if the stack is not empty: pop the top element from the stack (it represents a matched ()).
- continue this process for the entire sequence.

example: (())

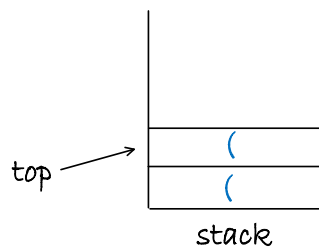
(i) push (



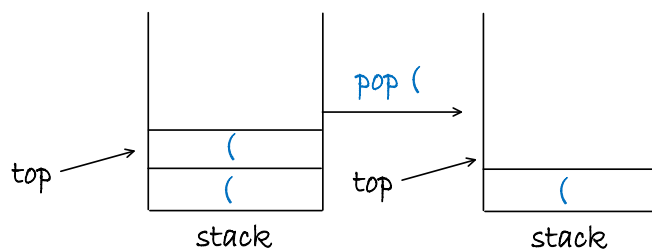
(ii) push (



(iii) push)

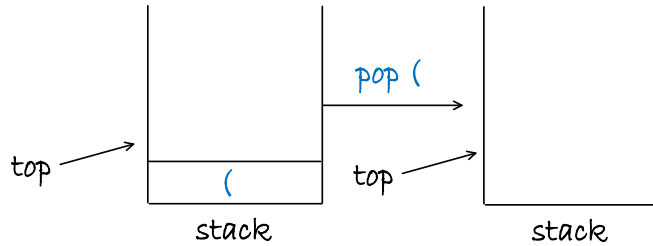


right parenthesis encountered, looks at the top of the stack, found its left parenthesis.



output: ()

(iv) push)

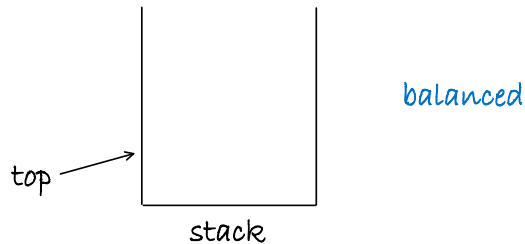


right parenthesis encountered, looks at the top of the stack, found its left parenthesis.

output: (())

(ii) final stack check:

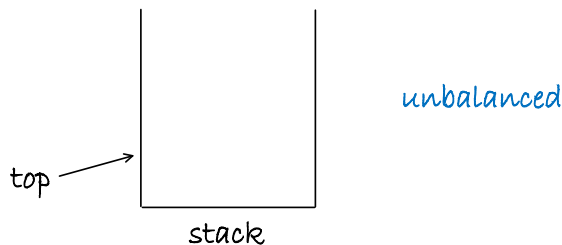
- if the stack is empty after processing the sequence, it is balanced.
- if the stack is not empty, it means there are unmatched (, so the sequence is unbalanced.



point to remember:

if a right parenthesis is encountered and stack is empty then do not perform further it is unbalanced

)()



q(i) which of the following is essential to convert an infix expression to postfix expression efficiently?

- (a) an operator stack
- (ii) an operand stack
- (iii) an operator and operand stack
- (iv) a tree

q(ii) the best data structure to check whether an arithmetic expression has balanced parenthesis is?

- (i) queue
- (ii) stack

- (iii) tree
- (iv) list

topic: queue

a queue is a linear data structure that follows the fifo principle, meaning first in, first out. the element inserted first will be removed first, similar to standing in a line or a queue.

characteristics of a queue:

- (i) enqueue: add an element to the rear (end) of the queue.
- (ii) dequeue: remove an element from the front of the queue.
- (iii) peek/front: view the front element without removing it.
- (iv) isempty: check if the queue is empty.

types of queues:

- (i) simple queue:

a simple queue is the most basic type of queue. it operates on the fifo principle, meaning "first in, first out." elements are inserted from the rear (end) of the queue and removed from the front (beginning) of the queue.

characteristics of a simple queue

- (i) fifo principle:
 - the first element inserted is the first one to be removed.
- (ii) linear structure:
 - elements are stored sequentially in a linear fashion.
- (iii) operations:
 - enqueue (insertion): add an element to the rear of the queue.
 - dequeue (deletion): remove an element from the front of the queue.
 - peek: view the element at the front without removing it.
- (iv) fixed size:
 - implemented using arrays or linked lists.
 - in array-based implementation, a simple queue may suffer from wasted memory if elements are dequeued repeatedly.

array implementation

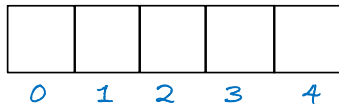
handling front and rear

- front: index of the element that can be deleted from queue
- rear: index of the most recently added element

so we need variables front and rear

```
#define size s (array of size s)
int queue[size]
int rear;
int front;
```

(i) initially the queue is empty

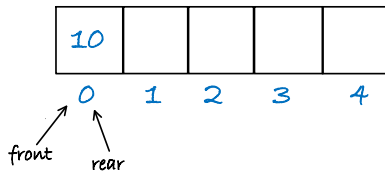


the front and rear both are pointing to nothing so,

rear = -1

front = -1

(ii) added the element

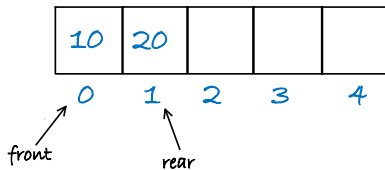


both are pointing to same index because this is the only element we have in the queue

rear = 0

front = 0

(iii) added the element

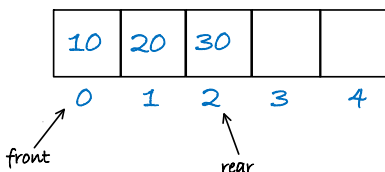


now rear is pointing to the index of most recently added element and front is still pointing to index 0

rear = 1

front = 0

(iv) added the element

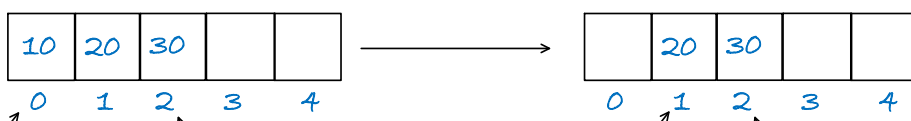


now rear is pointing to the index of most recently added element and front is still pointing to index 0

rear = 2

front = 0

(v) deleting the element 10



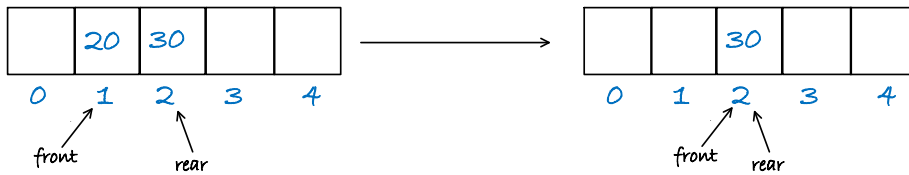


now rear is pointing to the index of most recently added element and as 10 is deleted so front moves to next element and is now pointing to index 1

rear = 2

front = 1

(vi) deleting the element 20



now rear is pointing to the index of most recently added element and as 10 is deleted so front moves to next element and is now pointing to index 2

rear = 2

front = 2

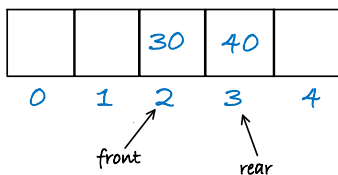
point to remember:

so, the cases when rear and front are empty is either when

(i) there is only one element in the queue.

(ii) queue is empty. (empty = -1)

(vii) added the element

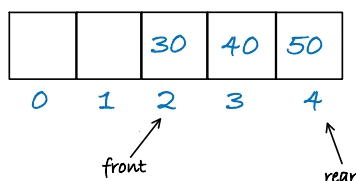


now rear is pointing to the index of most recently added element and front is still pointing to index 2

rear = 3

front = 2

(viii) added the element

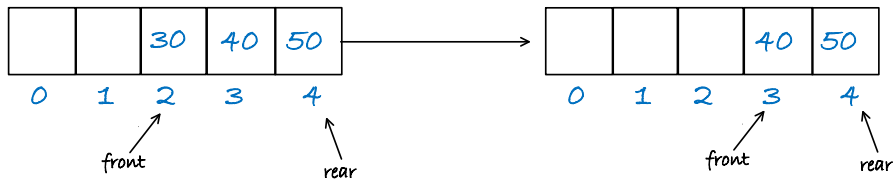


now rear is pointing to the index of most recently added element and front is still pointing to index 2

rear = 4

front = 2

(ix) deleting the element 30

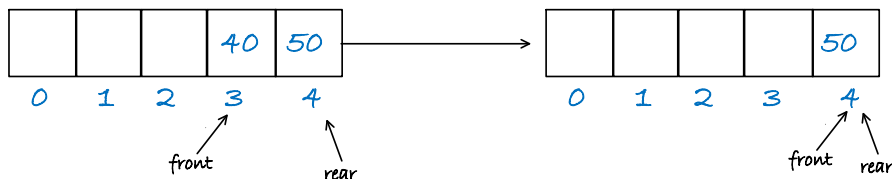


now rear is pointing to the index of most recently added element and as 10 is deleted so front moves to next element and is now pointing to index 3

rear = 4

front = 3

(x) deleting the element 40



now rear is pointing to the index of most recently added element and as 10 is deleted so front moves to next element and is now pointing to index 4

rear = 4

front = 4

(xi) deleting the element 50



now there is only one element in the queue and when we delete it the queue becomes empty so front and rear both become -1

rear = -1

front = -1

concept : need of circular queue

(ii) circular queue:

a circular queue is a linear data structure in which the last position connects back to the first position to form a circle. this eliminates the problem of unused space in a regular queue, especially in cases where elements are repeatedly dequeued and enqueued.

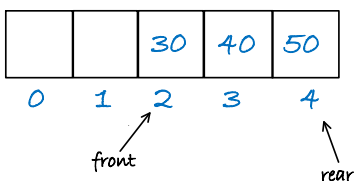
characteristics of a circular queue

- (i) fixed size: operates within a fixed memory size or array.
- (ii) circular movement: the rear pointer wraps around to the front when it reaches the end of the array.
- (iii) efficient memory usage: avoids the "wasted space" issue of a simple queue.
- (iv) two pointers:
 - front: points to the first element in the queue.
 - rear: points to the last element in the queue.

basic operations

- (i) enqueue (insertion):
 - add an element to the rear of the queue.
 - handle the circular increment of the rear pointer.
- (ii) dequeue (deletion):
 - remove an element from the front of the queue.
 - handle the circular increment of the front pointer.
- (iii) peek:
 - view the front element without removing it.
- (iv) is empty:
 - check if the queue is empty by comparing front and rear.
- (v) is full:
 - check if the queue is full by ensuring the rear wraps around and points just before the front.

(i) insert



rear is pointing to index 4 which is the last index in the queue so when you try to insert the element in the queue it will not allow because array is full as rear is pointing to the last index

but logically first three indexes are free that is why circular queue comes into the picture.

(i) insertion (enqueue)

- insertion happens at the rear of the queue.
- the rear pointer is moved forward after the new element is added.

function to enqueue an element into the queue (not the complete code)

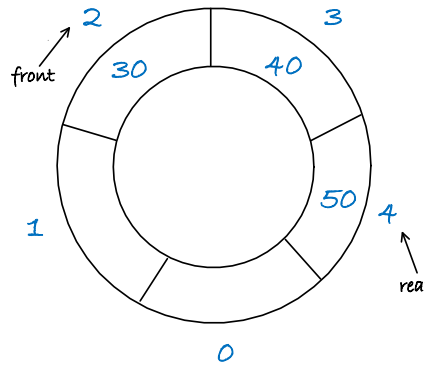
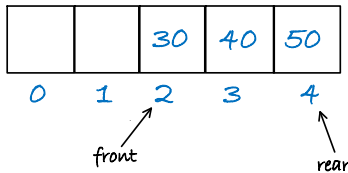
```
void enqueue(int x) {  
    if (rear == SIZE - 1) { (if rear is pointing to the last index, checks if the queue is full or not)  
        printf("queue overflow! cannot enqueue %d\n", x);  
        return;  
    }  
    else if (front == -1) { (if front is pointing to nothing/if the queue is empty, initialize front and rear)  
        front = 0;  
        rear = 0;  
    }  
    else {  
        rear = rear + 1;  
    }  
    arr[rear] = x;  
}
```

```

    rear = 0;
}
else { (when both conditions are false, means queue is not full but front pointing to some index means there is/are empty index)
    rear++; (move rear forward)
}
queue[rear] = x; (insert the element)
printf("enqueued: %d\n", x);
}

```

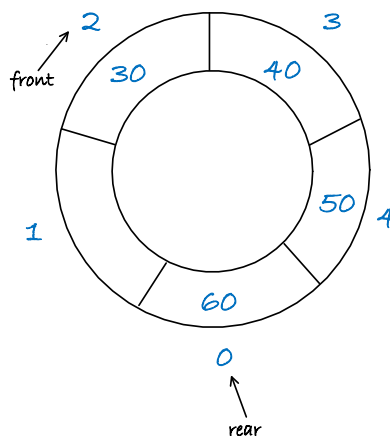
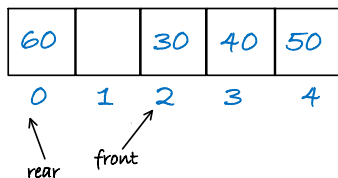
insert (60)



```

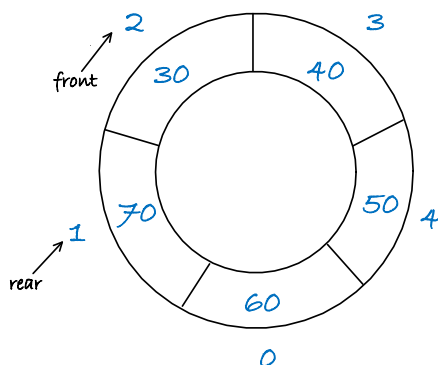
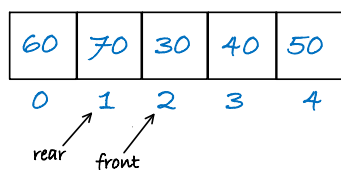
void enqueue(int x) {
    if (rear == SIZE - 1) { (no)
        printf("queue overflow! cannot enqueue %d\n", x);
        return;
    }
    else if (front == -1) { (no)
        front = 0;
        rear = 0;
    }
    else {
        rear++; moves to 0
    }
    queue[rear] = x; (queue[0]=60)
    printf("enqueued: %d\n", x);
}

```



insert (70)

```
void enqueue(int x) {  
    if (rear == SIZE - 1) { (no)  
        printf("queue overflow! cannot enqueue %d\n", x);  
        return;  
    }  
    else if (front == -1) { (no)  
        front = 0;  
        rear = 0;  
    }  
    else {  
        rear++; moves to 0  
    }  
    queue[rear] = x; (queue[0]=60)  
    printf("enqueued: %d\n", x);  
}
```



now queue is full,

there are two conditions to check if the queue is full.

(i) the next element of rear is front which means queue is full because rear points to the most recently added element and front is next means there is no more space to put any element.

$front == rear + 1$

(ii) $front = 0$ and $rear = size - 1$

means front is pointing to the start of the index and rear is pointing to the last index of the queue means the queue is full because if you consider circular queue, the queue move backs to 0 where front is already pointing means there is a element.

$f = 0 \ \&\& \ r = size - 1$

$if (front == (rear + 1) \% size)$

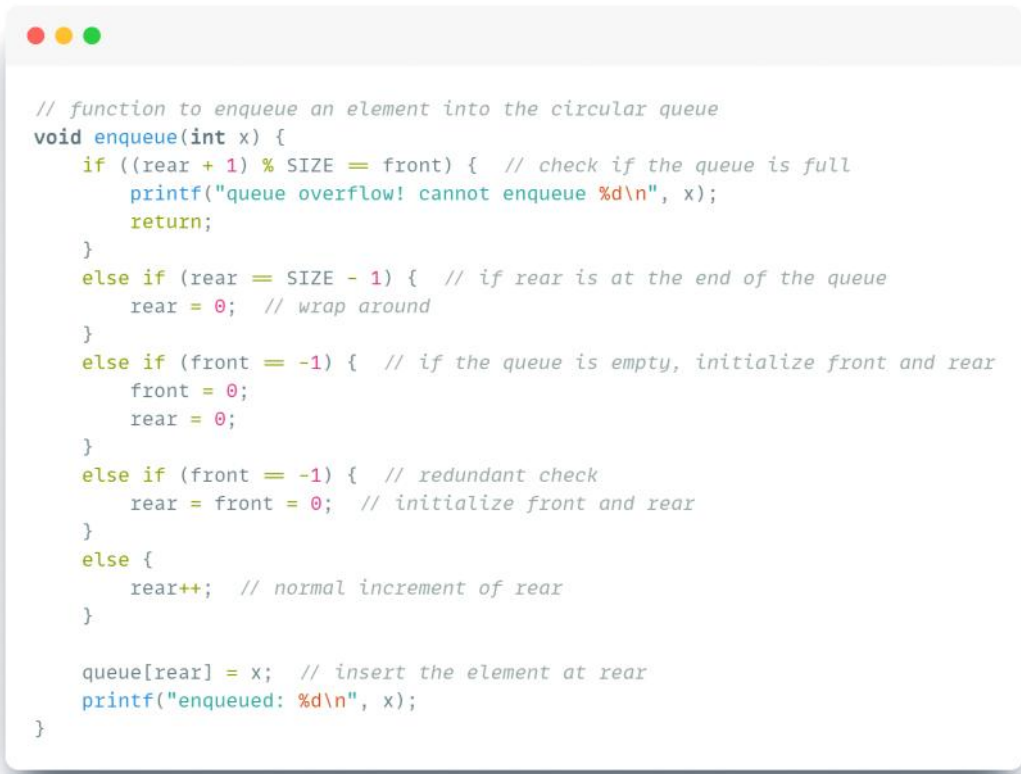
(function to enqueue an element into the circular queue)

```
void enqueue(int x) {  
    if (front == (rear + 1) \% size) { (check if the queue is full)
```

```

    printf("queue overflow! cannot enqueue %d\n", x);
    return;
} goes to this section if queue is not full
else if (rear == SIZE - 1) { (if rear is at the end of the queue/pointing to the last index)
    rear = 0; (wrap around, goes back to index 0)
} goes to this section if queue is not full and rear is not pointing to last index
else if (front == -1) { (if the queue is empty, initialize front and rear)
    front = 0;
    rear = 0;
} goes to this section if queue is not full, rear is not pointing to last index and queue is not empty
else if (front == -1) { ()
    rear = front = 0; (initialize front and rear)
} goes to this section if queue is not full and rear is not pointing to last index, queue is not empty and front is not pointing to first index
else {
    rear++; (normal increment of rear)
}
queue[rear] = x; (insert the element at rear)
printf("enqueued: %d\n", x);
}

```



```

// function to enqueue an element into the circular queue
void enqueue(int x) {
    if ((rear + 1) % SIZE == front) { // check if the queue is full
        printf("queue overflow! cannot enqueue %d\n", x);
        return;
    }
    else if (rear == SIZE - 1) { // if rear is at the end of the queue
        rear = 0; // wrap around
    }
    else if (front == -1) { // if the queue is empty, initialize front and rear
        front = 0;
        rear = 0;
    }
    else if (front == -1) { // redundant check
        rear = front = 0; // initialize front and rear
    }
    else {
        rear++; // normal increment of rear
    }

    queue[rear] = x; // insert the element at rear
    printf("enqueued: %d\n", x);
}

```

other code:

```

void enqueue(int x) {
    if ((rear + 1) % SIZE == front) { // check if the queue is full
        printf("queue overflow! cannot enqueue %d\n", x);
        return;
    }

    if (front == -1) { // if the queue is empty, initialize front and rear
        front = 0;
        rear = 0;
    }
    else {
        rear = (rear + 1) % SIZE; // circular increment of rear
    }

    queue[rear] = x; // insert the element at rear
    printf("enqueued: %d\n", x);
}

```

(ii) deletion (dequeue)

- deletion happens from the front of the queue.
- the front pointer is moved forward after the element is removed.

```

int dequeue() {
    int temp;

    if (front == -1) { // check if the queue is empty
        printf("queue underflow! cannot dequeue\n");
        return INT_MIN; // return a sentinel value to indicate underflow
    }

    temp = queue[front]; // store the front element to return it

    if (front == rear) { // if front == rear, the queue has only one element
        front = rear = -1; // reset the queue to empty (both front and rear to -1)
    } else { // if there are more than one element in the queue
        front = (front + 1) % SIZE; // circular increment of front
    }

    return temp; // return the dequeued element
}

```

(iii) priority queue

(iii) priority queue:

- elements are dequeued based on priority rather than arrival order.

a priority queue is a special type of queue where each element is associated with a priority. in a priority queue, the element with the highest priority (or lowest priority, depending on implementation) is dequeued first, regardless of its order of insertion.

characteristics of a priority queue

(i) priority-based access:

- elements are dequeued based on their priority.
- higher-priority elements are processed before lower-priority ones.

(ii) implementation:

- commonly implemented using:
 - array/list: a simple but less efficient approach.
 - heap: a binary heap provides better performance for insertion and deletion.
 - balanced binary search tree: for advanced use cases.

(iii) order of dequeue:

- o max-priority queue: the element with the highest priority is dequeued first.
- o min-priority queue: the element with the lowest priority is dequeued first.

(iv) use cases:

- scheduling tasks in an operating system.
- dijkstra's algorithm (shortest path in a graph).
- managing threads and network packets.

basic operations

(i) insertion (enqueue): add an element to the queue with a specific priority.

(ii) deletion (dequeue): remove the element with the highest or lowest priority.

(iii) peek: view the highest/lowest-priority element without removing it.

(iv) double-ended queue (deque):

a double-ended queue (deque) is a generalized version of a queue where elements can be inserted or removed from both ends of the queue. this flexibility makes it a versatile data structure.

characteristics of a deque

(i) bi-directional operations:

- elements can be added to or removed from both the front and rear.

(ii) types of deque:

- input-restricted deque: insertion is allowed only at one end, but deletion is allowed from both ends.
- output-restricted deque: deletion is allowed only at one end, but insertion is allowed from both ends.

(iii) flexibility:

- supports operations similar to both stacks (lifo) and queues (fifo).

(iv) implementation:

- typically implemented using arrays or linked lists.

tree

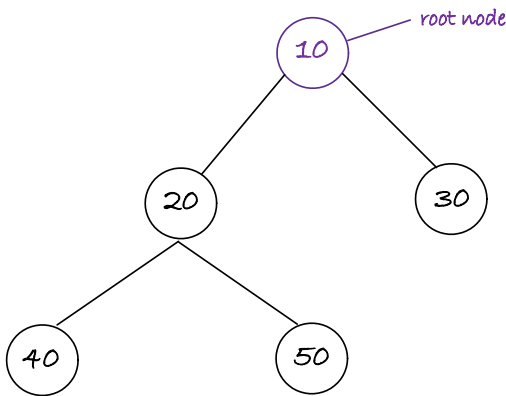
trees in data structures

a tree is a hierarchical data structure (non-linear) consisting of nodes, where each node has a parent node and zero or more child nodes. it is a widely used structure in computer science, especially for representing hierarchical data like file systems, xml/html documents, and more.

characteristics of a tree:

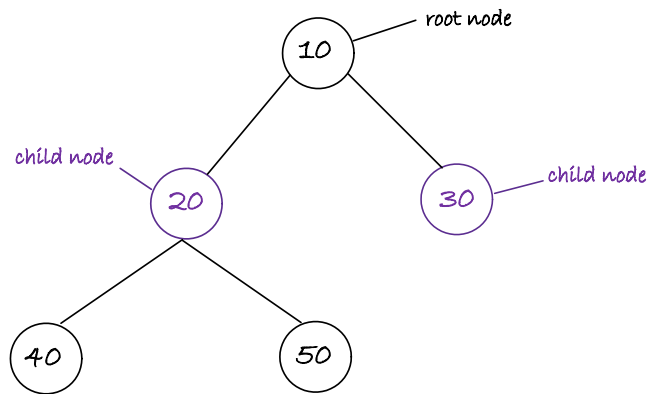
(i) root node:

- the topmost node of a tree.
- it has no parent.



(ii) child node:

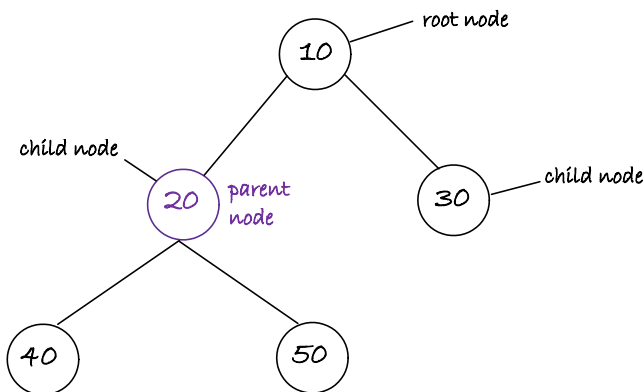
- a node directly connected to another node when moving away from the root.



20 and 30 are the child of 10

(iii) parent node:

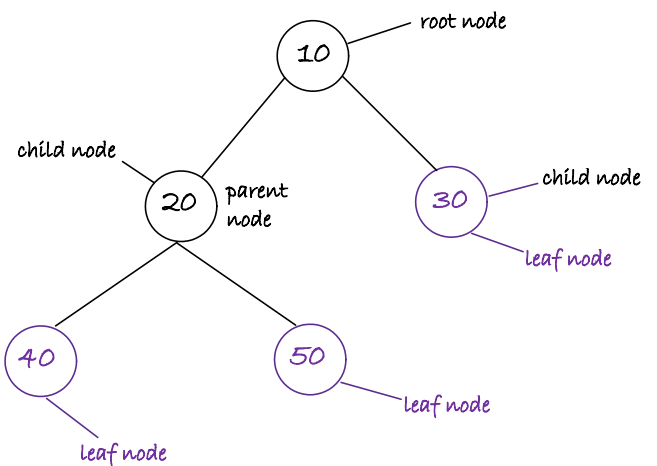
- a node directly connected to another node when moving toward the root.



20 is the parent of 40 and 50

(iv) leaf node:

- a node with no children.

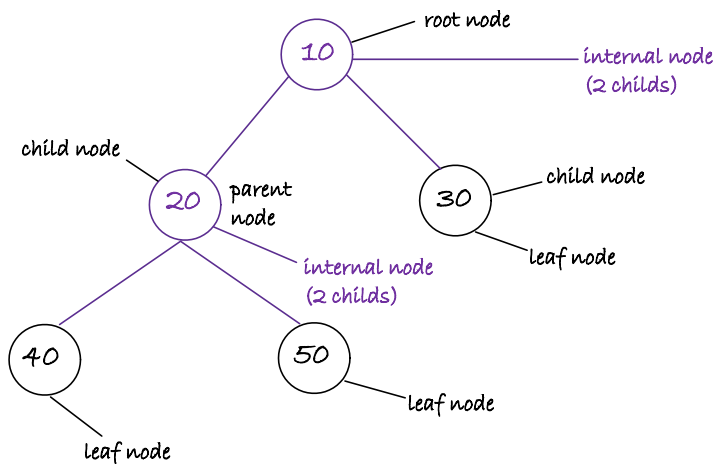


(vi) degree of node:

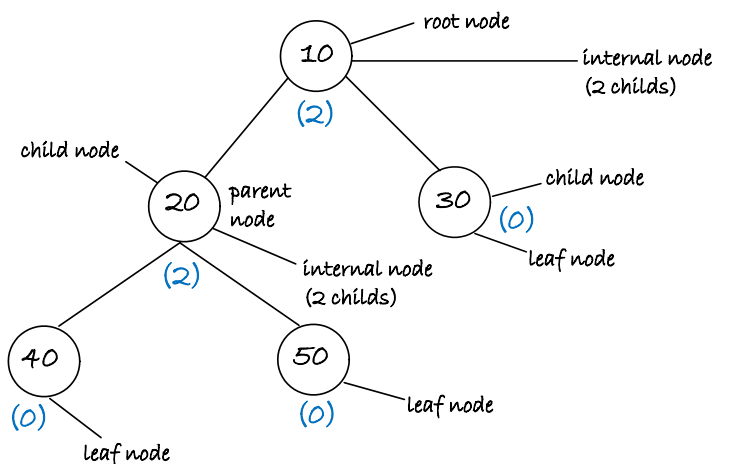
- number of childs



- nodes with atleast one child



- number of childs



(vii) height of a tree:

- the number of edges on the longest path from the X node to a leaf.

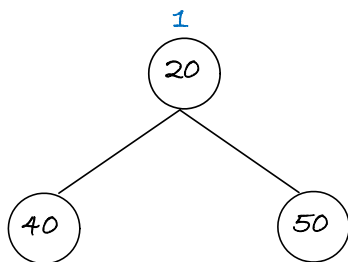
(a) height

$h(\text{leaf}) = 0$



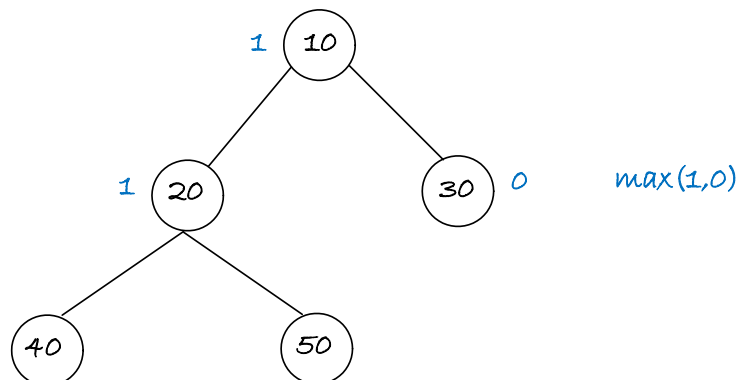
(b) height

$h(20) = 1$



(c) height

$h(10) = 2$



height of a node with 2 child = $1 + \max(1, 0)$

height of a node with 2 child = 2

point to remember:

height of tree = height of root node

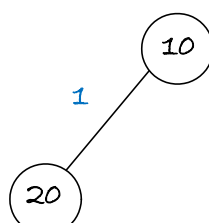
(viii) depth of a node:

- the number of edges from the root to the node.

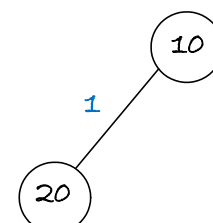
(a) level = 0

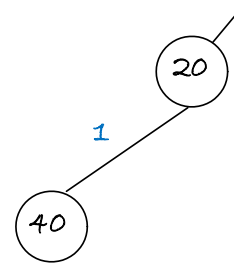


(b) level = 1



(c) level = 2





(ix) ancestor of a node:

- the number of edges from the root to the node.

an ancestor of a node in a tree is any node that is present on the path from the root to that node, excluding the node itself. the root node is considered the ancestor of all nodes in the tree.

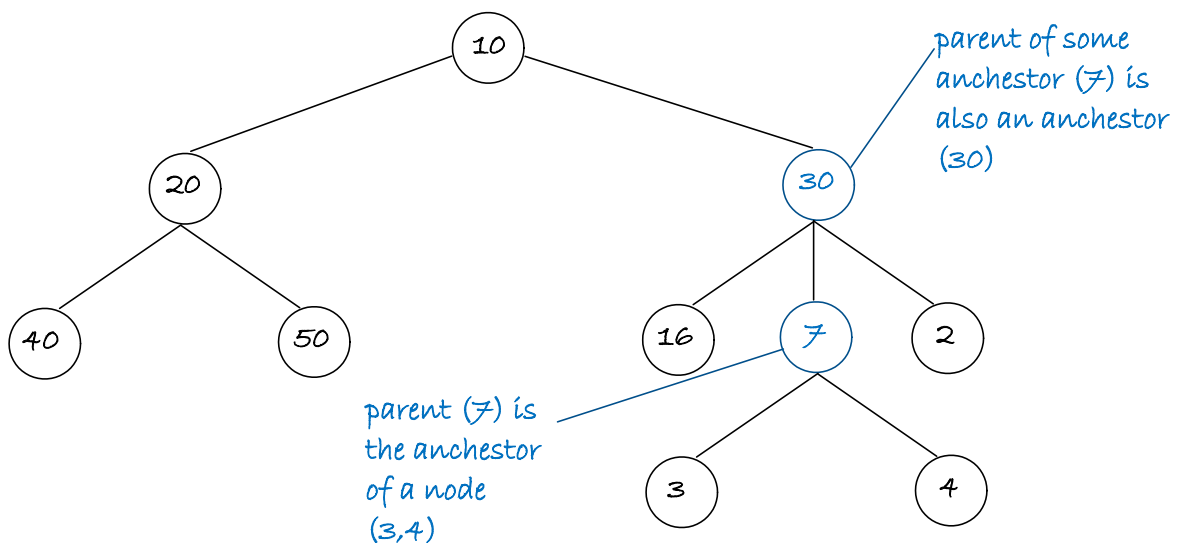
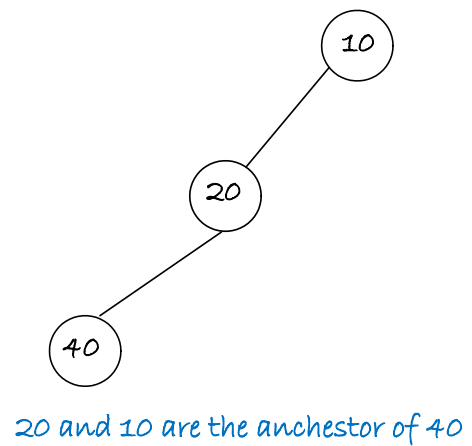
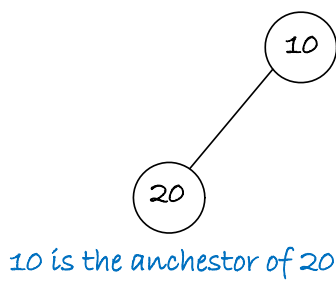
types of ancestors

(i) direct ancestor:

- the parent of the node.

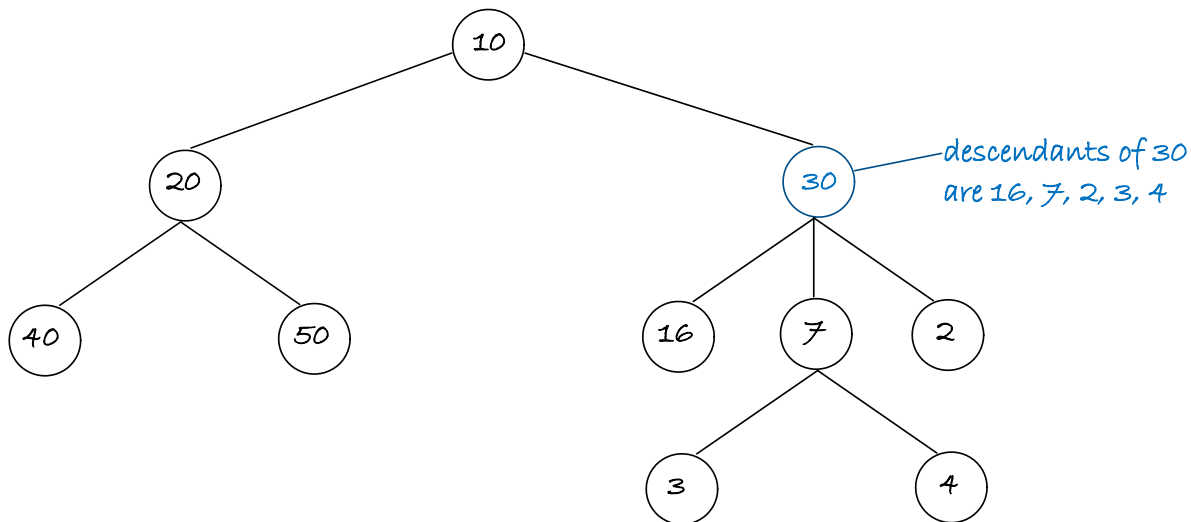
(ii) indirect ancestor:

- any node higher up in the hierarchy, other than the parent.



(x) descendant of a node:

a descendant of a node in a tree is any node that lies in the subtree rooted at that node, excluding the node itself. in simple terms, all child nodes, grandchildren, and so on, are descendants of a given node.



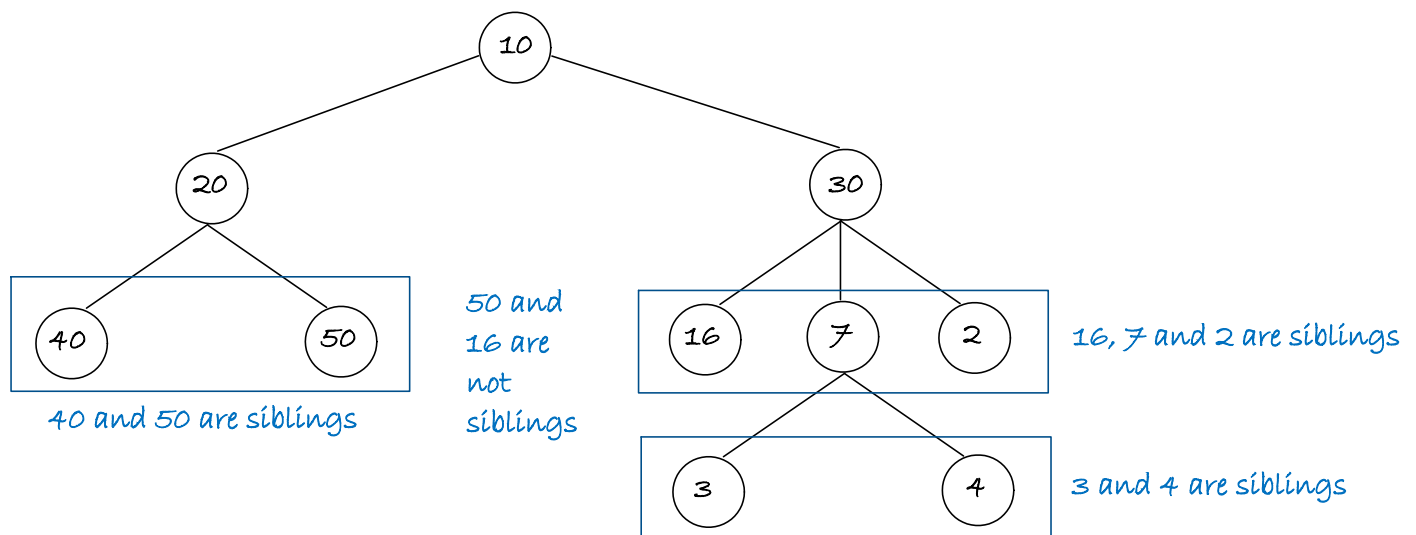
(xi) siblings:

siblings are nodes that share the same parent in a tree. in other words, two or more nodes are siblings if they are direct children of the same node.

to find the siblings of a node:

(i) locate the parent of the given node.

(ii) list all other children of the same parent.



(xii) generation in a tree

in the context of trees, the generation of a node refers to its level in the hierarchical structure, often described in terms of parent-child relationships.

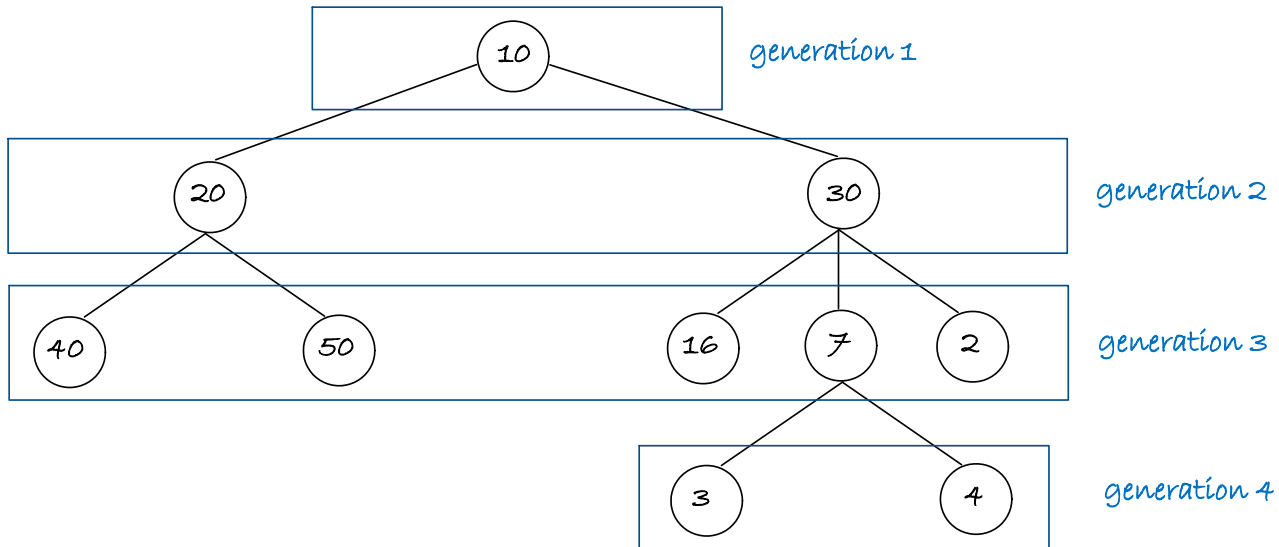
definition

(i) root generation:

the root of the tree is considered the first generation (generation 0 or 1 depending on conventions).

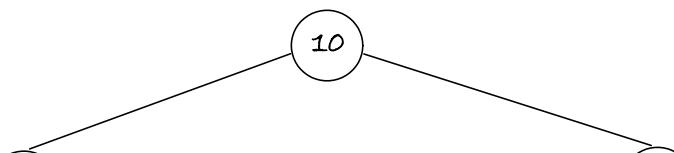
(ii) child generation:

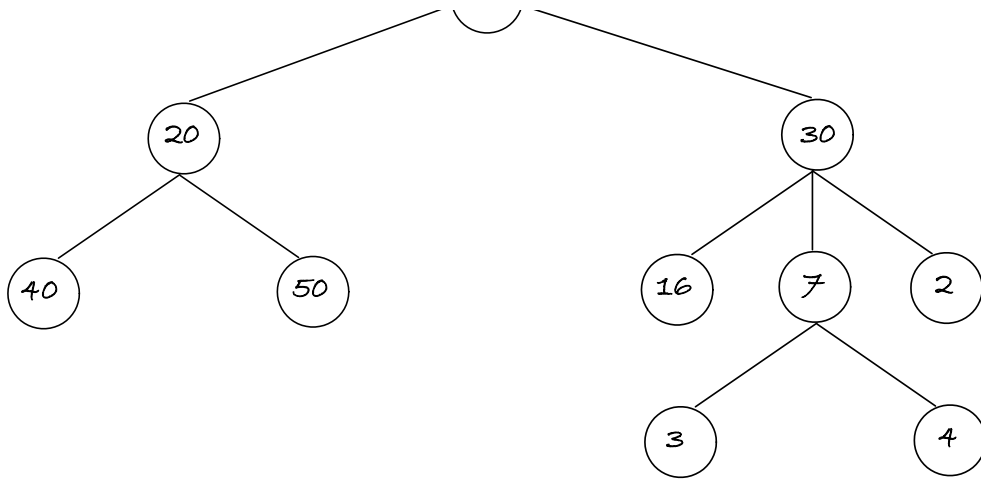
each level below the root represents a subsequent generation. nodes directly connected to the root are part of the second generation, and so on.



(xiii) size of a node in a tree

number of descendants of the node (including the node itself) is called as size of that node.



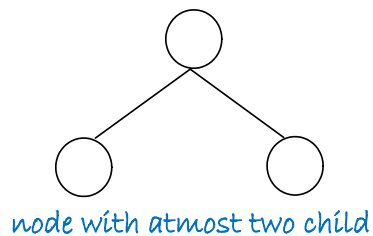
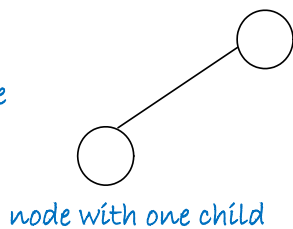


size of node with key 30 = 1 (root) + 5 (childs)
 size of node with key 30 = 6

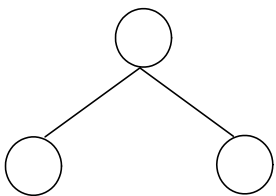
types of trees:

(i) binary tree:

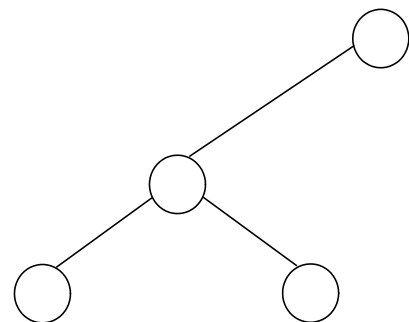
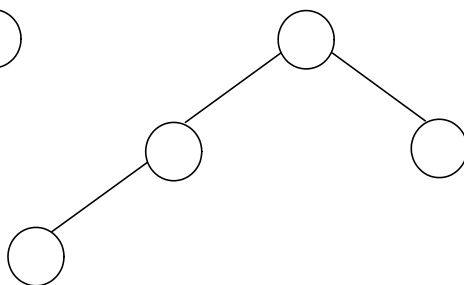
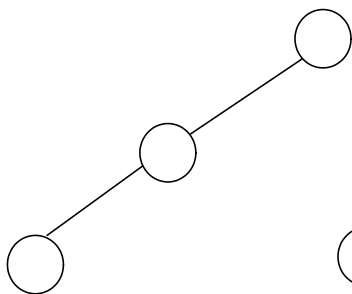
- each node has atmost two children.

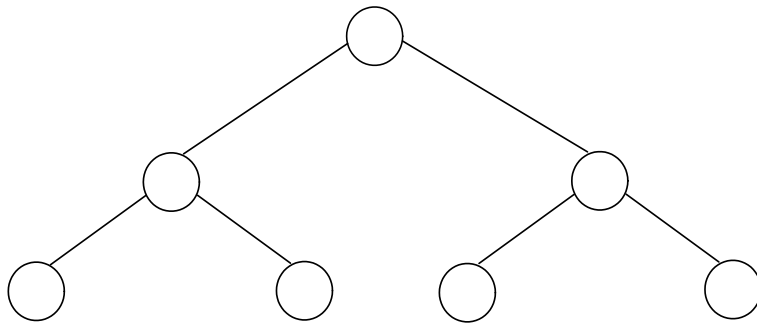
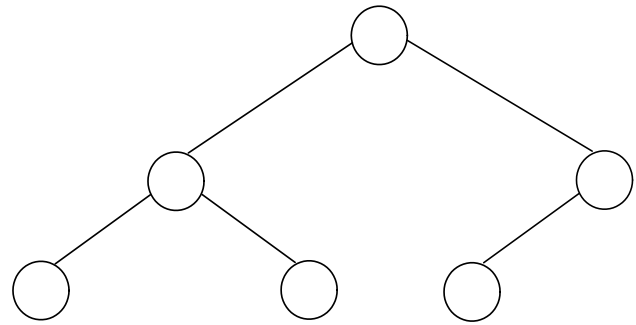
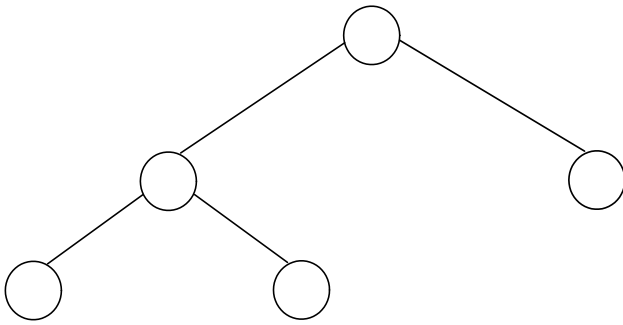


binary tree with height 1 : max 2 nodes



binary tree with height 2 : max 7 nodes

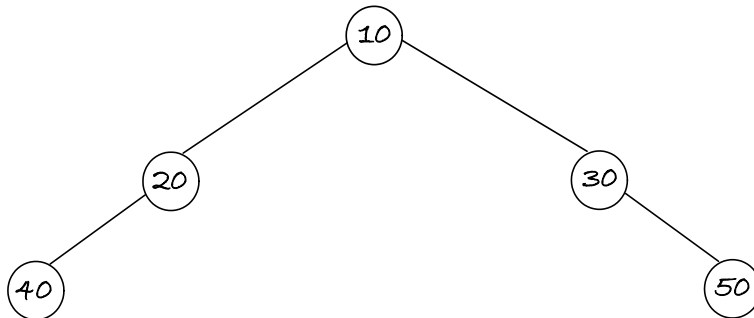




full binary tree/complete binary tree

when a binary tree is full binary tree then maximum number of nodes are $N_{\max} = 2^{n+1} - 1$
 minimum number of nodes in a binary tree are $N_{\min} = h + 1$

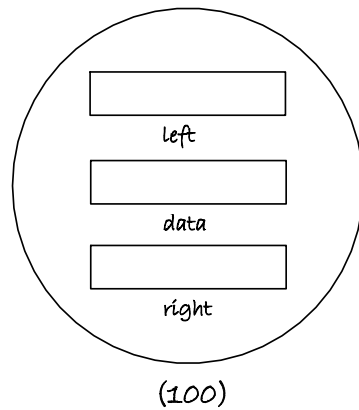
concept: binary tree in linked list



```

struct node {
    struct node *left;
    int data;
    struct node *right;
};
  
```

visualisation in memory:

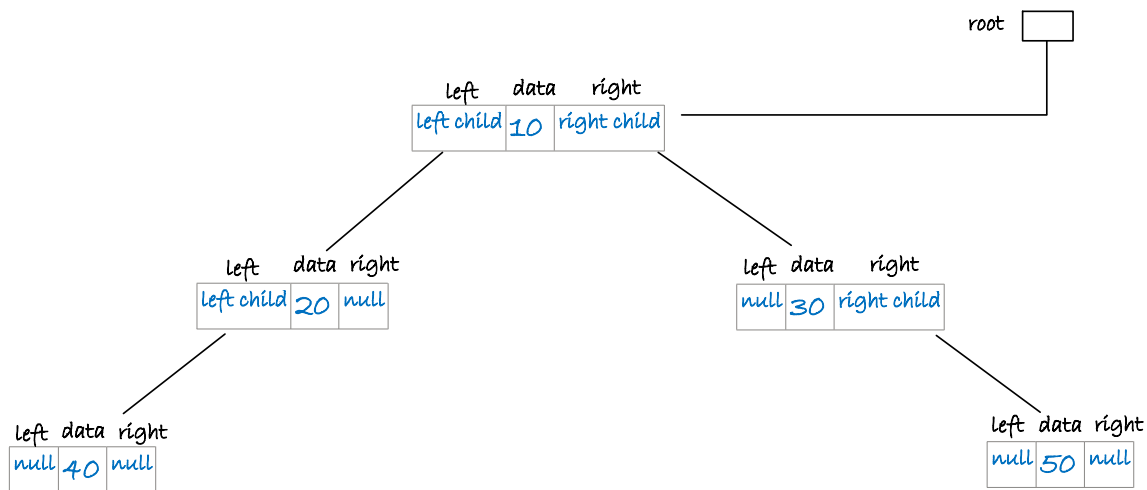


how to access members?

ptr->left

ptr->data

ptr->right



empty if

root null

(ii) k-ary tree:

- every internal node has exactly k childs

let i: number of internal nodes

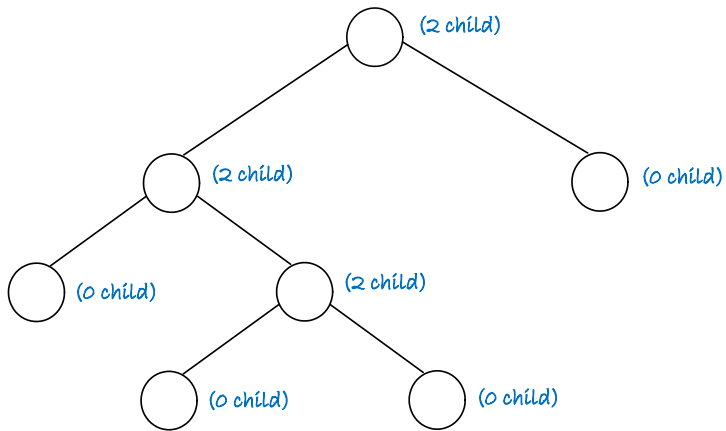
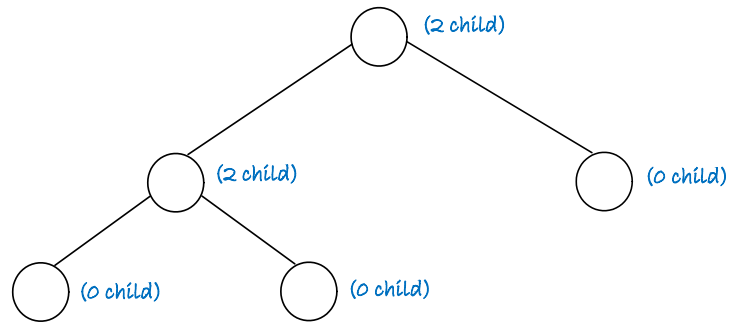
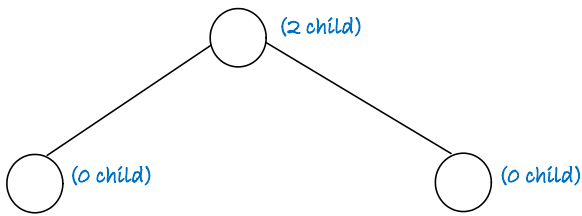
total nodes = $i \times k + 1$

total nodes = $k i + 1$

(a) 2-ary tree:

- a binary tree where each node can have 0 or 2 children.

- a binary tree where each internal node have exactly 2 nodes

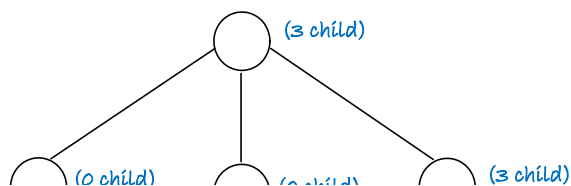
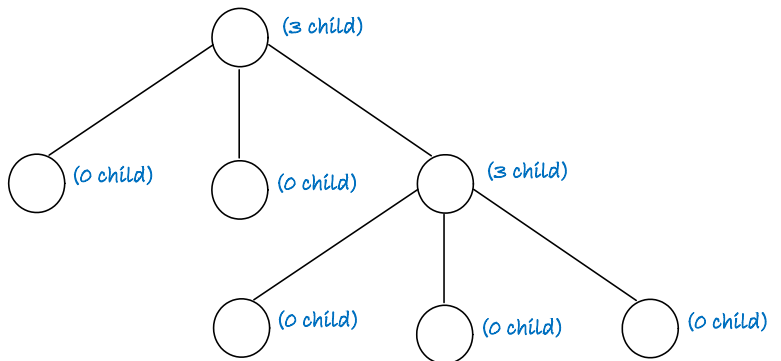
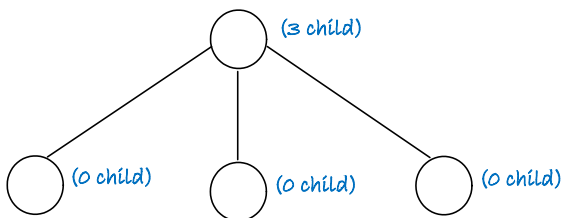


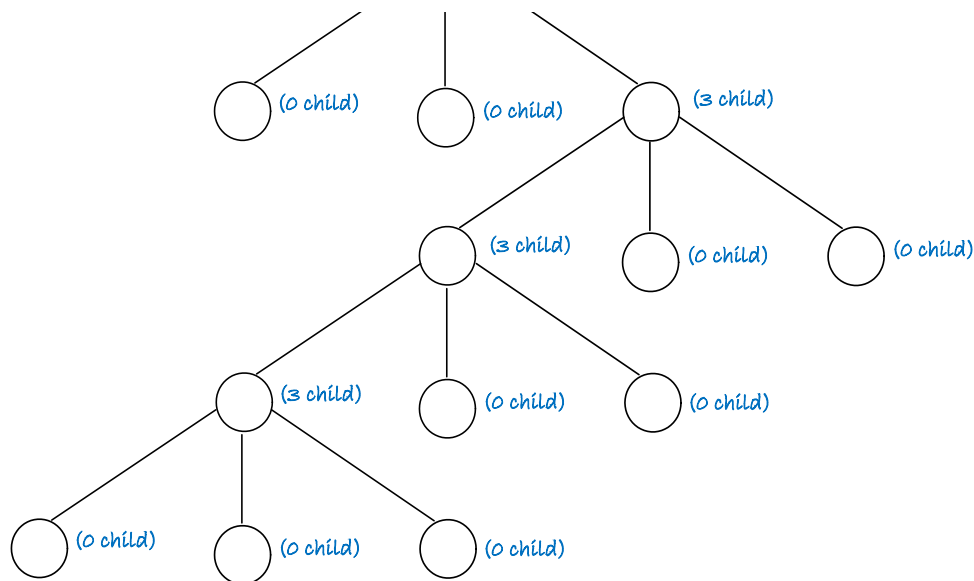
3 internal nodes
3 nodes of degree 2
3 nodes with 2 child

total nodes = $3 \times 2 + 1(\text{rootnode}) = 7$

(b) 3-ary tree:

- a binary tree where each node can have 0 or 3 children.
- a binary tree where each internal node have exactly 3 nodes





4 internal nodes

each node having 3 child

total nodes = $4 \times 3 + 1 (\text{root node}) = 13$

Let's derive the formula further more:

(i) total number of nodes in terms of internal node

total nodes = $ki + 1$

here, total nodes are basically number of internal nodes (having atleast one child) + number of leaf nodes

number of internal nodes (having atleast one child) + number of leaf nodes = $ki + 1$

(ii) total number of leaf nodes in terms of internal node

number of internal nodes (having atleast one child) + number of leaf nodes = $ki + 1$

$i + l = ki + 1$

$l = ki - i + 1$

$l = i(k-1) + 1$

total number of leaf nodes = $i(k-1) + 1$

(iii) total number of nodes in terms of leaf node

total number of leaf nodes = $i(k-1) + 1$

$l - 1 = i(k-1) + 1 - 1$

$(l-1) = i(k-1)$

$i = (l-1) / (k-1)$

substitute the value of i in $n = ki + 1$

$n = k(l-1) / (k-1) + 1$

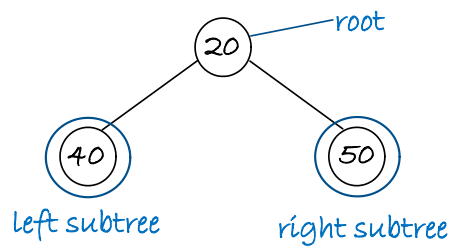
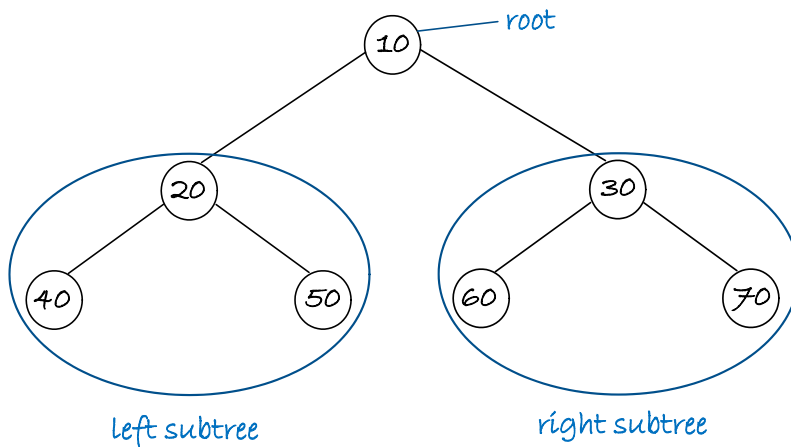
$n = (kl - k + k - 1) / (k-1)$

$n = (kl - 1) / (k-1)$

total number of nodes in terms of leaf node = $(kl - 1) / (k-1)$

topic: tree traversal

tree traversal refers to visiting all the nodes of a tree data structure in a systematic way.

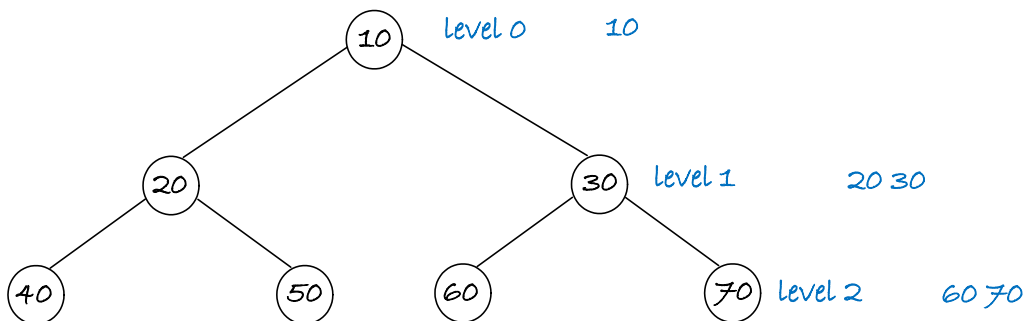


(i) breadth-first traversal (bft)

this approach visits nodes level by level, starting from the root and moving to each child.

level-order traversal

- visit nodes level by level from left to right.
- commonly implemented using a queue.



(ii) depth-first traversal (dft)

this approach explores as far down a branch as possible before backtracking.

there are three main types of depth-first traversal:

(a) in-order traversal

- visit the left subtree, then the root node, and finally the right subtree.

- applicable mostly to binary trees.

sequence: left $\rightarrow$ root $\rightarrow$ right.

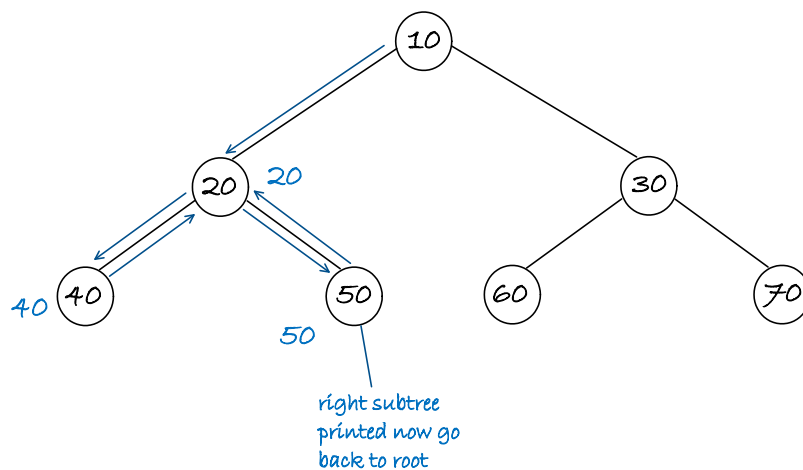
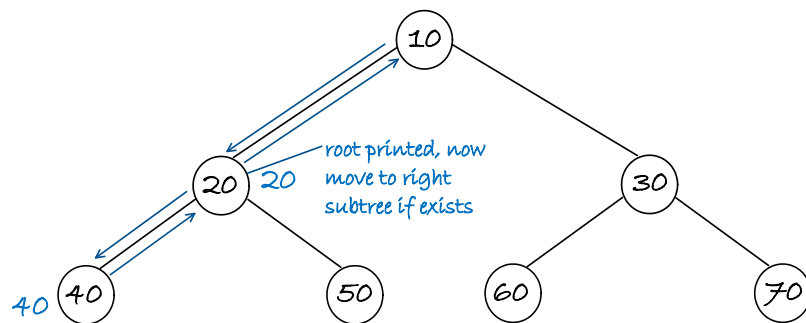
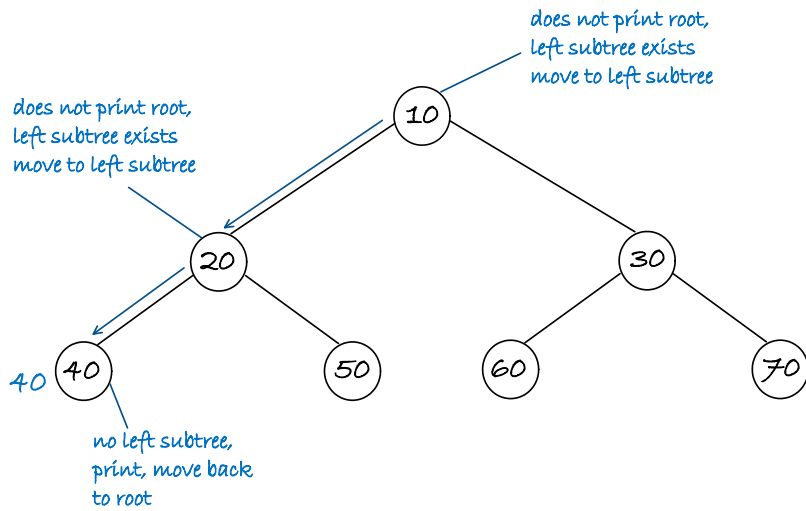
left, root, right

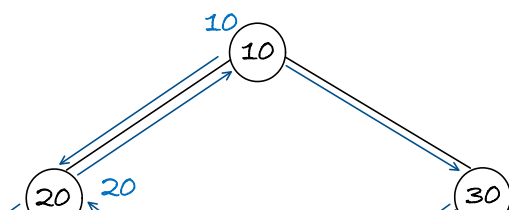
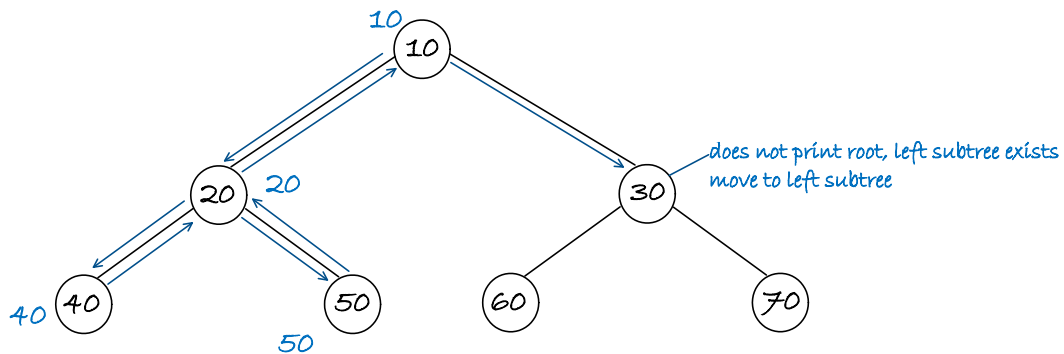
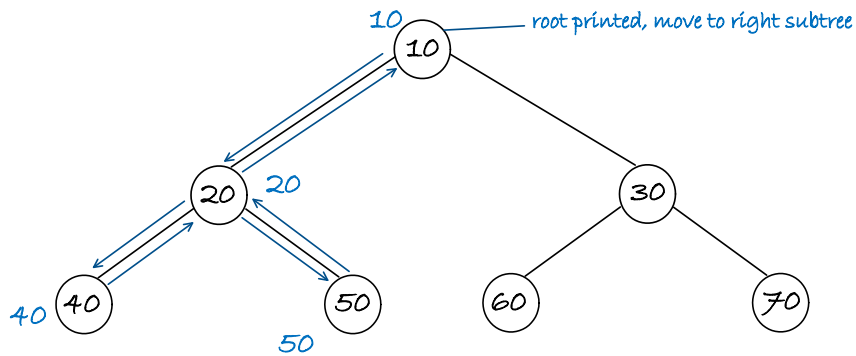
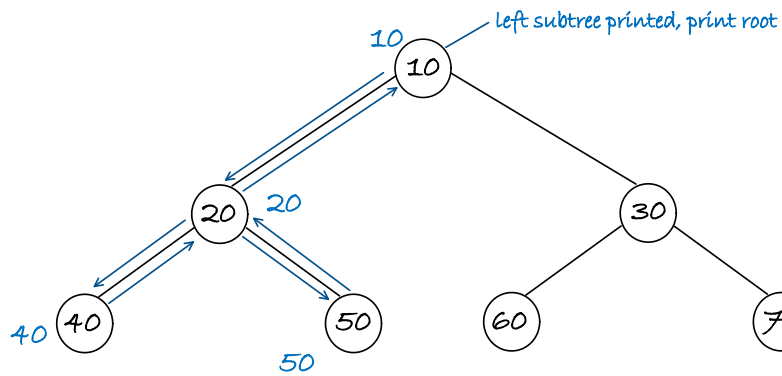
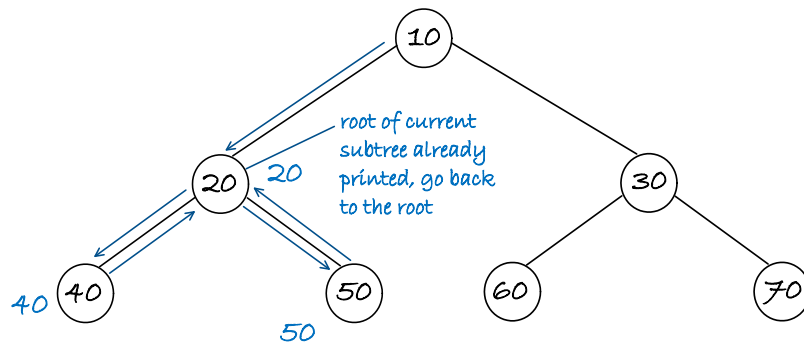
process

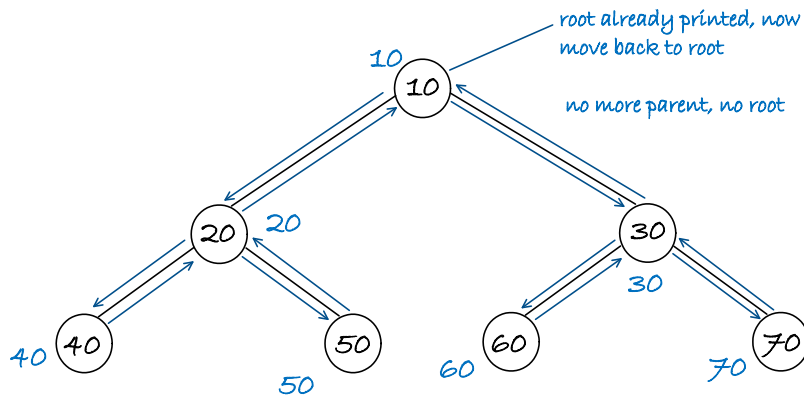
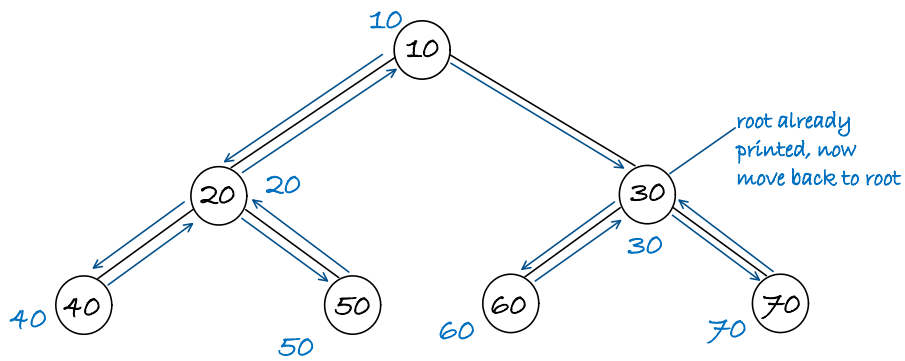
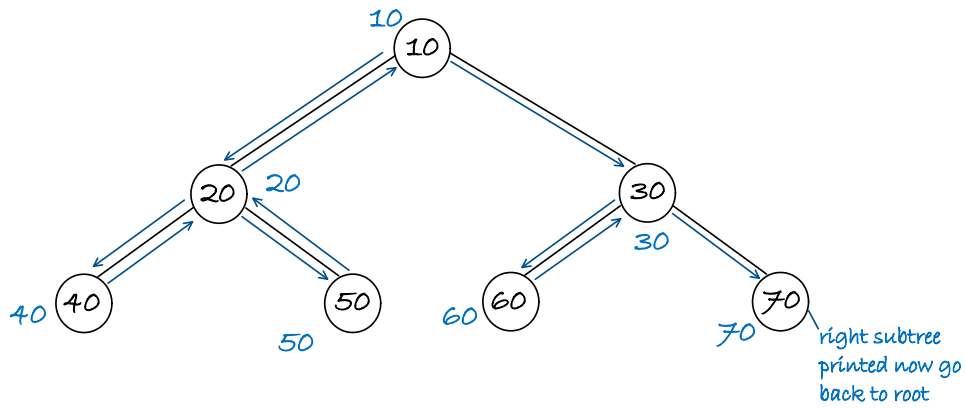
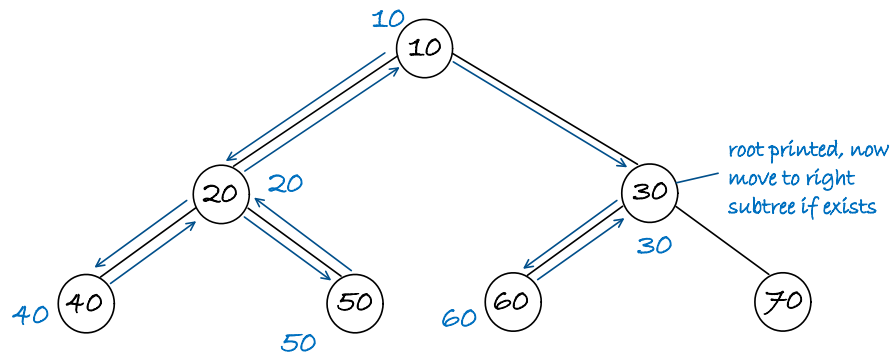
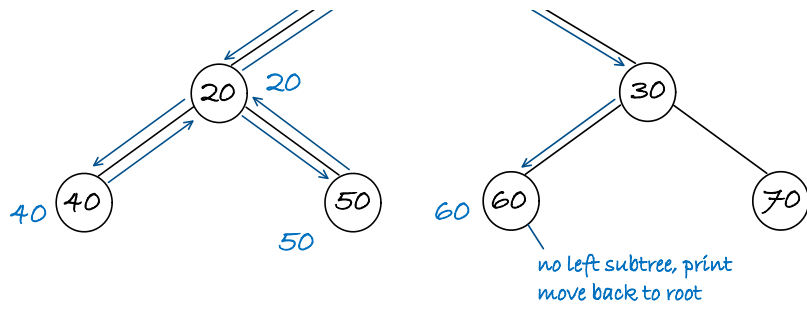
step (i) traverse the left subtree in in-order

step (ii) visit/print/process root node

step (iii) traverse the right subtree in in-order

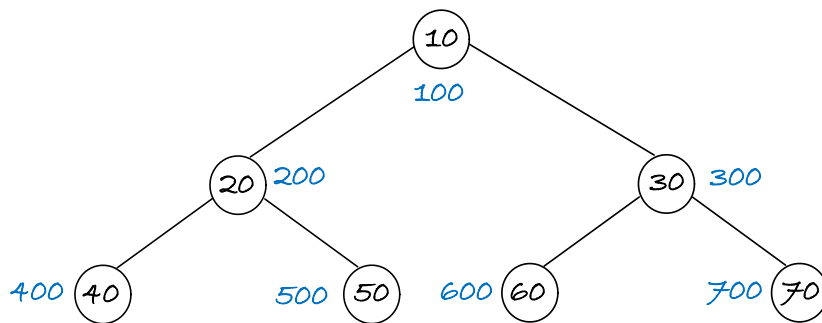




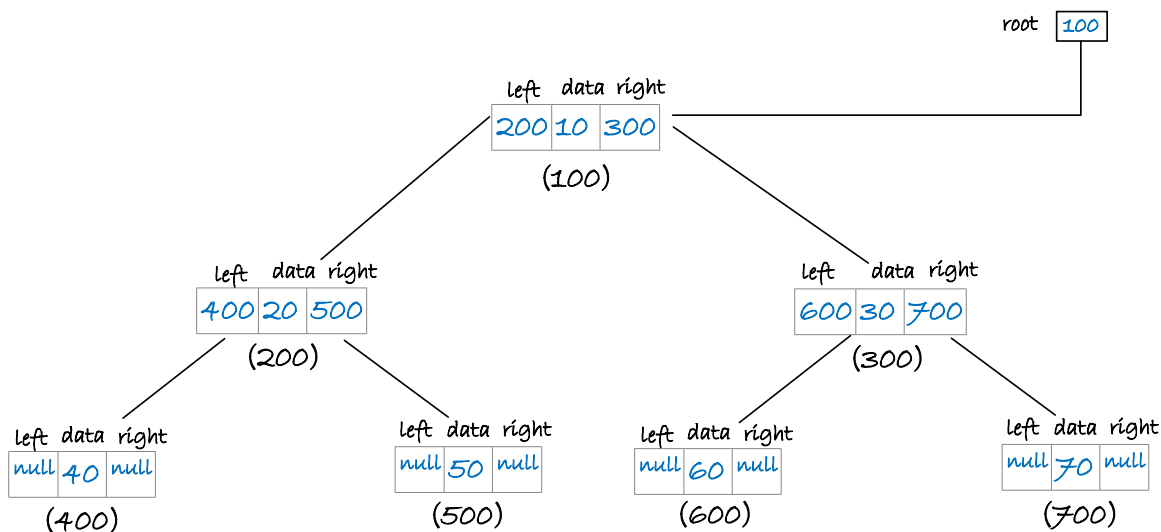


final output: 40 20 50 10 60 30 70

code for in-order traversal:



as we discussed before in memory tree is in linked list so lets write the code for pre-order



logic and analysis:

(i) root points to the node, so if root = null that means there is no tree so we must check the condition to ensure that tree exists

```
if (root == null) {  
    return;  
}
```

(ii) logic for in-order traversal

as we have discussed before the way we access members in structures are by using arrow ptr->data to print data.

- process

step (i) traverse the left subtree in in-order

```
preorder(root->left);
```

step (ii) visit/print/process root node
 print ("%c", root->data);

step (iii) traverse the right subtree in in-order
 preOrder(root->right);

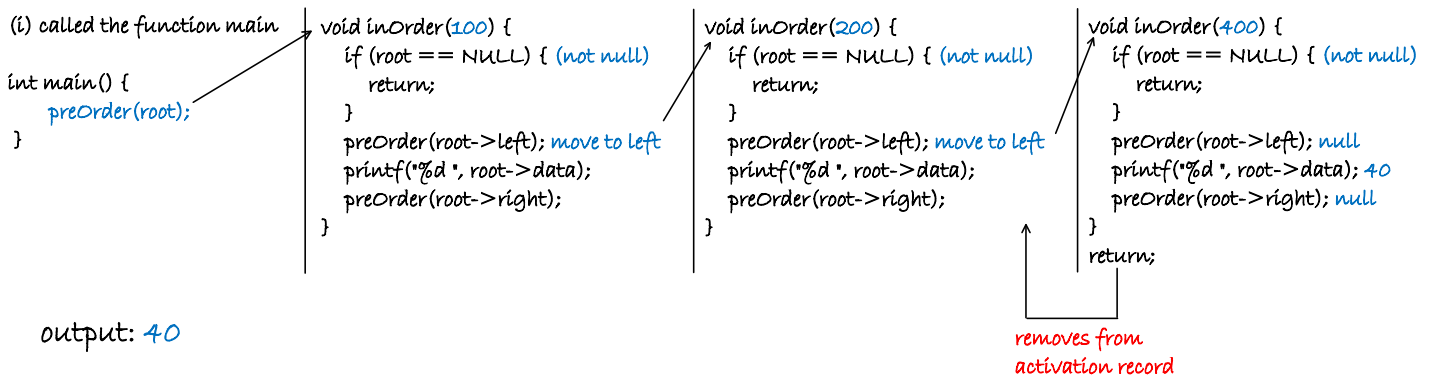
(function for in-order traversal)
 void inOrder(struct Node* root) {
 if (root == NULL) {
 return;
 }
 (traverse the left subtree)
 preOrder(root->left);

 (process the current node)
 printf("%d ", root->data);

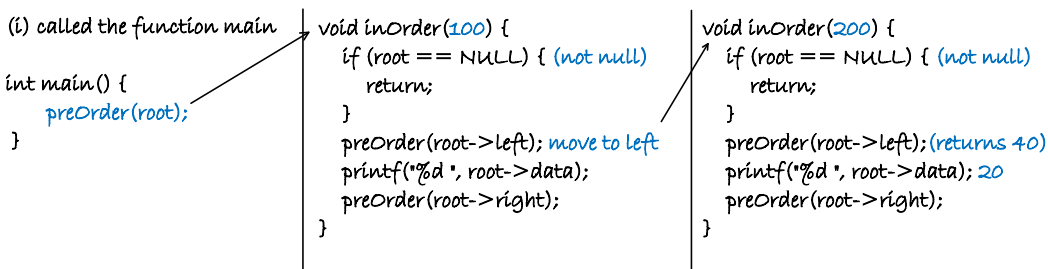
 (traverse the right subtree)
 preOrder(root->right);
 }

(main function to test the code)
 int main() {
 inOrder(root);
 }

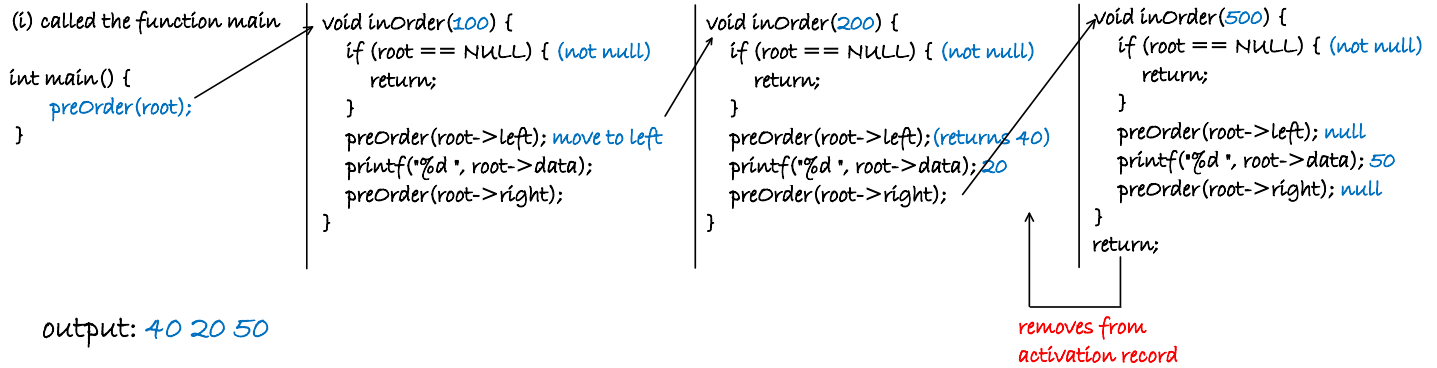
let's visualise this in stack:



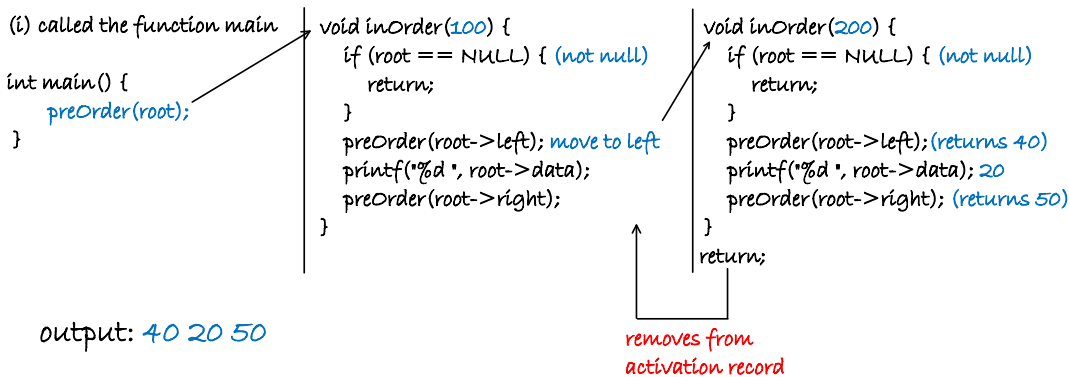
output: 40



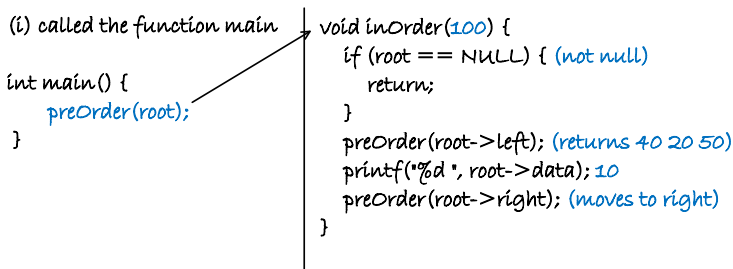
output: 40 20



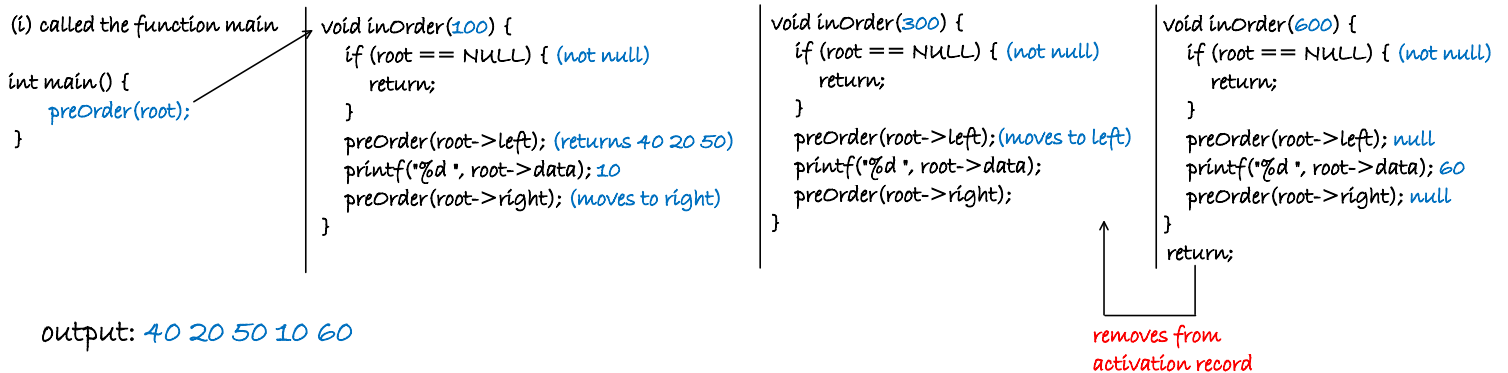
output: 40 20 50



output: 40 20 50



output: 40 20 50 10



output: 40 20 50 10 60

(i) called the function main

```
int main() {  
    preOrder(root);  
}
```

```
void inOrder(100) {  
    if (root == NULL) { (not null)  
        return;  
    }  
    preOrder(root->left); (returns 40 20 50)  
    printf("%d ", root->data); 10  
    preOrder(root->right); (moves to right)  
}
```

```
void inOrder(300) {  
    if (root == NULL) { (not null)  
        return;  
    }  
    preOrder(root->left); (returns 60)  
    printf("%d ", root->data); 30  
    preOrder(root->right); (moves to right)  
}
```

output: 40 20 50 10 60

(i) called the function main

```
int main() {  
    preOrder(root);  
}
```

```
void inOrder(100) {  
    if (root == NULL) { (not null)  
        return;  
    }  
    preOrder(root->left); (returns 40 20 50)  
    printf("%d ", root->data); 10  
    preOrder(root->right); (moves to right)  
}
```

```
void inOrder(300) {  
    if (root == NULL) { (not null)  
        return;  
    }  
    preOrder(root->left); (returns 60)  
    printf("%d ", root->data); 30  
    preOrder(root->right); (moves to right)  
}
```

```
void inOrder(700) {  
    if (root == NULL) { (not null)  
        return;  
    }  
    preOrder(root->left); null  
    printf("%d ", root->data); 70  
    preOrder(root->right); null  
}
```

output: 40 20 50 10 60 30 70

removes from
activation record

(i) called the function main

```
int main() {  
    preOrder(root);  
}
```

```
void inOrder(100) {  
    if (root == NULL) { (not null)  
        return;  
    }  
    preOrder(root->left); (returns 40 20 50)  
    printf("%d ", root->data); 10  
    preOrder(root->right); (moves to right)  
}
```

```
void inOrder(300) {  
    if (root == NULL) { (not null)  
        return;  
    }  
    preOrder(root->left); (returns 60)  
    printf("%d ", root->data); 30  
    preOrder(root->right); (returns 70)  
}
```

removes from
activation record

(i) called the function main

```
int main() {  
    preOrder(root);  
}
```

```
void inOrder(100) {  
    if (root == NULL) { (not null)  
        return;  
    }  
    preOrder(root->left); (returns 40 20 50)  
    printf("%d ", root->data); 10  
    preOrder(root->right); (returns 60, 30, 70)  
}
```

removes from
activation record

(i) called the function main

```
int main() {  
    preOrder(root); (returns 40 20 50 10 60 30 70)  
}
```

output: 40 20 50 10 60 30 70

```

(function for in-order traversal)
void inOrder(struct Node* root) {
    if (root == NULL) {
        return;
    }
    (traverse the left subtree)
    preOrder(root->left);

    (process the current node)
    printf("%d ", root->data);

    (traverse the right subtree)
    preOrder(root->right);
}

(main function to test the code)
int main() {
    inOrder(root);
}

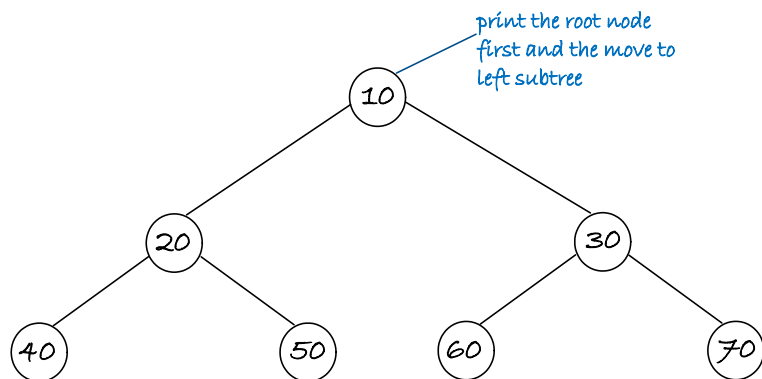
```

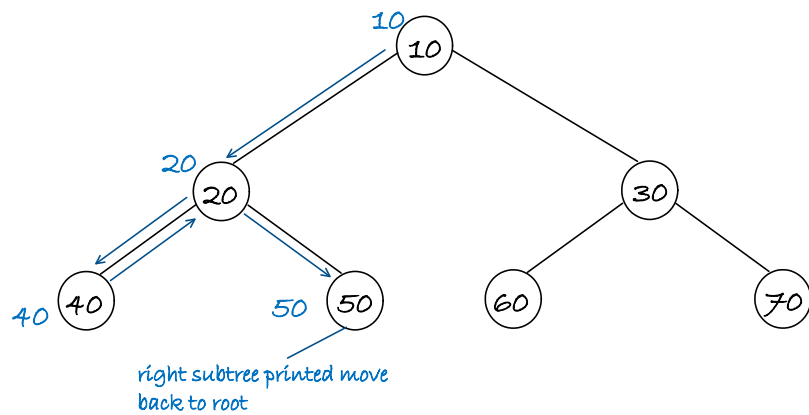
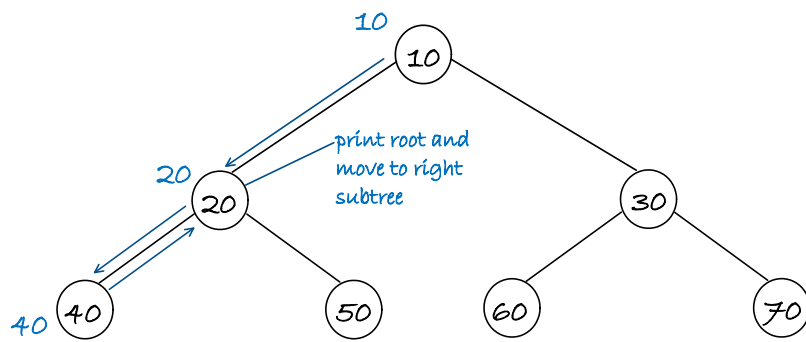
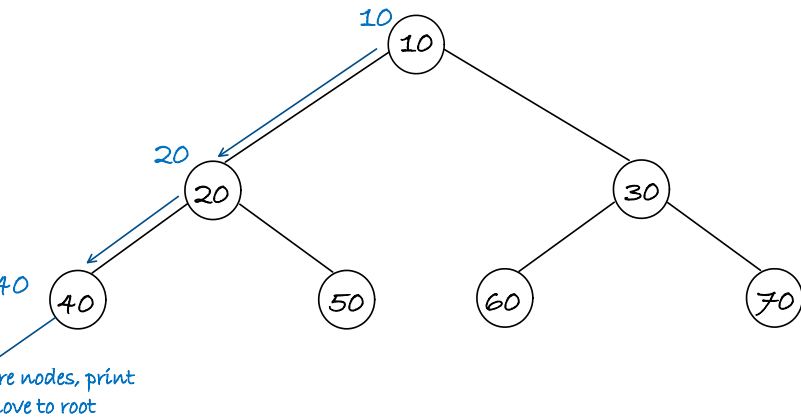
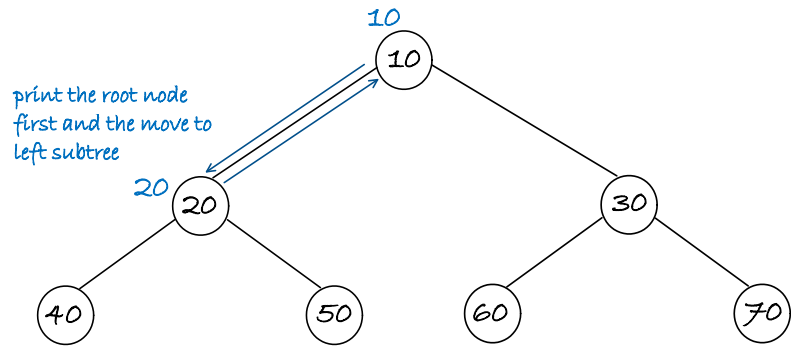
(b) pre-order traversal

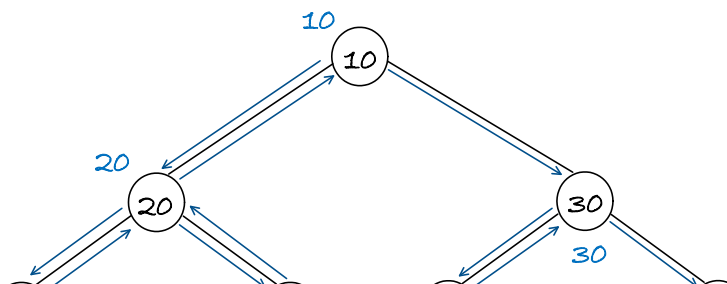
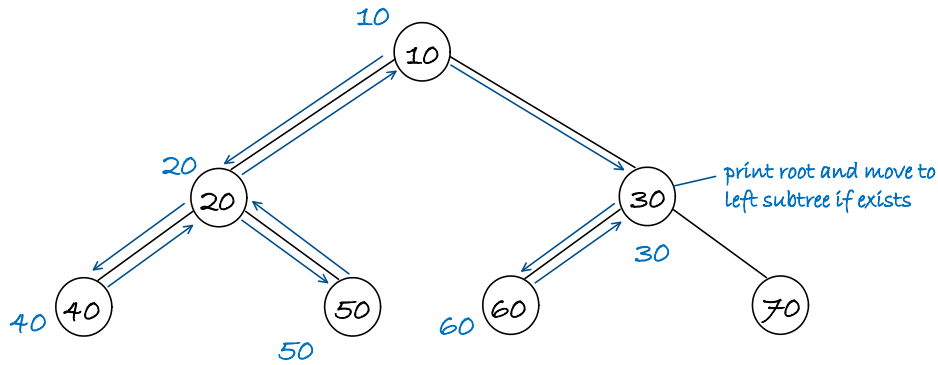
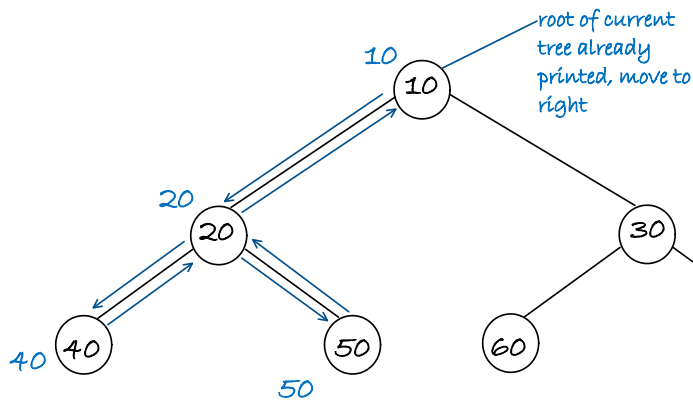
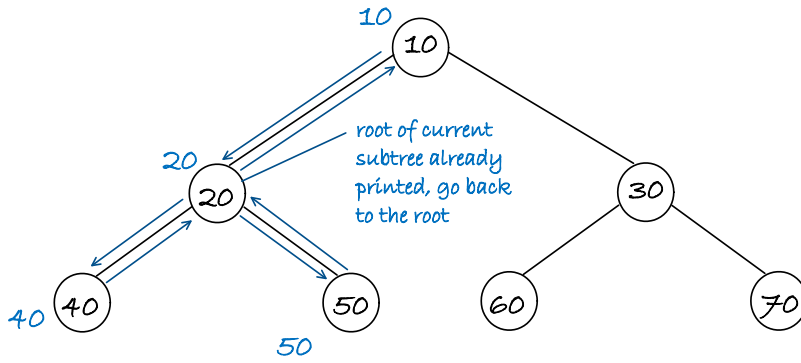
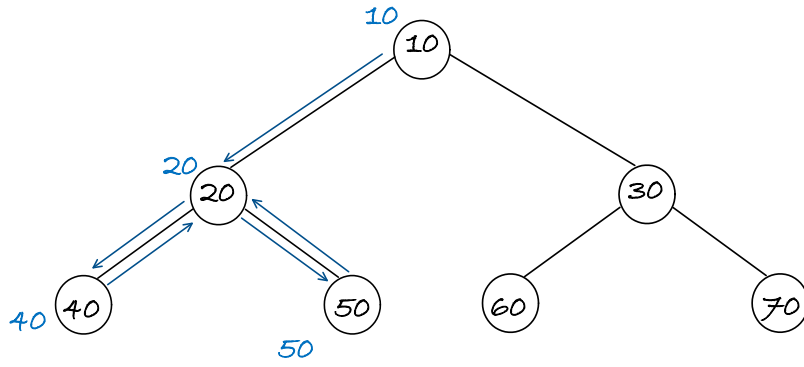
- visit the root node first, then the left subtree, and finally the right subtree.
- sequence: root → left → right.
- root, left, right

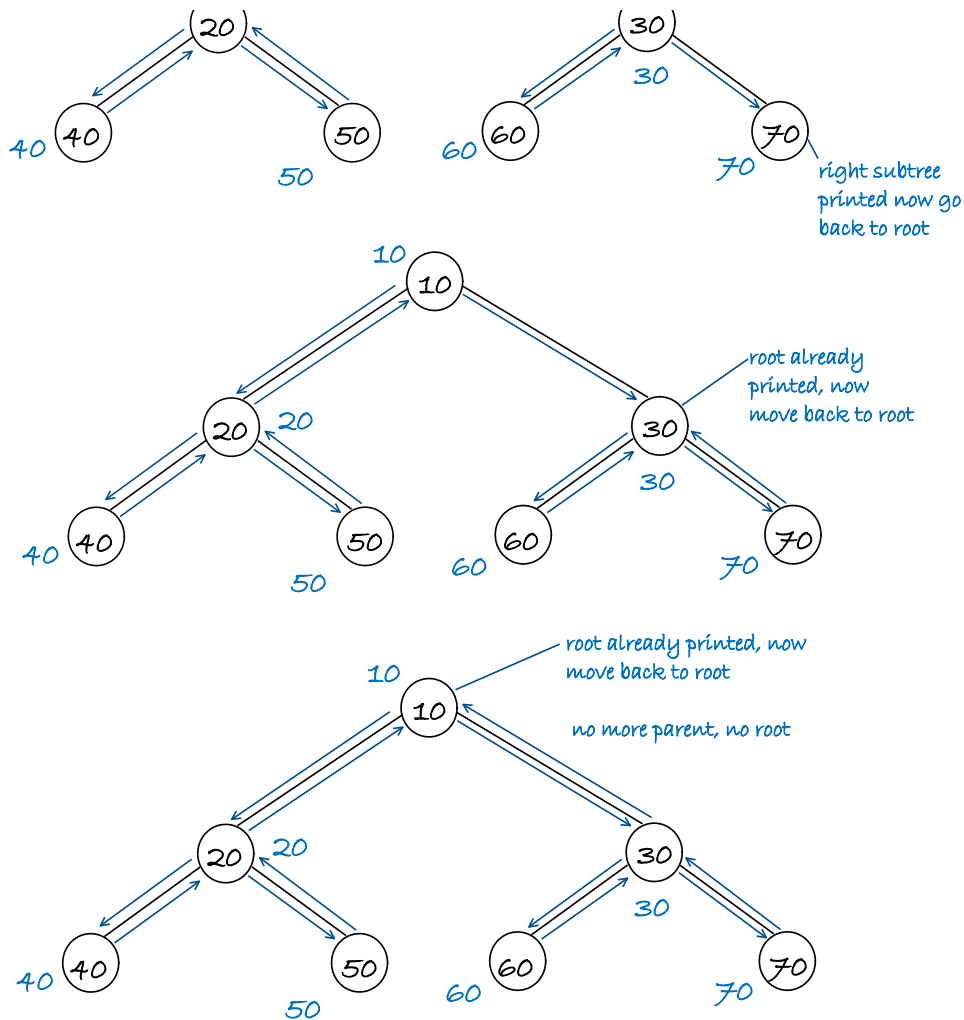
process

- step (i) visit/print/process root node
- step (ii) traverse the left subtree in in-order
- step (iii) traverse the right subtree in in-order



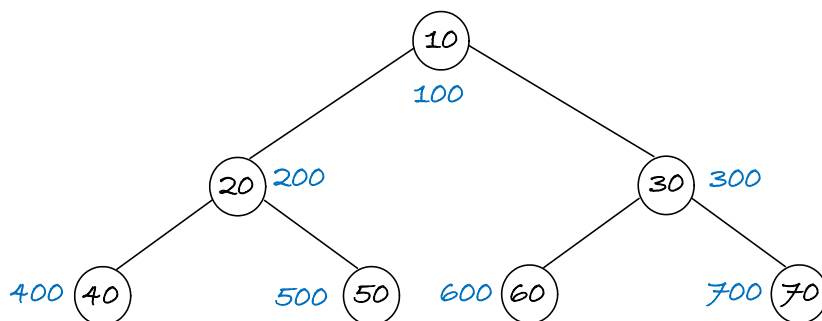






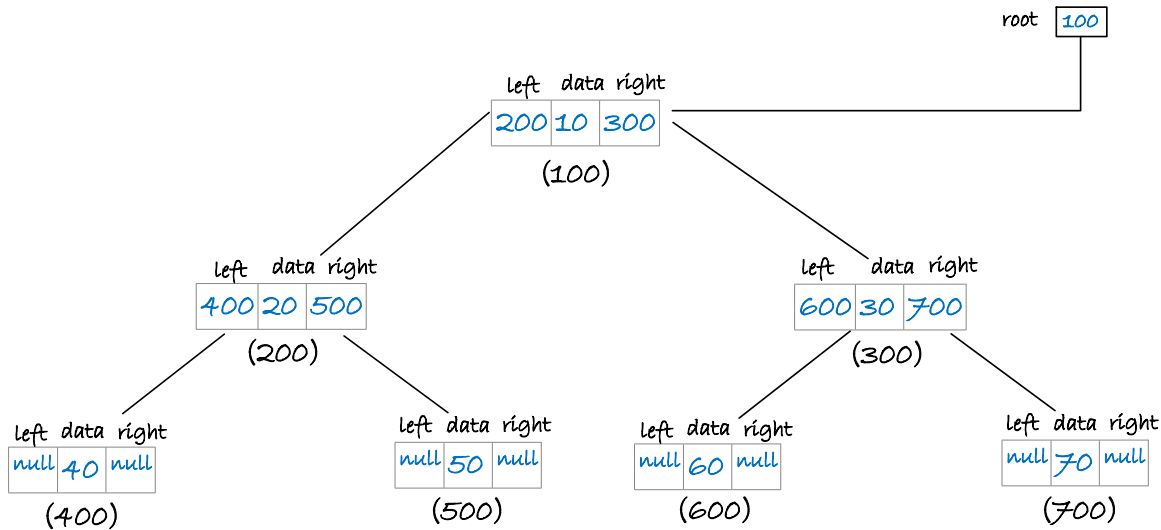
final output: 10 20 40 50 20 60 70

code for pre-order traversal:



as we discussed before in memory tree is in linked list so lets write the code for pre-order

root 100



logic and analysis:

(i) rootpoints to the node, so it root= null that means there is no tree so we must check the condition to ensure that tree exists

```

if (root==null){
return;
}

```

(ii) logic for pre-order traversal

as we have discussed before the way we access members in structures are by using arrow ptr->data to print data.

- process

step (i) visit/print/process root node

```

print ("%c", root->data);

```

step (ii) traverse the left subtree in in-order

```

preorder(root->left);

```

step (iii) traverse the right subtree in in-order

```

preorder(root->right);

```

(function for pre-order traversal)

```

void preOrder(struct Node* root) {
    if (root == NULL) {
        return;
    }
}

```

(process the current node)

```

printf("%d ", root->data);

```

(traverse the left subtree)

```

preOrder(root->left);

```

(traverse the right subtree)

```
preOrder(root->right);
```

```
}
```

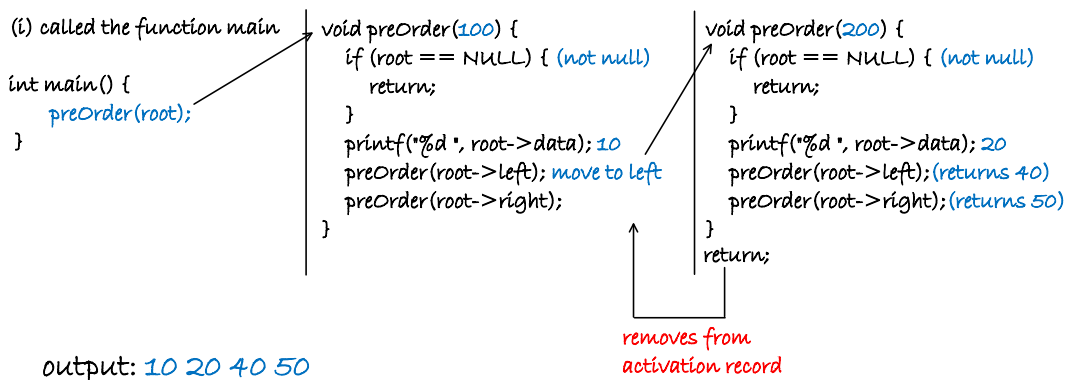
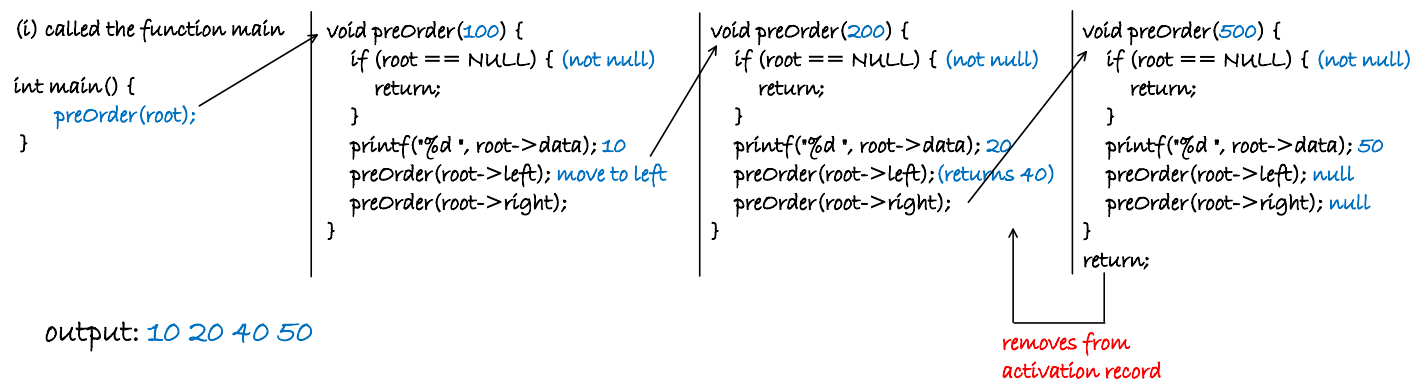
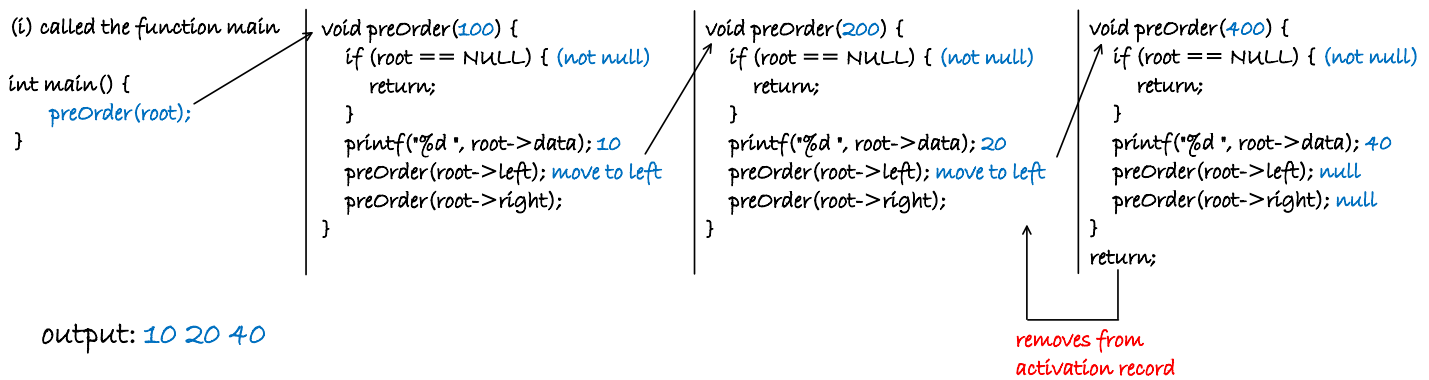
(main function to test the code)

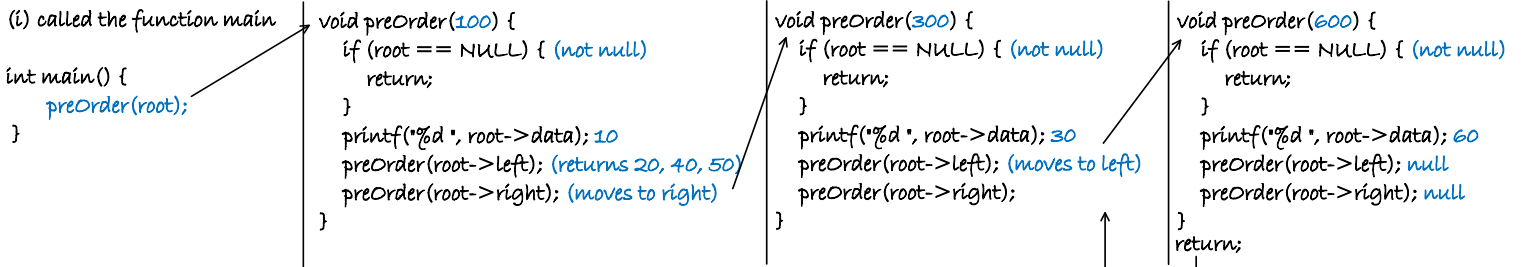
```
int main() {
```

```
    preOrder(root);
```

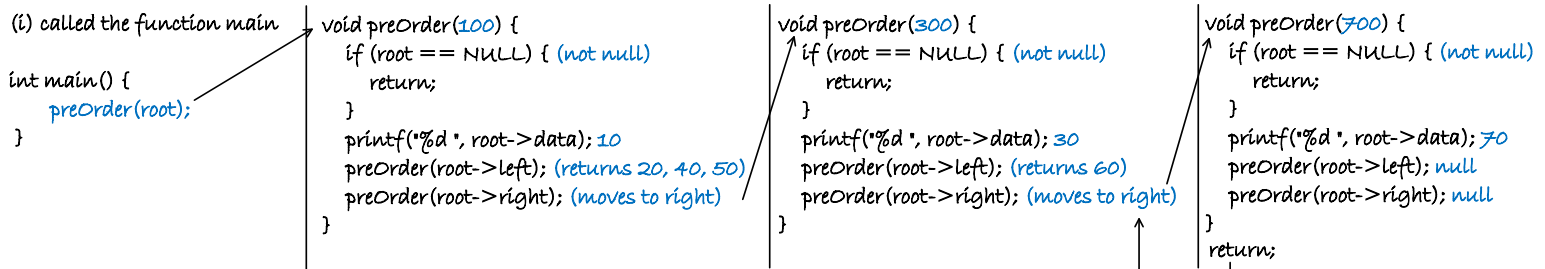
```
}
```

let's visualise this in stack:

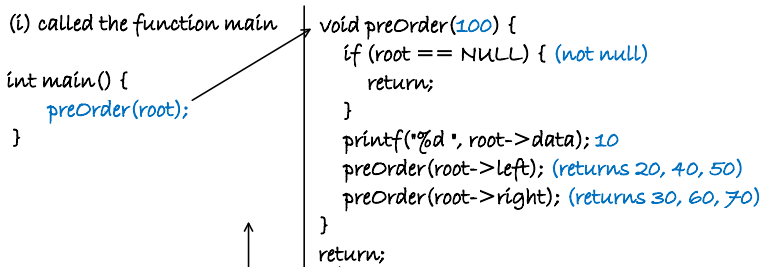
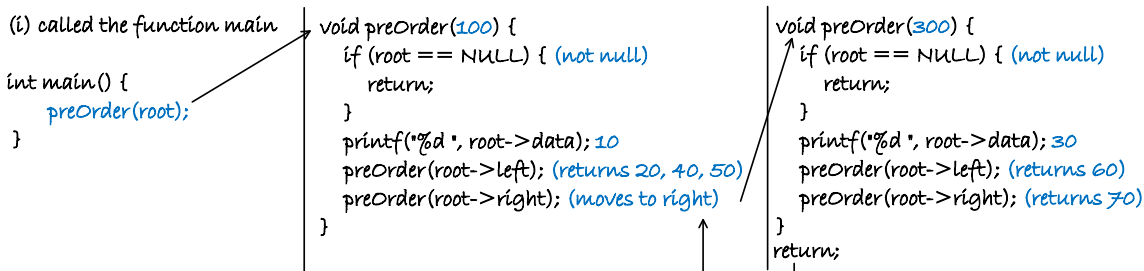




output: 10 20 40 50 30 60



output: 10 20 40 50 30 60 70



(i) called the function main

```

int main() {
    preOrder(root); (returns 10, 20, 40, 50, 30, 60, 70)
}

```

output: 10 20 40 50 30 60 70


```

// function for pre-order traversal
void preOrder(struct Node* root) {
    if (root == NULL) {
        return;
    }

    // process the current node
    printf("%d ", root->data);

    // traverse the left subtree
    preOrder(root->left);

    // traverse the right subtree
    preOrder(root->right);
}

// main function to test the code
int main() {
    preOrder(root);
}

```

(c) post-order traversal

- visit the left subtree, then the right subtree, and finally the root node.

sequence: left → right → root.

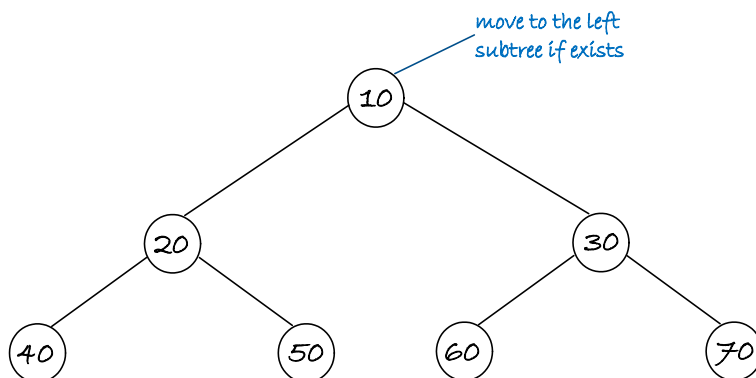
left, right, root

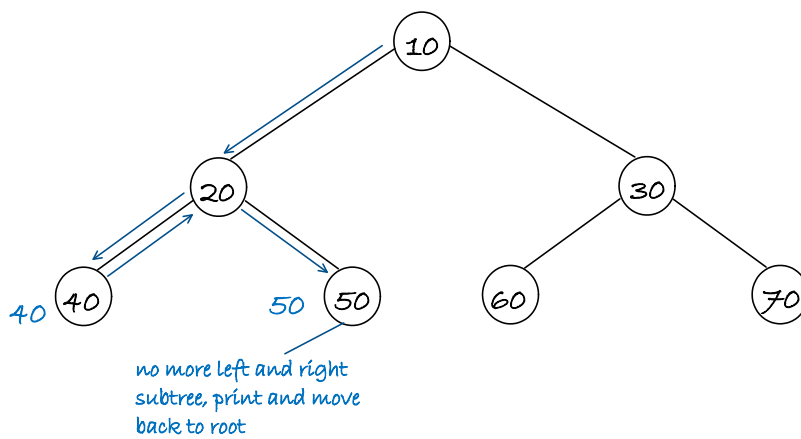
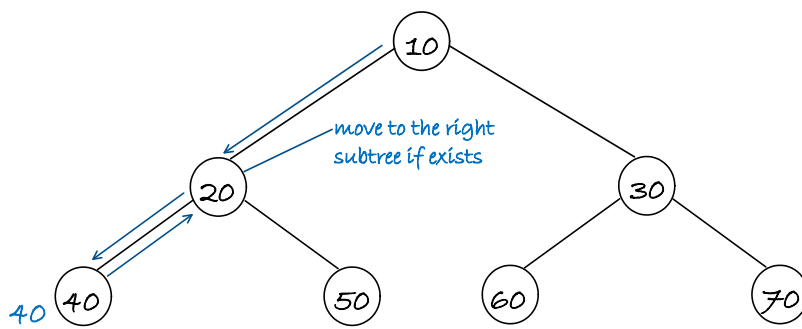
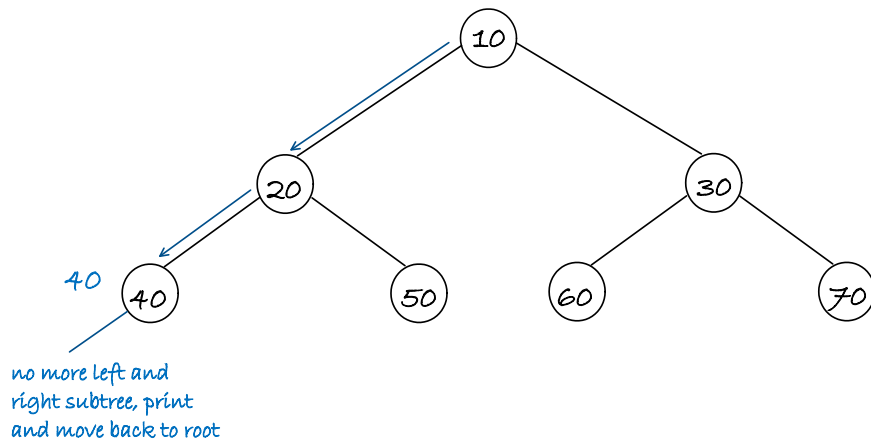
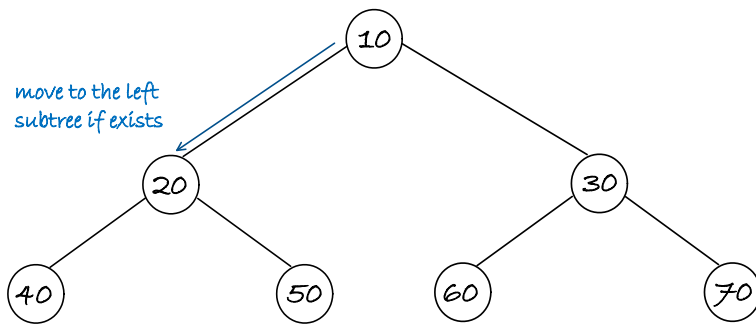
process

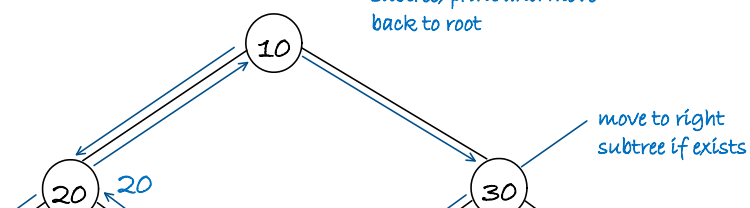
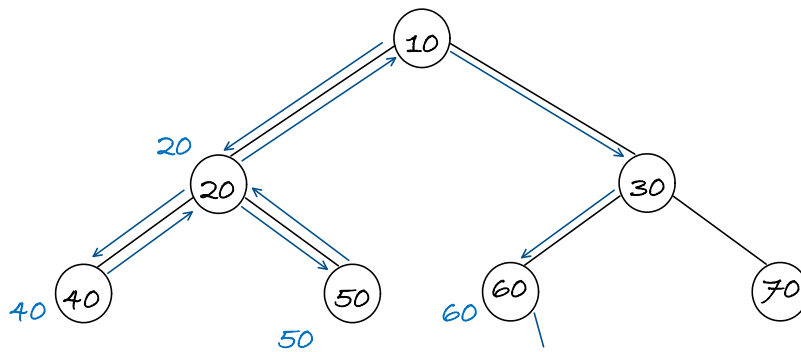
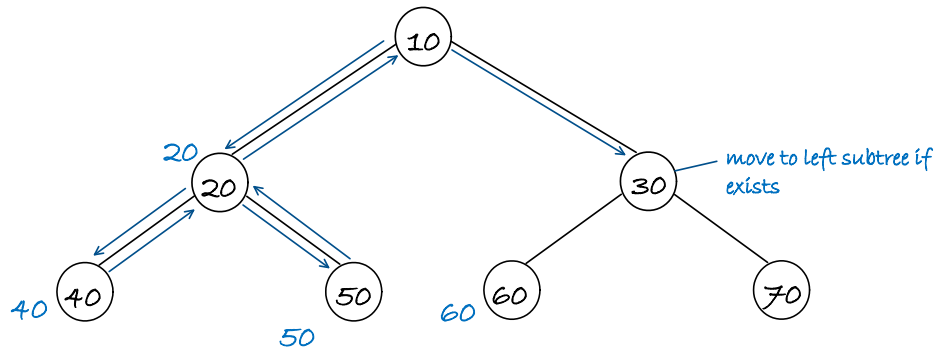
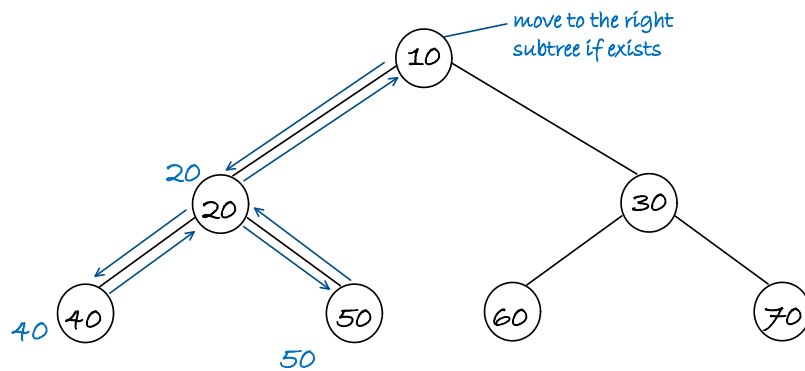
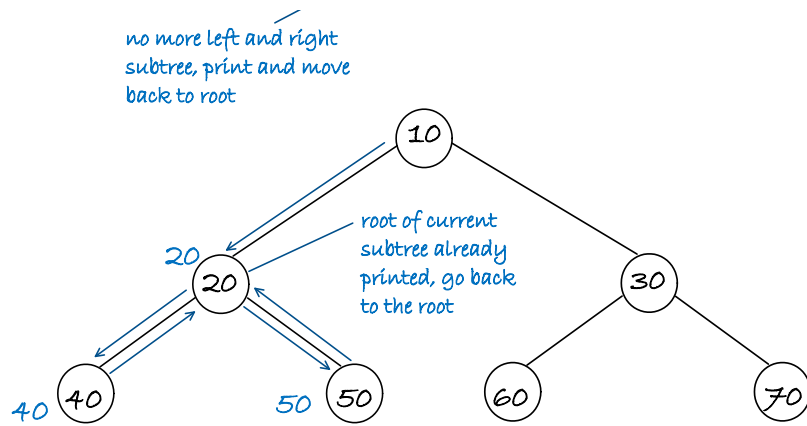
step (i) traverse the left subtree in in-order

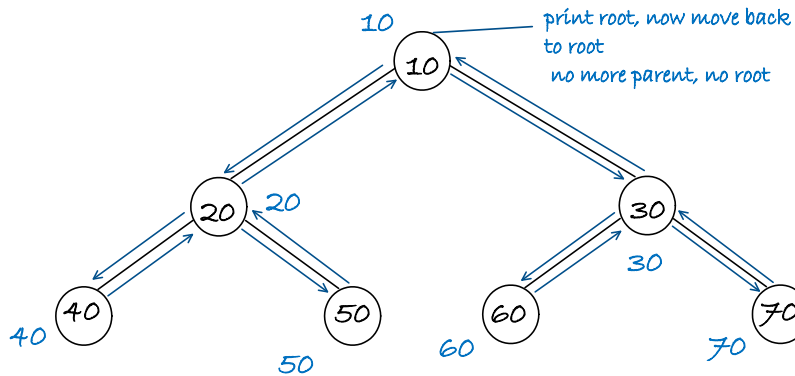
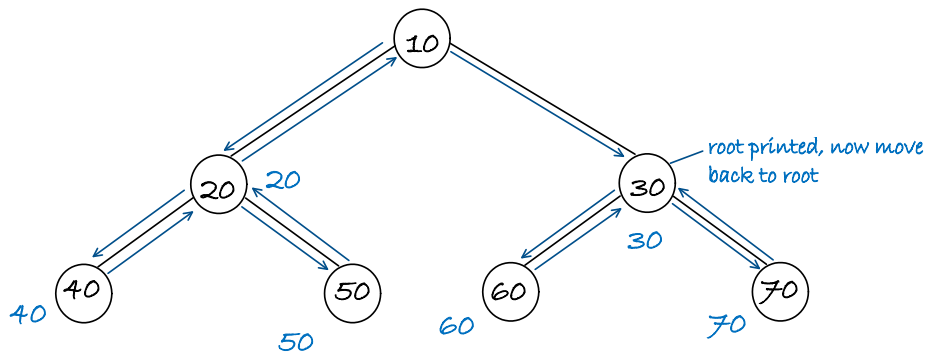
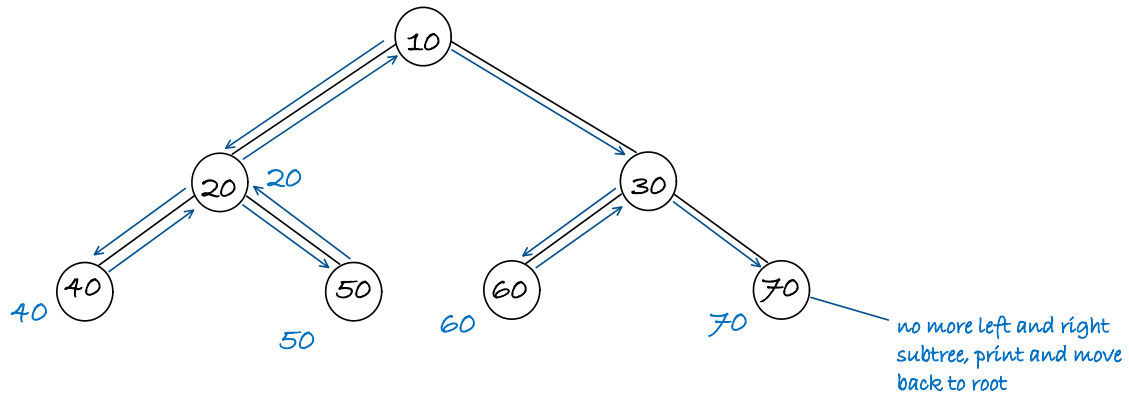
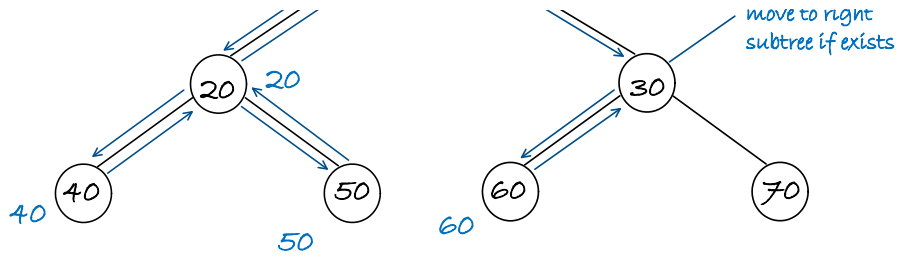
step (ii) traverse the right subtree in in-order

step (iii) visit/print/process root node



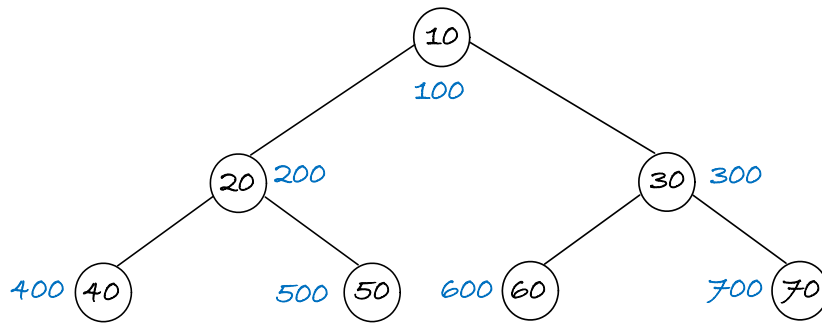




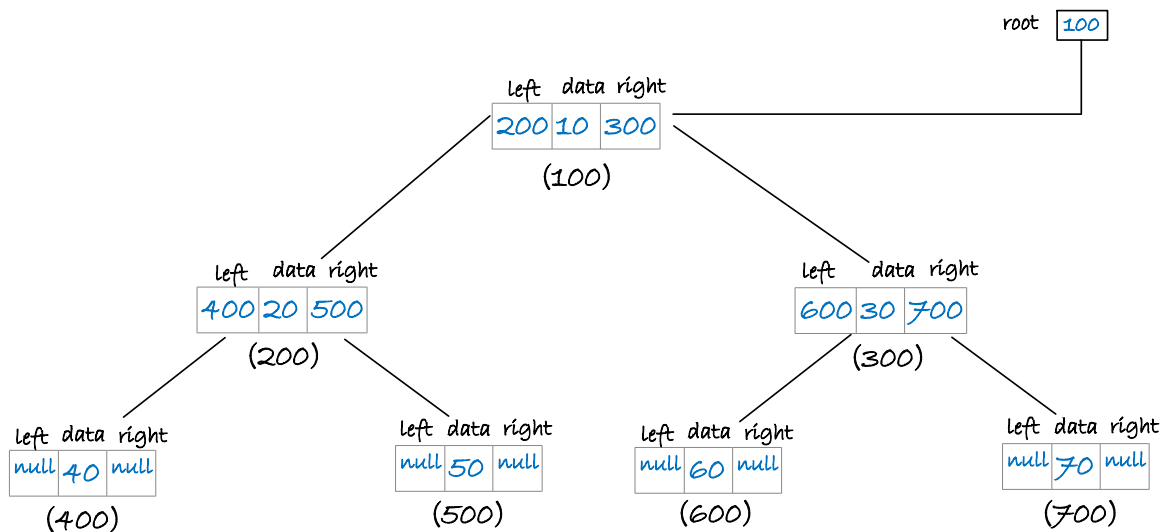


final output: 40 50 20 60 70 30 10

code for pre-order traversal:



as we discussed before in memory tree is in linked list so lets write the code for pre-order



logic and analysis:

(i) rootpoints to the node, so if root= null that means there is no tree so we must check the condition to ensure that tree exists

```
if (root==null){
return;
}
```

(ii) logic for pre-order traversal

as we have discussed before the way we access members in structures are by using arrow ptr->data to print data.

- process

step (i) visit/print/process root node

```
print ("%c", root->data);
```

step (ii) traverse the left subtree in in-order

```
preorder(root->left);
```

step (iii) traverse the right subtree in in-order

```
preorder(root->right);
```

```
(function for pre-order traversal)
void preOrder(struct Node* root) {
    if (root == NULL) {
        return;
    }
```

(traverse the left subtree)

```
preOrder(root->left);
```

(traverse the right subtree)

```
preOrder(root->right);
```

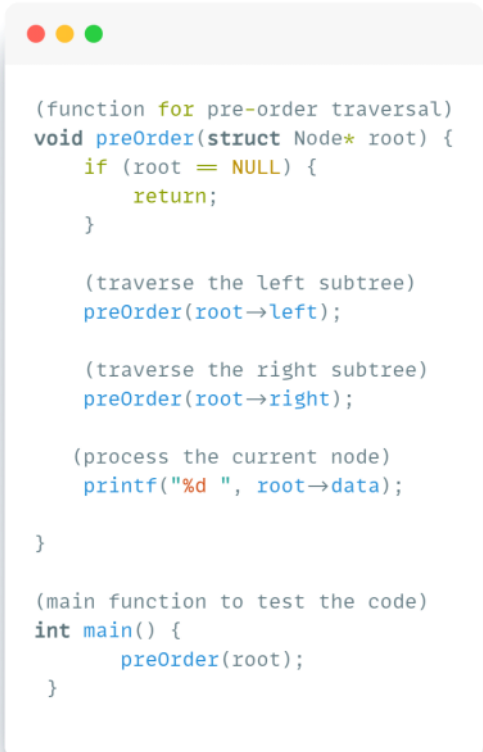
(process the current node)

```
printf("%d ", root->data);
```

```
}
```

(main function to test the code)

```
int main() {
    preOrder(root);
}
```



```
(function for pre-order traversal)
void preOrder(struct Node* root) {
    if (root == NULL) {
        return;
    }

    (traverse the left subtree)
    preOrder(root->left);

    (traverse the right subtree)
    preOrder(root->right);

    (process the current node)
    printf("%d ", root->data);
}

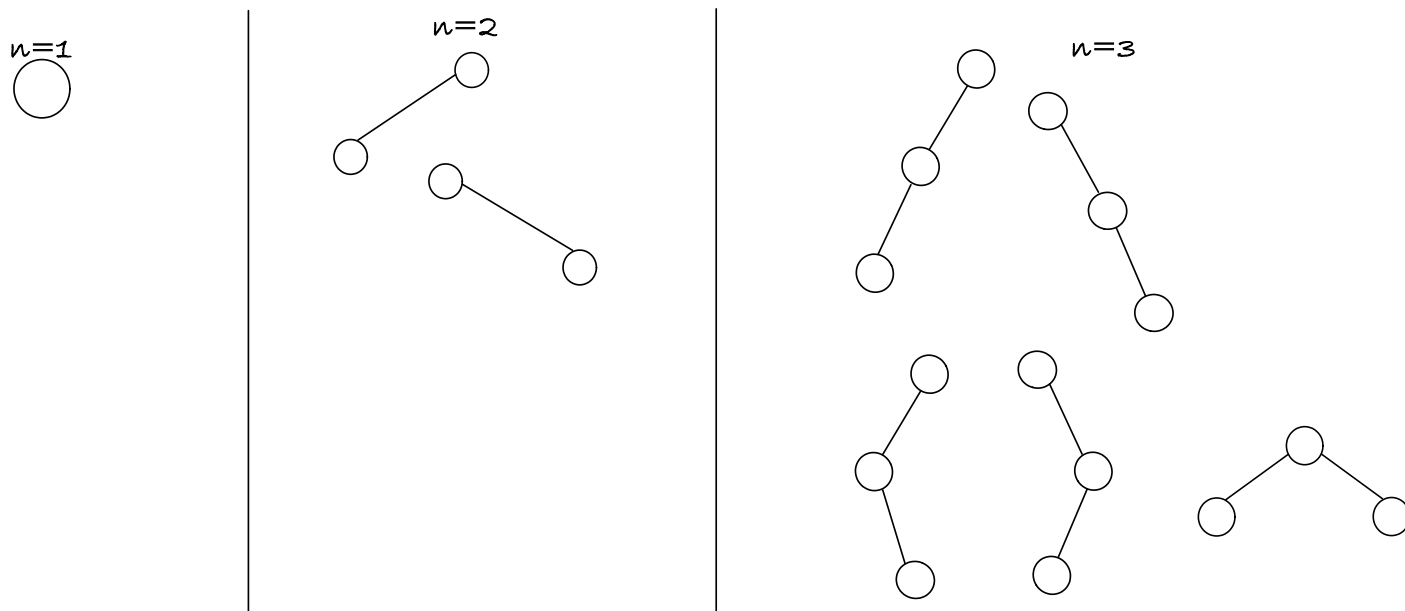
(main function to test the code)
int main() {
    preOrder(root);
}
```

concept: unlabelled and labelled

unlabelled: refers to structures where nodes (or vertices) do not have unique identifiers, meaning they are indistinguishable from each other.

labelled: refers to structure where nodes with unique identifiers, making it possible to differentiate between them.

concept: total number of unlabelled binary trees with n nodes

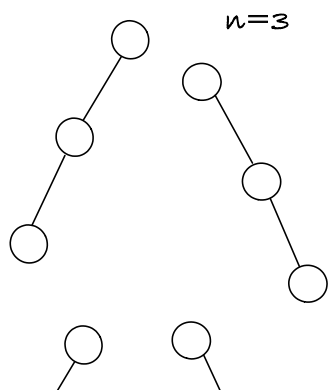


with any given traversal total number of unlabelled binary trees with n nodes $= \frac{2nC_n}{n+1}$ — catalyn number

with, $n = 3$

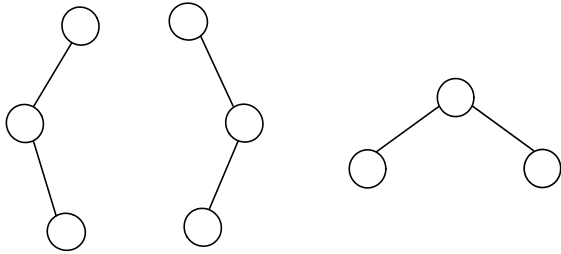
$$\text{total number of unlabelled binary trees with } 3 \text{ nodes} = \frac{2nC_n}{n+1} = \frac{2 \cdot 3C_3}{3+1} = \frac{6C_3}{4} = \frac{6!}{3! \cdot 3!} \times \frac{1}{4} = 5$$

$$ncr = \frac{n!}{n! (n-r)!}$$



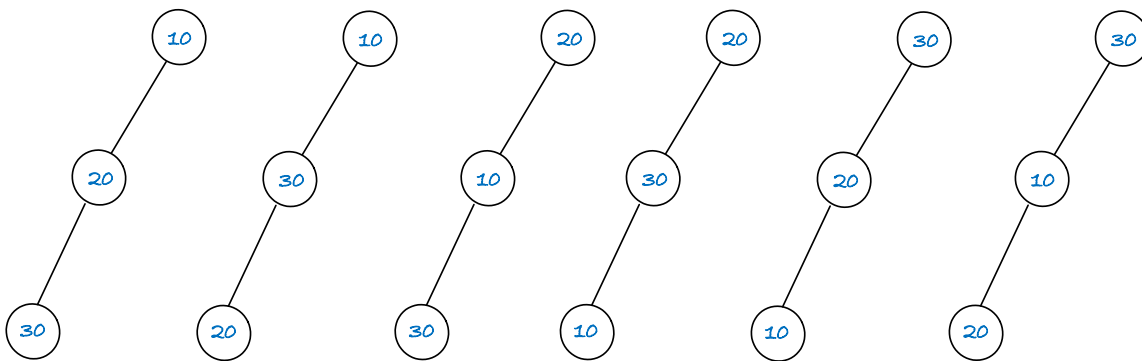
now consider any structure, how many ways we can label a single structure with 3 distinct keys?

with 3 distinct keys:



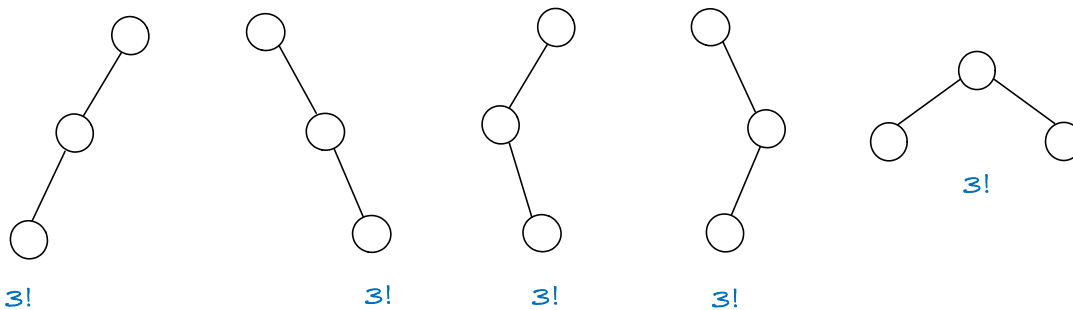
concept: number of labelled binary tree with n distinct keys

number of labelled binary tree with 3 distinct keys 10, 20, 30



3! for each structure

$n=3$



number of labelled binary tree with 3 distinct keys = (number of unlabelled binary trees with 3 nodes) $\times$ 3!

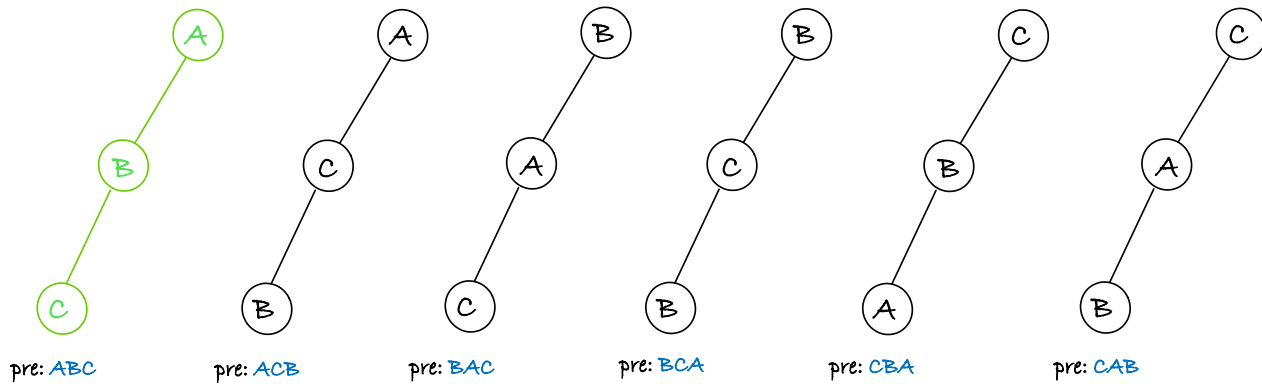
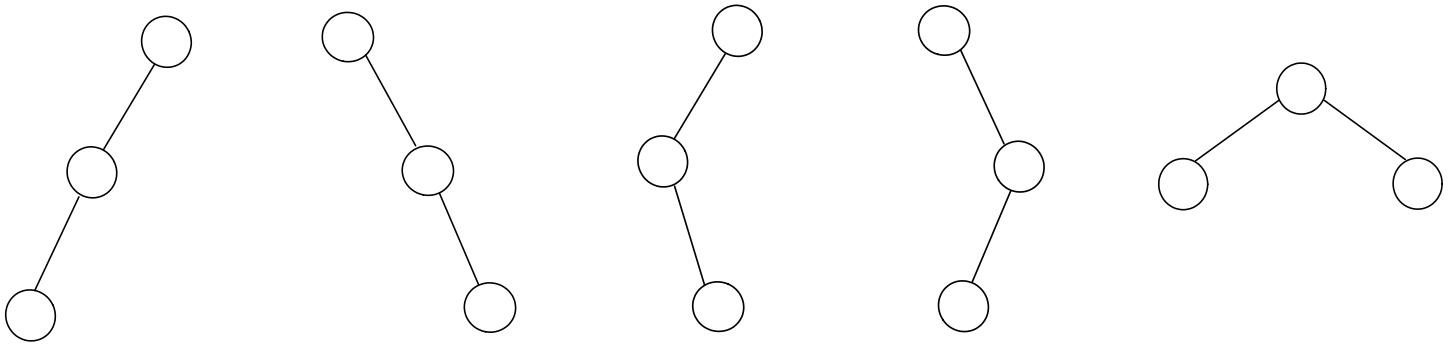
total number of unlabelled binary trees with 3 nodes = $\frac{2nC_n}{n+1} = \frac{2 \cdot 3C_3}{3+1} = \frac{6C_3}{4} = \frac{6!}{3! \times 3!} \times \frac{1}{4} = 5 \times 3!$

number of labelled binary tree with 3 distinct keys = $5 \times 3!$

number of labelled binary tree with 3 distinct keys = 30

total number of labelled binary trees with n distinct keys = $\left(\frac{2nC_n}{n+1} \right) n!$

concept: binary trees with pre-order ABC



for one structure only one such binary tree is possible.

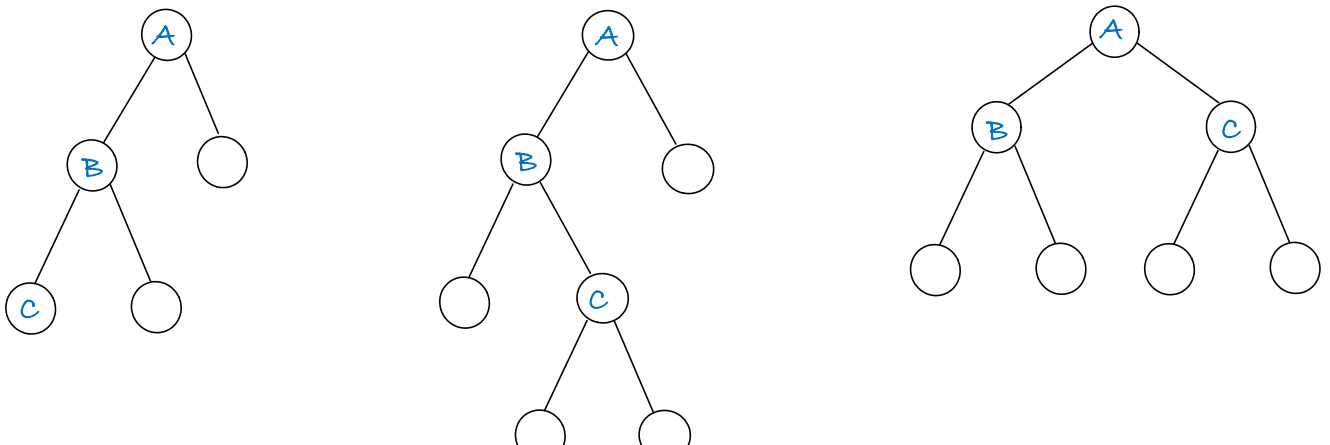
so $n=3$,

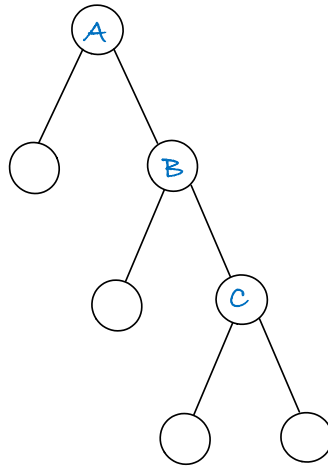
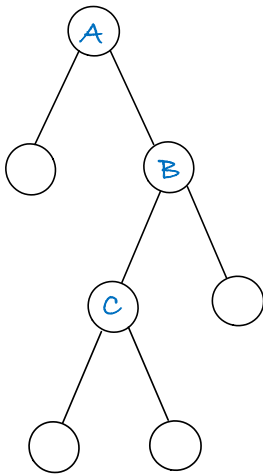
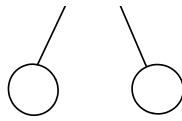
- 5 binary structures are possible and for each binary structure only 1 preorder with distinct keys are possible

so for 5 binary structure 5 binary tree structure possible with preorder ABC

number of binary tree possible with preorder ABC = 5

number of binary tree possible with preorder ABC = number of binary tree structure with 3 nodes

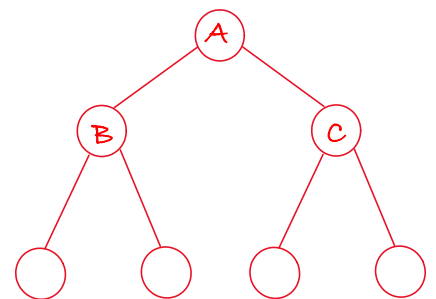
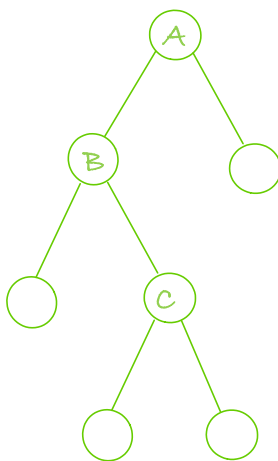
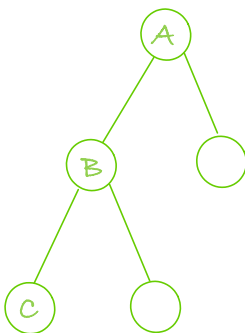




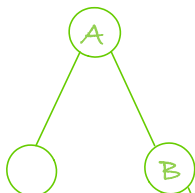
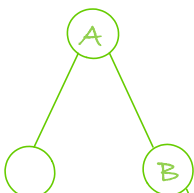
number of binary tree possible with a given distinct preorder of n length = $\left(\frac{2n C_n}{n+1}\right)$

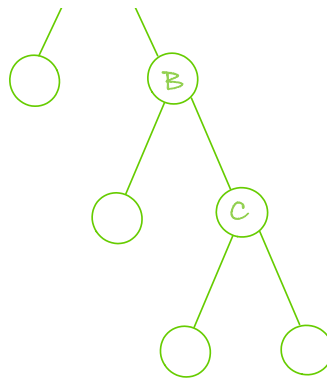
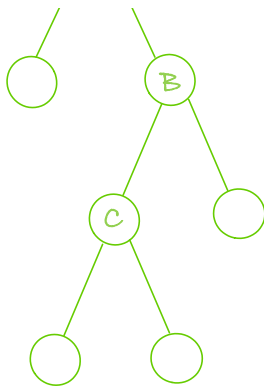
concept: number of binary tree that be constructed with given traversal

(i) number of binary trees possible with pre-order ABC and post-order CBA



post order: **BCA**
CBA not possible

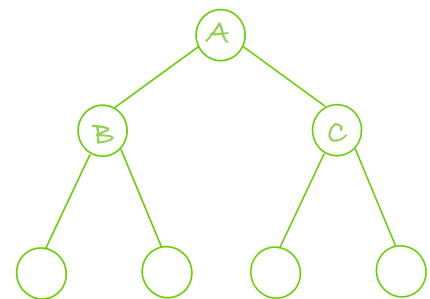
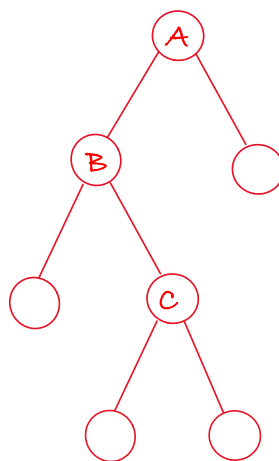
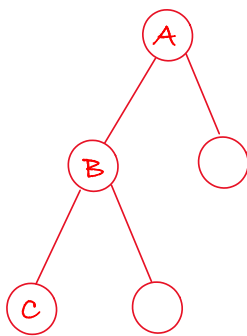




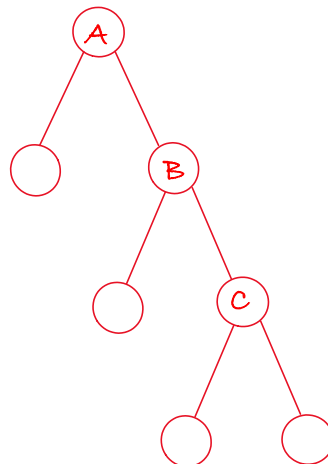
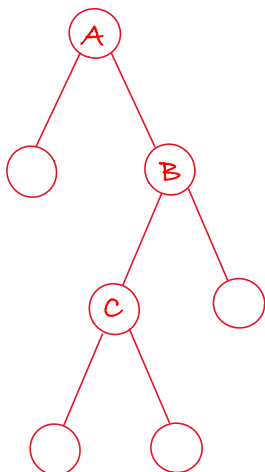
point to remember:

with valid pre-order (abc) and post-order (bca) traversals, multiple binary trees can exist because the traversals do not uniquely determine the tree structure. if the pre-order and post-order sequences are random, they may not correspond to a valid binary tree, resulting in zero or more possible trees.

(ii) number of binary trees possible with pre-order ABC and in-order BAC



in-order: BAC
only BAC possible



point to remember:

with a given pre-order and in-order traversal (or) a given post-order and in-order unique binary tree can be constructed.

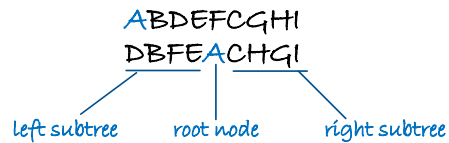
topic: how to construct a tree

(i) given a pre-order ABDEFCGHI and in-order DBFEACHGI

as per pre-order the first node is printed then we move to left subtree

ABDEFCGHI

A: root node



(i) root node: A



(ii) no root

left subtree

pre: BDEF

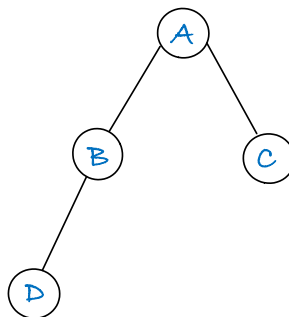
in: DBFE

D is in the left of B

right subtree

pre: CGHI

in: CHGI



(iii) no root

left subtree

pre: EF

in: FE

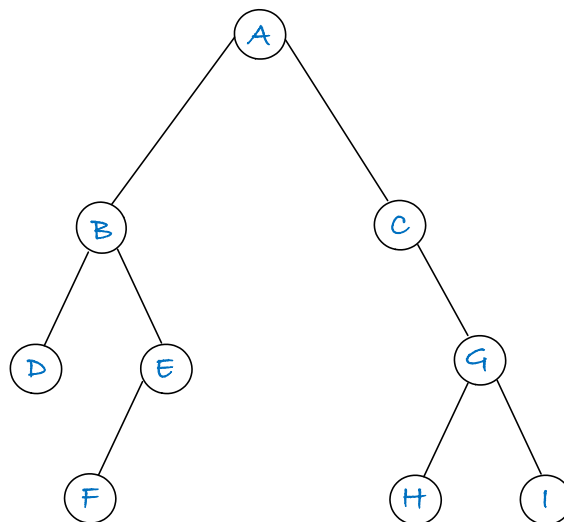
E is the right subtree and F is the left

right subtree

pre: GHI

in: HGI

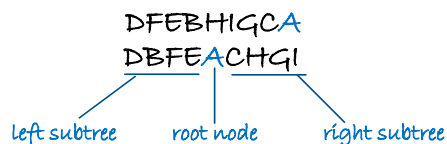
G is the root, H is the left and I is the right



point to remember:

check the pre-order and design according to in-order

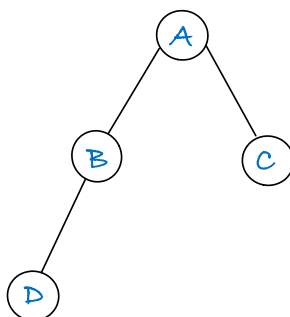
(ii) given a post-order DFEBHIGCA and in-order DBFEACHGI
 as per post-order the last node is printed then we move to left subtree
 DFEBHIGCA
 A: root node



(i) root node: A

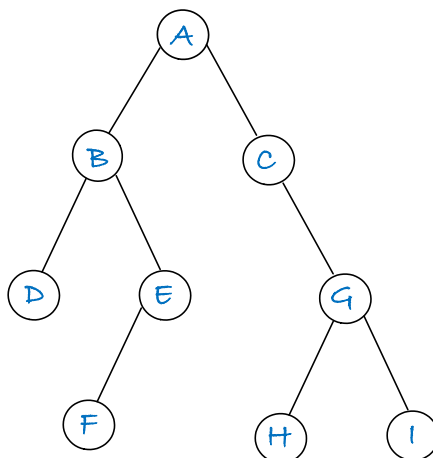


(ii) no root
 left subtree
 post: DFEB
 in: DBFE



right subtree
 post: HIG
 in: CHGI

(iii) no root
 left subtree
 post: FE
 in: FE



right subtree
 post: HIG
 in: HGI

point to remember:

check the post-order and design according to in-order

topic: complete binary tree

a complete binary tree is a binary tree in which all levels, except possibly the last, are completely filled.

properties

(i) node arrangement:

- nodes are filled level by level from left to right.
- the last level may not be completely filled, but any missing nodes are always on the right.

efficient storage:

- a complete binary tree can be efficiently stored in an array without any gaps.

applications

(i) heap data structure:

- a heap (min-heap or max-heap) is implemented as a complete binary tree.

(ii) searching and sorting:

- binary heaps (used in heapsort) are complete binary trees.

(iii) efficient storage:

- complete binary trees are stored in arrays for compact representation without pointers.

why complete binary trees are important

(i) space efficiency:

- minimizes unused memory since nodes are packed left to right.

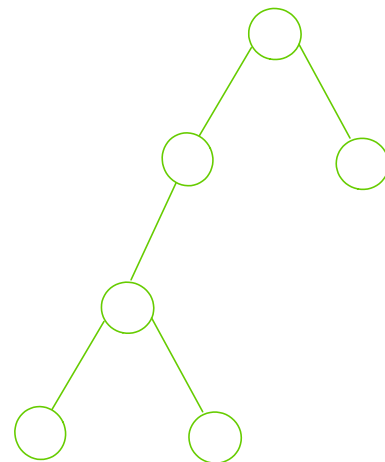
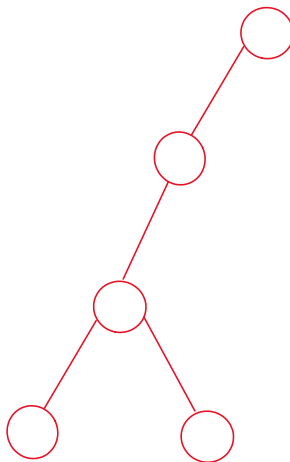
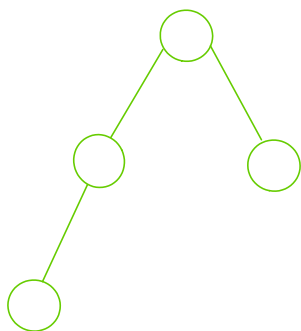
(ii) balanced structure:

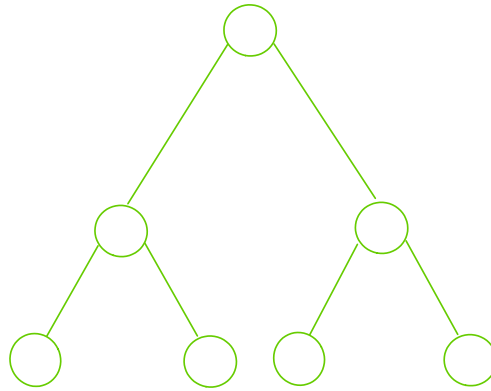
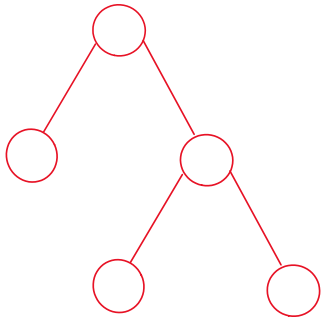
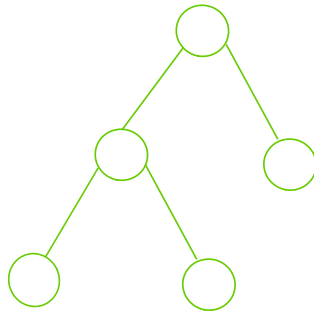
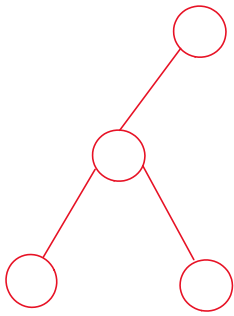
- ensures operations like insertion, deletion, or traversal are efficient (time complexity $O(\log n)$).

(iii) simplicity in implementation:

- can be easily represented in arrays, making algorithms like heapsort simple and efficient.

some examples of invalid and valid complete binary tree:

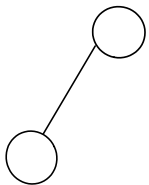




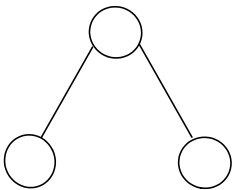
(a) structure of cbt with one node



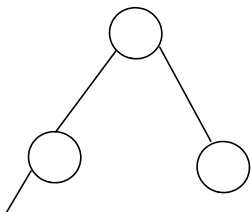
(b) structure of cbt with two node

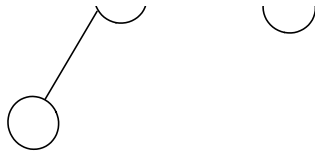


(c) structure of cbt with three node



(d) structure of cbt with four node





point to remember:

the structure of cbt with k nodes is always fixed (unique)

minimum number of nodes in a cbt of 'h' height = 2^h

maximum number of nodes in a cbt of 'h' height = $2^{h+1}-1$

minimum height of a complete binary tree = $\log_2(n+1)-1$

maximum height of a complete binary tree = $\log_2(n)$

explanation:

1. minimum number of nodes in a CBT of height 'h':

- A CBT with height h can have as few as 2^h nodes when only the root node and one node on each level are filled, making the tree unbalanced but still a valid CBT.

2. maximum number of nodes in a CBT of height 'h':

- The maximum number of nodes in a CBT of height h occurs when the tree is completely filled at all levels, which is equal to $2^{h+1} - 1$ nodes.

3. minimum height of a CBT:

- The minimum height of a CBT is calculated based on the total number of nodes, n . The formula $\log_2(n+1) - 1$ gives the height assuming the tree is as filled as possible.

4. maximum height of a CBT:

- The maximum height occurs when the tree is not completely filled and is as unbalanced as possible. The height is given by $\log_2(n)$.

why and how:

- The minimum and maximum formulas for nodes and height depend on how full the tree is. A complete binary tree can range from very compact (with all levels filled) to highly unbalanced (with only a few nodes per level).
- Using $\log_2(n)$ and $\log_2(n+1)$ helps determine the height because the logarithmic function represents the growth rate of a binary tree.

topic: binary search tree

a binary search tree (BST) is a type of binary tree where each node has at most two children, referred to as the left and right child. the key property that makes it a search tree is that for each node:

- all the values in the left subtree are less than the node's value.

- all the values in the right subtree are greater than the node's value.

this property makes searching for a value efficient, as it allows for a divide-and-conquer approach.

properties of a binary search tree:

- each node has at most two children: a left child and a right child.
- ordered structure:
 - for any node:
 - left subtree: contains values less than the node.
 - right subtree: contains values greater than the node.

no duplicate values: in most implementations, duplicate values are not allowed in a BST.

operations on a binary search tree:

(i) search:

to search for a value, start from the root and:

- if the value is smaller, move to the left subtree.
- if the value is larger, move to the right subtree.
- repeat until you find the value or reach a leaf node (i.e., a node with no children).

(ii) insert:

to insert a new value:

- start from the root and move left or right based on the value being inserted (as described above).
- once the correct position is found (i.e., an empty spot), insert the new node there.

(iii) delete:

deleting a node can be done in three cases:

- node is a leaf: simply remove the node.
- node has one child: replace the node with its only child.
- node has two children: find the in-order successor (the smallest node in the right subtree) or in-order predecessor (the largest node in the left subtree), and replace the node with that. Then delete the successor/predecessor.

(iv) traversals:

- in-order traversal (left-root-right): returns values in ascending order.
- pre-order traversal (root-left-right): visits the root before the children.
- post-order traversal (left-right-root): visits the root after the children.

time complexity of operations:

- search: $O(\log n)$ on average, $O(n)$ in the worst case (if the tree is unbalanced).
- insert: $O(\log n)$ on average, $O(n)$ in the worst case.
- delete: $O(\log n)$ on average, $O(n)$ in the worst case.
- traversals: $O(n)$ for all types (since every node must be visited).

why and how:

(i) efficiency: the BST allows for fast searching, insertion, and deletion because you can "skip" half of the tree based on comparisons at each step. this makes it much faster than searching through an unsorted list or array.

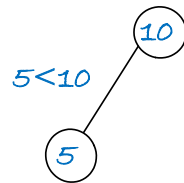
(ii) balanced vs. unbalanced trees: if a BST becomes unbalanced (i.e., the tree takes on a shape similar to a linked list), the performance can degrade to $O(n)$. this is why balanced versions of BSTs (like AVL trees or red-black trees) are often used in practice.

example: construct a bst by inserting keys 10, 5, 2, 9, 3, 4, in order into an initially empty BST

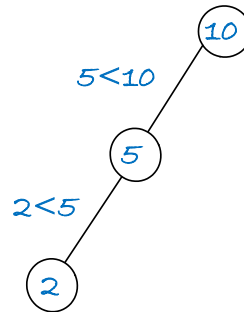
(i) insert 10



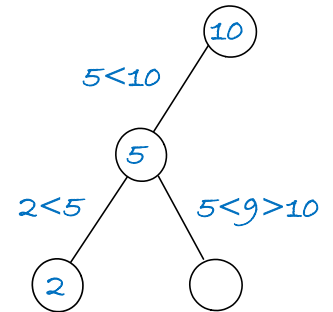
(ii) insert 5



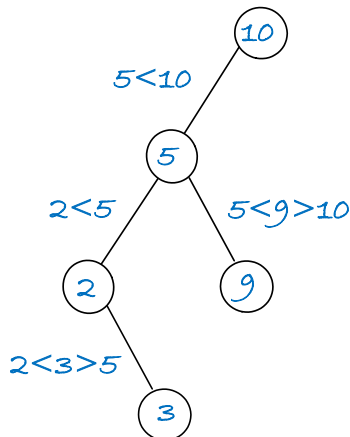
(iii) insert 2



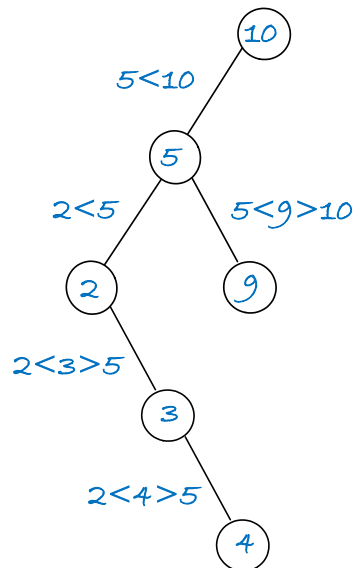
(iv) insert 9



(v) insert 3

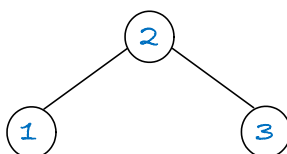


(vi) insert 4



number of BST when the insertion order of keys are fixed : 1

example: construct a bst by inserting keys 1, 2, 3 in any order into an initially empty BST

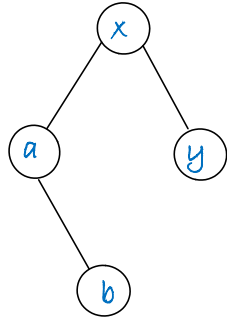


(q) given a binary tree structure with n nodes and also n distinct keys. in how many ways we can fill the structure so that it becomes bst.

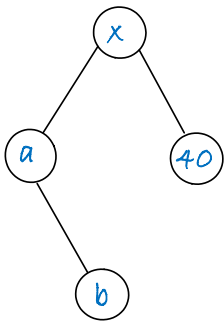
$n=4$

keys: 10, 20, 30, 40

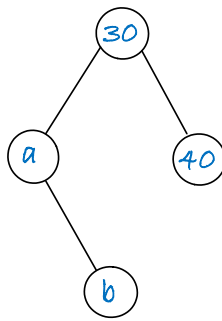
$a, b < x < y$



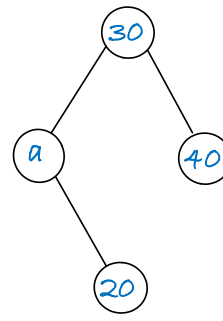
$a, b < x < y$
y: largest



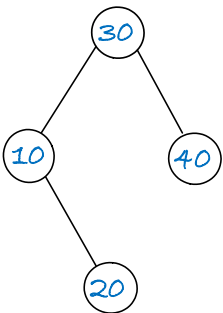
$a, b < x < y$
x: smaller than y



$a, b < x < y$
b smaller than x but greater than a



$a, b < x < y$



only one possibility.

number of bst's with distinct n keys = $\frac{2n C_n}{n+1}$

(i) if bst pre-order is given we can find the post order?

steps:

(i) rearranging the keys into ascending order:

- since the keys are distinct, rearranging them in ascending order will give you the in-order traversal of the BST.
- the reason for this is that in a binary search tree (BST), the in-order traversal visits the nodes in

ascending order.

(ii) in-order traversal:

- in the BST, the in-order traversal of the tree visits the nodes in the left subtree first, then the root, and then the right subtree. Since you've arranged the keys in ascending order, this gives you the exact in-order sequence.

(iii) reconstructing the tree:

- once you have the in-order sequence, you can use the pre-order sequence (which is given) to reconstruct the tree.
- pre-order traversal visits the root node first, then the left subtree, and finally the right subtree. Using this, you can identify the root and recursively build the left and right subtrees.
- by finding the root from the pre-order sequence and splitting the in-order sequence into left and right subtrees, you can reconstruct the entire tree.

(iv) post-order traversal:

- once the tree is reconstructed, you can then perform a post-order traversal to obtain the desired sequence.
- post-order traversal visits the left subtree, then the right subtree, and finally the root node.

why and how:

- the process works because the in-order traversal gives the correct arrangement of keys in a BST, and the pre-order traversal provides the sequence in which nodes are visited (starting with the root). combining these two allows us to reconstruct the tree.
- once the tree is reconstructed, performing the post-order traversal gives the sequence you need.

(ii) if bst post-order is given we can find the pre-order?

steps:

(i) understand the post-order traversal:

- in post-order traversal, we visit the left subtree first, then the right subtree, and finally the root node.
- so, the last element in the post-order sequence is always the root of the BST.

(ii) reconstruct the tree:

- start by identifying the root from the last element of the post-order sequence.
- once the root is identified, partition the remaining nodes into the left subtree and right subtree based on the BST property:
 - all nodes smaller than the root go into the left subtree.
 - all nodes greater than the root go into the right subtree.

(iii) recursively apply the same approach:

- for each subtree, repeat the process:
 - identify the root (last element of the post-order for that subtree).
 - partition the remaining nodes into left and right subtrees based on the root.
 - continue recursively for left and right subtrees.

(iv) generate the pre-order traversal:

- once the tree is reconstructed, generate the pre-order traversal by visiting the root node first, then the

left subtree, and then the right subtree.

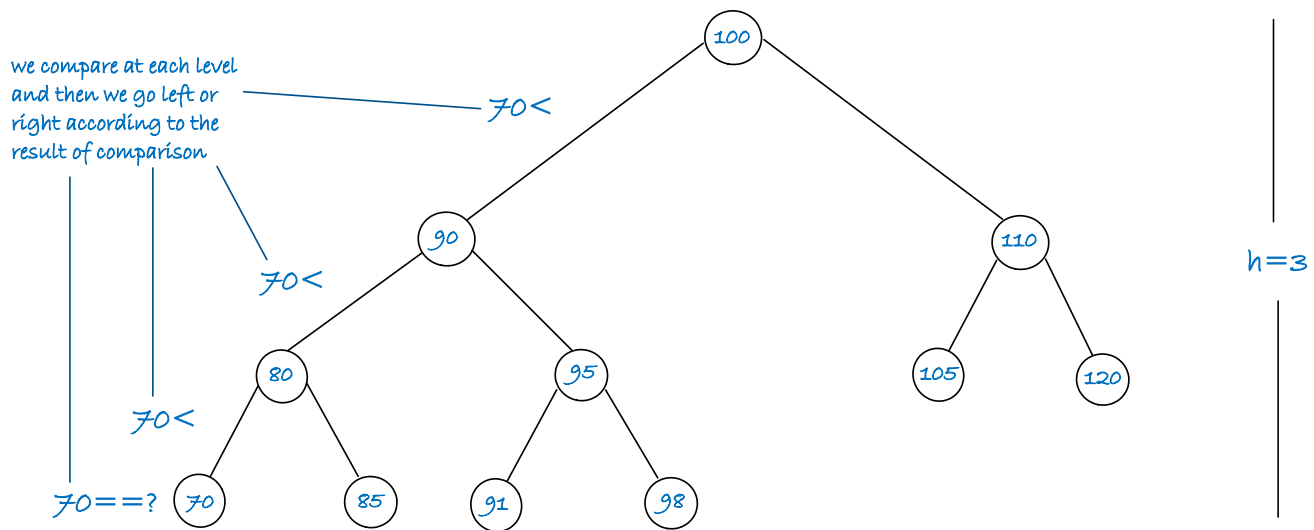
operation (i): search in a bst

to search for a value, start from the root and:

- if the value is smaller, move to the left subtree.
- if the value is larger, move to the right subtree.
- repeat until you find the value or reach a leaf node (i.e., a node with no children).

- find the key 70.

how many comparisons we needed?



height of the tree = $h(3)$

number of comparisons needed to find the key = 4

number of comparisons needed to find the key = $h(3) + 1$

number of comparisons needed to find the key = $O(h)$

concept: skewed tree

a skewed tree is a special type of binary tree in which all the nodes have only one child.

there are two types of skewed trees based on the direction of the child:

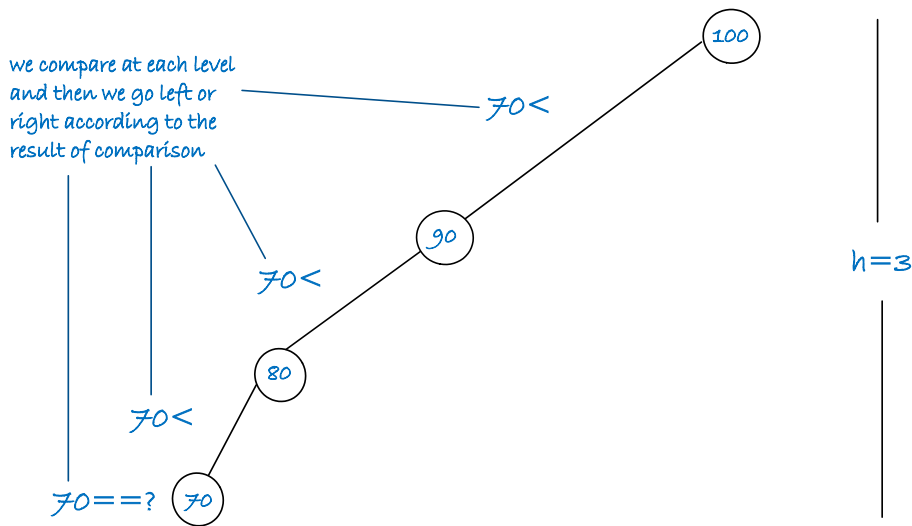
(i) left skewed tree:

- in a left skewed tree, every node has only a left child.
- this means that each node points to its left child, and the right child of every node is null.
- the tree grows only on the left side, and it essentially becomes a linear structure like a linked list.

(ii) right skewed tree:

- in a right skewed tree, every node has only a right child.
- this means that each node points to its right child, and the left child of every node is null.

- the tree grows only on the right side, again forming a linear structure like a linked list.



height = number of nodes

search time complexity

full binary tree/complete binary tree

height = $O(\log_2 n)$

number of comparison = $O(\log_2 n)$

in bst,

complexity of search

(i) best case (bst is balanced): $O(\log_2 n)$

(ii) worst case (bst is not balanced/skewed): $O(n)$

because you may need to search through every node in the tree, as there is no balance in the structure.

skewed tree

height = n

number of comparison = $O(n)$

in bst,

complexity of search

(i) no best case.

(ii) no worst case.

$O(n)$ because you may need to search through every node in the tree, as there is no balance in the structure.

operation (ii): insert

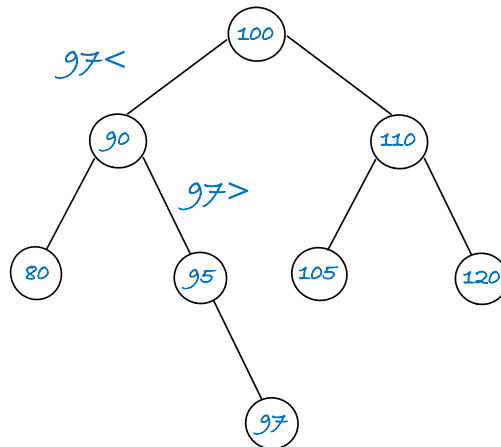
to insert a new value:

- search that particular number because if that number already exists then we cannot insert it.
- start from the root and move left or right based on the value being inserted.
- once the correct position is found (i.e., an empty spot), insert the new node there.

insert 97

100

insert 97



point to remember:

remember that in the memory we use linked lists for trees so if we are inserting the node the process of insertion of node is being followed as we discussed in previous chapter. create a new node with malloc and assigning it.

insertion time complexity

full binary tree/complete binary tree

height = $O(\log_2 n)$

number of comparison = $O(\log_2 n)$

in bst,

complexity of insertion

(i) best case (bst is balanced): $O(\log_2 n)$

(ii) worst case (bst is not balanced/skewed): $O(n)$

because you may need to search through every node in the tree, as there is no balance in the structure.

skewed tree

height = $O(n)$

number of comparison = $O(n)$

in bst,

complexity of insertion

(i) no best case.

(ii) no worst case

$O(n)$

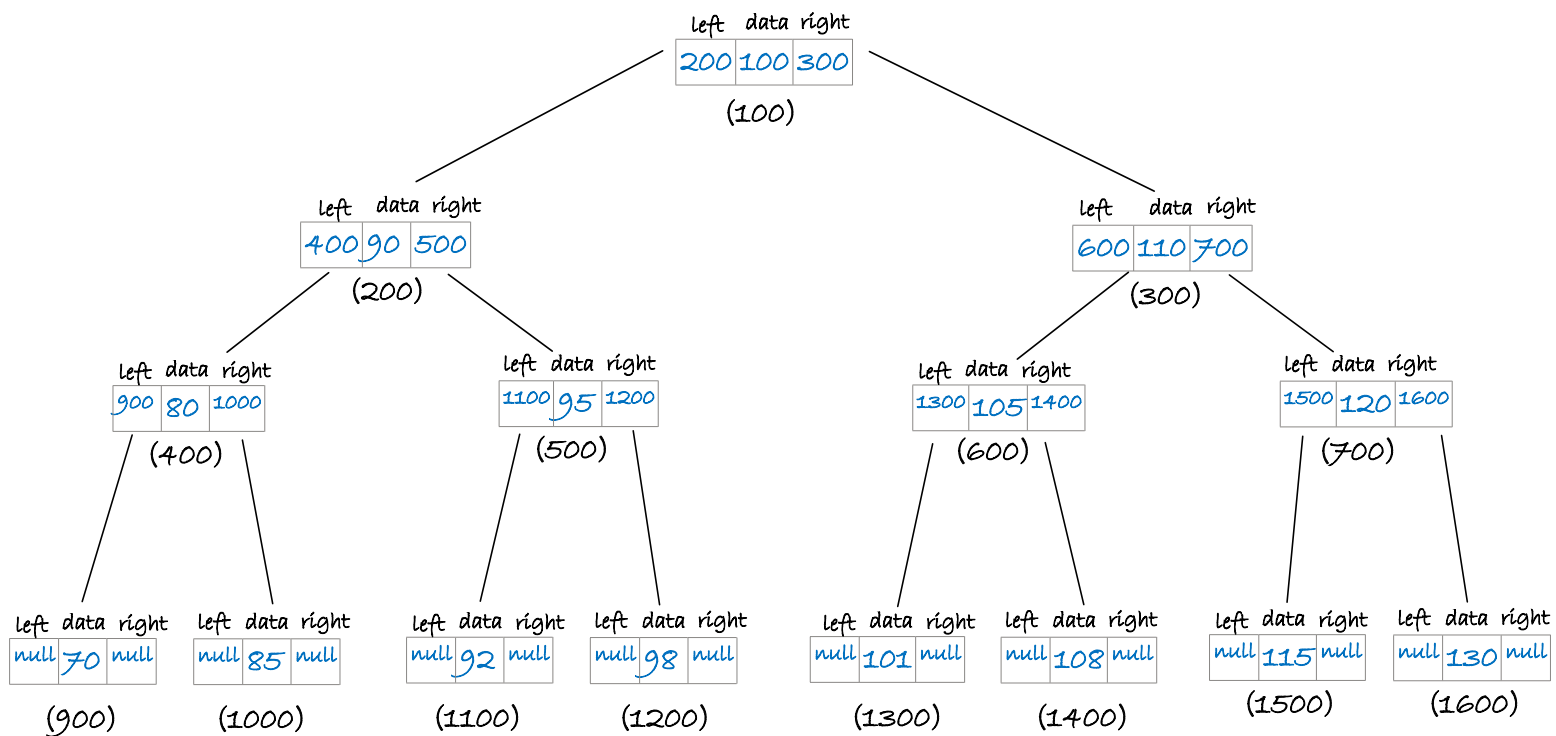
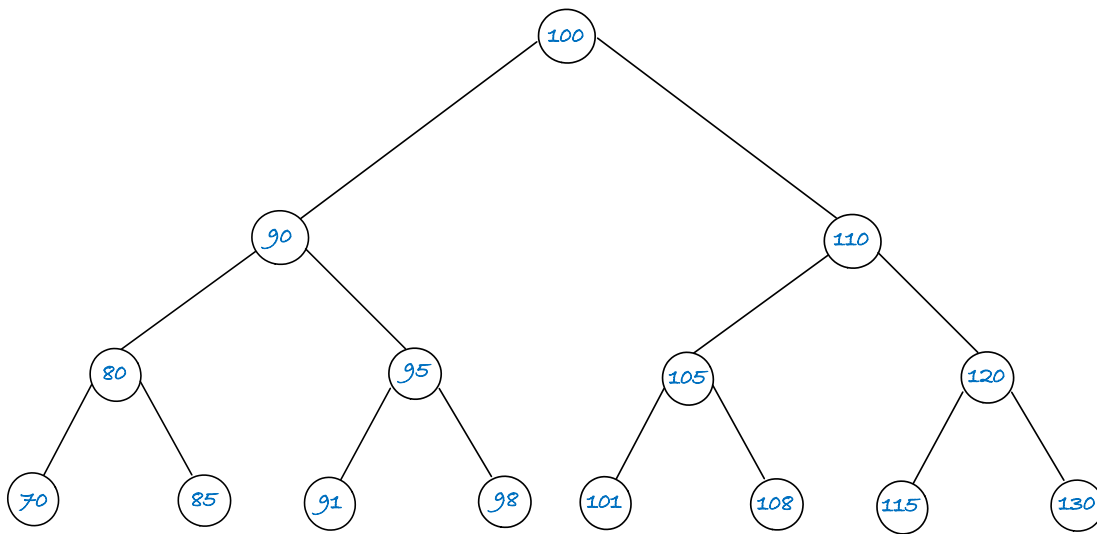
because you may need to search through every node in the tree, as there is no balance in the structure.

operation (iii): deletion from bst

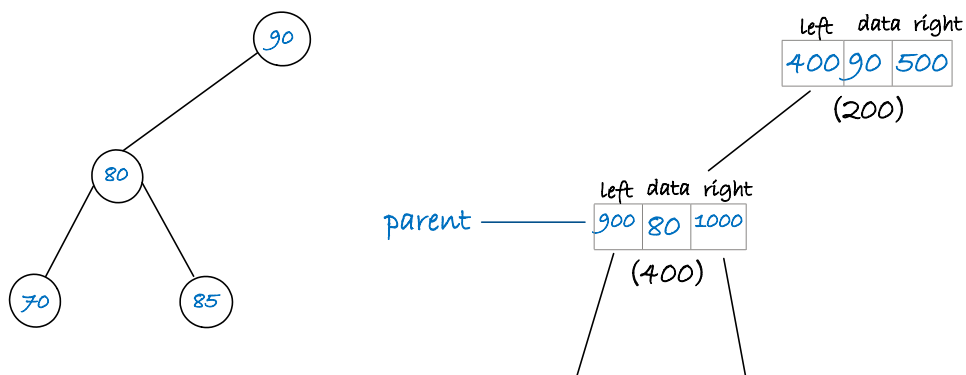
deleting a node can be done in three cases:

- node is a leaf: simply remove the node.
- node has one child: replace the node with its only child.
- node has two children: find the in-order successor (the smallest node in the right subtree) or in-order predecessor (the largest node in the left subtree), and replace the node with that. Then delete the successor/predecessor.

(i) deletion of leaf node

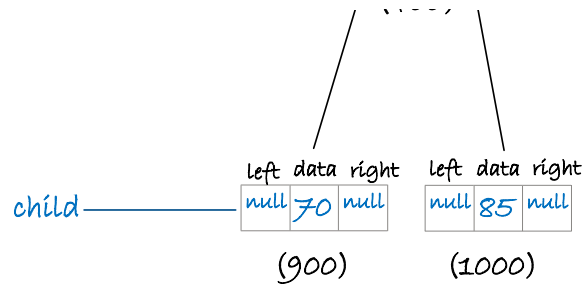


(i) identify the parent pointer (of the node to be deleted), which is pointing to deleted node.

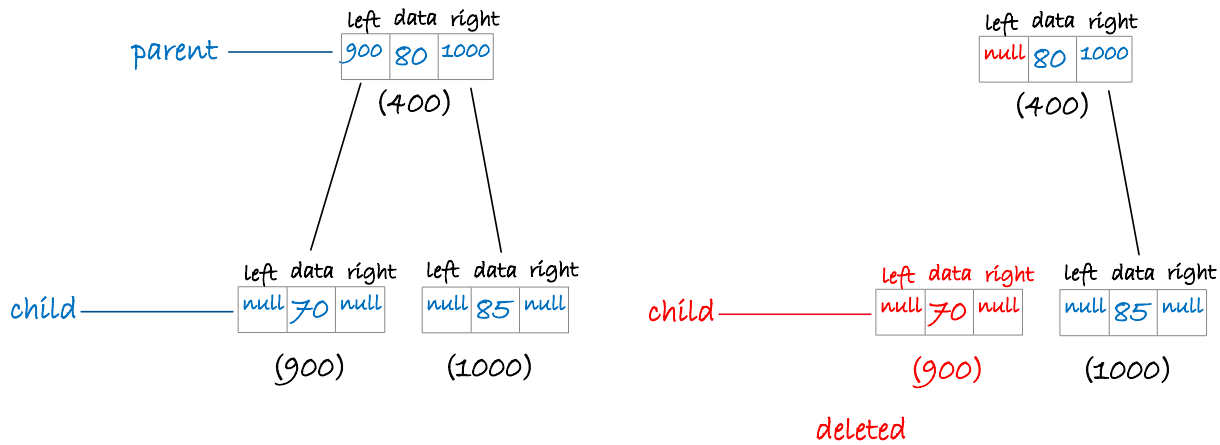


(70)

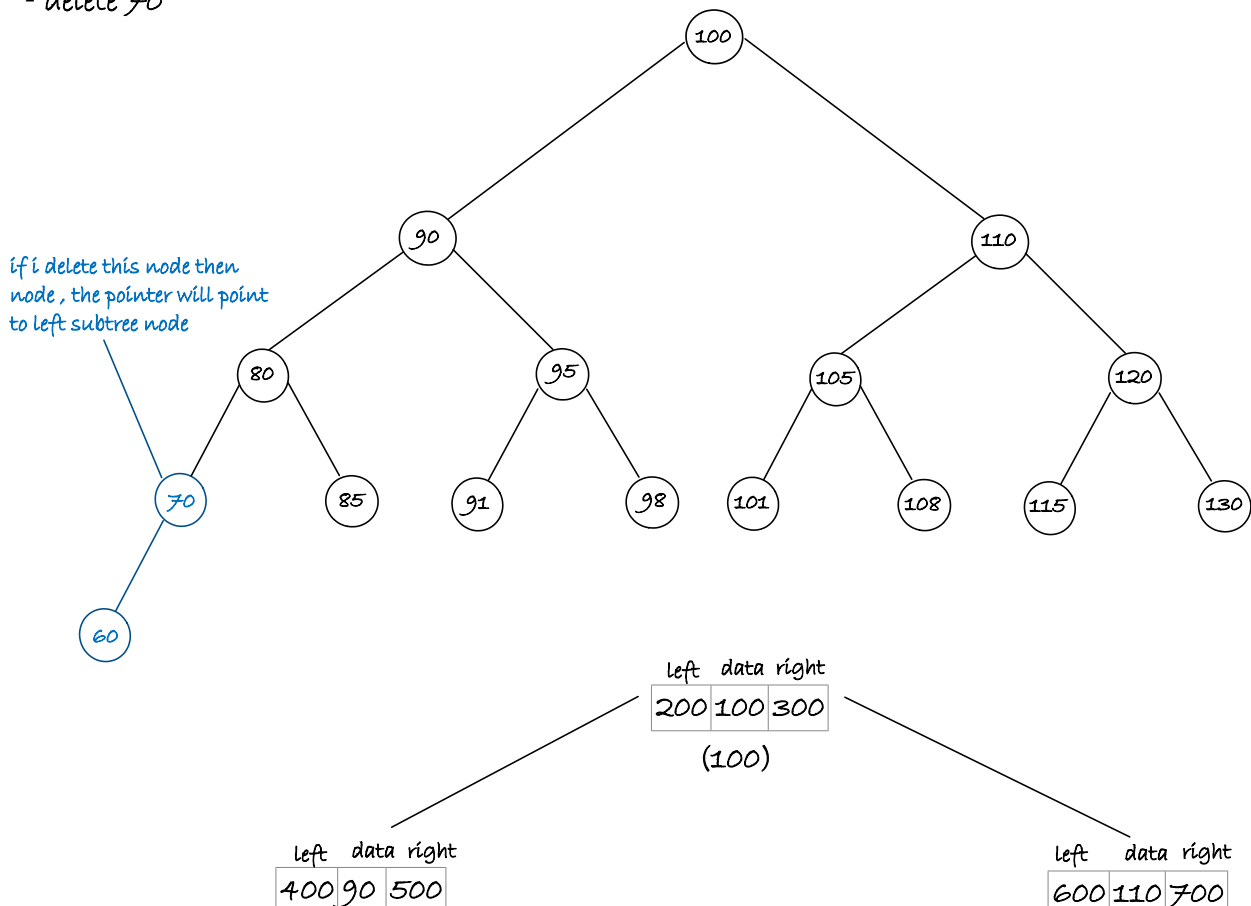
(85)

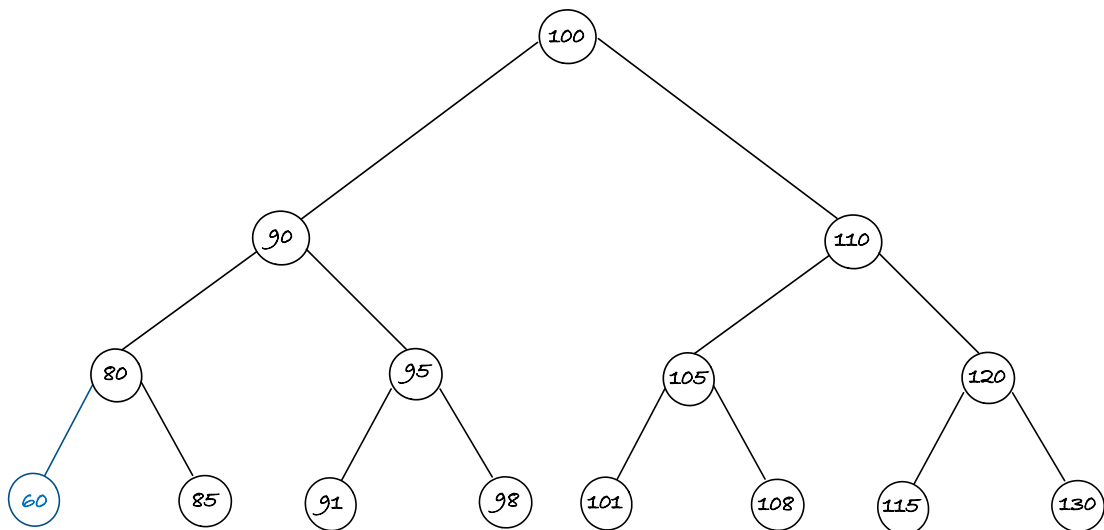
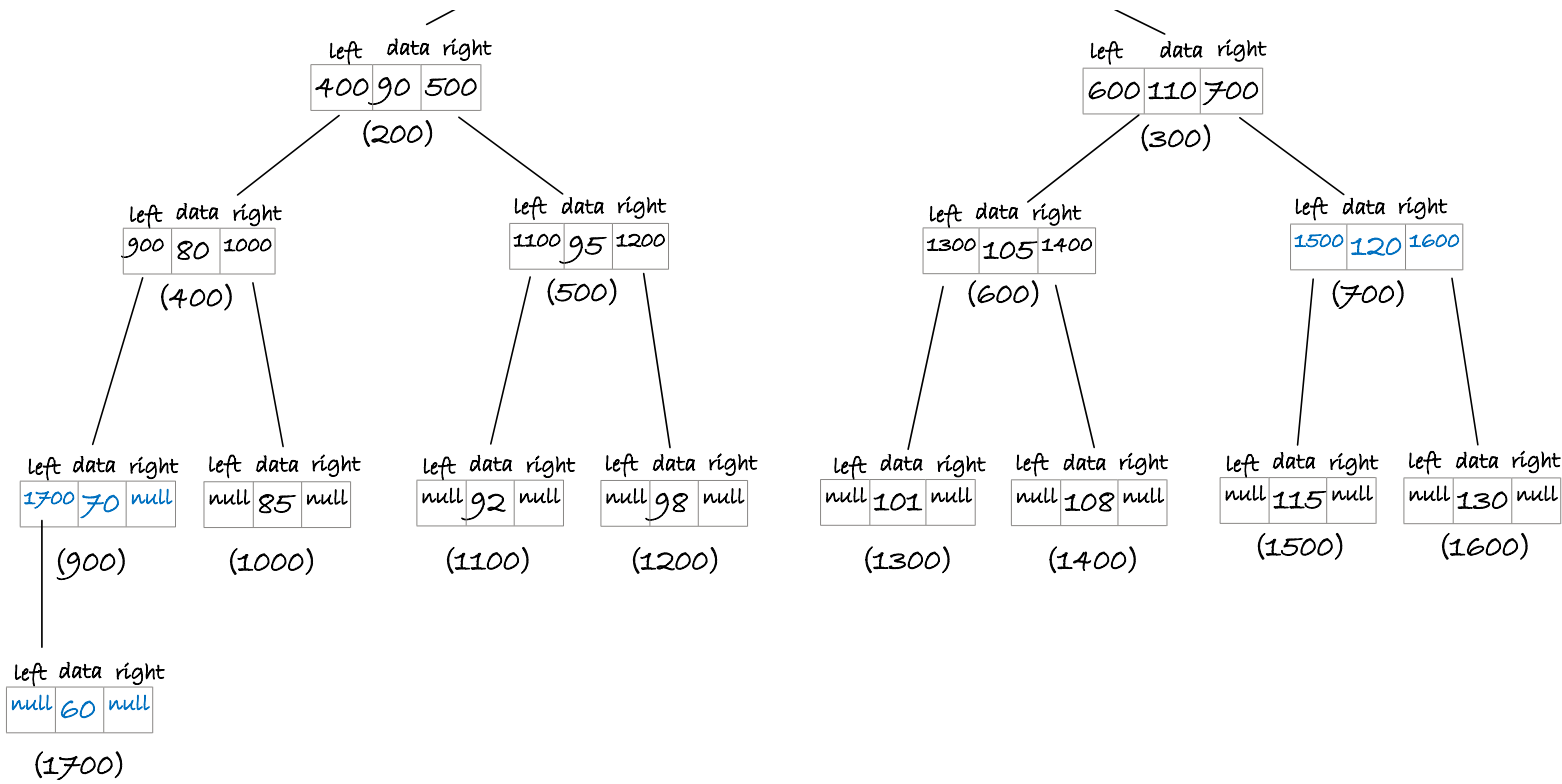


(ii) set this parent pointer to null

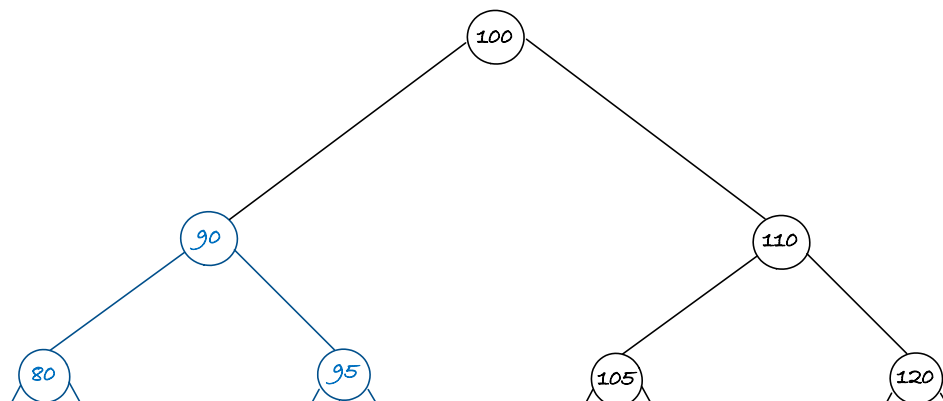


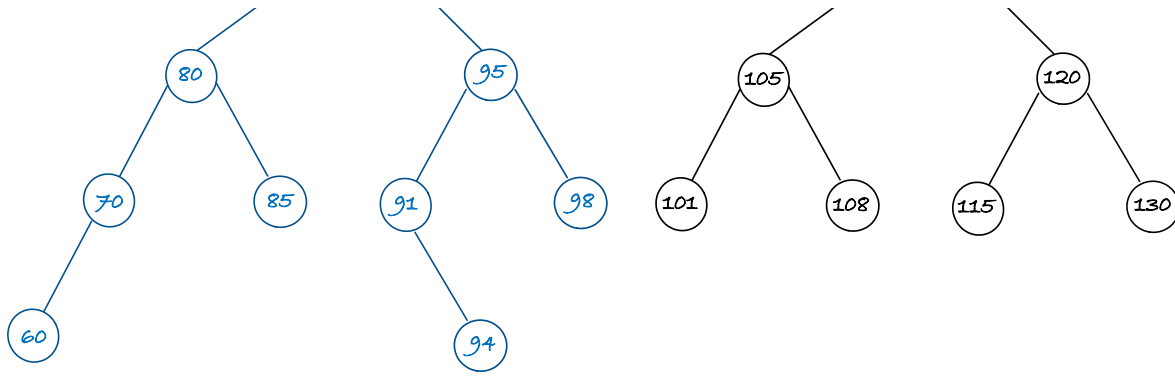
(ii) deletion of node with exactly one child
- delete 70





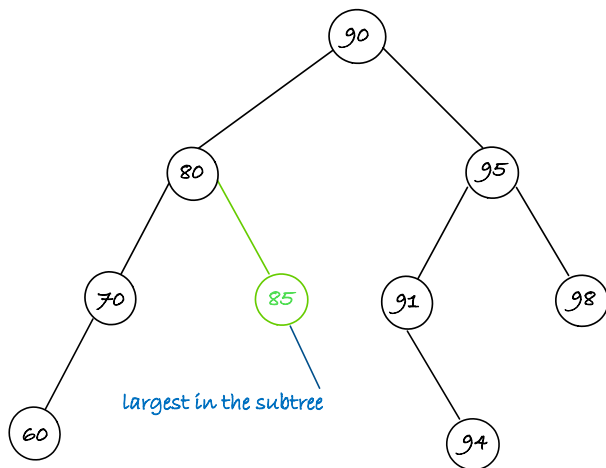
(iii) deletion of node with exactly two child
- delete 90



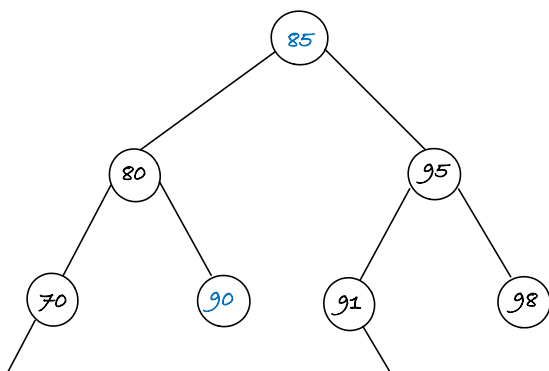


we have to protect both of the childs

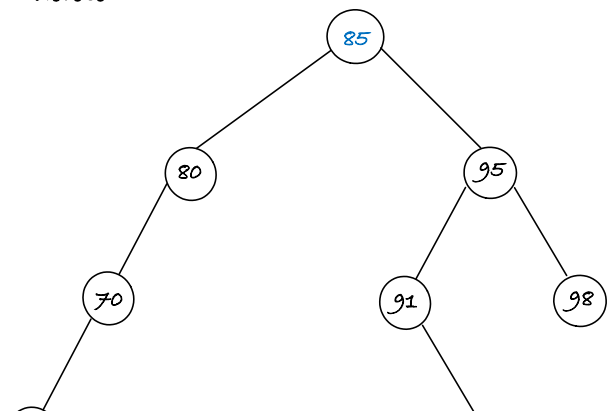
case(i) replace the key to be deleted by the largest key in the subtree of the node to be deleted and then perform the deletion

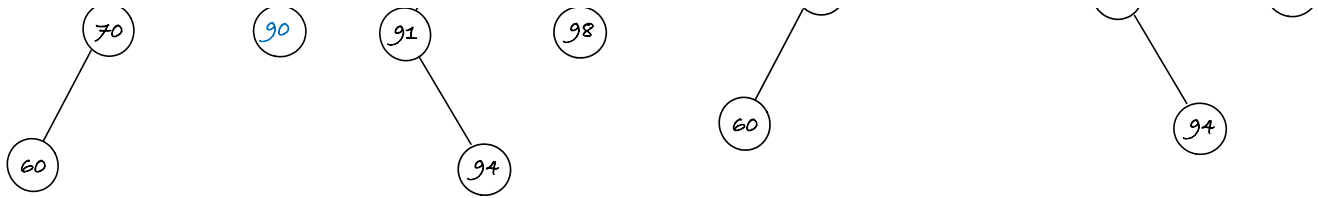


- swap



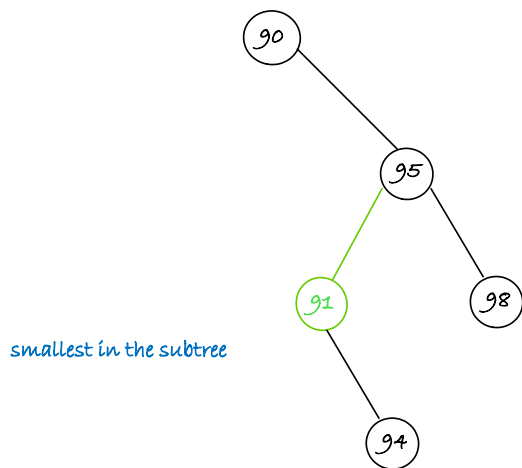
- delete



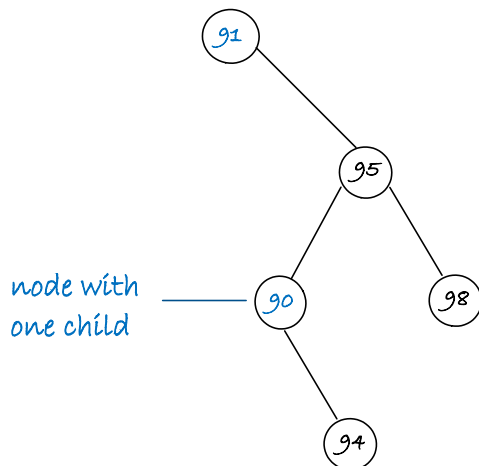


logic: i have placed the largest key in the left subtree on root, so the remaining keys are still of less values and when 90 is deleted there will be no problem.

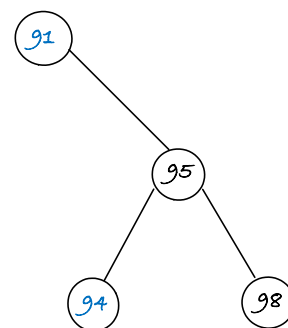
case(ii) replace the key to be deleted by the smallest key in the right subtree of node to be deleted and then perform deletion.



- swap



- delete



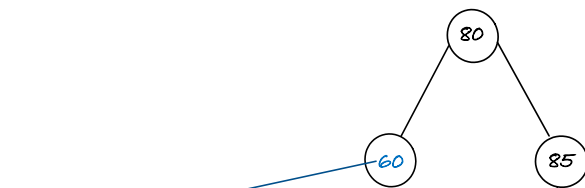
logic: i have placed the smallest key in the right subtree on root, so the remaining keys are still of more values and when 90 is deleted there will be no problem.

point to remember:

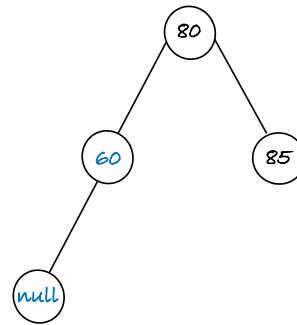
and when 90 is deleted there will be no problem.

point to remember:

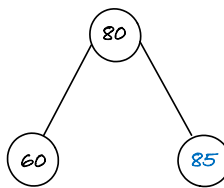
(i) node with minimum value can have 0 or 1 child



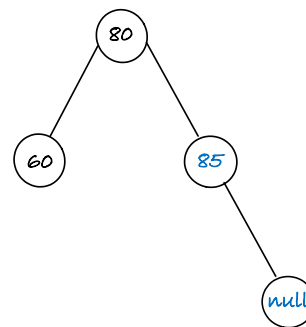
if i consider this to be minimum then left child will be null because only values lesser than 60 can be inserted in left child in bst.



(ii) node with maximum value can have 0 or 1 child



if i consider this to be maximum then right child will be null because only values more than 85 can be inserted in right child in bst.



deletion time complexity

full binary tree/complete binary tree

height = $O(\log_2 n)$

number of comparison = $O(\log_2 n)$

in bst,

complexity of deletion

(i) best case (bst is balanced): $O(\log_2 n)$

(ii) worst case (bst is not balanced/skewed): $O(n)$

because you may need to search through every node in the tree, as there is no balance in the structure.

skewed tree

height = $O(n)$

number of comparison = $O(n)$

in bst,

complexity of deletion

(i) no best case.

(ii) no worst case

$O(n)$

because you may need to search through every node in the tree, as there is no balance in the structure.

Operation	Full Binary Tree	Skewed Tree
Searching	Best: $O(1)$, Average: $O(\log n)$, Worst: $O(n)$	Best: $O(n)$, Average: $O(n/2)$, Worst: $O(n)$
Insertion	Best: $O(1)$, Average: $O(\log n)$, Worst: $O(n)$	Best: $O(n)$, Average: $O(n/2)$, Worst: $O(n)$
Deletion	Best: $O(1)$, Average: $O(\log n)$, Worst: $O(n)$	Best: $O(n)$, Average: $O(n/2)$, Worst: $O(n)$

why worst case is $O(n)$?

(i) operations depend on tree height (h):

- most operations in a binary tree (searching, insertion, deletion) have a time complexity of $O(h)$, where h is the height of the tree.
- in a skewed tree, the height becomes equal to the number of nodes (n), making the complexity $O(n)$ in the worst case.

(ii) skewed tree = linked list

- when the tree is skewed (all nodes are on one side), it effectively behaves like a linked list.
- traversing or performing any operation in such a structure requires visiting all nodes, hence $O(n)$ complexity.

(iii) height $\approx$ number of nodes in skewed trees

- in a skewed tree, the height is roughly the same as the number of elements. therefore, all operations — searching, insertion, and deletion, take $O(n)$ time in the worst case.

topic: balanced bst

a balanced binary search tree is a binary search tree in which the difference between the heights of the left and right subtrees of every node is minimal (commonly defined as at most 1). this balance ensures the height of the tree remains $O(\log n)$, improving the efficiency of search, insertion, and deletion operations.

features of a balanced bst:

(i) height balance:

- the tree's height is kept as small as possible to optimize operations.
- ensures that all nodes are distributed evenly across levels.

(ii) operation efficiency:

- search, insertion, and deletion all operate with a time complexity of $O(\log n)$ in the average and worst cases.

types of balanced bsts:

- avl trees
- red-black trees
- b-trees

concept: AVL tree

avl tree is a self-balancing binary search tree where the difference in height between the left and right subtrees (called the balance factor) of any node is at most 1. if this balance factor is violated during insertion or deletion, rotations are performed to restore the balance.

features of an avl tree:

(i) balance factor:

- balance factor = $\text{height}(\text{left subtree}) - \text{height}(\text{right subtree})$
- valid balance factor values: -1, 0, or 1.

(ii) rotations to maintain balance:

(a) single rotation:

- left rotation or right rotation.

(b) double rotation:

- left-right rotation or right-left rotation.

(iii) time complexity:

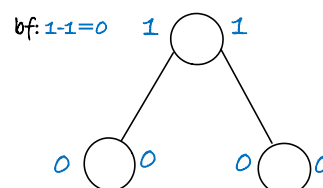
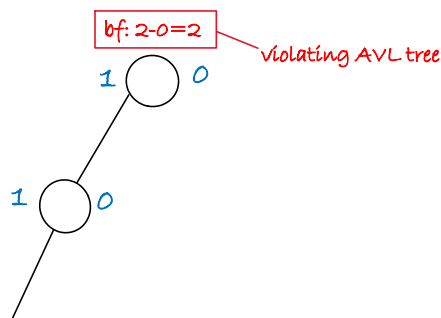
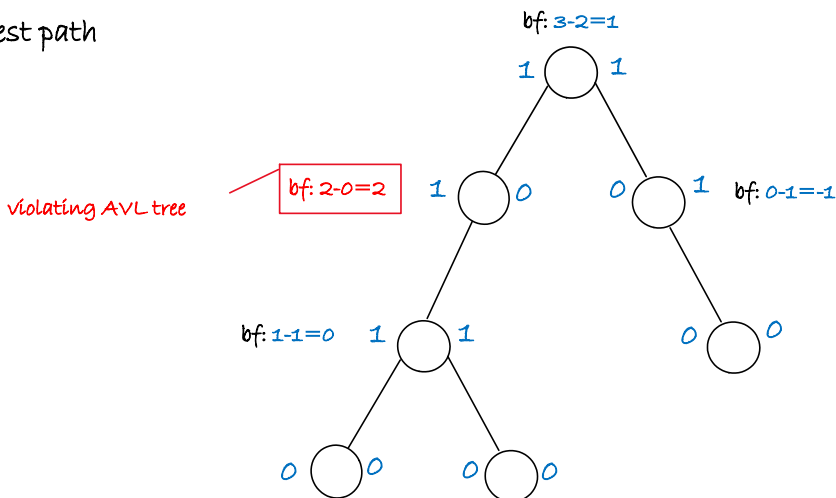
- search: $O(\log n)$
- insertion: $O(\log n)$ (may require rebalancing with rotations).
- deletion: $O(\log n)$ (may also require rebalancing).

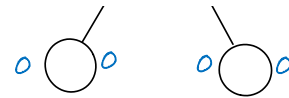
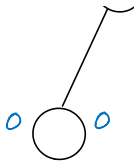
why use an avl tree:

- ensures the height of the tree is $O(\log n)$ even in the worst case.
- minimizes performance degradation compared to an unbalanced bst (which can degrade to a linked list with $O(n)$ operations).

- balance factor:

lengthiest path





to check AVL tree: structure is needed.

- to structure balance we need balancing factor

to check BST property: data is needed.

- to keep data in sorted way.

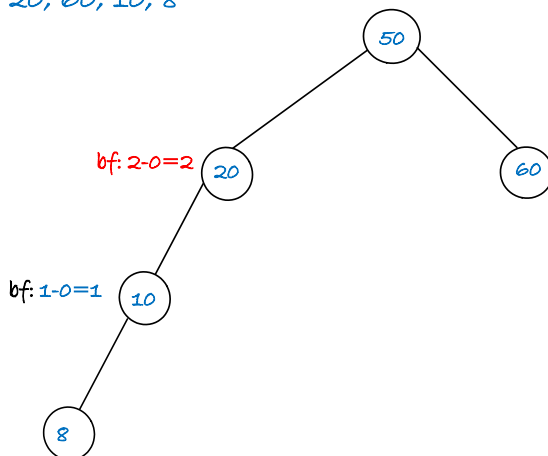
point to remember:

insertion of key may cause the balancing factor of node becomes other than +1, -1 and 0 (unbalanced), to balance it notations are performed.

- construct an AVL tree by inserting keys

50, 20, 60, 10, 8, 15, 32, 46, 11, 48

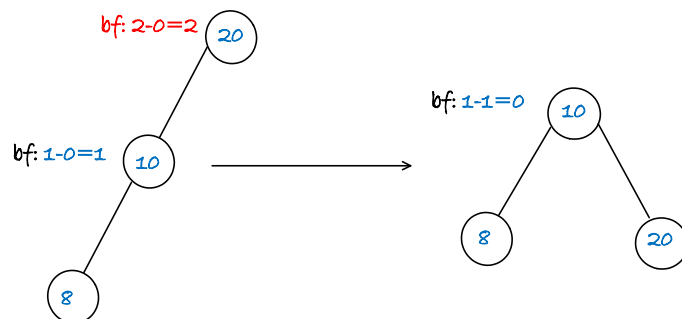
(i) insert : 50, 20, 60, 10, 8



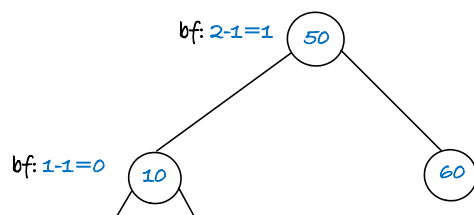
use tri-node structure:

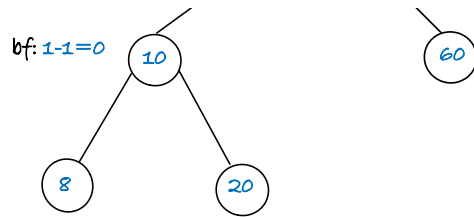
ascending order: 8 10 20

make 10 middle

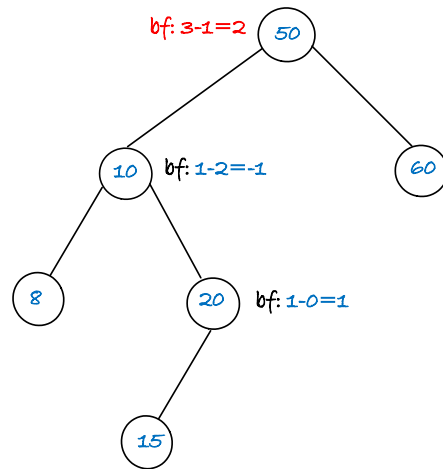


new structure:



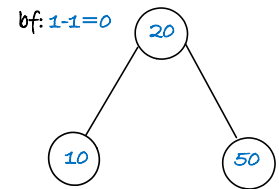
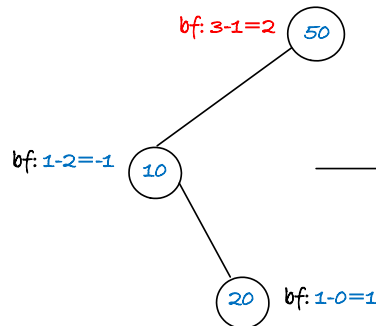


(ii) insert : 15

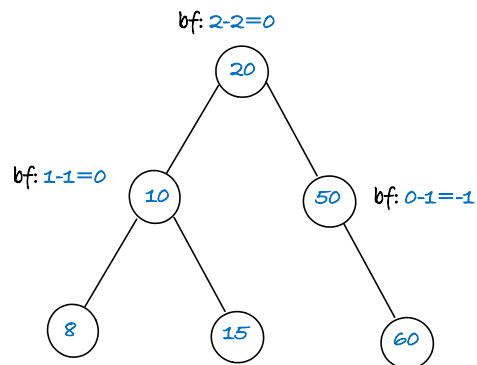


use tri-node structure:
ascending order: 10 20 50

make 20 middle

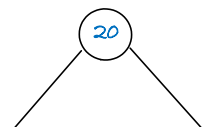


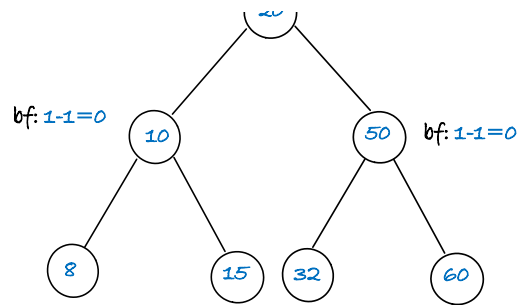
new structure:



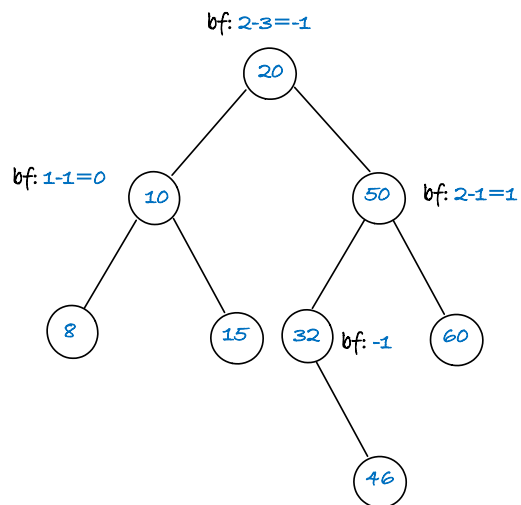
(iii) insert : 32

bf: 2-2=0

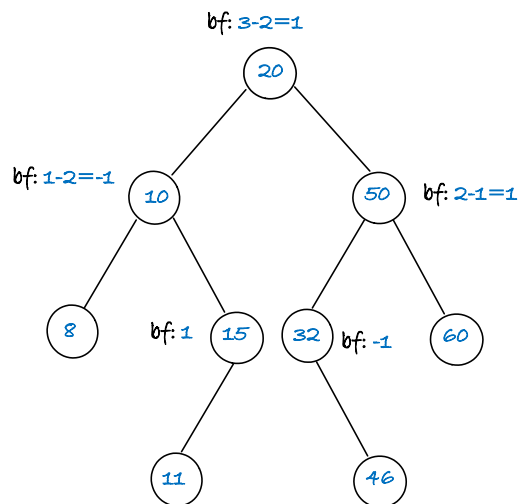




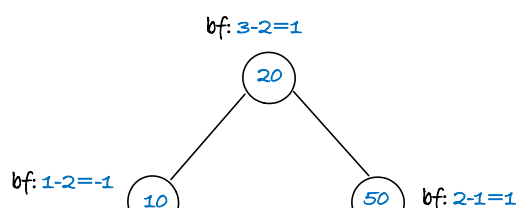
(iv) insert : 46

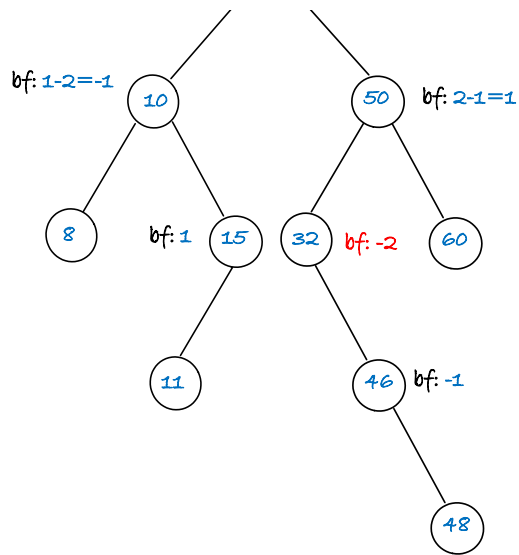


(v) insert : 11



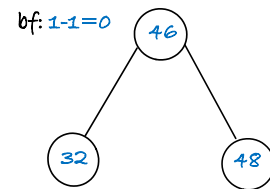
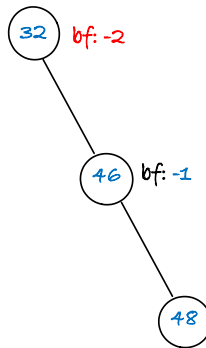
(vi) insert : 48



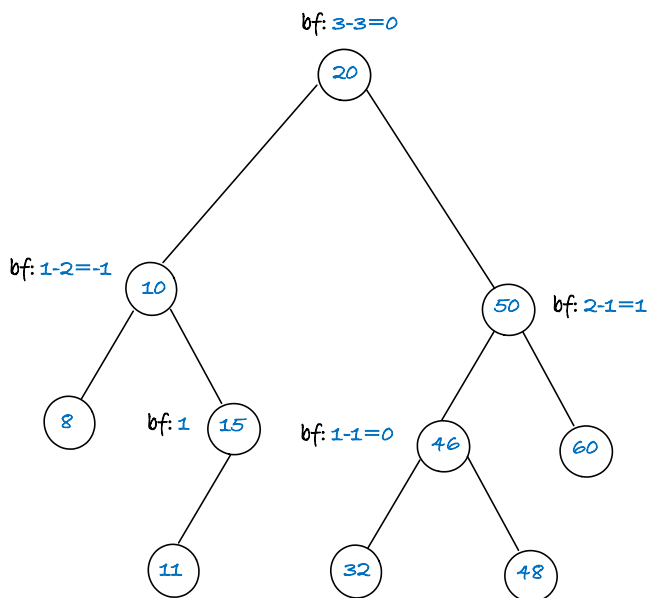


use tri-node structure:
ascending order: 32 46 48

make 46 middle



new structure:



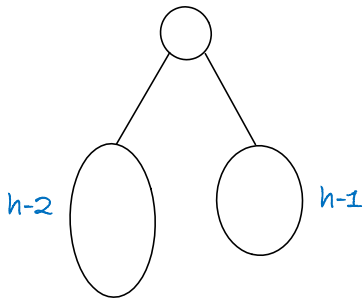
concept: height of AVL tree

maximum number of nodes in an avl tree: $2^{h+1}-1$

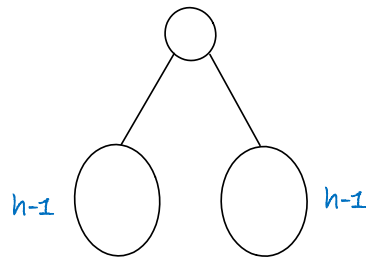
minimum number of nodes in an avl tree with height h

3 cases possible:

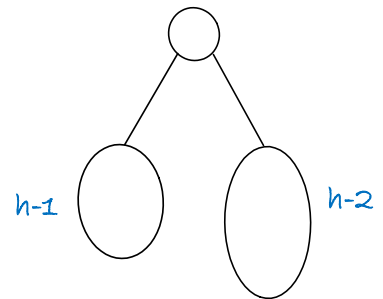
case (i)



case (ii)

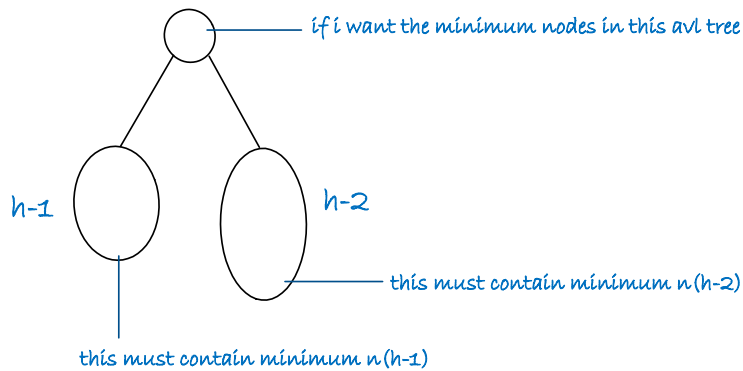


case (iii)



for minimum, either case (i) or (iii)

case (iii)



total nodes: n

so if (n) needs to be minimum

it should be equal to

$$n(h) = n(h-1) + n(h-2) + 1 \text{ (root)}$$

- $n(0)$

$$n(h) = n(h-1) + n(h-2) + 1 \text{ (root)}$$

$$n(0) = n(0-1) + n(0-2) + 1 \text{ (root)}$$

$$n(0) = 1$$

- $n(1)$

$$n(h) = n(h-1) + n(h-2) + 1 \text{ (root)}$$

$$n(1) = n(1-1) + n(1-2) + 1 \text{ (root)}$$

$$n(1) = n(0) + n(-1) + 1 \text{ (root)}$$

$$n(1) = 1 + 1$$

$$n(1) = 2$$

- $n(2)$

$$n(h) = n(h-1) + n(h-2) + 1 \text{ (root)}$$

$$n(2) = n(2-1) + n(2-2) + 1 \text{ (root)}$$

$$n(2) = n(1) + n(0) + 1 \text{ (root)}$$

$$n(2) = 2 + 1 + 1$$

$$n(2) = 4$$

- $n(3)$

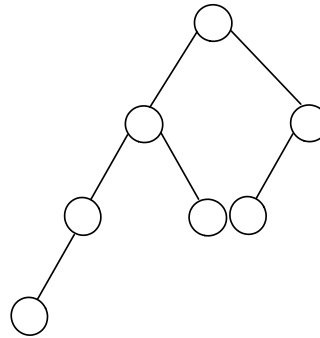
$$n(h) = n(h-1) + n(h-2) + 1 \text{ (root)}$$

$$n(3) = n(3-1) + n(3-2) + 1 \text{ (root)}$$

$$n(3) = n(2) + n(1) + 1 \text{ (root)}$$

$$n(3) = 4 + 2 + 1$$

$$n(3) = 7$$



h: 0 1 2 3 4 5 6....

$n(h)$: 1 2 4 7 12 20 33

every next term is the sum of last two terms + 1

q. consider a binary tree in which every node satisfies the property: the difference between the number of nodes in left tree of a node and number of nodes in right tree of node is at most 1.

$$|n(lt) - n(rt)| \leq 1$$

what is the minimum number of nodes in such a tree of height 5?

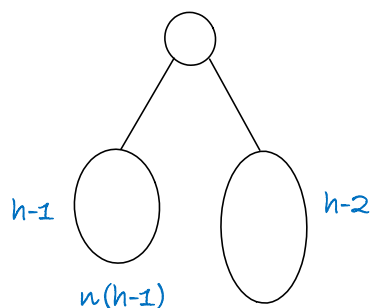
sol.

conditions for atmost 1

$$- n(h-1)$$

$$- n(h-1) + 1$$

$$- n(h-1) - 1$$



$$- n(h-1) : \text{same}$$

$$- n(h-1) + 1 : \text{same} + 1$$

$$- n(h-1) - 1 : \text{same} - 1$$

for minimum the condition (iii) is best

$$n(h-1) - 1$$

$$n(h) = n(h-1) + n(h-1) - 1 + 1$$

$$n(h) = n(h-1) + n(h-1)$$

$$n(h) = 2(n(h-1))$$

valid for >1 ?

$$n(1) = 2(n(1-1)) = 2(n(0)) = 2$$

$$n(2) = 2(n(2-1)) = 2(n(1)) = 2 \cdot 2 = 4$$

$$n(3) = 2(n(3-1)) = 2(n(2)) = 2 \cdot 4 = 8$$

$$n(4) = 2(n(4-1)) = 2(n(3)) = 2 \cdot 8 = 16$$

$$n(5) = 2(n(5-1)) = 2(n(4)) = 2 \cdot 16 = 32$$

q. what is the maximum height possible of an AVL tree with 7 nodes?

h: 0 1 2 3 4

n(h): 1 2 4 7 12

minimum: 3

with 7, 8, 9, 10, 11, maximum height possible of AVL tree is 3.

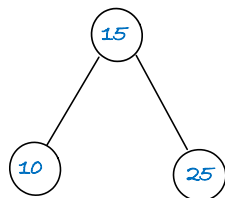
topic: heap

a heap is a specialized binary tree-based data structure that satisfies the heap property. it is primarily used to implement priority queues and is optimized for fast access to the smallest or largest element, depending on the type of heap. heap is a complete binary tree.

types of heaps:

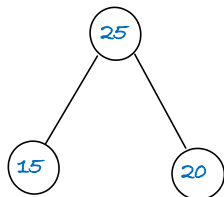
(i) min-heap:

- the value of the root node is the smallest among all the nodes.
- every parent node has a value smaller than or equal to its children.



(ii) max-heap:

- the value of the root node is the largest among all the nodes.
- every parent node has a value greater than or equal to its children.



key properties of a heap:

(i) complete binary tree:

- heaps are always represented as a complete binary tree, meaning all levels are fully filled except possibly the last, which is filled from left to right.

(ii) storage representation:

- heaps are typically stored in arrays for efficient access.
- the parent-child relationship can be derived as:
 - parent index: $(i - 1) / 2$
 - left child index: $2i + 1$
 - right child index: $2i + 2$

time complexities of heap operations:

(i) insertion: $O(\log n)$

- new elements are added to the end and then "heapified" up to restore the heap property.

(ii) deletion (removing root): $O(\log n)$

- the root is replaced by the last element, and then "heapified" down to restore the heap property.

(iii) peek (accessing the root): $O(1)$

- retrieving the minimum (min-heap) or maximum (max-heap) value takes constant time.

common applications of heaps:

(i) priority queues:

- heaps are widely used to implement efficient priority queues.

(ii) heap sort:

- a sorting algorithm based on the heap structure with $O(n \log n)$ complexity.

(iii) dijkstra's algorithm:

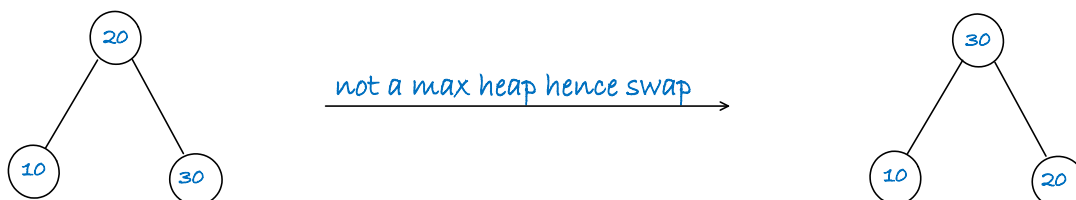
- used in shortest-path algorithms for graph traversal

q. construct a max heap by inserting keys in the order one after another
10, 20, 30, 40

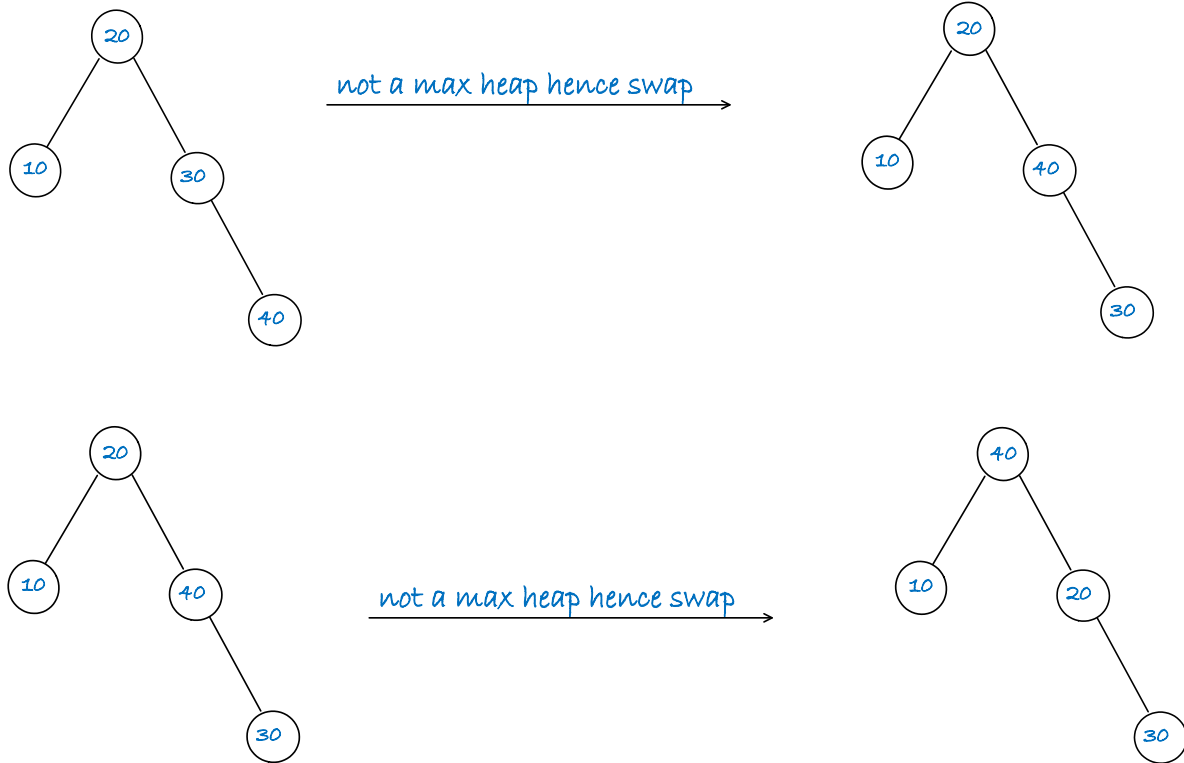
(i) insert 10 and 20



(ii) insert 30



(iii) insert 40



existing heap n nodes, insert the key

(i) best case: $O(1)$ (only one comparison)

- in the best case, the newly inserted element is larger (in a min-heap) or smaller (in a max-heap) than its parent, so no further comparisons are needed.
- the insertion will only require placing the element in the next available spot (usually at the last level), and no "heapify-up" operation is required.

(ii) worst case: $O(\log_2 n)$ (comparison at each level)

- in the worst case, the newly inserted key might require comparisons at each level to "heapify" and maintain the heap property.
- this happens if the newly inserted element has to be moved all the way up to the root, which requires $\log_2(n)$ comparisons, as the height of the heap is $\log_2(n)$. heap construction by inserting keys one after another.

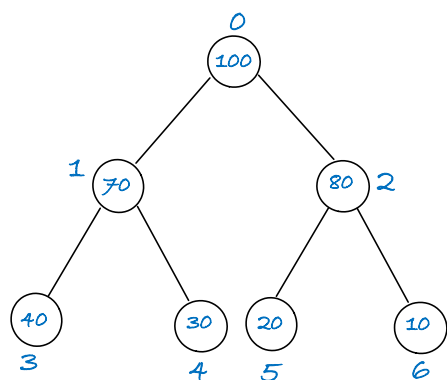
height of a complete binary tree (cbt)

- the height of a complete binary tree (cbt), which is the underlying structure of the heap, is $O(\log_2 n)$, where n is the number of nodes in the tree.

time complexity to insert one key

- the time complexity for inserting one key into an existing heap is $O(\log_2 n)$ in the worst case, because it depends on the height of the tree (which is logarithmic).

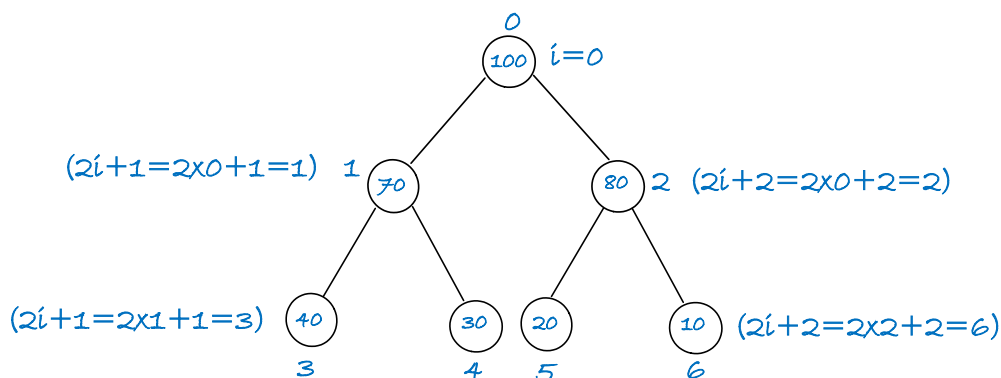
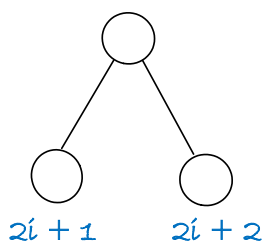
q. construct a heap by using build heap method given an array representation of a cbt convert it into a max heap (heapify method/algorithm)



array representation:

100	70	80	40	30	20	10
0	1	2	3	4	5	6

- parent index: i
- left child index: $2i + 1$
- right child index: $2i + 2$



we do not use this representation for binary trees causes a lot of wastage.

$n: O(2^n)$

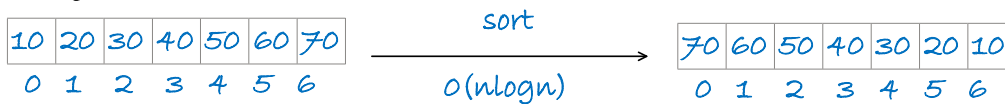
point to remember:

- array decreasing order: always represents a max heap
- array increasing order: always represents a min heap

q. given an array representation of a cbt convert it into a max heap

10, 20, 30, 40, 50, 60, 70

array representation:



but is there a better method?

yes, build heap method also known as heapify method.

build heap (heapify) method

the build heap method, also known as heapify, is a process used to convert an unordered array into a heap. this method is crucial for efficiently creating a heap structure from a given set of elements. it's typically used in heap sort or when you need to build a priority queue.

what is heapify?

- heapify is the process of rearranging the elements of a binary tree (or an array) to satisfy the heap property.
 - in a min-heap, each parent node must be smaller than its children.
 - in a max-heap, each parent node must be larger than its children.
- the build heap process ensures that a given array is transformed into a heap by applying the heapify process repeatedly.

time complexity of build heap

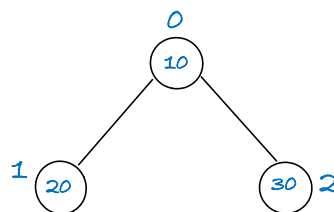
- heapify (for each individual node) takes $O(\log_2 n)$ time, as it might need to traverse the height of the tree.
- for $n/2$ non-leaf nodes (internal nodes), the total time complexity is $O(n)$.
 - this is because the cost of heapifying the nodes decreases as you move upwards in the tree (since nodes at lower levels have smaller subtrees to heapify).

index of internal nodes = 1 to $\lfloor n/2 \rfloor$ (n: size of heap)

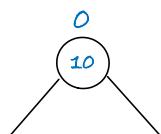
index of internal nodes = 1 to $\lfloor 7/2 \rfloor$

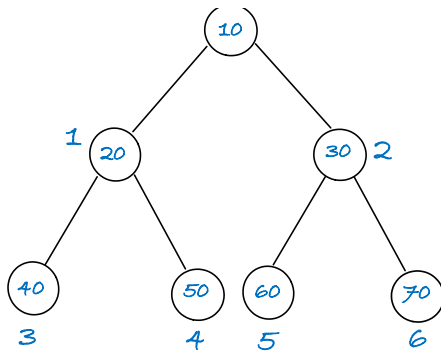
index of internal nodes = 1 to 3

index of internal nodes = 1, 2, 3



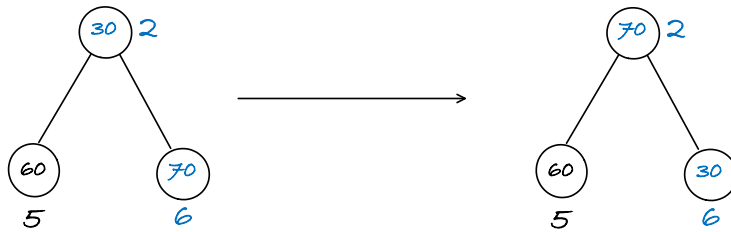
every leaf node always satisfy max heap property = 4, 5, 6, 7



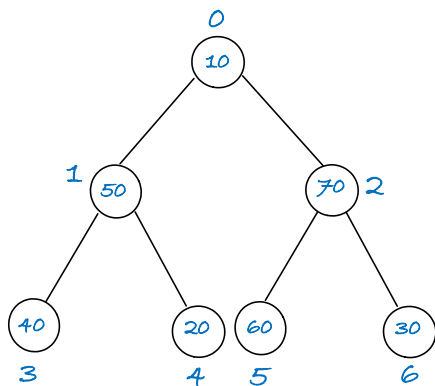
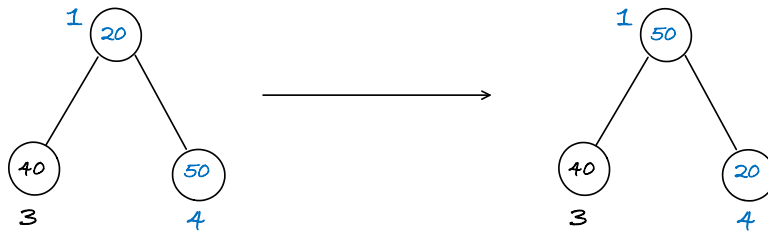


not a max heap

(i) find largest and swap

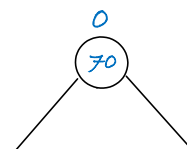
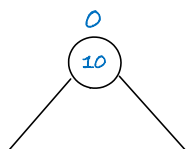


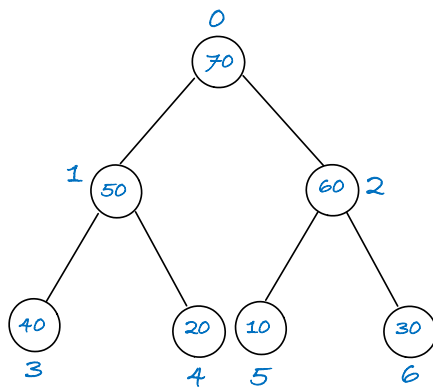
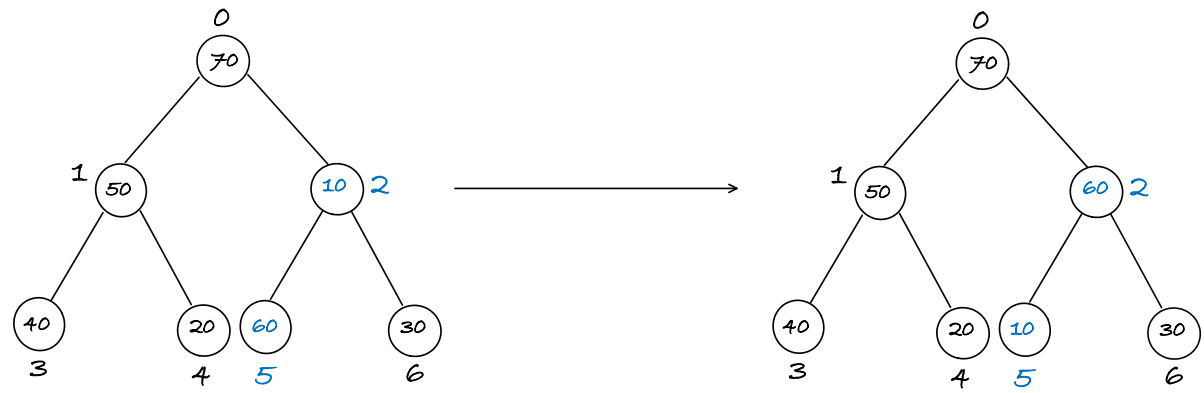
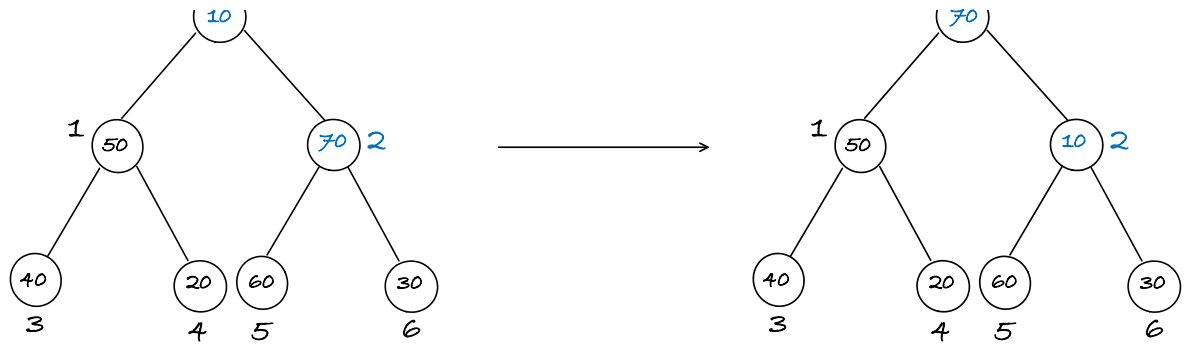
(ii) find largest and swap



still not a max heap

(iii) find largest and swap





```

def heapify(a, i, n):
    left = 2 * i      # left child index
    right = 2 * i + 1  # right child index
    largest = i        # assume the current node is the largest

    # check if left child exists and is greater than current largest
    if (left ≤ n and a[left] > a[largest]):
        largest = left;

    # check if right child exists and is greater than current largest
    if right ≤ n and a[right] > a[largest]:
        largest = right;

    # if largest is not the current node, swap and continue heapifying
    if largest ≠ i:
        a[i], a[largest] = a[largest], a[i] # swap values
        heapify(a, largest, n) # recursively heapify the affected subtree

```

Operation	Min-heap Time Complexity	Max-heap Time Complexity	Explanation
Find Min	$O(1)$	$O(n)$	- min-heap: minimum element is always at the root. - max-heap: requires searching all elements, as the minimum could be anywhere.
Find Max	$O(n)$	$O(1)$	- min-heap: maximum element can be anywhere in the heap, requiring a linear scan. - max-heap: maximum is always at the root, so it's constant time.
Insert	$O(\log n)$	$O(\log n)$	Inserting an element involves adding it at the last position and then restoring heap properties using "heapify", which takes logarithmic time.
Search	$O(n)$	$O(n)$	Searching for a specific element may require checking every node in the heap.
Extract Min	$O(\log n)$	$O(n)$	- min-heap: removing the root (min) involves swapping it with the last element and then reheapifying, which takes logarithmic time. - max-heap: there's no guaranteed way to remove the min element efficiently, so it requires checking all elements.
Extract Max	$O(n)$	$O(\log n)$	- min-heap: there's no guaranteed way to remove the max element efficiently, so it requires checking all elements. - max-heap: removing the root (max) involves swapping it with the last element and then reheapifying, which takes logarithmic time.

graph and hashing

topic: graphs

graph is a non-linear data structure

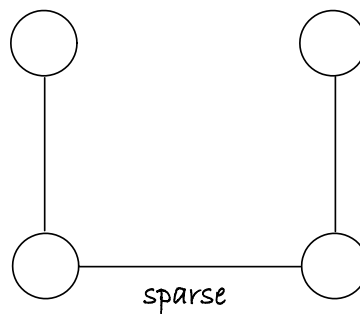
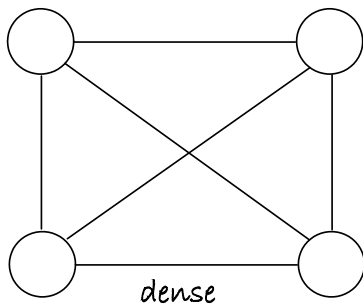
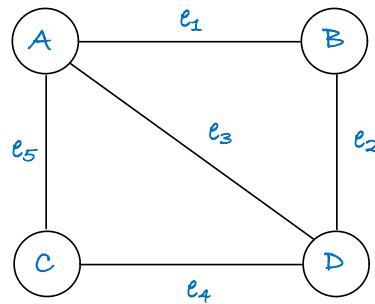
$G(V, E)$

V : set of vertices

E : set of edges

$V: \{A, B, C, D\}$

$E: \{e_1, e_2, e_3, e_4, e_5\}$



topic: graph representation

(i) adjacency matrix

(ii) adjacency list

concept: adjacency matrix

a 2d array (matrix) of size $n \times n$ is used, where n is the number of vertices in the graph. each cell (i, j) in the matrix represents whether an edge exists between vertex i and vertex j .

characteristics:

(i) space complexity:

- $O(n^2)$ irrespective of the number of edges.
- takes a lot of space for sparse graphs (graphs with fewer edges).

(ii) time complexity:

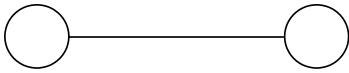
- check if an edge exists: $O(1)$ (direct access).
- add/remove edge: $O(1)$.
- traverse neighbors of a node: $O(n)$ (need to check all entries in the row).

(iii) suitable for:

- dense graphs: because all the possible edges are stored explicitly.
- graphs where edge queries (whether an edge exists) are frequent.

there are two types of edges:

(i) bi-directional/undirected edge



a bidirectional edge represents a connection between two vertices in both directions. in an undirected graph, the edge is symmetric.

characteristics:

(i) no direction: edge $u \rightarrow v$ implies both $u \rightarrow v$ and $v \rightarrow u$.

(ii) real-world examples:

- two-way streets in a city map.
- relationships in a social network (e.g., friends on facebook).

(iii) adjacency matrix:

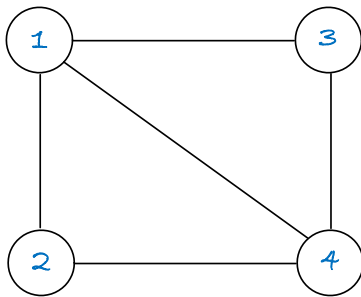
if $u \rightarrow v$ exists, then both $(u, v) = 1$ and $(v, u) = 1$.

(iv) adjacency list:

both u lists v and v lists u .

example:

do not count self loops

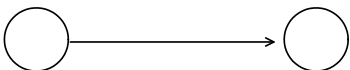


is there any edge from 1 to 1?

	1	2	3	4
1	0	1	1	1
2	1	0	0	1
3	1	0	0	1
4	1	1	1	1

$A =$

(ii) directional/directed edge



a directional edge (or directed edge) represents a connection between two vertices in a specific direction. in a directed graph, edges have an orientation, meaning they go from one vertex to another.

characteristics:

(i) direction matters: edge $u \rightarrow v$ does not imply $v \rightarrow u$.

(ii) real-world examples:

- one-way streets in a city map.

- tasks in a project schedule (task a must precede task b).

(iii) adjacency matrix:

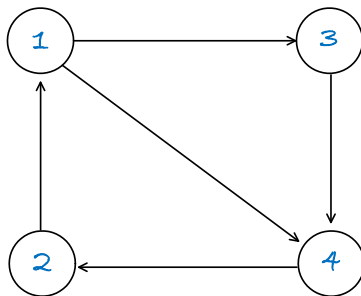
if $u \rightarrow v$ exists, then matrix entry $(u, v) = 1$; $(v, u) = 0$.

(iv) adjacency list:

u lists v as its neighbor, but v may not list u .

example:

do not count self loops



is there any edge from 1 to 2?

$A =$

	1	2	3	4
1	0	0	1	1
2	1	0	0	1
3	0	0	0	0
4	0	1	1	0

because there is no edge going out from 3

concept: adjacency list

a list of lists (or similar data structure) is used, where each vertex has a list containing its neighboring vertices. it stores only the edges that exist.

characteristics:

(i) space complexity:

- $O(v + e)$ where v is the number of vertices and e is the number of edges.
- much more space-efficient for sparse graphs.

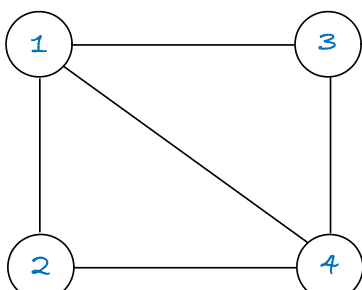
(ii) time complexity:

- check if an edge exists: $O(\text{degree of the vertex})$ (need to search the vertex's adjacency list).
- add/remove edge: $O(1)$ (amortized, depending on the implementation).
- traverse neighbors of a node: $O(\text{degree of the vertex})$.

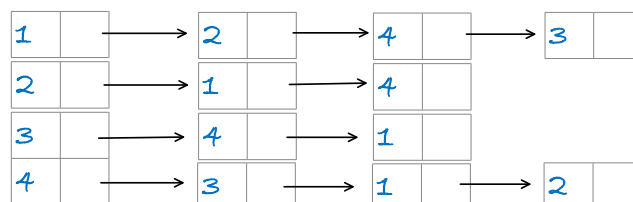
(iii) suitable for:

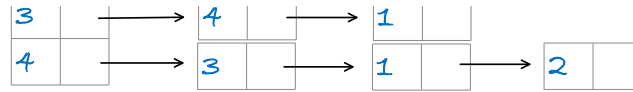
- sparse graphs: because it only stores edges that exist, saving space.
- graphs where traversal of neighbors is more frequent than edge existence queries.

example:



adjacency list





the more the dense graph is the more 1's in adjacency list will be.

aspect	directional edge	bidirectional edge
representation	(u, v)	$\{u, v\}$
direction	one-way	both ways
visual symbol	arrow ($\rightarrow$)	line ($-$)
adjacency matrix	asymmetric: $(u, v) = 1; (v, u) = 0$	symmetric: $(u, v) = (v, u) = 1$
adjacency list	u lists v , but not vice versa	u and v list each other
real-world example	one-way street	two-way street

topic: graph traversal

graph traversal refers to the process of systematically visiting all the vertices and edges in a graph. the concepts of visit and explore are key aspects of graph traversal algorithms like depth-first search (dfs) and breadth-first search (bfs).

(i) visit

visiting a node means that you have reached the node during traversal and marked it as discovered.

- purpose: visiting ensures that the node is acknowledged in the traversal sequence, and any associated action (like printing, marking, or processing) is performed.

characteristics:

- occurs when you first encounter a vertex.
 - a node is often marked as "visited" to prevent revisiting.
- example: in bfs, when you dequeue a node from the queue, it's visited.

use case: ensures no vertex is skipped during traversal.

(ii) explore

exploring a node means examining its neighbors or edges to determine further traversal paths.

- purpose: exploring ensures that all the connected components or adjacent vertices of a node are considered for traversal.

characteristics:

- involves traversing edges from the current vertex to its neighbors.
- typically, when a vertex is explored, its neighbors may be marked for a visit.

example: in dfs, when you recursively call dfs for a neighbor, you are exploring the vertex.

use case: ensures all paths and connections are fully traversed.

concept: breadth first search (graph traversal)

breadth-first search (bfs) is a graph traversal algorithm that visits all nodes at the current depth level before moving to the next depth level. bfs uses a queue data structure to keep track of nodes to visit, ensuring that nodes are explored in the order they are discovered.

how bfs works

(i) start from the source node:

- mark the source node as visited.
- enqueue it into the queue.

(ii) process nodes level by level:

- dequeue the front node from the queue.
- visit all its unvisited neighbors.
- mark those neighbors as visited and enqueue them.

(iii) repeat until the queue is empty:

- continue the process until all reachable nodes are visited.

time and space complexity

(i) time complexity:

- each vertex is enqueued and dequeued once $\rightarrow O(V)$
- all edges are explored once $\rightarrow O(E)$
- total time complexity $\rightarrow O(V + E)$

(ii) space complexity:

- for the queue $\rightarrow O(V)$
- for the visited set $\rightarrow O(V)$

applications of bfs

- shortest path: finding the shortest path in an unweighted graph.
- connected components: identifying all connected components in a graph.
- network flow: bfs is used in algorithms like edmonds-karp.
- ai and games: solving puzzles like mazes and shortest path problems.

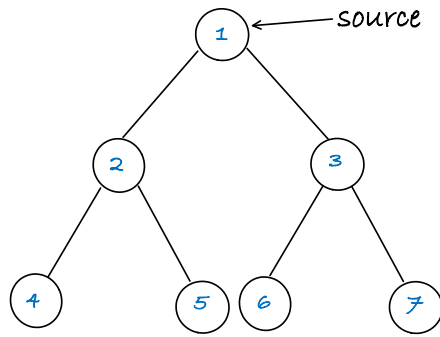
example: tree

in a tree, hierarchy is maintained, allowing us to reach any node by traversing through the left subtree or the right subtree.

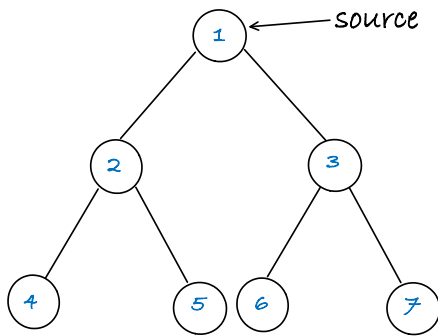
(i) start from source node



(i) start from source node



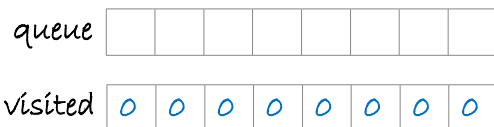
(ii) process nodes level by level



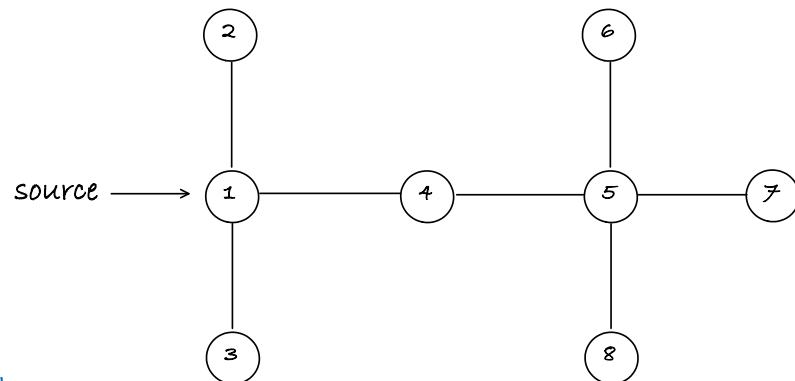
(a) from source at distance of 1: $1 \rightarrow \{2, 3\}$

(b) from source at distance of 2: $1 \rightarrow \{2, 3\} \rightarrow \{4, 5, 6, 7\}$

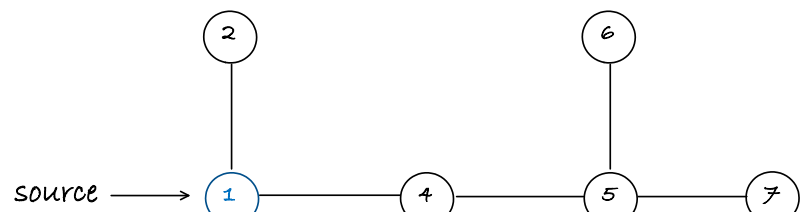
example: queue



in graphs, there may be cycles or multiple edges leading to the same node. revisiting nodes can lead to infinite loops or redundant work, the visited set keeps track of nodes that have already been processed, ensuring each node is visited only once.

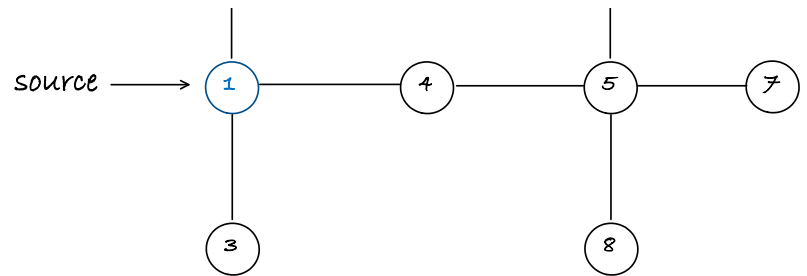


(i) start from source



queue	1							
-------	---	--	--	--	--	--	--	--

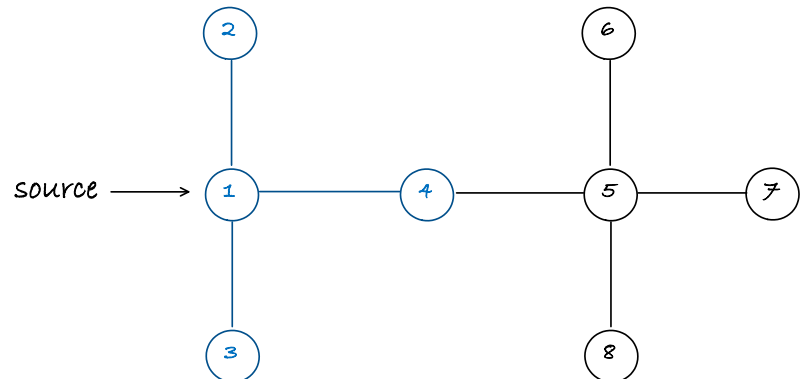
visited	1	0	0	0	0	0	0	0
	1	2	3	4	5	6	7	8



(ii) associated unvisited neighbours (2,3,4).

queue	1	2	3	4				
-------	---	---	---	---	--	--	--	--

visited	1	1	1	1	0	0	0	0
	1	2	3	4	5	6	7	8

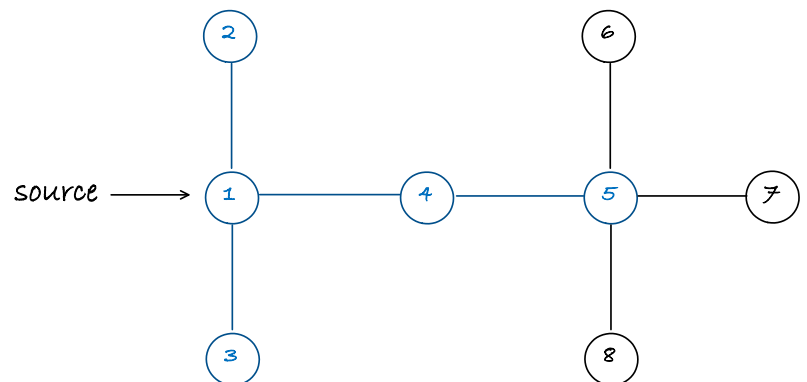


(iii) associated unvisited neighbours (5).

- 4 is also associated with 1 but it will not visit as in visited queue 1 is already marked as 1

queue	1	2	3	4	5			
-------	---	---	---	---	---	--	--	--

visited	1	1	1	1	1	0	0	0
	1	2	3	4	5	6	7	8

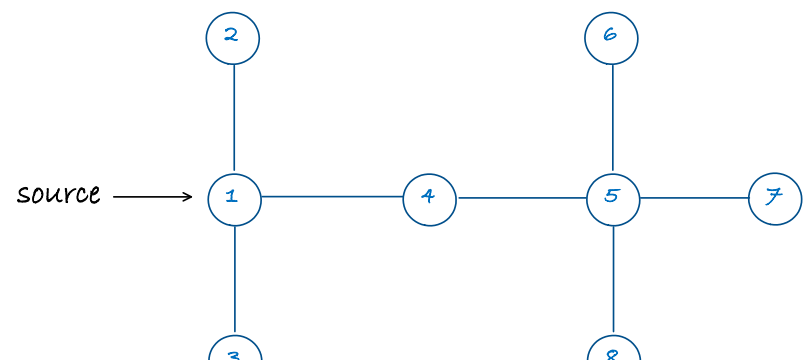


(iv) associated unvisited neighbours (6,7,8).

- 5 is also associated with 4 but it will not visit as in visited queue 4 is already marked as 1

queue	1	2	3	4	5	6	7	8
-------	---	---	---	---	---	---	---	---

visited	1	1	1	1	1	1	1	1
	1	2	3	4	5	6	7	8



1 2 3 4 5 6 7 8



$1 \rightarrow \{2, 3, 4\} \rightarrow \{5\} \rightarrow \{6, 7, 8\}$

concept: depth-first search (graph traversal)

depth-first search (dfs) is a graph traversal algorithm that explores as far as possible along each branch before backtracking. it's useful for searching trees or graphs and is implemented either using recursion or an explicit stack.

key points:

(i) explores deeper into the graph:

dfs starts at the root (or any arbitrary node) and explores as deep as possible along a branch before backtracking.

(ii) stack-based exploration:

dfs uses a stack to remember which nodes to visit next. this stack can be implicit with recursion or explicit using an actual stack data structure.

two main traversal types:

- pre-order dfs: visit the current node before its children.
- post-order dfs: visit the current node after its children.

dfs algorithm steps:

- visit the starting node: push it onto the stack.
- explore the current node's neighbors: pop the top node from the stack and explore its unvisited neighbors.
- repeat the process until all reachable nodes are visited.

dfs time and space complexity:

- time complexity:

$O(V + E)$, where V is the number of vertices and E is the number of edges. every node and edge is visited once.

- space complexity:

$O(V)$, for storing visited nodes and the stack.

key points:

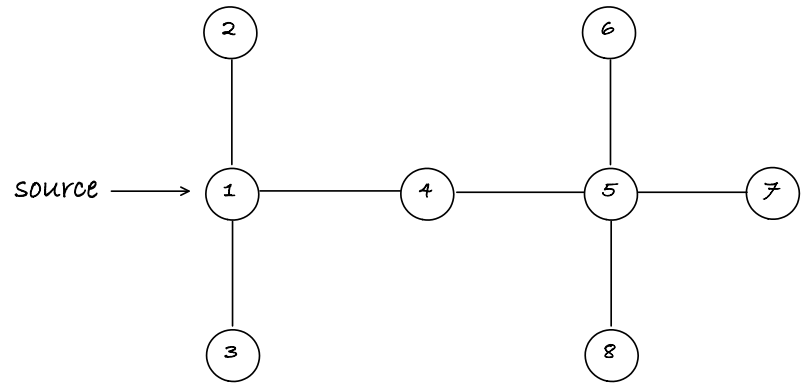
- dfs explores deeply into the graph, making it suitable for tasks like finding connected components, topological sorting, and cycle detection.
- recursion is a natural fit for dfs because the call stack automatically manages the traversal process.

(i) start from the source node



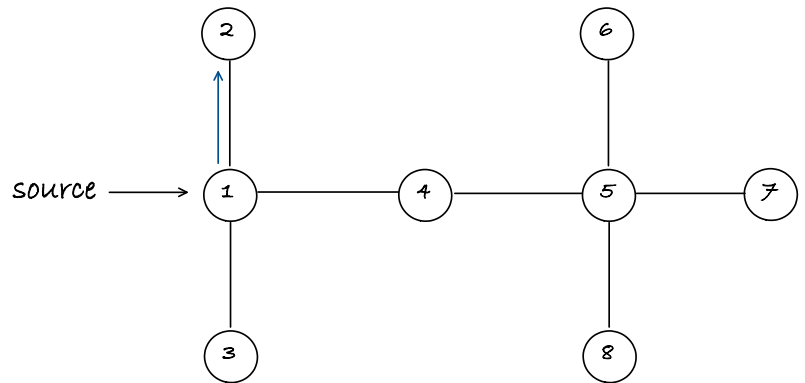
(i) start from the source node

✓
1



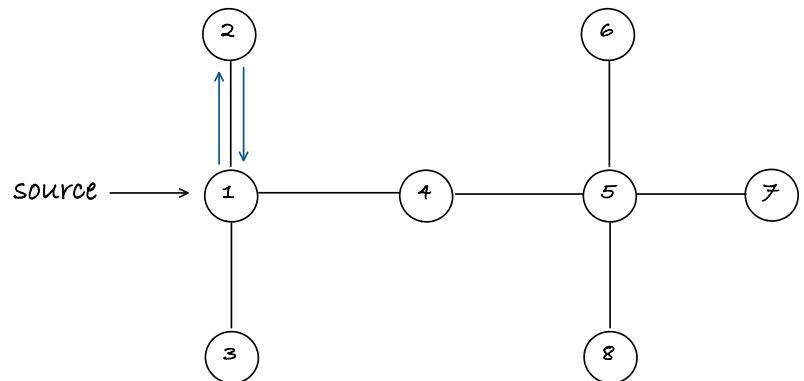
(ii) during the exploration of 1 we found 2 so stop exploring 1 and visited the unvisited node.

✗
✓ ✓
1 2



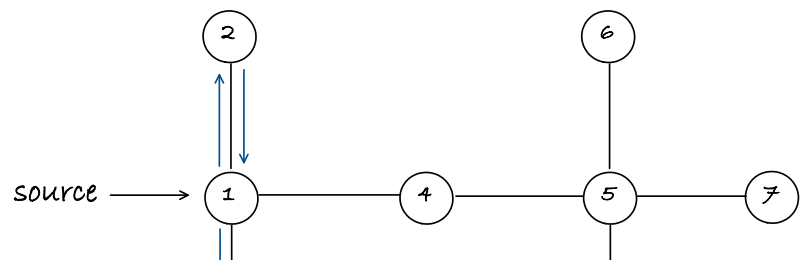
(iii) during the exploration of 2 we found nothing so stop exploring 2 and return back to 1.

✓
✗ ✗
✓ ✓
1 2

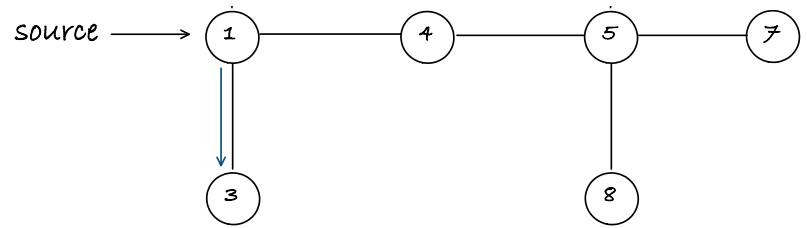


(iv) during the exploration of 1 we found 3 so stop exploring 1 and visit the unvisited node.

✗
✓
✗ ✗
✓ ✓ ✓
1 2 3

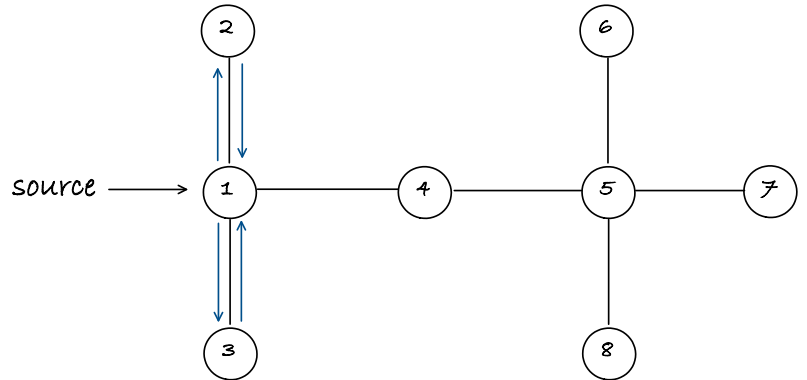


1 2 3



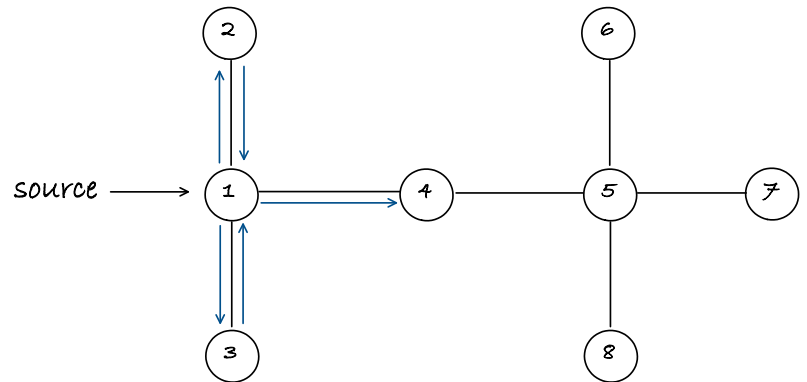
(v) during the exploration of 3 we found nothing so stop exploring 3 and return to 1.

✓
X
✓
X X X
✓ ✓ ✓
1 2 3



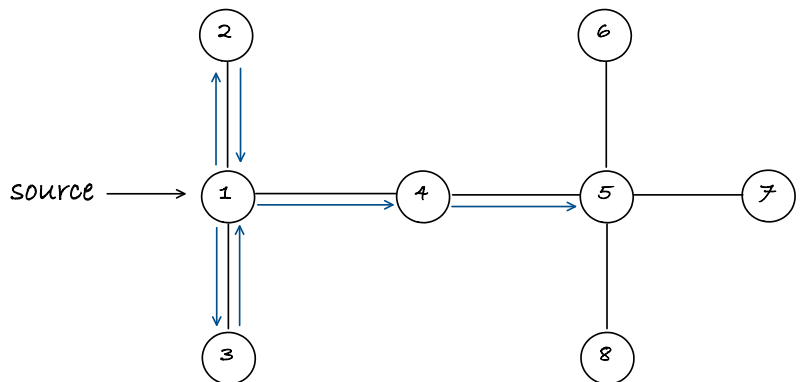
(vi) during the exploration of 1 we found 4 so stop exploring 1 and visit the unvisited node.

X
✓
X
✓
X X X
✓ ✓ ✓ ✓
1 2 3 4



(vi) during the exploration of 4 we found 5 so stop exploring 4 and visit the unvisited node.

X
✓
X
✓
X X X X
✓ ✓ ✓ ✓ ✓
1 2 3 4 5



(vii) during the exploration of 5 we found 8 so stop exploring 5 and visit the unvisited node.

X
.

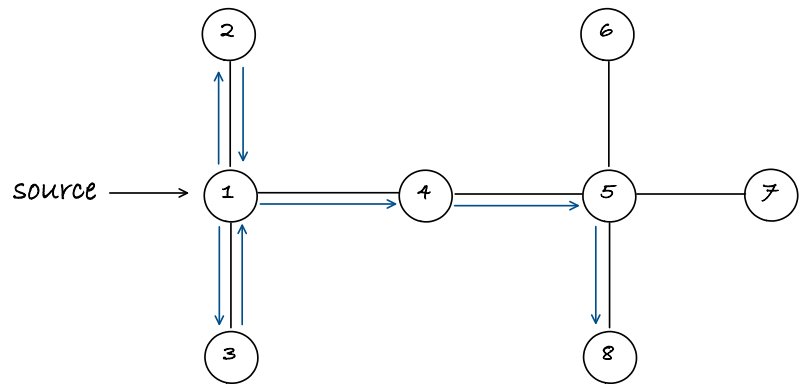
(2)

(6)

vii) during the exploration of 1 we found 2 so stop exploring 1 and visit the unvisited node.

```

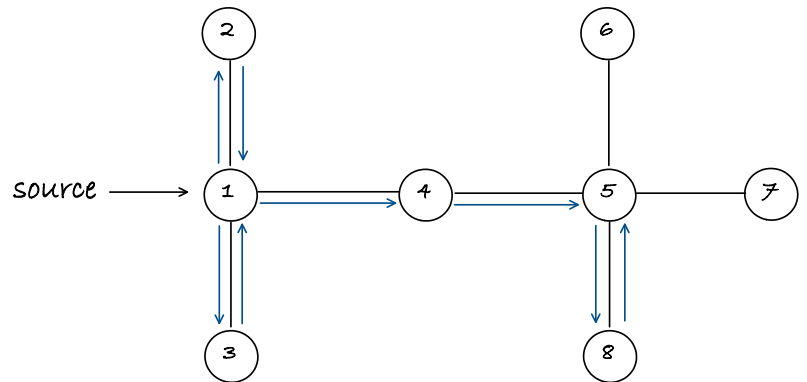
X
✓
X
✓
X X X X X X
✓ ✓ ✓ ✓ ✓ ✓
1 2 3 4 5 8
  
```



(viii) during the exploration of 8 we found nothing so stop exploring 8 and visit the return to 5.

```

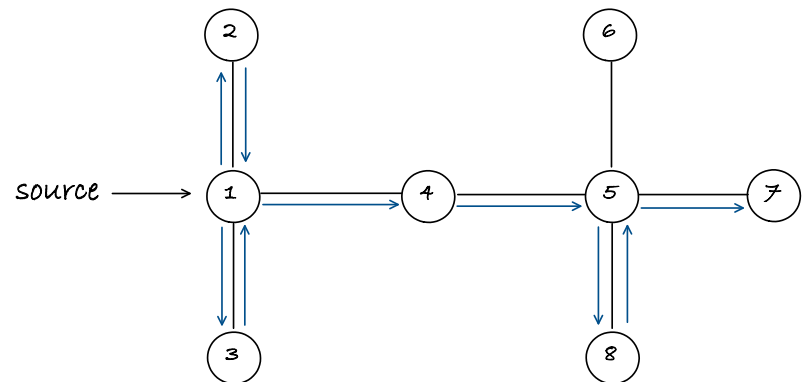
X
✓
X
✓
X X X X X X
✓ ✓ ✓ ✓ ✓ ✓
1 2 3 4 5 8
  
```



(ix) during the exploration of 5 we found 7 so stop exploring 5 and visit the unvisited node.

```

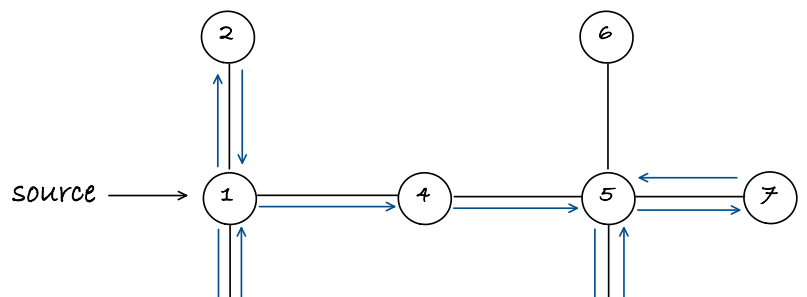
X
✓
X      X
✓      ✓
X X X X X X
✓ ✓ ✓ ✓ ✓ ✓ ✓
1 2 3 4 5 8 7
  
```



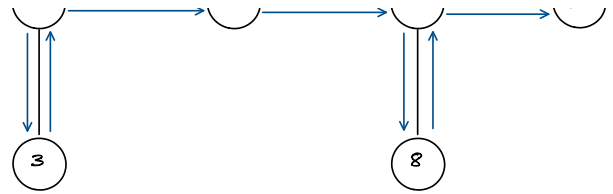
(x) during the exploration of 7 we found nothing so stop exploring 7 and return back to 5.

```

X
✓      ✓
X      X
✓      ✓
X X X X X X X
✓ ✓ ✓ ✓ ✓ ✓ ✓
1 2 3 4 5 8 7
  
```

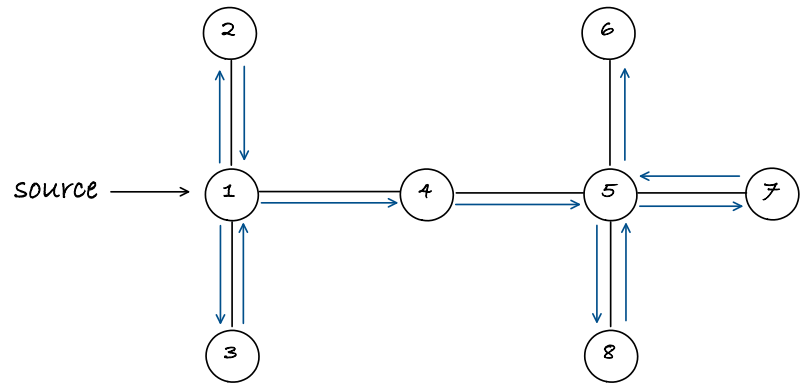


✓✓✓✓✓✓✓
1 2 3 4 5 8 7



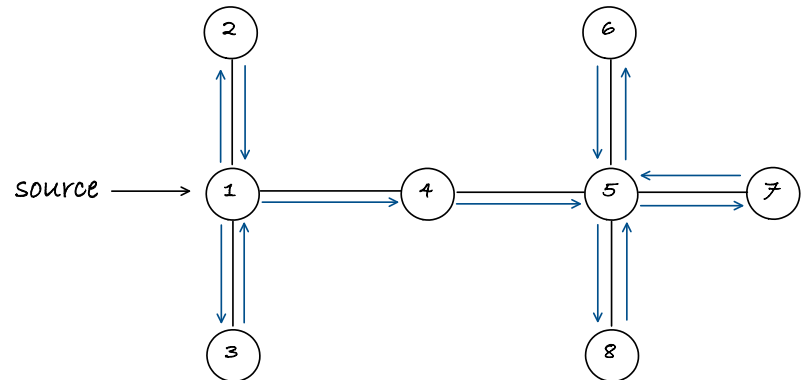
(xi) during the exploration of 5 we found 6 so stop exploring 5 and visit the unvisited node.

X X X X X X X X
✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓
1 2 3 4 5 8 7 6



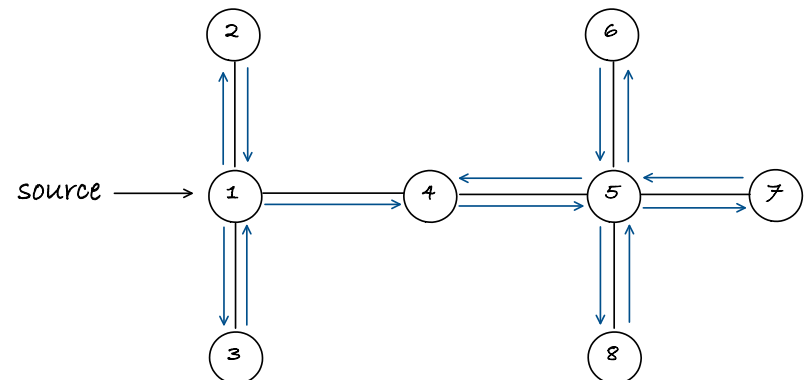
(xii) during the exploration of 6 we found nothing so stop exploring 6 and return back to 5.

X X X X X X X X
✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓
1 2 3 4 5 8 7 6



(xiii) during the exploration of 5 we found nothing so stop exploring 5 and return back to 4.

X X X X X X X X
✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓
1 2 3 4 5 8 7 6

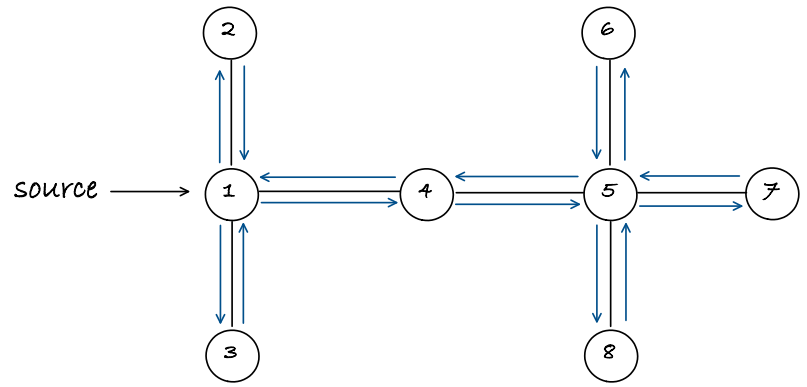


(xiv) during the exploration of 4 we found nothing so stop exploring 4 and return back to 1.

```

      X
  ✓      ✓
  X      X
  ✓      ✓
  X      X X
  ✓      ✓ ✓
  X X X X X X X X
  ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓
  1 2 3 4 5 8 7 6

```

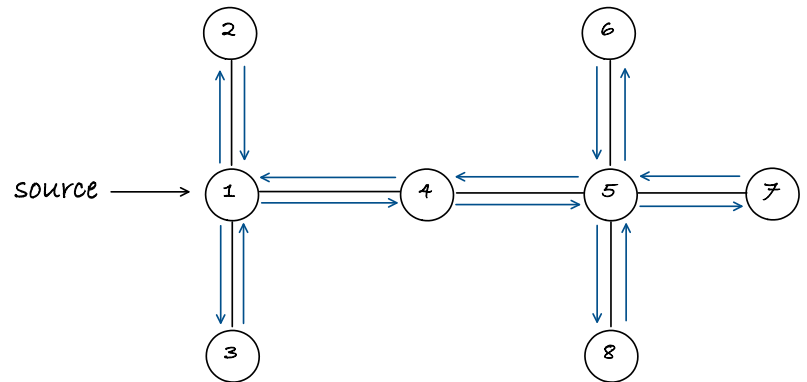


(xv) no more unvisited nodes.

```

  X      X
  ✓      ✓
  X      X
  ✓      ✓
  X      X X
  ✓      ✓ ✓
  X X X X X X X X
  ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓
  1 2 3 4 5 8 7 6

```



topic: hashing

hashing is a technique used to map data (keys) to fixed-size values, typically integers, through a hash function. it is commonly used for data retrieval and allows for efficient lookups, insertions, and deletions.

key concepts of hashing:

(i) hash function:

- a function that converts input (or key) into a fixed-size value called a hash code or hash value.
- the hash function should ideally distribute values uniformly to minimize collisions.

(ii) hash table:

- a data structure that uses a hash function to store data in an array-like structure.
- the index at which a key-value pair is stored is determined by the hash value.

(iii) collision:

- occurs when two different keys hash to the same index in the hash table.
- collision resolution techniques are used to handle this, such as chaining or open addressing.

types of hashing:

(i) direct hashing:

simple hash function that directly maps keys to an index (e.g., the key itself is used as the index).

(ii) modular hashing:

- the hash value is computed using a modulo operation:
 $\text{hash_value} = \text{key} \% \text{table_size}$
- this is one of the most common hash functions.

(iii) multiplicative hashing:

- uses a constant multiplier A to compute the hash value:
 $\text{hash_value} = \text{floor}(n * A) \% \text{table_size}$
- where n is the key and A is a constant (between 0 and 1).

(iv) division-based hashing:

- calculates the hash value by dividing the key by a prime number and taking the remainder.

applications of hashing:

(i) searching and retrieval:

provides constant time complexity $O(1)$ for average-case search, insertion, and deletion.

(ii) data encryption:

used in cryptography to map large messages to fixed-size hash values (e.g., SHA-256).

(iii) hash-based data structures:

used in hash maps, hash sets, and hash tables to store key-value pairs efficiently.

(iv) load balancing:

helps in distributing data across servers based on the hash of the data, ensuring an even load.

(v) caching:

used to quickly retrieve frequently accessed data from a cache.

time complexity:

(i) search, insert, and delete:

- average case: $O(1)$, assuming the hash function distributes keys uniformly and minimizes collisions.
- worst case: $O(n)$, if all keys hash to the same index, resulting in a chain or a full probe sequence.

(ii) space complexity:

$O(n)$ where n is the number of elements stored in the hash table.



example:

keys: 12, 18, 15, 14, 13, 29, 31, 57

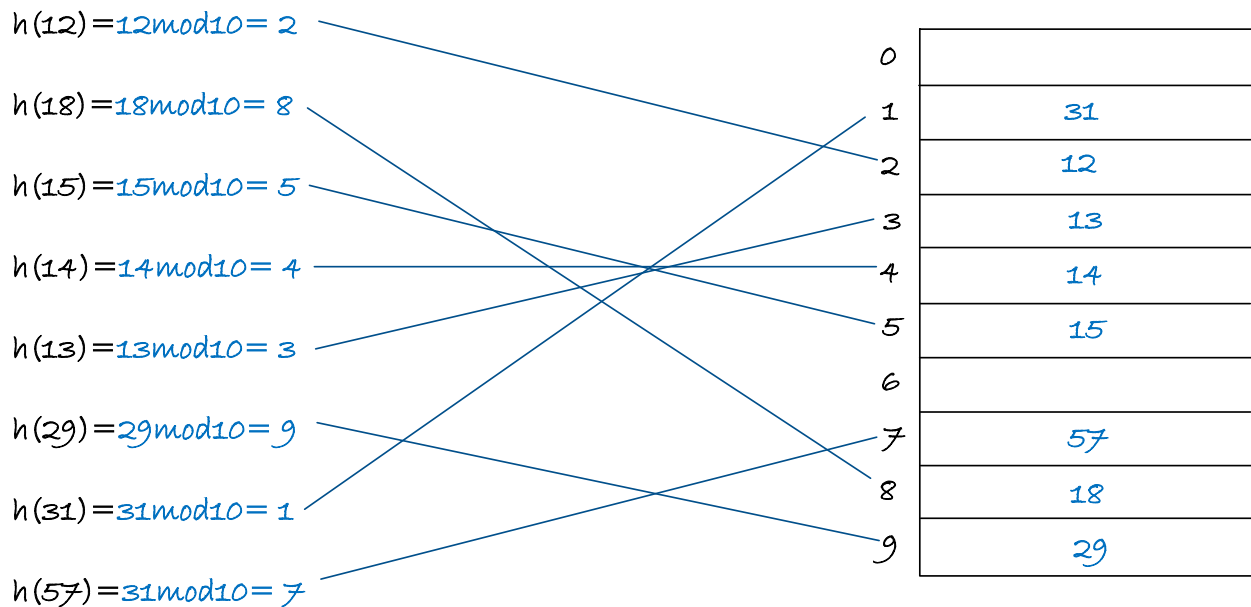
$m = 10$



i want to store these keys in such a way
that retrieval should be $O(1)$

using simple hash function:

$$h(k) = k \bmod n$$



problem with this approach:

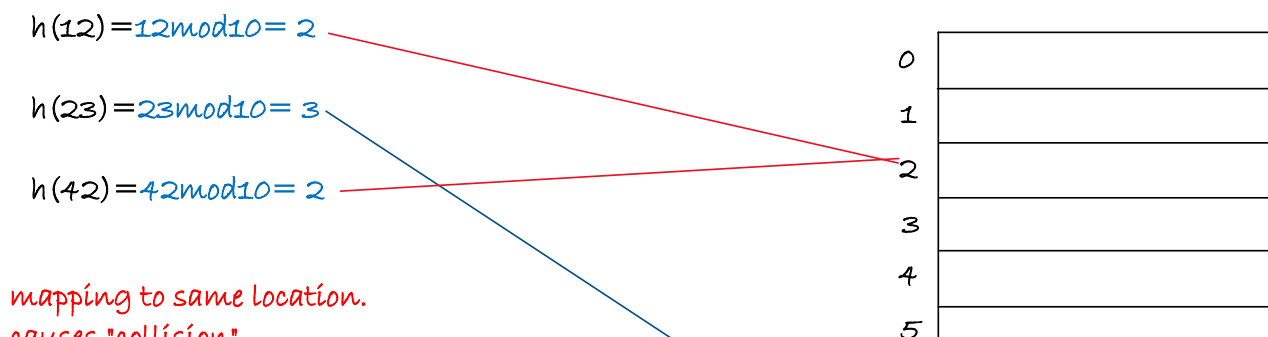
example:

keys: 12, 23, 42, 83, 54, 31, 82

$m = 10$

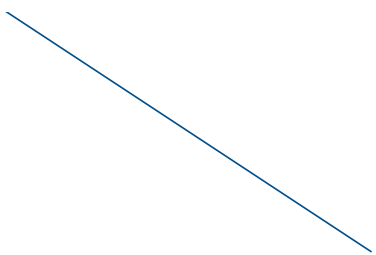
- using simple hash function:

$$h(k) = k \bmod 10$$



mapping to same location.
causes "collision"

- no 1 to 1 mapping
- no uniform distribution



4	
5	
6	
7	
8	
9	23

topic: collision resolution techniques in hashing

when two keys hash to the same index in a hash table, a collision occurs. various techniques are used to resolve collisions and ensure that keys are stored efficiently. the four main techniques for collision resolution are:

- (i) linear probing
- (ii) quadratic probing
- (iii) double hashing
- (iv) chaining

concept: linear probing

linear probing is an open addressing technique where, if a collision occurs at an index, we check the next available index in a linear sequence (i.e., increment the index by 1).

steps:

- when a key hashes to an index i and that index is already occupied, the algorithm checks the next index $(i + 1) \% \text{table_size}$.
- this process continues until an empty slot is found or the table is full.

pros:

- simple and easy to implement.
- good for small-sized hash tables or low load factors.

cons:

primary clustering: consecutive keys may cause primary clustering, where groups of consecutive occupied slots form, making future lookups slower.

function for collision:

$$h(k, i) = (h(k) + i) \bmod n$$

$$h(k_1) = l_1$$

but assume at l_1 a element already exists hence collision.

0	
1	
⋮	
⋮	
l_1	

but assume at l_1 a element already exists hence collision.

$$h(k, i) = (h(k) + i) \bmod n$$

$$h(k, 1) = (h(k) + 1) \bmod n$$

$$h(k, 1) = l_1 + 1$$

but assume at $l_1 + 1$ a element already exists hence collision number 2.

.	
l_1	
$l_1 + 1$	
$l_1 + 2$	
$l_1 + 3$	
8	
9	

why mod n?

$$n = 10$$

0, 1, 2, 3, 4, 5, 6, 7, 8, 9

let $h(k_1) = 4$ collision

$$h(k, 1) = (h(k_1) + 1) = (4 + 1) = 5 \text{ (already filled)}$$

$$h(k, 2) = (h(k_1) + 2) = (4 + 2) = 6 \text{ (already filled)}$$

$$h(k, 3) = (h(k_1) + 3) = (4 + 3) = 7 \text{ (already filled)}$$

$$h(k, 4) = (h(k_1) + 4) = (4 + 4) = 8 \text{ (already filled)}$$

$$h(k, 5) = (h(k_1) + 5) = (4 + 5) = 9 \text{ (already filled)}$$

$$h(k, 6) = (h(k_1) + 6) = (4 + 6) = 10 \text{ (doesn't exist)}$$

cells were not filled but we did not check, in order to check them we need to move circular and that is why we require mod n

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

example:

keys: 31, 26, 43, 22, 34, 46, 14, 58, 13

$$n = 12$$

(i) insert 31

simple hash function
 $h(k) = k \bmod n$
 $h(31) = 31 \bmod 12 = 7$

(ii) insert 26

simple hash function
 $h(k) = k \bmod n$
 $h(26) = 26 \bmod 12 = 2$

(iii) insert 43

simple hash function
 $h(k) = k \bmod n$
 $h(43) = 43 \bmod 12 = 7$
 collision occurred
 collision function
 $h(k, i) = (h(k) + i) \bmod n$
 $h(43, 1) = (h(43) + 1) \bmod 12$
 $h(43, 1) = (7 + 1) \bmod 12$
 $h(43, 1) = 8 \bmod 12 = 8$

(iv) insert 27

simple hash function
 $h(k) = k \bmod n$
 $h(27) = 27 \bmod 12 = 3$

(v) insert 34

simple hash function
 $h(k) = k \bmod n$
 $h(34) = 34 \bmod 12 = 10$

(vi) insert 46

simple hash function
 $h(k) = k \bmod n$
 $h(46) = 46 \bmod 12 = 10$
 collision occurred
 collision function
 $k = 46, i = 1$
 $h(k, i) = (h(k) + i) \bmod n$
 $h(46, 1) = (h(46) + 1) \bmod 12$
 $h(46, 1) = (10 + 1) \bmod 12$
 $h(46, 1) = 11 \bmod 12 = 11$

(vii) insert 14

simple hash function
 $h(k) = k \bmod n$

0	
1	
2	26
3	
4	
5	
6	
7	31
8	43
9	
10	
11	

0	
1	
2	26
3	27
4	
5	
6	
7	31
8	43
9	
10	34
11	46

0	
---	--

(vii) insert 14

simple hash function

$$h(k) = k \bmod n$$

$$h(14) = 14 \bmod 12 = 2$$

collision occurred

- collision function

$$k=14, i=1$$

$$h(k,i) = (h(k) + i) \bmod n$$

$$h(14,1) = (h(14) + 1) \bmod 12$$

$$h(14,1) = (2 + 1) \bmod 12$$

$$h(14,1) = 3 \bmod 12 = 3$$

collision occurred

- collision function

$$k=14, i=2$$

$$h(k,i) = (h(k) + i) \bmod n$$

$$h(14,2) = (h(14) + 2) \bmod 12$$

$$h(14,2) = (2 + 2) \bmod 12$$

$$h(14,2) = 4 \bmod 12 = 4$$

0	
1	
2	26
3	27
4	14
5	
6	
7	31
8	43
9	
10	34
11	46

(viii) insert 58

simple hash function

$$h(k) = k \bmod n$$

$$h(58) = 58 \bmod 12 = 10$$

collision occurred

collision function

$$k=58, i=1$$

$$h(k,i) = (h(k) + i) \bmod n$$

$$h(58,1) = (h(58) + 1) \bmod 12$$

$$h(58,1) = (10 + 1) \bmod 12$$

$$h(58,1) = 11 \bmod 12 = 11$$

collision occurred

collision function

$$k=58, i=2$$

$$h(k,i) = (h(k) + i) \bmod n$$

$$h(58,2) = (h(58) + 2) \bmod 12$$

$$h(58,2) = (10 + 2) \bmod 12$$

$$h(58,2) = 12 \bmod 12 = 0$$

0	58
1	
2	26
3	27
4	14
5	
6	
7	31
8	43
9	
10	34
11	46

(ix) insert 13

simple hash function
 $h(k) = k \bmod n$
 $h(13) = 13 \bmod 12 = 1$

0	58
1	13
2	26
3	27
4	14
5	
6	
7	31
8	43
9	
10	34
11	46

direct method:

use simple hash function, if collision occurs use collision function

$m = 12$

keys: 31, 26, 43, 22, 34, 46, 14, 58, 13

7 2 ~~7~~ 3 10 ~~10~~ ~~2~~ ~~10~~ 1
 8 11 ~~3~~ ~~11~~
 4 0

problem:

primary clustering: consecutive keys may cause primary clustering, where groups of consecutive occupied slots form, making future lookups slower.

probability that a new
key will get this slot
 $8/12$

0	58	
1	13	
2	26	
3	27	
4	14	
5		
6		
7	31	
8	43	
9		
10	34	
11	46	

concept: quadratic probing

quadratic probing is an open addressing technique where, after a collision, the next index to check is determined by a quadratic function. this helps reduce primary clustering.

steps:

- if a collision occurs at index i , the algorithm checks the index $(i + 1^2) \% \text{table_size}$, $(i + 2^2) \% \text{table_size}$, $(i + 3^2) \% \text{table_size}$, and so on, until an empty slot is found.

pros:

- reduces primary clustering compared to linear probing.
- more evenly distributes keys in the table.

cons:

- still may suffer from secondary clustering (where similar keys still collide).
- requires careful handling of table size to avoid infinite loops (if the table is nearly full).

simple hash function

$$h(k) = k \bmod n$$

collision occurred

collision function (quadratic probing)

$$h(k, i) = (h(k) + i^2) \bmod n$$

$$h(k, i) = (h(k) + i^2) \bmod n$$

$$- h(k, 1) = (h(k) + 1^2) \bmod n$$

$$h(k, 1) = l_1 + 1$$

$$- h(k, 2) = (h(k) + 2^2) \bmod n$$

$$h(k, 2) = l_1 + 4$$

does not form cluster

l_1	
l_1+1	
l_1+2	
l_1+3	
l_1+4	
l_1+5	
l_1+6	
l_1+7	
l_1+8	
l_1+9	

example:

keys: 24, 17, 32, 2, 13

$n = 11$

(i) insert 24

simple hash function

$$h(k) = k \bmod n$$

$$h(24) = 24 \bmod 11 = 2$$

(ii) insert 17

simple hash function

$$h(k) = k \bmod n$$

$$h(17) = 17 \bmod 11 = 6$$

(iii) insert 32

simple hash function

$$h(k) = k \bmod n$$

$$h(32) = 32 \bmod 11 = 10$$

(iv) insert 2

simple hash function

$$h(k) = k \bmod n$$

$$h(2) = 2 \bmod 11 = 2$$

collision occurred

collision function

$$k=2, i=1$$

$$h(k, i) = (h(k) + i^2) \bmod n$$

$$h(2, 1) = (h(2) + 1^2) \bmod 11$$

$$h(2, 1) = (2 + 1) \bmod 11$$

0	
1	
2	24
3	
4	
5	
6	17
7	
8	
9	
10	32
11	

0	
1	2
2	24
3	13
4	

$$h(2,1) = (h(2) + 1^2) \bmod 11$$

$$h(2,1) = (2 + 1) \bmod 11$$

$$h(2,1) = 3 \bmod 11 = 3$$

(v) insert 13

simple hash function

$$h(k) = k \bmod n$$

$$h(13) = 13 \bmod 11 = 2$$

collision occurred

collision function

$$k = 13, i = 1$$

$$h(k,i) = (h(k) + i^2) \bmod n$$

$$h(13,1) = (h(13) + 1^2) \bmod 11$$

$$h(13,1) = (13 + 1) \bmod 11$$

$$h(13,1) = 14 \bmod 11 = 3$$

3	15
4	
5	
6	17
7	
8	
9	
10	32
11	

problem: secondary clustering (where similar keys still collide).

key: 24, 2, 13

m: 11

(i) insert 24

simple hash function

$$h(k) = k \bmod n$$

$$h(24) = 24 \bmod 11 = 2$$

(ii) insert 2

simple hash function

$$h(k) = k \bmod n$$

$$h(2) = 2 \bmod 11 = 2$$

(iii) insert 13

simple hash function

$$h(k) = k \bmod n$$

$$h(13) = 13 \bmod 11 = 2$$

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	

collision (i)

collision occurred

collision function

$$k = 24, i = 1$$

$$h(k,i) = (h(k) + i^2) \bmod n$$

$$h(24,1) = (h(24) + 1^2) \bmod 11$$

$$h(24,1) = (24 + 1) \bmod 11$$

$$h(24,1) = 25 \bmod 11 = 3$$

collision (ii)

collision occurred

collision function

$$k = 24, i = 2$$

$$h(k,i) = (h(k) + i^2) \bmod n$$

$$h(24,2) = (h(24) + 2^2) \bmod 11$$

$$h(24,2) = (24 + 4) \bmod 11$$

$$h(24,2) = 28 \bmod 11 = 6$$

collision (iii)

collision occurred

collision function

$$k = 24, i = 3$$

$$h(k,i) = (h(k) + i^2) \bmod n$$

$$h(24,3) = (h(24) + 3^2) \bmod 11$$

$$h(24,3) = (24 + 9) \bmod 11$$

$$h(24,3) = 33 \bmod 11 = 0$$

$$h(24,1) = (h(24) + 1^2) \bmod 11$$

$$h(24,1) = (24 + 1) \bmod 11$$

$$h(24,1) = 25 \bmod 11 = 3$$

collision occurred

collision function

$$k=2, i=1$$

$$h(k,i) = (h(k) + i^2) \bmod n$$

$$h(2,1) = (h(2) + 1^2) \bmod 11$$

$$h(2,1) = (2 + 1) \bmod 11$$

$$h(2,1) = 3 \bmod 11 = 3$$

$$h(24,1) = (h(24) + 2^2) \bmod 11$$

$$h(24,1) = (24 + 4) \bmod 11$$

$$h(24,1) = 28 \bmod 11 = 6$$

collision occurred

collision function

$$k=2, i=2$$

$$h(k,i) = (h(k) + i^2) \bmod n$$

$$h(2,1) = (h(2) + 2^2) \bmod 11$$

$$h(2,1) = (2 + 4) \bmod 11$$

$$h(2,1) = 6 \bmod 11 = 6$$

$$h(24,1) = (h(24) + 3^2) \bmod 11$$

$$h(24,1) = (24 + 9) \bmod 11$$

$$h(24,1) = 33 \bmod 11 = 0$$

collision occurred

collision function

$$k=2, i=3$$

$$h(k,i) = (h(k) + i^2) \bmod n$$

$$h(2,1) = (h(2) + 3^2) \bmod 11$$

$$h(2,1) = (2 + 9) \bmod 11$$

$$h(2,1) = 11 \bmod 11 = 0$$

collision occurred

collision function

$$k=13, i=1$$

$$h(k,i) = (h(k) + i^2) \bmod n$$

$$h(13,1) = (h(13) + 1^2) \bmod 11$$

$$h(13,1) = (13 + 1) \bmod 11$$

$$h(13,1) = 14 \bmod 11 = 3$$

collision occurred

collision function

$$k=13, i=2$$

$$h(k,i) = (h(k) + i^2) \bmod n$$

$$h(13,1) = (h(13) + 2^2) \bmod 11$$

$$h(13,1) = (13 + 4) \bmod 11$$

$$h(13,1) = 17 \bmod 11 = 6$$

collision occurred

collision function

$$k=13, i=3$$

$$h(k,i) = (h(k) + i^2) \bmod n$$

$$h(13,1) = (h(13) + 3^2) \bmod 11$$

$$h(13,1) = (13 + 9) \bmod 11$$

$$h(13,1) = 22 \bmod 11 = 0$$

keys that are hashed to same locations follows the same resolution path because of which we are not able to utilize the table efficiently.

inspite of almost 50% available space we are not able to insert a new elements.

concept: double hashing

double hashing is an advanced open-addressing technique used to handle collisions in a hash table. it uses two hash functions to compute probe sequences, reducing clustering and improving performance.

why use double hashing?

- avoids clustering: unlike linear probing and quadratic probing, double hashing provides a better spread of values.
- efficient collision resolution: it ensures that probe sequences do not follow the same pattern, reducing collisions.
- good for large hash tables: works well when the table size m is a prime number, ensuring better distribution.

simple hash function

$$h(k) = k \bmod n$$

collision occurred

collision function (double hashing)

$$h(k,i) = (h(k) + i \cdot h'(k)) \bmod n$$

$$h'(k) = \neg(k \bmod \neg)$$

- $h'(k)$ never generates 0.

example:

keys: 13, 17, 21, 2, 57

$m=11$

(i) insert 13

simple hash function

$h(k) = k \bmod n$

$h(13) = 13 \bmod 11 = 2$

(ii) insert 17

simple hash function

$h(k) = k \bmod n$

$h(17) = 17 \bmod 11 = 6$

(iii) insert 21

simple hash function

$h(k) = k \bmod n$

$h(21) = 21 \bmod 11 = 10$

(iv) insert 2

simple hash function

$h(k) = k \bmod n$

$h(2) = 2 \bmod 11 = 2$

collision occurred

collision function

$k=2, i=1$

$h(k, i) = (h(k) + i \cdot h'(k)) \bmod n$

$h(2, 1) = (h(2) + 1 \cdot h'(2)) \bmod 11$

$h'(2) = 7 - (2 \bmod 7)$

$h'(2) = 7 - 2$

$h'(2) = 5$

$h(2, 1) = (h(2) + 1 \cdot 5) \bmod 11$

$h(2, 1) = (2 + 5) \bmod 11$

$h(2, 1) = 7 \bmod 11$

$h(2, 1) = 7$

problem:

overhead, because of two hash functions

concept: load factor

load factor = (n/m)

n : number of keys

m : table size